



Market Data Portal (MDP) — Complete Technical Documentation

[Coresight Research Portal — Engineering Reference](#)

Document version: 1.0

Generated: June 29, 2026

Classification: Confidential — Internal Use Only

Sections: 22 documents

CONFIDENTIAL — INTERNAL USE ONLY — DO NOT DISTRIBUTE

This document contains proprietary information belonging to Coresight Research. Unauthorized disclosure, copying, or distribution is strictly prohibited.

Table of Contents

Part 1

Market Data Portal (MDP) — Technical Documentation Index

Part 2

App Bootstrap, Page Registration, and Central Authentication

Part 3

Shared User Interface (UI) Components (`app/components/`)

Part 4

Core Infrastructure (`app/core/`)

Part 5

Data Layer (`app/data/`)

Part 6

Utilities & Caching (`app/utils/`)

Part 7

Login Page — Technical Documentation

Part 8

Logout Bridge Page — Technical Documentation

Part 9

Home Page — Technical Documentation

Part 10

Market Data — Technical Documentation

Part 11

Newsroom — Technical Documentation

Part 12

Earnings Calls — Technical Documentation

Part 13

Live Earnings Transcript — Technical Documentation

Part 14

Earnings Calendar — Technical Documentation

Part 15

Screening — Technical Documentation

Part 16

Company Filings — Technical Documentation

Part 17

Company Filings Add Files — Technical Documentation

Part 18

Logs Page — Technical Documentation

Part 19

Retailer Adding Page — Technical Documentation

Part 20

Forecasting Page (Revenue Estimates) — Technical Documentation

Part 21

Access Management Page — Technical Documentation

Part 22

Forecasting Admin Page — Technical Documentation

Application: Market Data Portal (MDP)

Stack: Streamlit multipage app · Python 3 · SQLAlchemy 2 · Azure MySQL Flexible Server · OpenID Connect (OIDC) via Coresight Identity Provider (IdP) · Azure Blob Storage · MCP servers

How to Read These Docs

This folder documents the **Market Data Portal (MDP)** for engineering review. Use it in this order:

1. **Start here (README)** — architecture, auth summary, page registry, warmup flags.
2. **Infrastructure (`00-infrastructure/`)** — shared bootstrap, components, core, data layer, and utils. Read before diving into any page doc.
3. **Page docs (`pages/`)** — one markdown file per registered route; each opens with architecture diagrams, then tables for dependencies, session state, and data layer.
4. **Diagrams first** — every doc leads with Mermaid flowcharts (`flowchart TD/LR`) for sequences and pipelines, and `sequenceDiagram` for multi-actor auth and Application Programming Interface (API) flows. Color legend is consistent across all docs (see Architecture Overview).
5. **PDFs** — generated exports live under `_pdf/` (individual per-doc PDFs in `_pdf/individual/`, merged bundle at `_pdf/Market-Data-Portal-Technical-Documentation-Complete.pdf`). Diagrams are **rendered as images** (not code blocks). Regenerate with `scripts/generate_technical_doc_pdf.py` (see [Generating PDFs](#)).

Conventions: paths are repo-relative (`app/pages/...`). Table names use the `coreiq` prefix. Auth cookie `auth_session` is the runtime source of truth; `market_data_user_sessions` is audit-only.

Architecture Overview

The portal is a Streamlit multipage application. Every browser request executes `app/main.py` first, which bootstraps authentication, database connections, and background warmups before delegating to the active page module.

High-level map from browser request through `main.py`, application layers, and external integrations.

Figure 1 Part A — Architecture layers

Figure 1 Part A — Architecture layers

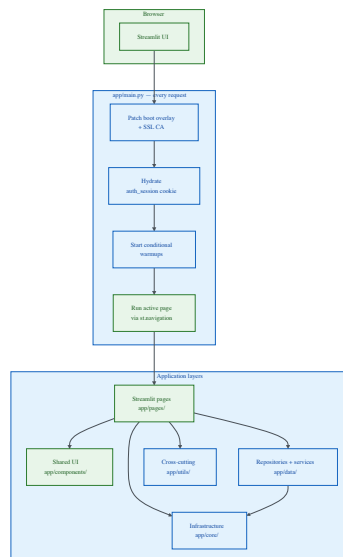
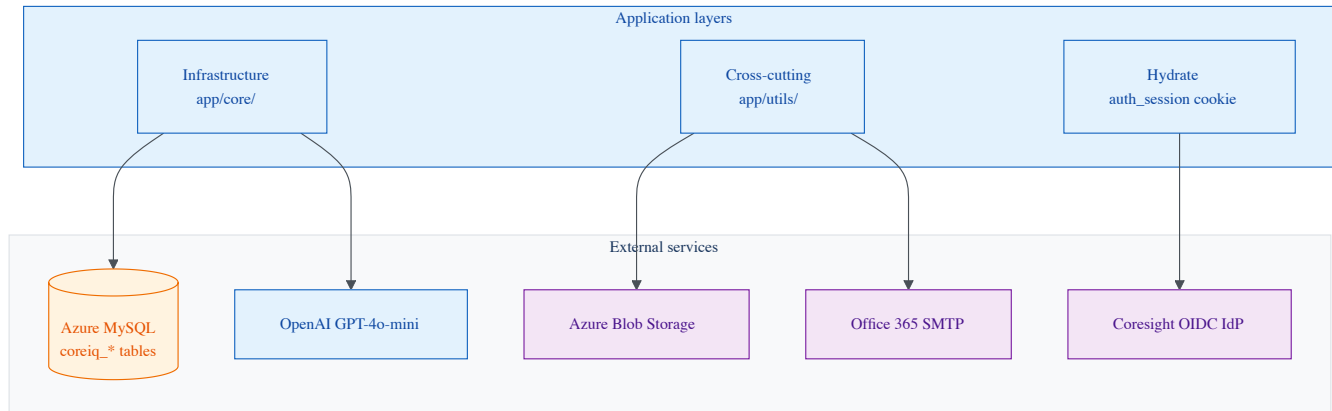


Figure 1 Part B — Data flow to external services

Figure 1 Part B — Data flow to external services



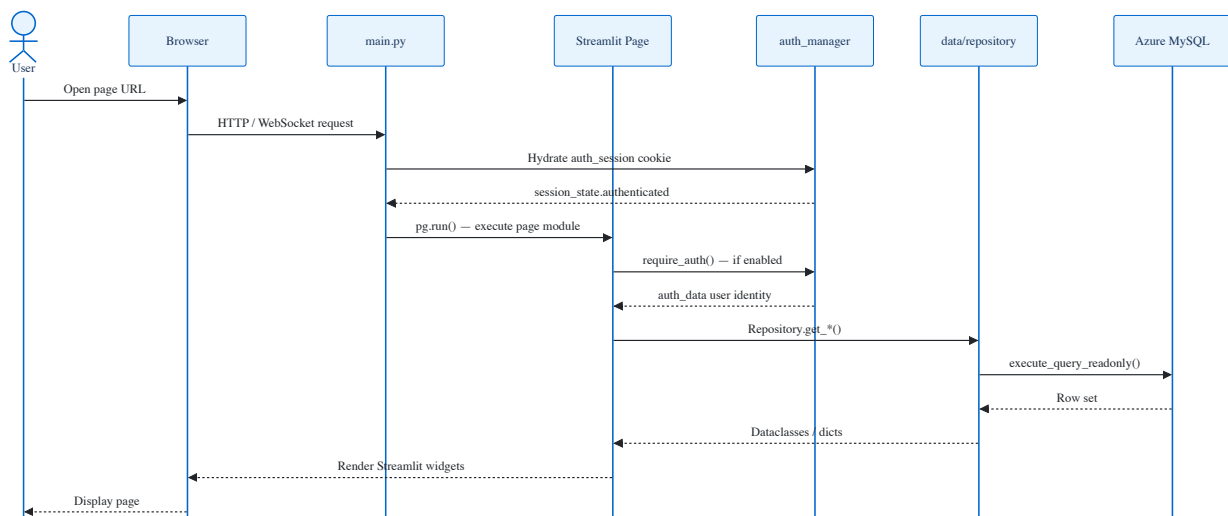
Layer responsibilities

Layer	Path	Responsibility
Entry	<code>app/main.py</code>	Page registration, auth bootstrap, startup warmups
Pages	<code>app/pages/</code>	One file = one Streamlit route; page-specific User Interface (UI) and orchestration
Components	<code>app/components/</code>	Shared UI: navigation, styles, toolbar, company profile
Data	<code>app/data/</code>	Models, repositories, screening engine, forecasting services
Core	<code>app/core/</code>	Config, database, auth, Access Control List (ACL), Large Language Model (LLM) extraction, search
Utils	<code>app/utls/</code>	Logging, caching, Azure Blob, Portable Document Format (PDF), email, constants

Data flow (typical authenticated page)

Typical authenticated request: cookie hydrate in `main.py`, optional `require_auth()` on the page, then a repository read via the AUTOCOMMIT read engine.

Typical authenticated request: cookie hydrate in `main.py`, optional `requireauth()` on the page, then a repository read via the AUTOCOMMIT read engine.



Shared Infrastructure Documentation

Document	Scope
00-app-bootstrap-and-auth.md	<code>main.py</code> startup, page registration, OIDC auth flow
01-shared-components.md	<code>app/components/</code> — navigation, styles, toolbar, charts, tables
02-core-infrastructure.md	<code>app/core/</code> — config, database, auth, ACL, LLM, search
03-data-layer.md	<code>app/data/</code> — models, repository, screening, forecasting
04-utils-and-caching.md	<code>app/utils/</code> — logger, cache, Azure Blob, PDF, email

Page Registry (`main.py`)

Pages are registered in `_PAGES_CONFIG` and exposed via `st.navigation(..., position="hidden")`. The default landing page is **Login**.

Page file	Title	URL path	Detailed doc
<code>pages/login.py</code>	Login	<code>/login</code>	login.md
<code>pages/logout_bridge.py</code>	Logout	<code>/logout_bridge</code>	logout-bridge.md
<code>pages/home.py</code>	Home	<code>/home</code>	home.md
<code>pages/market_data.py</code>	Market Data	<code>/market_data</code>	market-data.md
<code>pages/newsroom.py</code>	Newsroom	<code>/newsroom</code>	newsroom.md
<code>pages/earnings_calls.py</code>	Earnings Calls	<code>/earnings_calls</code>	earnings-calls.md
<code>pages/live_earnings_transcript.py</code>	Live Transcript	<code>/live_earnings_transcript</code>	live-earnings-transcript.md
<code>pages/earnings_calendar.py</code>	Earnings Calendar	<code>/earnings_calendar</code>	earnings-calendar.md
<code>pages/screening.py</code>	Screening	<code>/screening</code>	screening.md
<code>pages/company_filings.py</code>	Company Filings	<code>/company_filings</code>	company-filings.md
<code>pages/company_filings_add_files.py</code>	Add Files	<code>/company_filings_add_files</code>	company-filings-add-files.md
<code>pages/logs.py</code>	Logs	<code>/logs</code>	logs.md
<code>pages/retailer_adding.py</code>	Retailers	<code>/retailer_adding</code>	retailer-adding.md
<code>pages/forecasting.py</code>	Forecasting	<code>/forecasting</code>	forecasting.md
<code>pages/access_management.py</code>	Access Management	<code>/access_management</code>	access-management.md

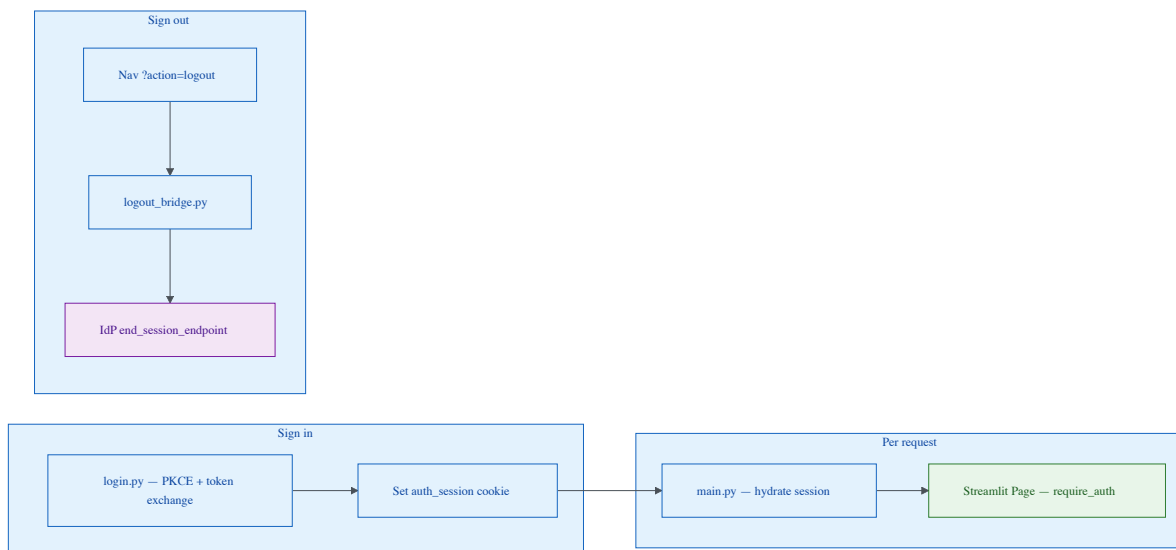
Unregistered pages (exist in `app/pages/` but not in `main.py`)

Page file	Notes	Detailed doc
<code>admin_iam.py</code>	Admin IAM — ACL-gated	—
<code>audit.py</code>	Audit log viewer — admin	—
<code>forecasting_admin.py</code>	Forecast admin — ACL-gated	forecasting-admin.md
<code>reports.py</code>	Reports — admin	—
<code>settings.py</code>	Settings — admin	—

Authentication Summary

OpenID Connect (OIDC)-only via Coresight Identity Provider (IdP). Cookie `auth_session` is the source of truth; database `market_data_user_sessions` is audit-only.

OpenID Connect (OIDC)-only via Coresight Identity Provider (IdP). Cookie `authsession` is the source of truth; database `marketdatausersessions` is audit-only.



Authentication is enforced via `main.py` cookie hydration (`is_authenticated()` on every request) and page-level `require_auth()` where enabled. Pages that do not call `require_auth()` still receive identity from the central bootstrap but do not redirect unauthenticated visitors to `/login`. See [00-app-bootstrap-and-auth.md](#) for the full page matrix.

See [00-app-bootstrap-and-auth.md](#) and [02-core-infrastructure.md](#) for full detail.

Database Summary

Two SQLAlchemy engines in `core/database.py` :

Engine	Use	RTT on Azure
Read (<code>_read_engine</code> , AUTOCOMMIT)	All SELECTs, single-statement writes	~1 RTT
Main (<code>_engine</code> , transactional)	<code>get_session()</code> , multi-query batches	~3 RTTs

All tables prefixed `coreiq_` (55 tables). See [02-core-infrastructure.md](#) and [03-data-layer.md](#).

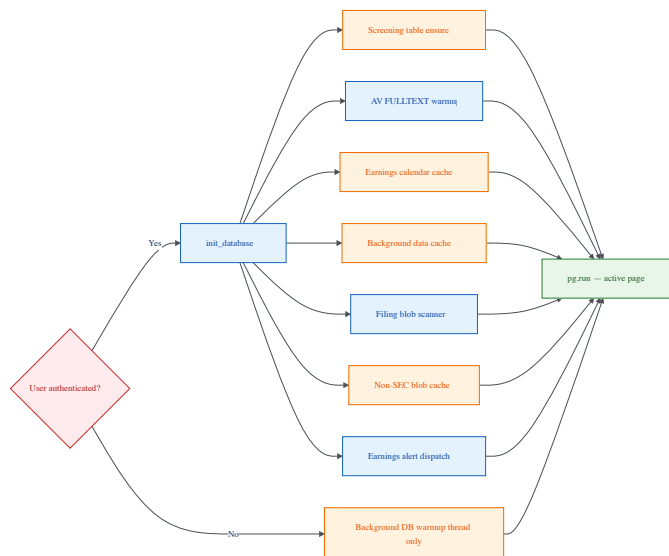
Startup Warmups (authenticated only)

Warmup	Module	Env gate
Database init	<code>core.database.init_database</code>	Always when authenticated
Screening tables	<code>watchlist/saved_criteria/portal/earnings_alert_ensure_tables</code>	<code>APP_SCREENING_PREWARM_STARTED</code>
Alpha Vantage (AV) FULLTEXT	<code>warmup_av_fulltext</code>	<code>ENABLE_AV_FT_WARMUP=1</code>
Earnings calendar cache	<code>repository.warmup_ec_caches</code>	<code>ENABLE_EC_WARMUP=1</code>
Background data cache	<code>cache_manager.start_background_warmup</code>	<code>ENABLE_BG_WARMUP=1</code>
Filing blob scanner	<code>background_scanner</code>	<code>ENABLE_BG_SCANNER=1</code>
Non-U.S. Securities and Exchange Commission (SEC) filing cache	<code>non_sec_blob_fallback</code>	Once per process
Earnings alert dispatch	<code>earnings_alert_dispatcher</code>	<code>EARNINGS_ALERT_DISPATCH=1</code>

Unauthenticated requests (login page) only get a lightweight background Database (DB) warmup thread.

Authenticated warmups run in parallel background threads after `init_database()`:

Authenticated warmups run in parallel background threads after `initdatabase()`:



Key External Integrations

Integration	Env vars	Purpose
Coresight OIDC	IDP_BASE_URL , OIDC_CLIENT_ID , OIDC_CLIENT_SECRET , OIDC_REDIRECT_URI	Authentication
Azure MySQL	STG_DB_* or PROD_DB_*	All persistent data
Azure Blob	AZURE_STORAGE_ACCOUNT_NAME , AZURE_STORAGE_ACCOUNT_KEY , AZURE_BLOB_CONTAINER	Filing PDFs
OpenAI	OPENAI_API_KEY	eXtensible Business Reporting Language (XBRL) metric LLM extraction
Simple Mail Transfer Protocol (SMTP)	SMTP_SERVER , EMAIL_PASSWORD , FROM_EMAIL	Earnings alert emails

Documentation Folder Map

```
docs/technical-documentation/
├─ README.md                # This index
├─ FOLDER-STRUCTURE.md      # Reusable taxonomy template for future products
├─ 00-infrastructure/       # Mirrors app/core, app/data, app/utils, app/components, main.py
│   └─ 00-app-bootstrap-and-auth.md
│   └─ 01-shared-components.md
│   └─ 02-core-infrastructure.md
│   └─ 03-data-layer.md
│   └─ 04-utils-and-caching.md
├─ pages/                   # One doc per Streamlit route (mirrors app/pages/)
│   └─ login.md
│   └─ home.md
│   └─ ...
├─ _pdf/                    # Generated PDF exports (do not edit by hand)
│   └─ individual/          # Per-document PDFs
│       └─ Market-Data-Portal-Technical-Documentation-Complete.pdf
├─ _templates/              # PDF HTML/CSS templates for Playwright pipeline
├─ _assets/                 # Logo and static assets for PDF cover pages
├─ _diagram_cache/          # Cached Mermaid SVG renders
└─ _mermaid_config.json     # Mermaid CLI theme config
```

Application Directory Map

```
app/
├─ main.py                  # Entry point
├─ pages/                   # One file per route
├─ components/              # Shared UI
├─ core/                    # Infrastructure
├─ data/                    # Models + repositories + services
├─ utils/                   # Cross-cutting utilities
└─ assets/                  # Static SVG/PNG icons
```

PDF Versions

PDF exports mirror markdown sources under `_pdf/individual/`. PDFs include a branded cover page, table of contents, rendered Mermaid diagrams, headers/footers with confidentiality notice, and Coresight styling.

Complete merged PDF

A single production bundle combines all 22 documents with a master cover, clickable table of contents, and section dividers:

Market-Data-Portal-Technical-Documentation-Complete.pdf

Individual per-document PDFs are in `_pdf/individual/` for targeted reference. Regenerate the merged bundle with:

```
python scripts/merge_technical_doc_pdf.py
```

Generating PDFs

Prerequisites (one-time):

```
pip install markdown          # in project venv
npx @mermaid-js/mermaid-cli  # auto-installed on first run via npx
playwright install chromium   # if not already installed for UI tests
```

Regenerate all 22 PDFs:

```
python scripts/generate_technical_doc_pdf.py
```

Regenerate one document:

```
python scripts/generate_technical_doc_pdf.py screening.md
```

Verify diagrams embedded (not raw code):

```
python scripts/generate_technical_doc_pdf.py --verify screening.pdf
```

Validate all PDFs (blank pages, diagram splits, clickable TOC samples):

```
python scripts/validate_technical_doc_pdfs.py
```

Pipeline: Markdown → Mermaid pre-render (`mmdc`) → SVG → branded HyperText Markup Language (HTML) (`_templates/_pdf_template.html` + `_templates/_pdf_styles.css`) → Playwright PDF with cover, clickable table of contents (H1/H2 with dotted leaders and page numbers), headers, and footers → PyMuPDF internal link annotations.

Assets: Logo at `_assets/coresight-logo.png` ; diagram cache at `_diagram_cache/` ; TOC validation screenshots at `_validation-toc/`.

Infrastructure

Markdown	PDF
README.md	_pdf/individual/README.pdf
00-app-bootstrap-and-auth.md	_pdf/individual/00-app-bootstrap-and-auth.pdf
01-shared-components.md	_pdf/individual/01-shared-components.pdf
02-core-infrastructure.md	_pdf/individual/02-core-infrastructure.pdf
03-data-layer.md	_pdf/individual/03-data-layer.pdf
04-utils-and-caching.md	_pdf/individual/04-utils-and-caching.pdf

Registered pages

Markdown	PDF
login.md	_pdf/individual/login.pdf
logout-bridge.md	_pdf/individual/logout-bridge.pdf
home.md	_pdf/individual/home.pdf
market-data.md	_pdf/individual/market-data.pdf
newsroom.md	_pdf/individual/newsroom.pdf
earnings-calls.md	_pdf/individual/earnings-calls.pdf
live-earnings-transcript.md	_pdf/individual/live-earnings-transcript.pdf
earnings-calendar.md	_pdf/individual/earnings-calendar.pdf
screening.md	_pdf/individual/screening.pdf
company-filings.md	_pdf/individual/company-filings.pdf
company-filings-add-files.md	_pdf/individual/company-filings-add-files.pdf
logs.md	_pdf/individual/logs.pdf
retailer-adding.md	_pdf/individual/retailer-adding.pdf
forecasting.md	_pdf/individual/forecasting.pdf
access-management.md	_pdf/individual/access-management.pdf

Unregistered pages

Markdown	PDF
forecasting-admin.md	_pdf/individual/forecasting-admin.pdf

Documentation Conventions

Element	Convention
Diagrams	Mermaid <code>flowchart TD</code> for vertical flows, <code>flowchart LR</code> for pipelines, <code>sequenceDiagram</code> for multi-actor auth/API
Colors	<code>start</code> blue · <code>ui</code> green · <code>db</code> orange · <code>ext</code> purple · <code>error</code> red · <code>process</code> grey
Paths	Relative to repository root (<code>app/...</code>)
Tables	<code>coreiq_*</code> prefix on all MySQL tables
Auth	<code>auth_session</code> cookie = runtime truth; DB session table = audit only

— End of Market Data Portal (MDP) — Technical Documentation Index —

PART 2 OF 22

App Bootstrap, Page Registration, and Central Authentication

Application entry point, OIDC bootstrap, cookie hydration, and per-page auth enforcement.

Continued from previous section →

Scope: `app/main.py`, `app/core/auth_manager.py`, `app/core/auth_environment.py`

Application: Market Data Portal (MDP)

Stack: Streamlit multipage app, SQLAlchemy 2, Azure MySQL, OpenID Connect (OIDC) via Coresight Identity Provider (IdP)

Overview

`app/main.py` is the single Streamlit entry point for every page request. Before any page script runs, it:

1. Configures the Python/runtime environment (no bytecode cache, path setup, boot overlay).
2. Registers all multipage routes via `st.navigation`.
3. Hydrates authentication from HTTP cookies into `st.session_state`.
4. Routes logout requests to `logout_bridge.py` (OIDC) or `login.py` (legacy JSON Web Token (JWT) path).
5. Starts conditional background warmups (Database (DB), screening tables, caches, scanners) only when the user is authenticated.
6. Invokes `pg.run()` to execute the active page module.

`auth_manager.py` implements the session-token authentication pattern: **cookie is the source of truth** for auth checks; the database (`market_data_user_sessions`) is used for audit only. `auth_environment.py` is the environment gate that historically chose OIDC vs JWT; as of the current codebase, **OIDC is always enabled**.

URL / Route Model

Streamlit multipage navigation is configured in `main.py` via `_PAGES_CONFIG`. Each tuple is `(file_path, title, url_path, is_default)`.

File	Title	URL path	Default?
<code>pages/login.py</code>	Login	<code>/login</code>	Yes (landing)
<code>pages/logout_bridge.py</code>	Logout	<code>/logout_bridge</code>	No
<code>pages/home.py</code>	Home	<code>/home</code>	No
<code>pages/market_data.py</code>	Market Data	<code>/market_data</code>	No
...	...	...	...

The default page is **Login** (`is_default=True`). Unauthenticated users hitting `/` land on login.

Global query parameters handled in `main.py`:

Parameter	Handler	Effect
<code>action=logout</code>	Logout routing block	OIDC → <code>logout_bridge.py</code> ; else clear session → <code>login.py</code>
<code>login_handoff=1</code>	Auth bootstrap	Verifies <code>auth_session</code> cookie was written after login
<code>auth_flow_id</code>	Auth bootstrap	Correlation ID for login journey logging
<code>browser_cookie_ok</code>	Auth bootstrap	<code>1</code> = handoff success, <code>0</code> = cookie write failed

Dependencies

`main.py` imports

Module	Purpose
<code>core.boot_overlay.patch_streamlit_index_html</code>	Patches Streamlit <code>index.html</code> to hide grey skeleton, show branded loader
<code>utils.server_logger</code>	<code>log_error</code> , <code>log_exception</code> , <code>log_timing</code> , <code>new_rerun_id</code> — no stdlib logging
<code>core.ssl_setup.ensure_ca_cert</code>	Azure MySQL SSL CA setup
<code>core.database.init_database</code>	Engine/pool initialization (authenticated path)
<code>core.auth_manager</code>	Cookie hydration, logout stash, handoff verification
<code>core.auth_environment.is_oidc_enabled</code>	Logout routing decision
<code>utils.cache_manager.start_background_warmup</code>	Background data cache warmup
<code>utils.background_scanner</code>	Filing blob cache purge + scanner
<code>utils.non_sec_blob_fallback</code>	Non-U.S. Securities and Exchange Commission (SEC) filing cache warmup thread
<code>utils.earnings_alert_dispatcher</code>	Earnings alert email dispatch tick + background thread
<code>data.repository.warmup_ec_caches</code>	Earnings calendar cache warmup
<code>data.watchlist_service</code> , <code>saved_criteria_service</code> , <code>portal_users_service</code> , <code>earnings_alert_service</code>	Screening-related table ensures (background thread)

`auth_manager.py` imports

Module	Purpose
<code>streamlit</code> / <code>st.context.cookies</code>	Sync cookie read from HTTP headers
<code>streamlit_cookies_controller.CookieController</code>	Fallback cookie read (same WebSocket session)
<code>core.database.db_manager</code>	<code>market_data_user_sessions</code> INSERT on login
<code>utils.server_logger</code>	Structured errors and timing

`auth_environment.py`

Function	Behavior
<code>is_production_deploy()</code>	True if <code>ENV</code> , <code>ENVIRONMENT</code> , or <code>APP_ENV</code> equals <code>production</code>
<code>is_oidc_enabled()</code>	Always returns <code>True</code> (JWT disabled; OIDC for all deploys)
<code>IS_OIDC_ENV</code>	Module-level snapshot: <code>is_oidc_enabled()</code> at import time

Startup Sequence (`main.py`)

Phase 1 — Process hygiene

- `sys.dont_write_bytecode = True` and `PYTHONDONTWRITEBYTECODE=1`

- One-time `__pycache__` tree deletion (guarded by `APP_PYCACHE_CLEANED`)
- `sys.path.insert(0, app_dir)` so `core.*` , `pages.*` , `components.*` resolve

Phase 2 — Boot overlay

`patch_streamlit_index_html()` modifies Streamlit's static `index.html` before the server serves it, injecting CSS/JS to suppress `[data-testid="stSkeleton"]` and show a Coresight-branded boot overlay. This runs at **process start**, before any page CSS can load.

Phase 3 — Streamlit config

```
st.set_page_config(
    page_title="Market Data Portal",
    page_icon="https://coresight.com/.../favico-32x32.png",
    layout="wide",
    initial_sidebar_state="collapsed",
)
```

Early skeleton-suppression CSS is also injected via `st.markdown` .

Phase 4 — Optional cookie reset

If `AUTH_CLEAR_COOKIES_ONCE=1` , injects JS that clears `auth_session` and `auth_logout_bridge` once per browser (`localStorage` marker `__csr_cookie_reset_v1`), then reloads.

Phase 5 — Page registration

All page files are verified to exist; missing files cause `FileNotFoundError` . Pages are wrapped in `st.Page(..., position="hidden")` and composed into `st.navigation(_st_pages, position="hidden")` .

Phase 6 — Central auth bootstrap

See [Authentication Bootstrap](#) below.

Phase 7 — Logout routing

If `?action=logout` :

- **OIDC:** `stash_logout_context()` → `st.switch_page("pages/logout_bridge.py")`
- **Non-OIDC (legacy):** Clear `auth_data` / `authenticated` , set `_auth_invalidated` , → `login.py`

Phase 8 — Conditional warmups

Condition	Work started
Not authenticated	Background DB warmup thread only (<code>APP_DB_WARMUP_STARTED</code>) — speeds OIDC callback
Authenticated	<code>init_database</code> , screening table prewarm thread, Alpha Vantage (AV) fulltext warmup, EC cache warmup, <code>start_background_warmup</code> , background scanner, non-SEC blob warmup thread, earnings alert dispatch

Environment flags: `ENABLE_BG_WARMUP` , `ENABLE_BG_SCANNER` , `ENABLE_EC_WARMUP` , `EARNINGS_ALERT_DISPATCH` , `EARNINGS_ALERT_DISPATCH_THREAD` .

Phase 9 — Run page

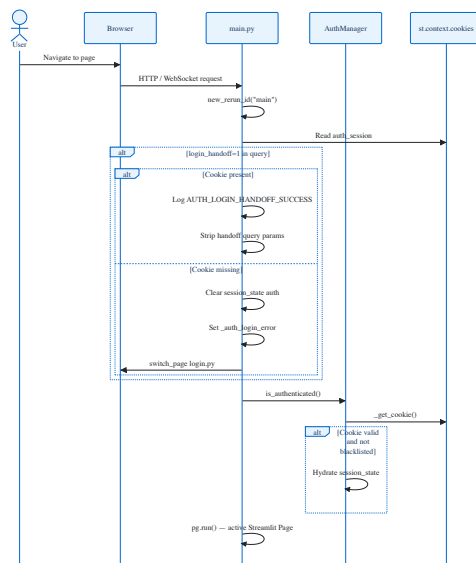
`pg.run()` executes the routed page module.

Authentication Bootstrap

On every request, before `pg.run()` :

`main.py` reads the `auth_session` cookie, handles login handoff verification, and hydrates `session_state` before routing to the active page.

main.py reads the authsession cookie, handles login handoff verification, and hydrates sessionstate before routing to the active page.



Key behaviors:

- login_handoff=1 verification** — After `render_auth_cookie_handoff_redirect()` from login, the browser navigates to `/home?login_handoff=1&auth_flow_id=...&browser_cookie_ok=1`. `main.py` confirms the server sees `auth_session` in `st.context.cookies`. If not, auth state is cleared and user is sent back to login with an error.
- is_authenticated() side effect** — Calling `get_auth_manager().is_authenticated()` populates `st.session_state.authenticated` and `auth_data` from the slim cookie if not already present. This is **not** a redirect; individual pages call `require_auth()` for enforcement.
- Logging** — `MAIN_AUTH_BOOTSTRAP_ENTRY` and `MAIN_AUTH_BOOTSTRAP` timing events include host, referer, origin, query param keys, cookie presence, and routing target.

Shared Auth Flow (`auth_manager.py`)

Design principles

Principle	Implementation
Fast auth check	No DB read on <code>is_authenticated()</code>
Slim cookie	<code>auth_session</code> holds only <code>session_id</code> , <code>user_email</code> , <code>user_display_name</code> , <code>login_at</code> (< 4 KB)
<code>id_token</code> storage	Temp file <code>/tmp/_oidc_idtok_{hash}.json</code> + full <code>auth_data</code> in <code>session_state</code>
Logout authority	Server-side session blacklist file <code>/tmp/_oidc_invalid_{hash}.json</code>
Cookie write	Parent-frame JS injection (not <code>CookieController.set</code>) via <code>render_auth_cookie_handoff_redirect</code>

Cookie: `auth_session`

Shape (JSON, URL-encoded in browser):

```
{
  "session_id": "<32-char hex uuid>",
  "user_email": "user@coresight.com",
  "user_display_name": "Display Name",
  "login_at": "2026-06-26T12:00:00+00:00"
}
```

Attributes: `path=`, `SameSite=Lax`, `Secure` on HTTPS non-localhost, **host-only** (no `Domain` attribute on set), 7-day expiry (`COOKIE_MAX_AGE_DAYS`).

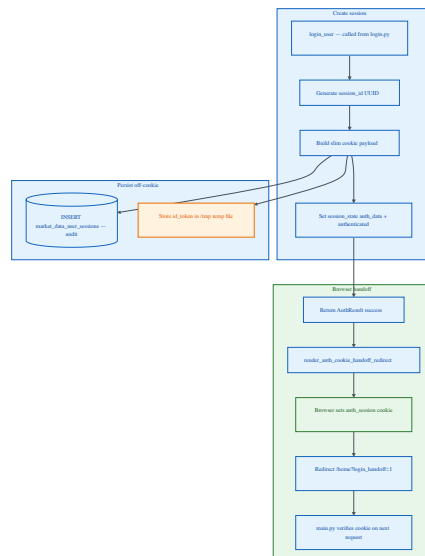
Cookie: `auth_logout_bridge`

Short-lived (~120s, `AUTH_LOGOUT_BRIDGE_MAX_AGE_SECONDS`) bridge cookie holding `{id_token, user_email, created_at}` for `logout_bridge.py` to recover OIDC logout hint after navigation.

Login flow (`AuthManager.login`)

`AuthManager.login` creates a session, writes an audit row, stores `id_token` off-cookie, and hands off a slim `auth_session` cookie to the browser.

AuthManager.login creates a session, writes an audit row, stores idtoken off-cookie, and hands off a slim authsession cookie to the browser.



Database write (audit only):

```
INSERT INTO market_data_user_sessions
  (user_email, user_nickname, user_display_name, token, login_at, last_activity)
VALUES
  (:user_email, :user_nickname, :user_display_name, :token, NOW(), NOW())
```

Login **succeeds even if DB insert fails** — cookie/session_state remain authoritative.

Auth check (`is_authenticated`)

Order:

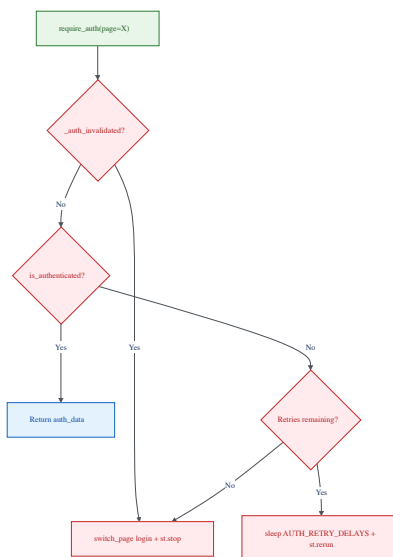
1. If `_auth_invalidated` in `session_state` → `False`
2. If `authenticated` + `auth_data` in `session_state` → `True`
3. Read `auth_session` from `st.context.cookies` (fast path)
4. Fallback: `CookieController.get("auth_session")`

5. Reject if `session_id` is in server-side blacklist (`_is_session_invalidated`)

Protected pages (`require_auth`)

Page modules call `require_auth()` to enforce identity — retry cookie mount on handoff, or redirect to login when the session is invalid or absent.

Page modules call `requireauth()` to enforce identity — retry cookie mount on handoff, or redirect to login when the session is invalid or absent.



Page-level `require_auth` enforcement

Identity is always established by `main.py` auth bootstrap (`is_authenticated()` hydrates session from the `auth_session` cookie on every request). Individual pages may additionally call `require_auth()` to redirect unauthenticated visitors to `/login`. Pages without an active `require_auth()` call rely on navigation flow and upstream network controls.

Page module	<code>require_auth</code> enforcement
<code>home.py</code> , <code>market_data.py</code> , <code>newsroom.py</code> , <code>earnings_calls.py</code> , <code>live_earnings_transcript.py</code> , <code>screening.py</code> , <code>company_filings.py</code> , <code>forecasting.py</code> , <code>logs.py</code> , <code>earnings_calendar.py</code>	Cookie hydrate only at module level
<code>access_management.py</code> , <code>forecasting_admin.py</code> , <code>admin_iam.py</code> , <code>audit.py</code> , <code>reports.py</code> , <code>settings.py</code> , <code>company_filings_add_files.py</code>	Cookie hydrate; <code>require_auth()</code> inside <code>main()</code> guard when enabled
<code>retailer_adding.py</code>	Active — <code>auth_user = require_auth()</code>
<code>login.py</code> , <code>logout_bridge.py</code>	N/A (public / logout bridge)

Security note: `/logs` is a hidden route without page-level `require_auth()` or ACL. Restrict reachability via network controls and monitor access in production.

Environment overrides:

Variable	Effect
<code>AUTH_RETRY_DELAYS</code>	Comma-separated seconds (default <code>0.15,0.35</code>)
<code>AUTH_FLOW_VERBOSE=1</code>	Extra auth timing logs

Cookie handoff (`render_auth_cookie_handoff_redirect`)

Injected parent-frame JS:

1. Clears state `auth_session` and `auth_logout_bridge` (host + legacy `.coresight.com`)
2. Sets new host-only `auth_session`
3. Polls `document.cookie` up to 20×50ms
4. `window.location.replace` to success URL (`/home?login_handoff=1&...`) or fail URL (`/login?auth_error=cookie_write_failed`)

Logout stash (`stash_logout_context`)

Called from `main.py` before routing to `logout_bridge.py` :

1. Recover `id_token` from `auth_data` , cookie, or temp file
2. `invalidate_session(session_id)` — blacklist before cookie clear
3. Store `__logout_bridge_data` in `session_state`
4. Set short-lived `auth_logout_bridge` cookie via JS injection

Session state keys (auth-related)

Key	Set by	Purpose
<code>authenticated</code>	<code>login</code> , <code>is_authenticated</code> , fast-restore	Boolean auth flag
<code>auth_data</code>	<code>login</code> , cookie restore	Full user payload (+ <code>id_token</code> in memory)
<code>auth_flow_id</code>	<code>get_or_create_auth_flow_id</code>	Login journey correlation ID
<code>_auth_invalidated</code>	Logout (JWT path), denied flows	Block cookie restore
<code>_auth_retry_count</code>	<code>require_auth</code>	Cookie mount retry counter
<code>_auth_retry_in_progress</code>	<code>require_auth</code>	Retry rerun marker
<code>_auth_cookie_source</code>	<code>_get_cookie</code>	<code>st.context.cookies</code> or <code>CookieController</code>
<code>__auth_cookie_controller</code>	<code>_get_controller</code>	Cached <code>CookieController</code> instance
<code>__logout_bridge_data</code>	<code>stash_logout_context</code>	OIDC logout handoff payload
<code>_auth_login_error</code>	<code>main.py</code> handoff failure	Surfaced on login page

External integrations

Integration	Role
Coresight OIDC IdP (<code>https://coresight.com/csr-idp/</code>)	Authorization, token, logout (via login/logout_bridge)
Azure MySQL	<code>market_data_user_sessions</code> audit insert
Browser cookies	<code>auth_session</code> , <code>auth_logout_bridge</code>
<code>/tmp</code> ephemeral files	Exchange cache, <code>id_token</code> cache, session blacklist, return-origin handoff

Error Handling

Location	Failure	Behavior
Missing page file	<code>FileNotFoundError</code> at startup	App won't start; logged as FATAL
SSL setup	Exception	Logged; app continues
Auth bootstrap	Exception	Swallowed (<code>pass</code>); page may run unhydrated
Logout routing	Exception	<code>log_exception</code> ; re-raise
<code>pg.run()</code>	Exception	<code>log_exception</code> FATAL; re-raise
DB init (authenticated)	Exception	Logged; warmups may be skipped
Background threads	Exception	Logged inside thread; non-blocking

Access Control List (ACL) / Permissions

`main.py` does **not** perform page-level ACL. ACL is enforced per-page via `AccessControlManager.check_access(page_name, user_email)` on gated pages (e.g. `access_management` , `retailer_adding`). Auth bootstrap only establishes **identity**, not **authorization**.

Hardcoded admin emails (used elsewhere): `mohdsaeedafri@coresight.com` , `philipmoore@coresight.com` , `shashankgupta@coresight.com` .

Flowcharts

Full request lifecycle

Every HTTP request passes through boot, auth hydrate, optional logout routing, conditional warmups, then the active Streamlit page.

Figure 1 Part A — Process boot and auth hydrate

Figure 1 Part A — Process boot and auth hydrate

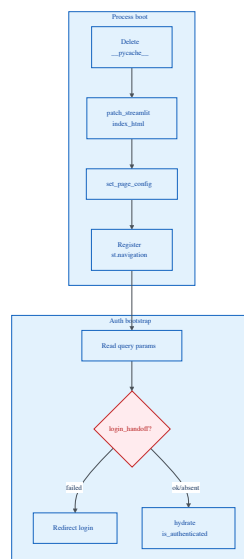
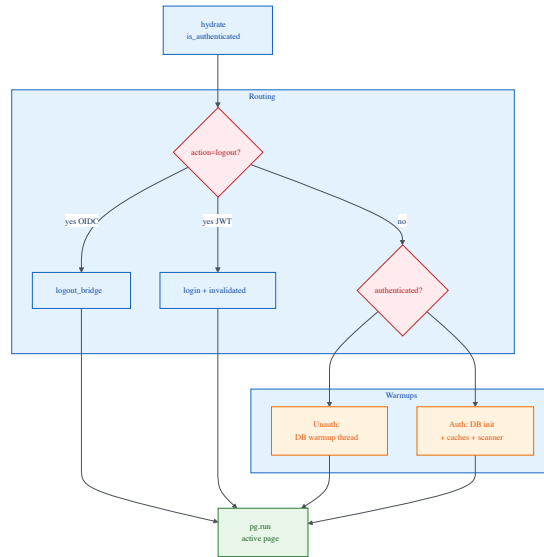


Figure 1 Part B — Routing, warmups, and page run

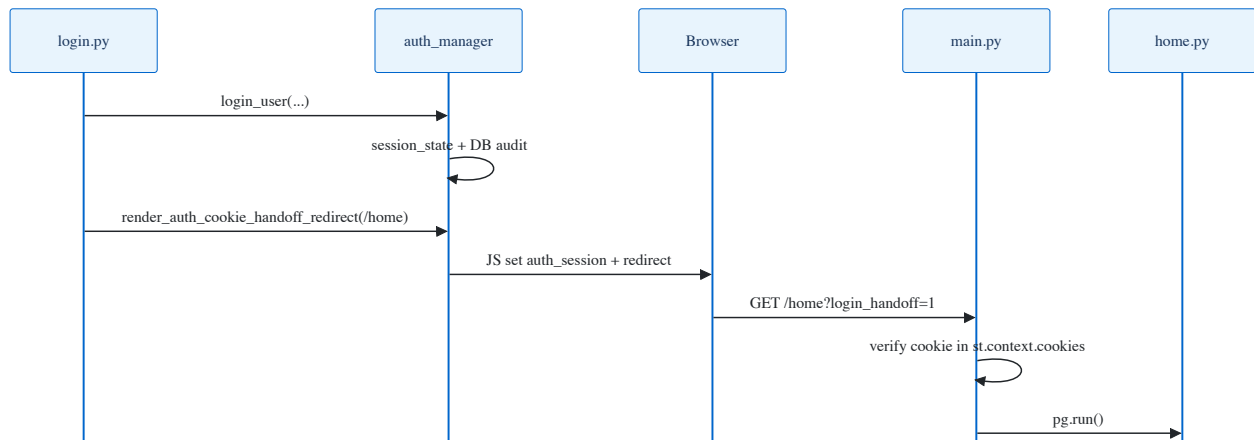
Figure 1 Part B — Routing, warmups, and page run



Login → Home cookie handoff (cross-module)

After OIDC login, the browser sets `auth_session` via JS and `main.py` verifies the cookie before rendering Home.

After OIDC login, the browser sets `authsession` via JS and `main.py` verifies the cookie before rendering Home.



Key Code References

Concern	File	Lines (approx.)
Page registration	<code>app/main.py</code>	175–228
Auth bootstrap	<code>app/main.py</code>	236–345
Logout routing	<code>app/main.py</code>	350–380
Authenticated warmups	<code>app/main.py</code>	414–553
OIDC env gate	<code>app/core/auth_environment.py</code>	18–33
Login + cookie handoff	<code>app/core/auth_manager.py</code>	347–429, 473–548
<code>require_auth</code>	<code>app/core/auth_manager.py</code>	800–1002
Cookie read + blacklist	<code>app/core/auth_manager.py</code>	1004–1156
Logout stash	<code>app/core/auth_manager.py</code>	635–718

Environment Variables (auth & bootstrap)

Variable	Default	Purpose
<code>APP_ENV</code>	<code>staging</code>	Deployment environment; footer staging links
<code>DEBUG</code>	—	Verbose diagnostics when <code>true / 1 / yes</code>
<code>AUTH_CLEAR_COOKIES_ONCE</code>	<code>0</code>	One-time browser cookie reset
<code>AUTH_RETRY_DELAYS</code>	<code>0.15, 0.35</code>	<code>require_auth</code> cookie wait retries
<code>AUTH_COOKIE_SYNC_WAIT_MS</code>	<code>350</code>	Cookie sync budget
<code>AUTH_FLOW_VERBOSE</code>	<code>0</code>	Verbose auth logging
<code>AUTH_LOGOUT_BRIDGE_MAX_AGE_SECONDS</code>	<code>120</code>	Bridge cookie TTL
<code>ENABLE_BG_WARMUP</code>	<code>1</code>	Background cache warmup
<code>ENABLE_BG_SCANNER</code>	<code>1</code>	Filing background scanner
<code>ENABLE_EC_WARMUP</code>	<code>1</code>	Earnings calendar cache warmup
<code>EARNINGS_ALERT_DISPATCH</code>	<code>1</code>	Alert dispatch tick
<code>EARNINGS_ALERT_DISPATCH_THREAD</code>	<code>1</code>	Background alert thread

Generated from source analysis of Market Data Portal (MDP). MCP `code-review-graph` was unavailable in this environment; documentation is based on direct file reads.*

— End of App Bootstrap, Page Registration, and Central Authentication —

PART 3 OF 22

Shared User Interface (UI) Components (`app/components/`)

Shared Streamlit UI — navigation, charts, tables, styles, and toolbar patterns.

Continued from previous section →

Application: Market Data Portal (MDP)

Stack: Streamlit multipage app with custom HyperText Markup Language (HTML)/CSS branding

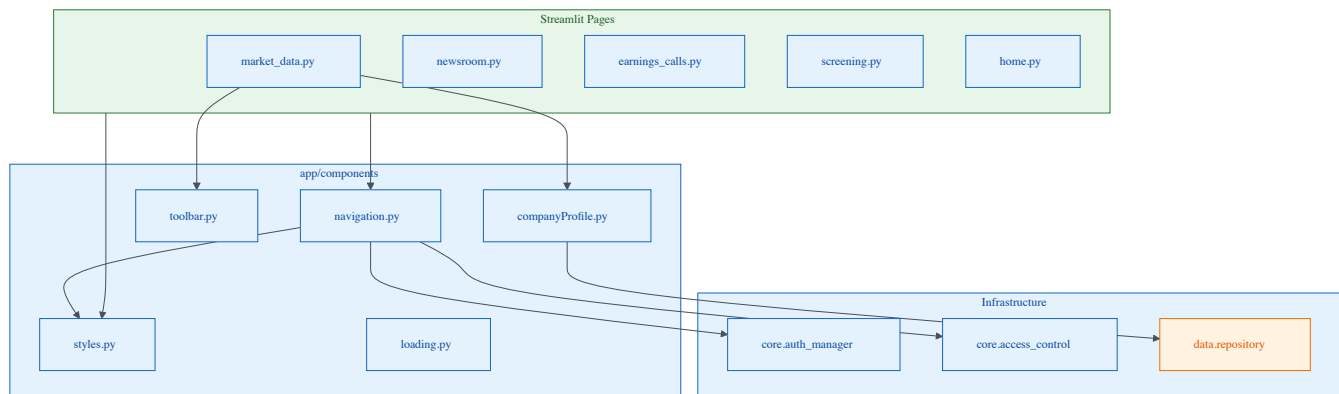
Scope: All modules under `app/components/` — navigation, styles, charts, tables, toolbar, loading, layout, company profile, auth utilities

Overview

The `app/components/` package provides reusable Streamlit UI primitives that give every page a consistent Coresight look and feel. Components hide Streamlit chrome (sidebar, default header), inject global CSS design tokens, render the branded site header/footer, and supply Market Data-specific building blocks (company header, tab bar, profile tables).

Shared UI modules sit between Streamlit pages and core infrastructure — pages import components; components call auth, Access Control List (ACL), and repositories as needed.

Shared UI modules sit between Streamlit pages and core infrastructure — pages import components; components call auth, Access Control List (ACL), and repositories as needed.



Typical page bootstrap sequence

Every authenticated page follows this pattern:

1. `hide_sidebar()` — immediately after `st.set_page_config()` (or at page top)
2. `render_styles()` + `set_page_layout()` — global CSS and content width
3. `render_header(current_page="...")` — sticky Coresight header
4. Page-specific content
5. `render_coresight_footer()` — four-column footer

Market Data adds: `render_company_header()` → `render_tabs()` → tab content.

Module Index

File	Lines (approx.)	Status	Primary consumers
<code>styles.py</code>	729	Active — core	17 pages, 2 utils
<code>navigation.py</code>	1,234	Active — core	15 pages, 2 utils
<code>toolbar.py</code>	218	Active	<code>market_data.py</code>
<code>companyProfile.py</code>	617	Active	<code>market_data.py</code>
<code>loading.py</code>	439	Active	<code>company_filings.py</code> , <code>forecasting*.py</code>
<code>tables.py</code>	446	Library (unused by pages)	—
<code>charts.py</code>	410	Library (unused by pages)	—
<code>layout.py</code>	313	Library (unused by pages)	—
<code>auth_utils.py</code>	517	Legacy / dead	—
<code>login_header.html</code>	—	Static asset	<code>login.py</code> only
<code>login_footer.html</code>	—	Static asset	<code>login.py</code> only
<code>login_layout.css</code>	—	Static asset	<code>login.py</code> only
<code>header_logout.html</code>	—	Static asset	Unused (legacy logout snippet)
<code>__init__.py</code>	2	Package marker	—

Login static assets are loaded via `Path.read_text()` in `login.py` and injected with `st.markdown(..., unsafe_allow_html=True)`. See [login.md](#) for the full login chrome layout.

`styles.py` — Design System & Global CSS

Purpose: Single source of truth for Coresight design tokens and global CSS injection. Hides Streamlit sidebar/header, applies card shadows, smooth content animations, and page layout constraints.

Design token exports

Constant	Contents
<code>COLORS</code>	Primary (<code>#D62E2F</code>), semantic (success/warning/danger/info), neutrals, chart palette
<code>TYPOGRAPHY</code>	Font families (Inter), sizes, weights, line heights
<code>SPACING</code>	<code>space_0</code> ... <code>space_16</code> rem scale
<code>BORDER_RADIUS</code>	<code>rounded_sm</code> ... <code>rounded_full</code>
<code>SHADOWS</code>	<code>shadow_sm</code> ... <code>shadow_card</code>

Public Application Programming Interface (API)

Function	Signature	Purpose
<code>get_global_css</code>	<code>() -> str</code>	Returns ~9KB global CSS string; cached in <code>st.session_state</code>
<code>render_styles</code>	<code>()</code>	Injects global CSS via <code>st.markdown(unsafe_allow_html=True)</code>
<code>set_page_layout</code>	<code>(header_full_width=True, footer_full_width=True, body_padding="0 20px", max_content_width="1350px", remove_top_padding=True, footer_at_bottom=True)</code>	Adjustable content width and padding CSS
<code>hide_sidebar</code>	<code>()</code>	Hides Streamlit sidebar and <code>stHeader</code> ; must run right after <code>set_page_config()</code>

Usage by pages

Page	Imports
<code>admin_iam.py</code> , <code>access_management.py</code> , <code>audit.py</code> , <code>reports.py</code> , <code>settings.py</code>	<code>hide_sidebar</code> , <code>render_styles</code> , <code>set_page_layout</code>
<code>company_filings.py</code> , <code>screening.py</code>	<code>hide_sidebar</code> , <code>set_page_layout</code> , <code>render_styles</code>
<code>earnings_calls.py</code> , <code>newsroom.py</code>	Above + <code>COLORS</code> , <code>TYPOGRAPHY</code> , <code>SPACING</code> for inline HTML
<code>market_data.py</code>	<code>hide_sidebar</code> , <code>render_styles</code> , <code>set_page_layout</code> , <code>COLORS</code>
<code>home.py</code> , <code>earnings_calendar.py</code>	<code>hide_sidebar</code> , <code>render_styles</code>
<code>login.py</code>	<code>hide_sidebar</code> only
<code>logs.py</code>	<code>hide_sidebar</code> , <code>render_styles</code> , <code>set_page_layout</code> (no header/footer)

Pages not using styles: `company_filings_add_files.py` , `retailer_adding.py` , `logout_bridge.py`

Dependencies

- `utils.server_logger.log_structured_error` only

`navigation.py` — Header, Footer, Company Selector

Purpose: Coresight-branded site header and footer, client-side page transitions, company selector header for Market Data, and legacy sidebar helpers.

Key exports

Name	Type	Description
<code>Page</code>	<code>Enum</code>	Legacy: <code>MARKET_DATA</code> , <code>NEWSROOM</code>
<code>PAGE_CONFIG</code>	<code>dict</code>	Labels/icons for <code>Page</code> enum

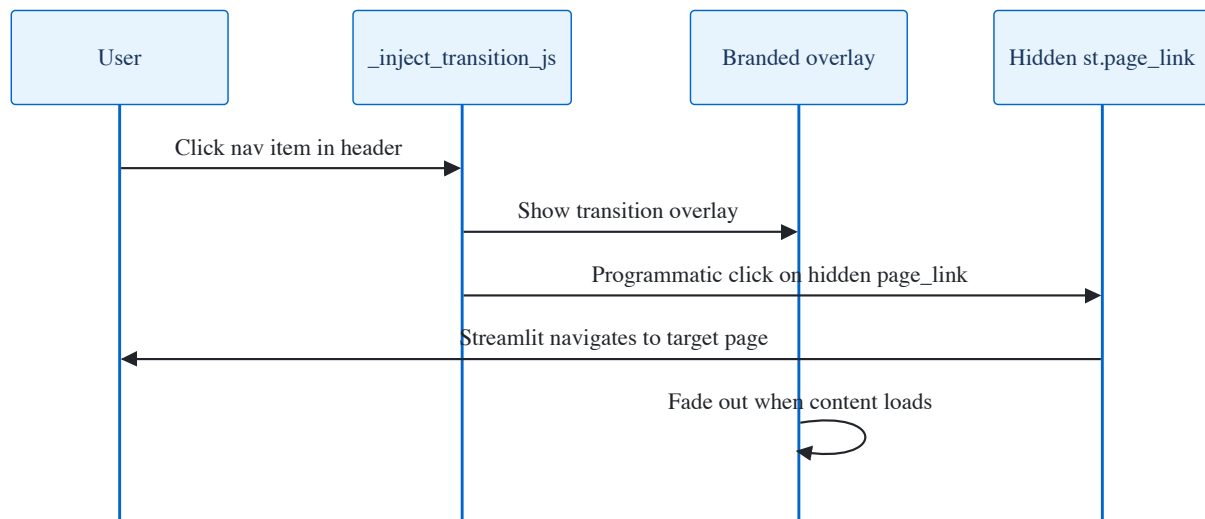
Public API

Function	Purpose
<code>render_header(full_width=True, current_page="market_data", ticker="M")</code>	Figma-matched sticky header; active nav highlighting; hidden <code>st.page_link</code> targets for client-side navigation; injects transition JS
<code>render_coresight_footer(full_width=True, stick_to_bottom=True)</code>	Four-column footer; staging-only admin links; Access Management for admins
<code>render_company_header(company_name, ticker, exchange="NYSE")</code>	"CORESIGHT MARKET DATA" title + invisible company <code>st.selectbox</code> + Company Documents link
<code>render_logo</code> , <code>render_user_state</code> , <code>render_divider</code> , <code>render_footer</code> , <code>render_top_bar</code>	Legacy sidebar helpers
<code>get_page_title(page: Page) -> str</code>	Icon + label for page enum

Client-side navigation flow

Header navigation uses parent-frame JavaScript to click hidden Streamlit page links while showing a branded transition overlay.

Header navigation uses parent-frame JavaScript to click hidden Streamlit page links while showing a branded transition overlay.



The transition overlay prevents the grey Streamlit skeleton flash during multipage navigation. Navigation targets: `market_data`, `earnings_calls`, `earnings_calendar`, `screening`, `newsroom`.

ACL in footer

- **Staging links** (`APP_ENV=staging` + email allowlists): Retailer admin, Forecast admin
- **Access Management:** `_can_access_mgmt(user_email)` Via `UserRolesManager.is_admin_or_super_user()` with hardcoded fallback admins

Session state side effects

- Sets `st.session_state["_active_page"]`
- Company selector updates `active_ticker` and `st.query_params["ticker"]`

Usage by pages

All authenticated pages except `login.py`, `logs.py`, `company_filings_add_files.py`, `retailer_adding.py`, `logout_bridge.py` import `render_header` + `render_coresight_footer`. `market_data.py` also imports `render_company_header`.

Dependencies

- `components.styles` — design tokens
- `core.auth_manager` — `is_authenticated` , `get_current_user`
- `core.access_control.UserRolesManager` — footer ACL (lazy import)
- `data.forecast_admin_service.is_forecast_admin` — staging forecast admin link
- `data.repository.CompanyRepository` — company list for selector (lazy in `render_company_header`)

`toolbar.py` — Market Data Tab Bar

Purpose: Horizontal tab bar for the Market Data page (10 tabs). Active tab renders as a styled label; inactive tabs are `st.button` widgets that set `st.query_params["tab"]` .

Public API

Function	Purpose
<code>render_tabs(selected_tab: str) -> str</code>	Renders tab row; returns selected tab key

Tab keys

Key	Label	Notes
<code>company_profile</code>	Company Profile	
<code>key_stats</code>	Key Stats	
<code>income_statement</code>	Income Statement	
<code>balance_sheet</code>	Balance Sheet	
<code>cash_flow</code>	Cash Flow	
<code>ratios</code>	Ratios	
<code>estimates</code>	Estimates	
<code>forecasting</code>	Forecasting	"Testing in Progress" badge
<code>segment_data</code>	Segment Data	"Testing in Progress" badge
<code>ratings</code>	Ratings	"Testing in Progress" badge

Tab selection flow

Market Data tab changes persist in query params and local storage, then rerun the page to load the selected tab content.

Market Data tab changes persist in query params and local storage, then rerun the page to load the selected tab content.



Consumers

- `pages/market_data.py` — primary
- `utils/market_data.py` — deprecated duplicate

Dependencies

- `utils.local_storage.set_marketdata_tab`

companyProfile.py — Company Profile Tab Content

Purpose: Renders the Company Profile tab on Market Data — Figma-matched info table, business description, reference expander, and Excel export row builder.

Public API

Function	Purpose
<code>get_company_css() -> str</code>	Page-scoped CSS for profile tables
<code>render_info_table(company: CompanyOverview) -> str</code>	4-column HTML info table (hides sector/industry in main table)
<code>render_business_description(description) -> str</code>	Business description HTML block
<code>render_reference_table(company) -> str</code>	Sector/Industry reference table (in expander)
<code>get_profile_rows_for_excel(company) -> list</code>	<code>[(label, plain_value), ...]</code> for Excel export
<code>render_company_profile_content(company)</code>	Streamlit render: info + description + "For Reference" expander
<code>render_company_profile(ticker="M")</code>	Standalone page path (header + Database (DB) fetch)
<code>object_to_dict</code> , <code>company_to_label_value</code> , <code>format_market_cap</code> , <code>to_title_case</code>	Formatting helpers

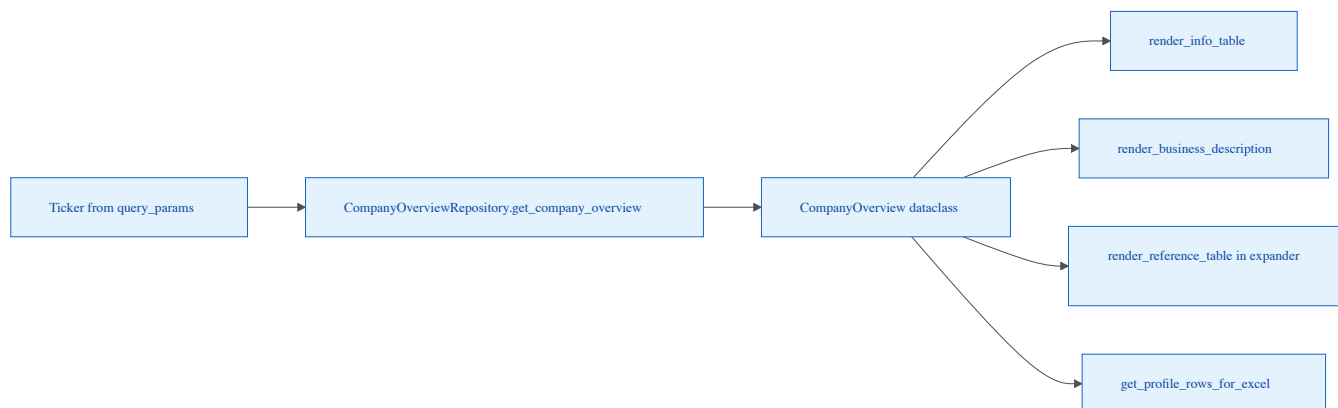
Internal constants

- `HIDDEN_FIELDS` — sector, industry, CIK, etc. excluded from main table
- `LABEL_MAP` — display overrides (e.g. "Pe Ratio" → "P/E Ratio")

Data flow

Company Profile loads overview data from the repository layer and renders HTML table blocks for the active ticker.

Company Profile loads overview data from the repository layer and renders HTML table blocks for the active ticker.



Consumers

- `pages/market_data.py` — primary integration hub
- `utils/market_data.py`, `utils/companyProfile.py` — legacy duplicates

Dependencies

- `data.models.CompanyOverview`
- `data.repository.CompanyOverviewRepository`
- `utils.constants` — currency, exchange, country formatters

`loading.py` — Branded Spinners & Overlays

Purpose: Coresight-red (`#D62E2F`) spinner CSS, full-screen overlays, skeleton placeholders, and Azure filing scan progress overlay.

Public API

Function	Purpose
<code>inject_red_spinner_css()</code>	Global CSS override for <code>stSpinner</code>
<code>show_loading_overlay(message, submessage, show_progress, progress_percent)</code>	Full-screen branded overlay
<code>hide_loading_overlay()</code>	No-op placeholder (overlay clears on rerun)
<code>show_inline_loading(message)</code>	Inline spinner row
<code>show_skeleton_card()</code> / <code>show_skeleton_text(lines)</code>	Placeholder skeletons
<code>show_progress_bar(label, percent, show_percent)</code>	Styled progress bar
<code>show_red_spinner(text)</code>	Context manager wrapping <code>st.spinner</code>
<code>render_scan_progress_overlay(progress: dict)</code>	Azure filing scan progress from <code>background_scanner</code>

Consumers

Page	Usage
<code>company_filings.py</code>	Lazy import; <code>inject_red_spinner_css()</code>
<code>forecasting.py</code>	<code>inject_red_spinner_css()</code>
<code>forecasting_admin.py</code>	<code>inject_red_spinner_css()</code> (2 call sites)

`tables.py` — Reusable Data Tables (Library)

Purpose: Typed `st.dataframe` wrappers with column formatting presets. **Not currently imported by any page** — pages build tables inline.

Formatters

Function	Output example
<code>format_currency(value, decimals=2)</code>	\$1,234.56
<code>format_percentage(value, decimals=2)</code>	Colored HTML <code>+/-x.xx%</code>
<code>format_number(value, decimals=0)</code>	Thousand separators
<code>format_volume(value)</code>	K/M/B suffix
<code>format_market_cap(value)</code>	<code>\$X.XXB/T/M</code>
<code>format_datetime</code> / <code>format_date</code>	Date formatting

Table builders

Function	Preset for
<code>create_data_table(data, columns_config, ...)</code>	Generic styled dataframe
<code>create_sec_filings_table(filings_data, ...)</code>	U.S. Securities and Exchange Commission (SEC) filings
<code>create_market_data_table(market_data, ...)</code>	Market data quotes
<code>create_news_table(articles, ...)</code>	News articles
<code>render_sortable_header(columns, sort_column, sort_order, on_sort)</code>	Clickable sort header row

charts.py — Plotly Chart Builders (Library)

Purpose: Plotly `go.Figure` factories with design-system colors. **Not currently imported by any page** — pages build Plotly charts inline.

Constants

- `CHART_CONFIG` — mode bar config (no logo, no lasso)
- `CHART_LAYOUT` — transparent bg, Inter font, margins

Public API

Function	Chart type
<code>create_candlestick_chart(metrics, title, height)</code>	OHLC + volume subplots
<code>create_line_chart(data, x_column, y_columns, ...)</code>	Multi-line
<code>create_bar_chart(...)</code>	Bar (horizontal option)
<code>create_pie_chart(...)</code>	Pie/donut (<code>hole=0.4</code>)
<code>create_area_chart(...)</code>	Area with gradient fill
<code>render_chart(fig, width, key)</code>	<code>st.plotly_chart</code> wrapper

Dependency: `data.models.MarketMetric`

layout.py — Layout Primitives (Library)

Purpose: Generic cards, metrics, badges, empty states, tabs, filters, pagination. **Not currently imported by any page.**

Function	Purpose
<code>render_card(content, title, ...)</code>	Styled HTML card
<code>render_metric_card(label, value, change, ...)</code>	KPI metric card
<code>render_badge(text, variant, icon)</code>	Pill badge
<code>render_empty_state(title, description, icon, action_label, action_callback)</code>	Centered empty state
<code>render_loading_skeleton(height)</code>	Pulsing placeholder
<code>render_tabs(tabs_config, default_tab)</code>	<code>st.tabs</code> with render callbacks (different from <code>toolbar.render_tabs</code>)
<code>render_filter_bar(filters, on_change)</code>	Filter bar shell
<code>render_pagination(current_page, total_pages, on_page_change)</code>	Prev/Next pagination

`auth_utils.py` — Legacy Auth (Dead Code)

Purpose: Pre-OpenID Connect (OIDC) cookie-based auth via `streamlit_cookies_controller`. **Superseded by** `app/core/auth_manager.py`. No imports from `app/pages/` or `app/utils/`.

Function	Notes
<code>require_auth(redirect_to="login.py")</code>	Legacy guard
<code>login_user(...)</code> , <code>logout()</code>	Cookie + DB session
<code>restore_session_from_cookie()</code>	Cookie hydration
<code>create_db_connection()</code>	Direct <code>mysql.connector</code> (legacy env vars)

Do not use for new code. Uses stdlib `logging` (violates `server_logger` convention).

`__init__.py`

Package marker: `"""UI components module."""`

Cross-Cutting Patterns

1. Design tokens in inline HTML

Content-heavy pages (`newsroom`, `earnings_calls`, `market_data`) import `COLORS`, `TYPOGRAPHY`, `SPACING` directly for bespoke HTML blocks rather than using `layout.py`.

2. Market Data integration hub

Only `market_data.py` combines all three Market Data components:

```
render_company_header → render_tabs → render_company_profile_content (and other tab renderers)
```

3. Duplicate utility modules

`app/utils/market_data.py` and `app/utils/companyProfile.py` mirror page logic and import the same components. Treat as deprecated entry points.

4. Early skeleton suppression

`main.py` injects skeleton-hide CSS at the top of every run. `navigation._inject_transition_js()` adds the branded overlay. Together they eliminate Streamlit's grey loading flash.

Component → Page Consumer Matrix

Component	Pages using it
<code>styles.hide_sidebar</code>	17 pages
<code>styles.render_styles</code>	16 pages
<code>navigation.render_header</code>	15 pages
<code>navigation.render_coresight_footer</code>	15 pages
<code>toolbar.render_tabs</code>	<code>market_data</code>
<code>companyProfile.*</code>	<code>market_data</code>
<code>loading.inject_red_spinner_css</code>	<code>company_filings</code> , <code>forecasting</code> , <code>forecasting_admin</code>
<code>tables.*</code>	—
<code>charts.*</code>	—
<code>layout.*</code>	—
<code>auth_utils.*</code>	—

— End of Shared User Interface (UI) Components ('app/components/') —

PART 4 OF 22

Core Infrastructure (`app/core/`)

Core modules — configuration, database engines, auth manager, access control, and LLM extraction.

Continued from previous section →

Application: Market Data Portal (MDP)

Stack: Streamlit + SQLAlchemy 2 + Azure MySQL + OpenID Connect (OIDC)

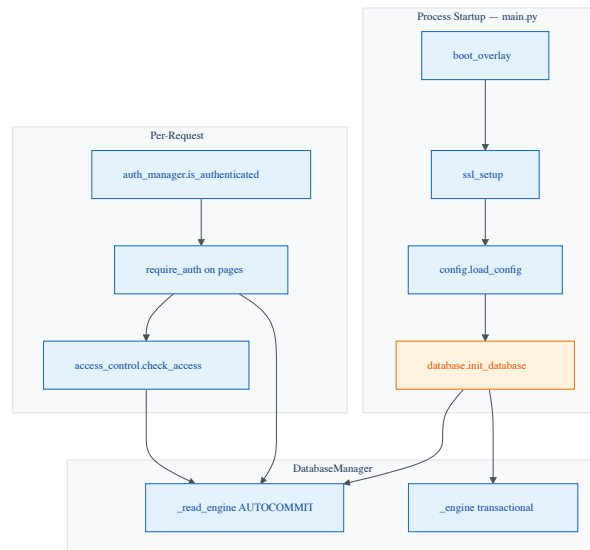
Scope: `app/core/` — configuration, database, authentication, access control, Large Language Model (LLM) extraction, search, SSL, boot overlay

Overview

The `app/core/` package is the application infrastructure layer. It sits below pages (`app/pages/`) and the data layer (`app/data/`), providing:

- Environment-based configuration and SSL
- Dual-engine database access (read vs transactional)
- OIDC session-token authentication (cookie-first)
- Page-level and system-role access control
- On-demand LLM filing metric extraction
- In-memory Term Frequency-Inverse Document Frequency (TF-IDF) transcript search (utility, currently unwired)

- In-memory Term Frequency-Inverse Document Frequency (TF-IDF) transcript search (utility, currently unwired)



Module Index

Module	Purpose	Primary consumers
<code>config.py</code>	Env-based <code>AppConfig</code> / <code>DatabaseConfig</code>	<code>database.py</code> (indirect: entire app)
<code>database.py</code>	Singleton <code>DatabaseManager</code> , dual engines	30+ modules across pages/data/utils
<code>auth_manager.py</code>	Cookie-first OIDC session auth	All protected pages, <code>navigation</code> , <code>main.py</code>
<code>access_control.py</code>	Page Access Control List (ACL) + system roles	ACL pages, <code>navigation</code> , <code>forecast_refresh_service</code>
<code>auth_environment.py</code>	OIDC/production flags	<code>auth_manager</code> , <code>main</code> , <code>login</code> , <code>logout_bridge</code>
<code>ssl_setup.py</code>	DigiCert CA provisioning	<code>main.py</code> , <code>config.py</code>
<code>boot_overlay.py</code>	Streamlit index.html patch	<code>main.py</code>
<code>llm_extractor.py</code>	GPT-4o-mini filing extraction	<code>repository.py</code> , <code>company_filings.py</code>
<code>search_engine.py</code>	TF-IDF segment search	Unused (orphan utility)
<code>__init__.py</code>	Package marker	—

`config.py` — Application Configuration

Classes

Class	Role
<code>Environment</code> (Enum)	<code>LOCAL</code> , <code>STAGING</code> , <code>PRODUCTION</code>
<code>DatabaseConfig</code> (dataclass)	MySQL connection params, pool settings, SSL
<code>AppConfig</code> (dataclass)	<code>env</code> , <code>debug</code> , <code>database</code> , <code>secret_key</code> , session timeout, caching, pagination

Key functions

Function	Role
<code>load_config()</code>	Build <code>AppConfig</code> from environment
<code>DatabaseConfig.connection_string</code>	<code>mysql+pymysql://...</code> property
<code>DatabaseConfig.connect_args</code>	SSL <code>{"ssl": {"ca": path}}</code> with late-binding CA resolution

Environment variables

Variable	Default / notes
<code>APP_ENV</code>	<code>local</code> → drives <code>Environment</code> enum
<code>DB_HOST</code> , <code>DB_PORT</code> , <code>DB_NAME</code> , <code>DB_USER</code> , <code>DB_PASSWORD</code>	Local fallbacks
<code>STG_DB_*</code>	Staging Azure MySQL
<code>PROD_DB_*</code>	Production Azure MySQL
<code>ENABLE_SSL</code>	<code>true</code> (stg/prod), <code>false</code> (local)
<code>SSL_CA</code>	Optional; late-bound via <code>ssl_setup</code> if missing
<code>DB_POOL_SIZE</code>	<code>5</code>
<code>DB_MAX_OVERFLOW</code>	<code>10</code>
<code>DEBUG</code>	<code>true</code> local; <code>false</code> prod
<code>SECRET_KEY</code>	Warns if default in non-local
<code>SESSION_TIMEOUT</code>	<code>3600</code>
<code>ENABLE_CACHING</code>	<code>true</code>
<code>CACHE_TTL</code>	<code>300</code>
<code>NAVIGATION_MODE</code>	<code>same</code>

Global singleton: `config = load_config()` at module import.

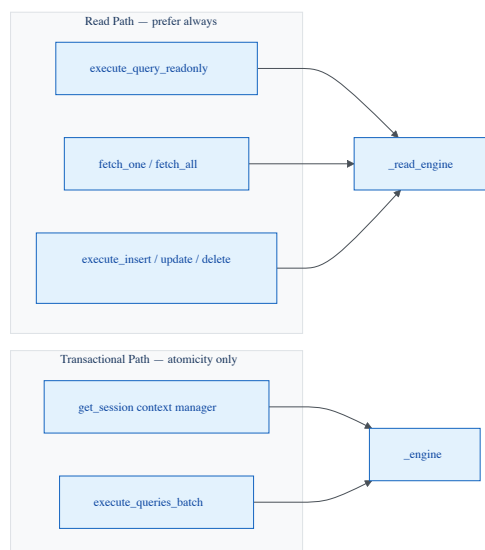
`database.py` — Dual-Engine Database Access

Design philosophy

On Azure MySQL (~250ms round-trip per connection operation), minimizing RTTs is critical:

Engine	Attribute	Isolation	Use when	Azure RTTs
Read	<code>_read_engine</code>	<code>AUTOCOMMIT</code> + <code>skip_autocommit_rollback=True</code>	All SELECTs, single-statement writes	~1 RTT
Main	<code>_engine</code>	Default (implicit BEGIN)	Multi-statement atomicity, <code>get_session()</code>	~3 RTTs

On Azure MySQL (~250ms round-trip per connection operation), minimizing RTTs is critical:



DatabaseManager Application Programming Interface (API)

Method	Engine	Returns	Notes
<code>connect()</code>	Both	—	Creates engines; parallel SSL warmup (6 conns on Azure)
<code>get_session()</code>	Main	Context manager	commit/rollback
<code>execute_query_readonly(query, params)</code>	Read	<code>List[Dict]</code>	Swallows errors → <code>[]</code>
<code>execute_query_readonly_raising(...)</code>	Read	<code>List[Dict]</code>	Raises on error (cache-safe)
<code>fetch_one</code> / <code>fetch_all</code>	Read	<code>Optional[Dict]</code> / <code>List[Dict]</code>	Thin wrappers
<code>execute_insert</code> / <code>execute_update</code> / <code>execute_delete</code>	Read (AUTOCOMMIT)	<code>int</code> rowcount	Single-statement writes
<code>execute_insert_returning_id</code>	Read	<code>Optional[int]</code>	<code>lastrowid</code>
<code>execute_scalar</code>	Main session	scalar	
<code>execute_queries_batch</code>	Main session	—	Multi-query, one connection
<code>health_check()</code>	Main	<code>bool</code>	<code>SELECT 1</code>
<code>_dispose_and_reconnect()</code>	Both	—	SSL recovery

Pool tuning (Azure)

- `pool_pre_ping=False` on Azure — avoids 250ms `SELECT 1` per checkout
- `pool_recycle=3600` — avoids 4s SSL re-handshake on stale connections

Module-level exports

Symbol	Role
<code>db_manager</code>	Global <code>DatabaseManager()</code> singleton
<code>with_db_session</code>	Decorator injecting <code>session</code> kwarg
<code>init_database()</code>	<code>db_manager.connect()</code> — fail fast
<code>warmup_av_fulltext()</code>	Gated by <code>ENABLE_AV_FT_WARMUP=1</code> ; warms FULLTEXT indexes

Usage pattern (from project conventions)

```
# READ (1 RTT — always prefer):
rows = db_manager.execute_query_readonly(query, params)
row = db_manager.fetch_one(query, params)

# WRITE (1 RTT — AUTOCOMMIT):
db_manager.execute_insert(query, params)
db_manager.execute_update(query, params)

# TRANSACTIONAL (3 RTTs — only for atomicity):
with db_manager.get_session() as session:
    session.execute(text(query), params)
```

`auth_manager.py` — Session-Token Authentication

Design principle

Cookie is the source of truth. `is_authenticated()` never hits the database. `market_data_user_sessions` is audit-only on login.

Constants

Name	Value
<code>COOKIE_NAME</code>	<code>"auth_session"</code>
<code>COOKIE_MAX_AGE_DAYS</code>	<code>7</code>
<code>LOGOUT_BRIDGE_COOKIE_NAME</code>	<code>"auth_logout_bridge"</code>

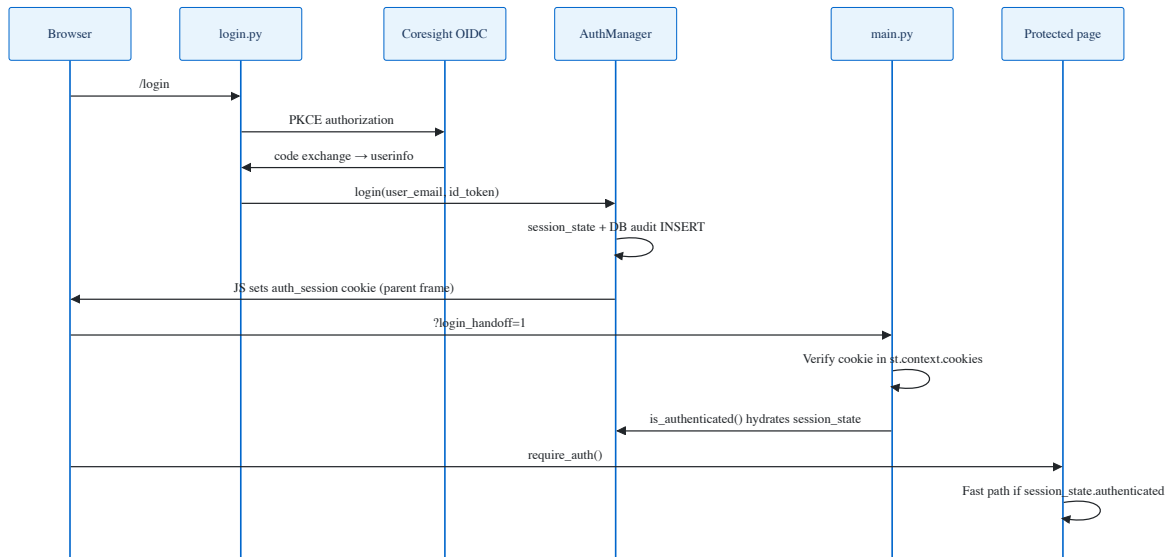
Classes

Class	Role
<code>AuthResult</code>	<code>success</code> , <code>session_id</code> , <code>user_email</code> , <code>error_message</code>
<code>AuthManager</code>	Login, auth check, logout, cookie I/O

Auth flow

End-to-end OIDC login: Identity Provider (IdP) authorization, cookie handoff via parent-frame JavaScript, and per-page identity checks.

End-to-end OIDC login: Identity Provider (IdP) authorization, cookie handoff via parent-frame JavaScript, and per-page identity checks.



`require_auth()` check order

1. `st.session_state._auth_invalidated` → redirect login
2. `st.session_state.authenticated` + `auth_data` → fast path
3. `__get_cookie()` from `st.context.cookies` → `CookieController` fallback
4. Blacklist check (`_is_session_invalidated`)
5. Retry with `AUTH_RETRY_DELAYS` (default `0.15, 0.35 s`) → `st.rerun()`
6. Exhausted → `st.switch_page(login)`

Cookie design

Slim cookie JSON only: `session_id`, `user_email`, `user_display_name`, `login_at`

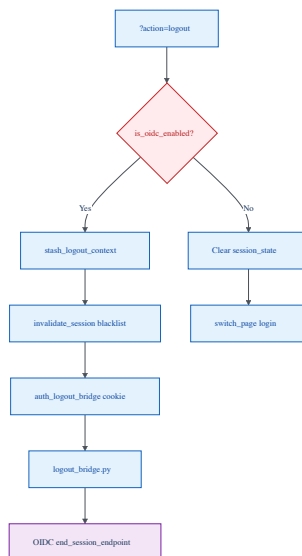
Not in cookie: `id_token` stored in `/tmp/_oidc_idtok_{hash20}.json` and `session_state.auth_data`

Cookie write: Parent-frame JS injection (proven on Azure STG; `CookieController.set` unreliable cross-frame)

Logout flow

Logout routing branches on OIDC mode — stash context and RP-initiated IdP logout, or legacy session clear.

Logout routing branches on OIDC mode — stash context and RP-initiated IdP logout, or legacy session clear.



Module-level exports

`get_auth_manager` , `login_user` , `stash_logout_context` , `require_auth` , `is_authenticated` , `logout` , `get_current_user` , `get_session_id` , `get_auth_data` , `get_current_domain` , `build_cookie_clear_js_lines` , `render_auth_cookie_handoff_redirect`

Environment variables

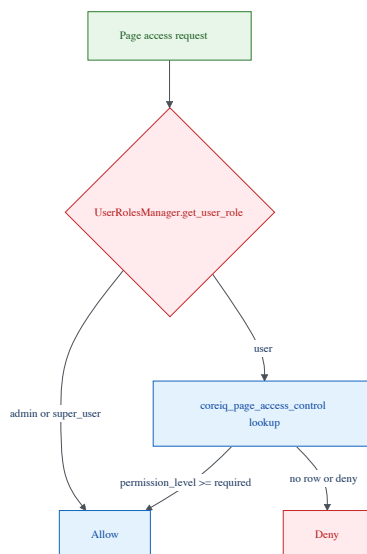
Variable	Role
<code>APP_ENV</code>	Deployment environment (<code>LOCAL</code> , <code>STAGING</code> , <code>PRODUCTION</code>)
<code>DEBUG</code>	Verbose diagnostics
<code>AUTH_FLOW_VERBOSE</code>	Verbose auth timing logs
<code>AUTH_RETRY_DELAYS</code>	Cookie retry schedule
<code>AUTH_COOKIE_SYNC_WAIT_MS</code> / <code>AUTH_COOKIE_SYNC_POLL_MS</code>	Handoff timing
<code>AUTH_LOGOUT_BRIDGE_*</code>	Bridge cookie sync

`access_control.py` — Page ACL & System Roles

Two-layer model

Access checks consult system role first (admin bypass), then page-level ACL rows in MySQL.

Access checks consult system role first (admin bypass), then page-level ACL rows in MySQL.



AccessControlManager

Method	Role
<code>check_access(page_name, user_email, required_permission)</code>	Hierarchy check; admins bypass
<code>get_page_users(page_name)</code>	List ACL rows
<code>set_user_access(...)</code>	UPSERT permission
<code>remove_user_access(...)</code>	DELETE ACL row

Permission hierarchy: `view_only(1) < upload_only(2) < edit(3) < delete_admin(4) = allow(4)` . Explicit `deny` blocks all.

Table: `coreiq_page_access_control`

UserRolesManager

Method	Role
<code>get_user_role</code> / <code>set_user_role</code> / <code>delete_user</code>	Create, Read, Update, Delete (CRUD) on <code>coreiq_user_roles</code>
<code>is_admin</code> / <code>is_admin_or_super_user</code> / <code>can_delete_users</code>	Convenience checks

Roles: `admin` > `super_user` > `user`

Fallback admins (if Database (DB) down): `mohdsaeedafri@coresight.com` , `philipmoore@coresight.com` , `shashankgupta@coresight.com`

Consumers

- Pages: `access_management` , `forecasting` , `forecasting_admin` , `admin_iam` , `audit` , `reports`
- `components/navigation.py` — footer Access Management link
- `data/forecast_refresh_service.py`

auth_environment.py — OIDC Gate

Function	Current behavior
<code>is_production_deploy()</code>	<code>True</code> if <code>ENV / ENVIRONMENT / APP_ENV == production</code>
<code>is_oidc_enabled()</code>	Always <code>True</code> (JSON Web Token (JWT) path disabled)
<code>IS_OIDC_ENV</code>	Snapshot at import

ssl_setup.py — Azure MySQL SSL CA

Function	Role
<code>ensure_ca_cert()</code>	Search bundled DigiCert G2 → Azure <code>\$HOME/site/wwwroot/</code> → CWD → download

Sets `os.environ["SSL_CA"]` on success. Called at startup (`main.py`) and lazily from `DatabaseConfig.connect_args` .

boot_overlay.py — Streamlit Boot Loader

Function	Role
<code>patch_streamlit_index_html()</code>	Idempotent patch of Streamlit static <code>index.html</code>

Injects:

- CSS to hide `[data-testid='stSkeleton']`
- Branded Coresight overlay HyperText Markup Language (HTML) + JS
- `MutationObserver` removes overlay when `stMain` contains real widgets
- 15s safety timeout

Marker: `cs-boot-overlay-v1` — re-applied on cold start after Streamlit upgrades.

llm_extractor.py — GPT Filing Metric Extraction

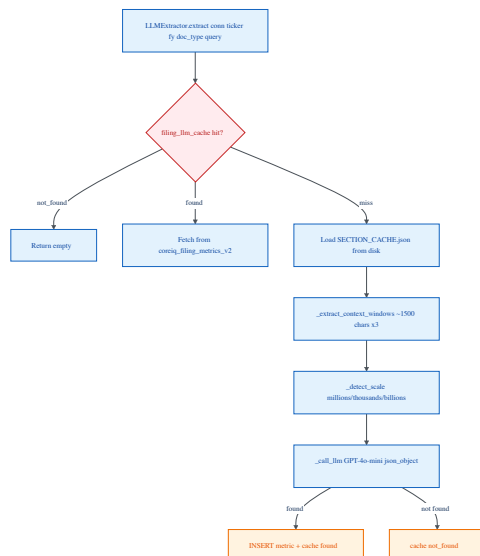
Purpose

On-demand GPT-4o-mini extraction for U.S. Securities and Exchange Commission (SEC) filing metrics missing from eXtensible Business Reporting Language (XBRL). Results cached in `filing_llm_cache` and persisted to `coreiq_filing_metrics_v2` with `source='llm'` .

Extraction flow

LLM metric extraction checks caches first, then builds context windows and calls GPT-4o-mini before persisting results.

LLM metric extraction checks caches first, then builds context windows and calls GPT-4o-mini before persisting results.



Class

Class	Method
LLMExtractor	<code>extract(conn, ticker, fy, doc_type, query) -> List[FilingMetricResult]</code>

Requires: Open SQLAlchemy `conn` from caller (not `db_manager` directly).

Env: `OPENAI_API_KEY` (missing → empty results)

Cost: ~\$0.00012 per unique query; \$0 when cached.

Consumers

- `data/repository.py` — `FilingMetricRepository.search_with_llm_fallback`
- `pages/company_filings.py` — on-demand extraction User Interface (UI)

`search_engine.py` — TF-IDF Transcript Search

Purpose

In-memory TF-IDF + cosine similarity over earnings call transcript segments (scikit-learn).

Classes

Class	Role
SearchResult	<code>index</code> , <code>speaker</code> , <code>text</code> , <code>score</code> , <code>snippet</code>
TranscriptSearchEngine	<code>__init__(segments)</code> , <code>search(query, top_k, min_score)</code>

Algorithm

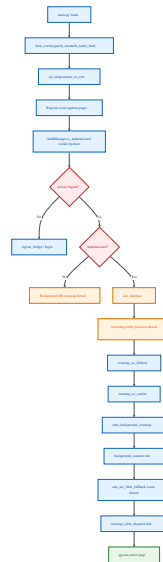
1. Fit `TfidfVectorizer` (unigrams+bigrams, 10k features, English stopwords)
2. Cosine similarity ranking
3. Guard: result must contain at least one query word
4. Fallback: keyword count if sklearn missing

Status

Not wired to any page. `earnings_calls.py` uses MySQL `FULLTEXT` via `EarningsCallRepository.search_transcripts_fulltext()` instead.

Startup Integration (`main.py`)

Startup Integration ('main.py')



Quick Reference: Import Map

Module	Imported by
config	database.py only
database	All data services, most pages, auth_manager , access_control , utils
auth_manager	All protected pages, navigation , main , login , logout_bridge
access_control	ACL pages, navigation , forecast_refresh_service
llm_extractor	repository , company_filings
search_engine	— (unused)
auth_environment	auth_manager , main , login , logout_bridge
ssl_setup	main , config
boot_overlay	main , scripts/patch_streamlit_boot.py

— End of Core Infrastructure (`app/core/') —

PART 5 OF 22

Data Layer (`app/data/`)

Data models, repositories, screening services, and forecast pipelines.

Continued from previous section →

Application: Market Data Portal (MDP)

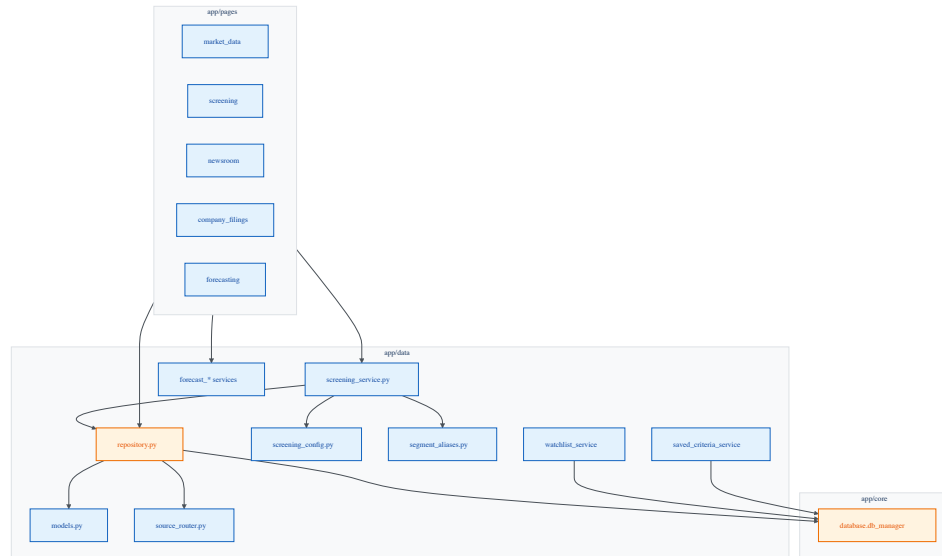
Stack: SQLAlchemy 2 via `core.database.db_manager`, Streamlit `@st.cache_data`

Scope: Models, repositories, screening engine, source routing, segment aliases, user persistence, forecasting pipeline

Overview

`app/data/` is the domain and data-access layer between Streamlit pages and Azure MySQL. All persistent tables use the `coreiq_` prefix. Reads prefer `db_manager.execute_query_readonly` (AUTOCOMMIT, 1 RTT on Azure).

app/data/ is the domain and data-access layer between Streamlit pages and Azure MySQL. All persistent tables use the coreiq prefix. Reads prefer dbmanager.executequeryreadonly (AUT



Module Index

File	Role
<code>models.py</code>	Dataclasses + fiscal calendar helpers
<code>repository.py</code>	19 repository classes — primary Database (DB) read layer (~11k lines)
<code>screening_config.py</code>	Screening constants (zero DB calls)
<code>screening_service.py</code>	Screening SQL/pandas engine
<code>source_router.py</code>	U.S. Securities and Exchange Commission (SEC) vs YFinance per ticker
<code>segment_aliases.py</code>	Geographic segment canonicalization
<code>watchlist_service.py</code>	Watchlist Create, Read, Update, Delete (CRUD) + Access Control List (ACL)
<code>saved_criteria_service.py</code>	Saved screening criteria + ACL
<code>portal_users_service.py</code>	Internal user registry for ACL pickers
<code>earnings_alert_service.py</code>	Earnings email alert prefs + dedupe
<code>earnings_alert_store.py</code>	Re-export shim → <code>earnings_alert_service</code>
<code>forecast_store.py</code>	<code>coreiq_model_forecasts</code> read/write cache
<code>forecast_admin_service.py</code>	Admin bulk forecast sync
<code>forecast_refresh_service.py</code>	Forecast refresh popup + email
<code>revenue_forecast_service.py</code>	Revenue Estimates / Forecasting page data
<code>revenue_estimates_service.py</code>	Alias: <code>RevenueEstimatesService</code> = <code>RevenueForecastService</code>
<code>live_earnings_transcript_store.py</code>	Live transcript upsert
<code>dummy_data.py</code>	Legacy stub — incompatible with current <code>models.py</code>
<code>__init__.py</code>	Package marker

models.py — Domain Dataclasses

Dataclass catalog

Class	Source / usage
<code>TickerSentiment</code>	Per-ticker sentiment on news articles
<code>NewsArticle</code>	<code>coreiq_av_market_news_sentiment (+ source_tag : av yf)</code>
<code>Company</code>	<code>coreiq_companies</code>
<code>IncomeStatementLineItem</code>	Display row for income statement grid
<code>FiscalPeriod</code>	Column header (annual/quarterly, Calendar Quarter (CQ)/Fiscal Quarter (FQ), estimate/forecast flags)
<code>IncomeStatementData</code>	Company + periods + line items bundle
<code>CompanyOverview</code>	Alpha Vantage (AV)/Yahoo Finance (YF) company overview payload
<code>BalanceSheetLineItem</code> / <code>BalanceSheetData</code>	Balance sheet display
<code>CashFlowLineItem</code> / <code>CashFlowData</code>	Cash flow display
<code>EarningsCall</code>	<code>coreiq_av_earnings_call_transcripts</code>
<code>FilingMetricResult</code>	<code>coreiq_filing_metrics_v2</code> / v5 filing metrics

Key properties

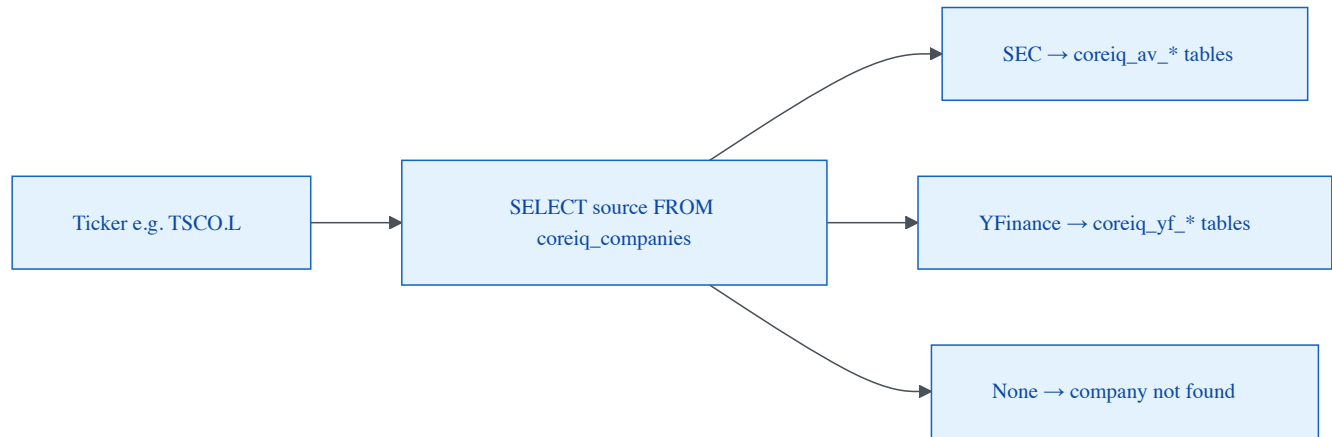
- `NewsArticle` : `formatted_date` , `company_tickers`
- `Company` / `CompanyOverview` : `display_name` , `formatted_market_cap` , `website_display`
- `EarningsCall` : `display_title` , `formatted_quarter` , `speaker_sections` (transcript parser)
- `FilingMetricResult` : `formatted_value` , `display_label` , `display_statement_type`

Fiscal calendar functions

Function	Returns
<code>get_calendar_quarter(month)</code>	CQ1–4
<code>get_fiscal_quarter(month, fiscal_year_end_month)</code>	FQ1–4
<code>parse_fiscal_year_end(str)</code>	Month 1–12
<code>FiscalPeriod.from_date(dt, period_type, fiscal_year_end)</code>	Labeled period column

source_router.py — SEC vs YFinance Routing

'source_router.py' — SEC vs YFinance Routing



Function	Returns	Cache
<code>get_company_source(ticker)</code>	<code>'SEC' 'YFinance' None</code>	<code>@st.cache_data(ttl=600)</code>
<code>is_sec_company(ticker)</code>	<code>bool</code>	
<code>is_yf_company(ticker)</code>	<code>bool</code>	

Table: `coreiq_companies.source`

Composite tickers: `TSCO.L` resolved via `exchange_acronym='L'`

Nearly every financial repository branches on `get_company_source(ticker)`.

segment_aliases.py — Geographic Segment Canonicalization

Purpose

Conservative US-only geographic grouping for screening segment filters.

Constants

Constant	Role
<code>GEO_CANONICAL_GROUPS</code>	United States, Canada, UK, North America, Non-US, Americas
<code>GEO_ALIAS_TO_CANONICAL</code>	Case-insensitive raw label → canonical
<code>GEO_SUPPRESSED_DROPDOWN_LABELS</code>	Hide duplicate aliases in User Interface (UI)
<code>GEO_PENSION_SKIP_LABELS</code>	Exclude pension-plan contamination
<code>GEO_KNOWN_BIZ_LABELS</code>	Business labels incorrectly in geo presets

Functions

Function	Role
<code>canonicalize_geo_label(raw)</code>	Single label normalization
<code>expand_geo_canonical_segments(selected)</code>	Canonical → all raw aliases for Structured Query Language (SQL) <code>IN</code>
<code>is_geo_pension_label()</code> / <code>is_geo_known_biz_label()</code>	Filter guards

Consumers: `screening_service.py`, `screening.py`, `repository.py` (`SegmentDataRepository`)

`screening_config.py` — Screening Constants

Rule: Zero DB calls. All constants for UI and SQL generation.

Core constants

Constant	Meaning
<code>OPERATORS</code> / <code>OPERATOR_SQL</code>	UI operators → SQL (<code>Between</code> handled separately)
<code>PERIOD_TYPES</code>	<code>Fiscal Year (FY)</code> , <code>CQ</code> , <code>FQ</code>
<code>TRAILING_QUARTERS</code> (<code>TQ</code>)	Display-only last N quarters (4/8/12, default 8)
<code>DB_SCALE</code>	<code>1_000_000</code> — UI \$mm ↔ raw DB dollars
<code>SCREENING_YEARS</code>	Historical FY range (current year → 2015)
<code>FORWARD_SCREENING_YEARS</code>	Includes future FY up to current+5

`STATEMENT_CONFIG`

Maps statement types to DB tables/columns:

Statement type	Data source
Income Statement, Balance Sheet, Cash Flow	AV + YF financial tables
Key Stats, Ratios	Repository tabular output (<code>metric_key</code>)
Business/Geographical Segments	<code>coreiq_filing_metrics_v5</code>
Estimates	AV wide + YF long earnings estimates
Forecasting	<code>coreiq_model_forecasts</code> <code>model_key</code>

News & Key Developments

- `NEWSDEV_TOPICS` — 15 AV NLP topic keys
- `NEWSDEV_TIMEFRAMES` — 7/30/90/365 days
- `KEYDEV_CATEGORIES` — 12-category CIQ taxonomy
- `KEYDEV_CATEGORIES_ALL` — 24 Excel-style DB categories
- `KEYDEV_TIMEFRAMES` — 1 day through All History

Helpers

- `get_metric_labels(statement)`, `get_metric_info(statement, label)`

screening_service.py — Screening Execution Engine

Rule: No Streamlit UI. Returns pandas DataFrames + debug metadata.

Pipeline

Screening narrows the company universe through progressive criterion filters until the working set matches all active rules.

Screening narrows the company universe through progressive criterion filters until the working set matches all active rules.



Criterion applicators

Function	Criterion type
<code>apply_industry_criteria()</code>	Industry multi-select
<code>apply_geography_criteria()</code>	Country filter
<code>apply_financial_criteria()</code>	FY/CQ/FQ financial metrics
<code>apply_key_stats_criteria()</code> / <code>apply_ratios_criteria()</code>	Tabular market data
<code>apply_estimates_criteria()</code> / <code>apply_forecast_criteria()</code>	Forward-looking
<code>apply_segment_statement_criteria()</code>	Business/Geo segment statements
<code>apply_keydevs_criteria()</code>	Key developments events
<code>apply_biz_segments_criteria()</code> / <code>apply_geo_segments_criteria()</code>	Legacy segment filters

Segment cache tables

- `coreiq_screening_segment_member_cache`
- `coreiq_screening_segment_values_cache`

Functions: `build_segment_values_cache()`, `read_segment_member_options_cache()`, `get_segment_member_options()`, etc.

Orchestration

Function	Role
<code>recompute_working_set(active_criteria, criterion_cache, allowed_tickers, allowed_members)</code>	Main pipeline; per-criterion fingerprint cache
<code>filter_universe_to_members()</code>	<code>(ticker, company_name)</code> composite key filter
<code>criteria_stack_fingerprint()</code>	Cache key for full stack

Scale rule: User enters \$mm; compared against raw DB dollars × `DB_SCALE` (1e6).

`repository.py` — Repository Catalog

Central read layer. All classes use static methods, `db_manager`, `source_router`, and `@st.cache_data`.

Repository classes

#	Class	Primary tables	Key methods
1	<code>CompanyRepository</code>	<code>coreiq_companies</code>	<code>get_companies_map</code> , <code>get_companies_rows</code> , <code>get_company_by_ticker</code> , <code>get_all_sectors</code>
2	<code>IncomeStatementRepository</code>	<code>coreiq_av/yf_financials_income_statement</code>	<code>get_income_statement_data</code> , <code>get_date_range</code> , <code>get_reported_currency</code>
3	<code>NewsRepository</code>	<code>coreiq_av/yf_market_news_sentiment</code>	<code>get_articles</code> , <code>get_yf_articles</code> , <code>get_news_tickers</code> , <code>get_sectors</code>
4	<code>CompanyOverviewRepository</code>	<code>coreiq_av/yf_company_overview</code>	<code>get_company_overview</code> , <code>company_exists</code>
5	<code>EarningsCallRepository</code>	<code>coreiq_av_earnings_call_transcripts</code>	<code>get_earnings_calls</code> , <code>search_transcripts_fulltext</code> , <code>get_non_sec_transcript_*</code>
6	<code>BalanceSheetRepository</code>	AV/YF balance sheet	<code>get_balance_sheet_data</code>
7	<code>CashFlowRepository</code>	AV/YF cash flow	<code>get_cash_flow_data</code>
8	<code>KeyStatsRepository</code>	Computed from statements + overview	<code>get_key_stats_data</code> , <code>extract_metric_value</code> , <code>get_estimated_data</code>
9	<code>AnalystEstimatesRepository</code>	AV/YF earnings estimates	<code>get_estimates_data</code>
10	<code>ModelForecastsRepository</code>	<code>coreiq_model_forecasts</code>	<code>get_forecasts_data</code> (ensemble + scenarios)
11	<code>RatiosRepository</code>	Computed in Python	<code>get_ratios_data</code> (liquidity, leverage, margins, ROA/ROE, P/E)
12	<code>ForexRepository</code>	<code>coreiq_av_forex_daily</code>	<code>get_conversion_rate</code> , <code>get_conversion_rates_bulk</code>
13	<code>StockQuoteRepository</code>	AV/YF time series daily	<code>get_latest_quote</code> , <code>get_price_history</code> , <code>get_market_cap_chart_data</code>
14	<code>FilingMetricRepository</code>	<code>coreiq_filing_metrics_v5</code>	<code>search_with_llm_fallback</code> , <code>get_header_metadata</code> , <code>SYNONYM_MAP</code>
15	<code>AVFinancialsEarningsRepository</code>	<code>coreiq_av_financials_earnings</code>	<code>get_earnings_date_pair_for_fiscal_quarter_e</code>
16	<code>EarningsCalendarRepository</code>	NASDAQ + YF calendar, IR websites	<code>get_calendar_events</code> , <code>get_transcript_for_calendar_event</code> , <code>get_earnings_alert_preferences</code>
17	<code>SegmentDataRepository</code>	<code>coreiq_filing_metrics_v5</code> + edgartools	<code>get_segment_data</code> , <code>get_square_footage_data</code>
18	<code>RatingsDataRepository</code>	Filing metrics (credit_rating, store_count)	<code>get_ratings_data</code>
19	<code>ExecutiveCompensationRepository</code>	<code>coreiq_executives_compensation</code> , YF overview	<code>get_sec_latest_compensation</code> , <code>get_yf_latest_compensation</code>

Module-level helpers

- `normalize_title_for_dedupe(title)` — Unicode-safe news dedupe
- `_get_fiscal_year_end_cached(ticker)` — 30 min cache
- `warmup_ec_caches()` — Earnings calendar cache warmup (called from `main.py`)

Caching TTL conventions

TTL	Use
60s	Earnings calls
300s	Financial statements, quotes, segments
600s	Company map, overview, source router
3600s	Companies rows, fiscal year end, news reference
43200s	News articles list

User Persistence Services

`watchlist_service.py`

Tables: `coreiq_watchlists`, `coreiq_watchlist_access`, `coreiq_watchlist_companies`

Function	Action
<code>ensure_tables()</code>	Idempotent DDL
<code>create_watchlist()</code>	Insert metadata
<code>get_user_watchlists()</code>	Own + granted (120s cache)
<code>get_watchlist_companies()</code> / <code>get_watchlist_tickers()</code>	Members
<code>add_companies_to_watchlist()</code> / <code>remove_company_from_watchlist()</code>	Membership
<code>grant_watchlist_access()</code> / <code>revoke_watchlist_access()</code>	ACL

Consumers: `screening.py`, `newsroom.py`, `earnings_calls.py`, `earnings_calendar.py`

`saved_criteria_service.py`

Tables: `coreiq_saved_criteria`, `coreiq_saved_criteria_access`

Function	Action
<code>save_criteria()</code> / <code>get_user_criteria()</code> / <code>get_criteria_by_id()</code>	CRUD
<code>update_criteria()</code> / <code>delete_criteria()</code>	Soft delete
<code>grant_access()</code> / <code>revoke_access()</code>	ACL

Consumer: `screening.py`

`portal_users_service.py`

Table: `coreiq_portal_users` — seeded with ~40 Coresight team members

Function	Role
<code>get_all_portal_users(exclude_email)</code>	ACL user-picker dropdown
<code>upsert_portal_user()</code>	Add/update user

Consumers: `screening.py`, `access_management.py`

Earnings Alert Services

`earnings_alert_service.py`

Tables:

- `coreiq_earnings_alert_preferences_history` — append-only; latest row = current settings
- `coreiq_earnings_alert_delivery_log` — unique `dedupe_key` per sent email

Function	Role
<code>load_preference(user_email)</code> / <code>save_preference(...)</code>	enabled, days_before, selection_mode, tickers, sectors
<code>list_enabled_preferences()</code>	Users with latest <code>enabled=1</code>
<code>delivery_exists(dedupe_key)</code> / <code>try_insert_delivery()</code>	Idempotent send guard

Consumers: `earnings_calendar.py`, `utils/earnings_alert_dispatcher.py`

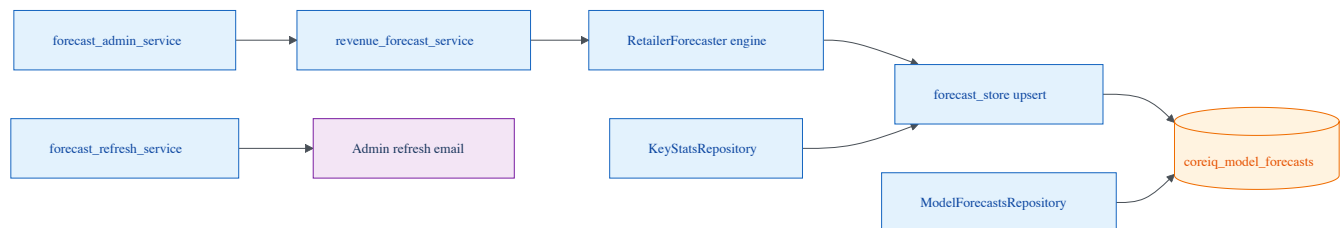
`earnings_alert_store.py`

Re-export shim only — imports from `earnings_alert_service`.

Forecasting Pipeline

Revenue forecasts flow from the forecasting engine into `coreiq_model_forecasts`, with admin sync and Key Stats consumption paths.

Revenue forecasts flow from the forecasting engine into coreiqmodelforecasts, with admin sync and Key Stats consumption paths.



`forecast_store.py`

Function	Role
<code>needs_update(ticker, last_actual_date)</code>	Staleness check (1 RTT)
<code>upsert_forecasts(...)</code>	<code>INSERT ... ON DUPLICATE KEY UPDATE</code>
<code>get_forecasts(ticker, model_key='ensemble')</code>	Read for Key Stats F- columns
<code>delete_forecasts_beyond_horizon(max_periods_ahead=5)</code>	Cleanup

Model keys: `ensemble`, `linear`, `cagr`, `exp_smoothing`, `holt`, `ma_trend`, `weighted_avg`

Scenario keys: `scenario_pessimistic`, `scenario_baseline`, `scenario_optimistic`

`revenue_forecast_service.py`

Class: `RevenueForecastService`

Method	Description
<code>get_companies()</code>	Companies with ≥3 annual revenue points
<code>get_company_dashboard(ticker, periods=5)</code>	Actuals, backtest, forecast, summary

Engine: `utils.retailer_forecaster.RetailerForecaster`

Fast path: Reconstructs forecast from `coreiq_model_forecasts` when `needs_update()` is false

`forecast_admin_service.py`

Function	Role
<code>is_forecast_admin(email)</code>	Hardcoded admin allowlist (4 emails)
<code>sync_forecast_for_ticker(display_ticker, force, periods=5)</code>	Single-ticker sync
<code>sync_all_eligible(force, progress_callback)</code>	Bulk sync
<code>clear_revenue_forecast_caches()</code>	Clear Streamlit caches

`forecast_refresh_service.py`

Function	Role
<code>get_refresh_table_data(period_type)</code>	All-ticker refresh status table
<code>send_model_refresh_email()</code>	HyperText Markup Language (HTML) Simple Mail Transfer Protocol (SMTP) notification to admins
<code>validate_q4_reporting_dates()</code>	Q4 date consistency audit
<code>ensure_forecast_columns()</code>	DDL migrations

`live_earnings_transcript_store.py`

Table: `coreiq_earnings_call_live_transcripts`

Source tag: `live_streamlit`

Unique key: `(source, ticker, quarter)` where quarter = `2024Q1`

Function	Role
<code>export_period_key(fiscal_year, quarter)</code>	<code>2024Q1</code> format
<code>build_earnings_transcript_payload()</code>	JSON transcript structure
<code>upsert_live_transcript_row()</code>	INSERT ... ON DUPLICATE KEY UPDATE

Consumer: `live_earnings_transcript.py`

`dummy_data.py` — Legacy Stub

Status: Broken — imports types (`SEC filing`, `MarketData`) that no longer exist in `models.py`. Production uses `repository.py`.

Design Conventions

- Logging:** `utils.server_logger` only (`log_structured_error`, `log_db_timing`)

2. **No UI in services:** `screening_service.py` is pure data; pages own Streamlit
3. **ACL:** Watchlists and saved criteria enforce server-side grants
4. **Idempotent DDL:** Services use `CREATE TABLE IF NOT EXISTS` + threading locks
5. **Source routing:** SEC → `coreiq_av_*` wide tables; YFinance → `coreiq_yf_*` long/pivot; composite tickers use `yf_symbol` or `TICKER.EXCHANGE`

Page → Data Module Map

Page	Primary data imports
<code>market_data.py</code>	<code>repository.*</code> , <code>models</code> , <code>source_router</code>
<code>screening.py</code>	<code>screening_config</code> , <code>screening_service</code> , <code>watchlist_service</code> , <code>saved_criteria_service</code> , <code>portal_users_service</code> , <code>segment_aliases</code>
<code>newsroom.py</code>	<code>repository.NewsRepository</code> , <code>models</code> , <code>watchlist_service</code>
<code>earnings_calls.py</code>	<code>repository.EarningsCallRepository</code> , <code>watchlist_service</code>
<code>company_filings.py</code>	<code>repository.FilingMetricRepository</code>
<code>forecasting.py</code>	<code>revenue_forecast_service</code> , <code>forecast_admin_service</code> , <code>forecast_refresh_service</code>
<code>live_earnings_transcript.py</code>	<code>live_earnings_transcript_store</code>
<code>access_management.py</code>	<code>portal_users_service</code> (via ACL UI)
<code>main.py</code>	<code>repository.warmup_ec_caches</code> , <code>watchlist/saved_criteria/portal/earnings_alert</code> <code>ensure_tables</code>

— End of Data Layer ('app/data/') —

PART 6 OF 22

Utilities & Caching (`app/utils/`)

Logging, caching, Azure Blob, PDF generation, email dispatch, and background warmups.

Continued from previous section →

Application: Market Data Portal (MDP)

Scope: Logging, Streamlit cache management, Azure Blob, Portable Document Format (PDF) generation, email dispatch, market data helpers, background scanning, local storage, constants

Overview

`app/utils/` provides cross-cutting infrastructure used by pages, core modules, data services, and components. Every module imports `server_logger` instead of `stdlib logging`. Each module exposes a consistent Application Programming Interface (API).

Figure 1 Part A — Startup triggers and core utils

Figure 1 Part A — Startup triggers and core utils

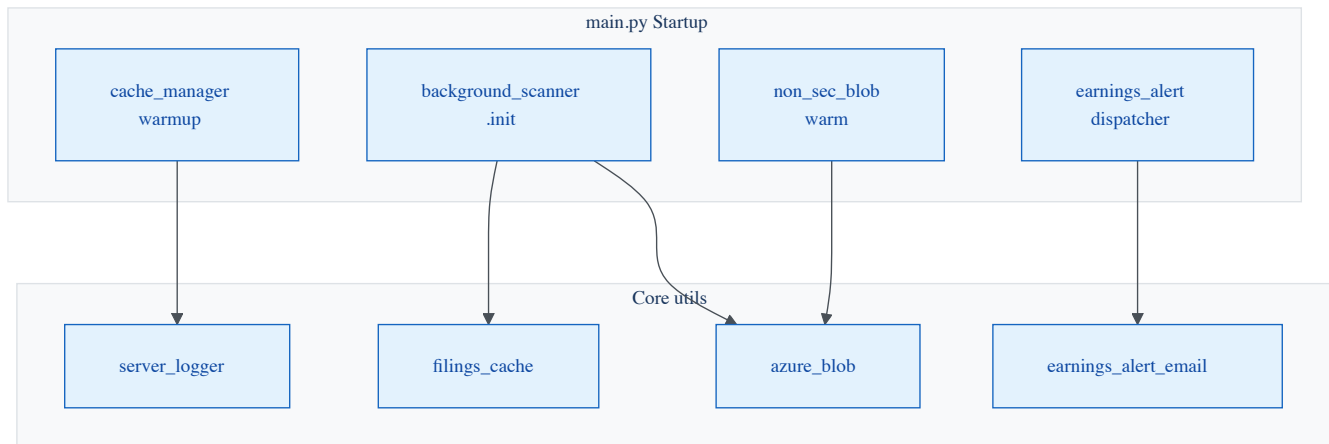
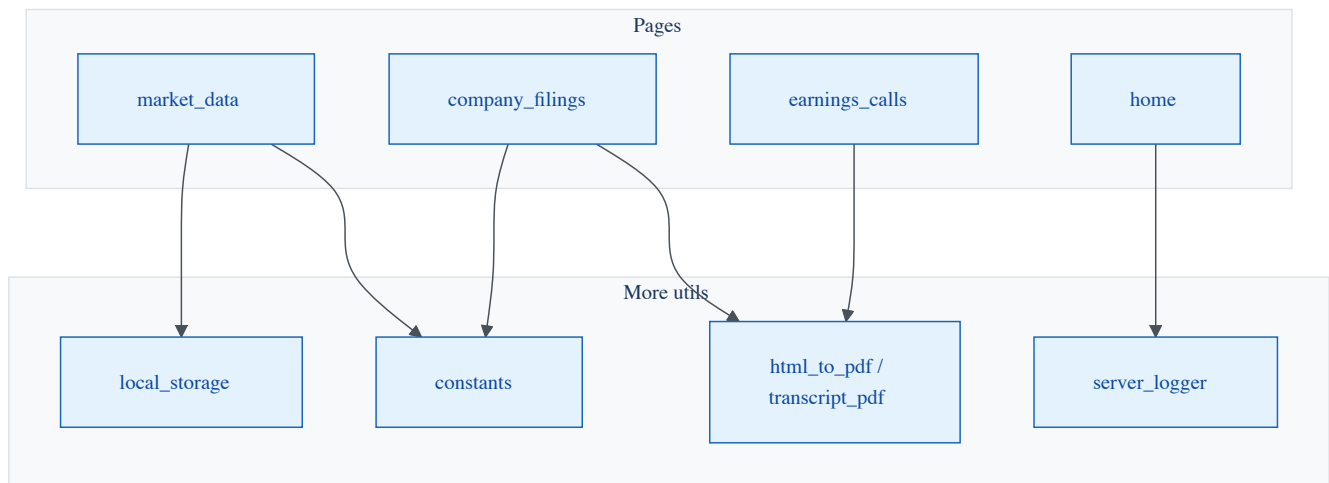


Figure 1 Part B — Pages consume utils

Figure 1 Part B — Pages consume utils



Module Index

Module	Status	Purpose
<code>server_logger.py</code>	Core	Central application logging
<code>cache_manager.py</code>	Active	Streamlit cache control + background warmup
<code>azure_blob.py</code>	Active	Azure Blob Storage helpers
<code>filings_cache.py</code>	Active	Persistent blob metadata cache
<code>background_scanner.py</code>	Active	Background Azure filing pre-fetch
<code>non_sec_blob_fallback.py</code>	Active	Non-U.S. Securities and Exchange Commission (SEC) filing local index
<code>html_to_pdf.py</code>	Active	SEC filing PDF (PyMuPDF)
<code>transcript_pdf.py</code>	Active	Earnings transcript PDF (fpdf2)
<code>earnings_alert_email.py</code>	Active	Simple Mail Transfer Protocol (SMTP) earnings alert emails
<code>earnings_alert_dispatcher.py</code>	Active	Background alert dispatch
<code>excel_export.py</code>	Active	Branded Excel export
<code>local_storage.py</code>	Active	Session-state preference keys
<code>local_storage_manager.py</code>	Active	Browser persistence via streamlit-local-storage
<code>ticker_utils.py</code>	Active	Ticker validation + fallback
<code>constants.py</code>	Active	Currency, exchange, country, topic lookups
<code>filing_units.py</code>	Active	Multi-filing document support (8-K, 6-K, etc.)
<code>retailer_forecaster.py</code>	Active	Revenue forecasting engine
<code>cache_cleaner.py</code>	Active	<code>__pycache__</code> purge on rerun
<code>blob_doc_type_discovery.py</code>	Standalone tool	Azure doc_type gap scanner
<code>filing_pdf.py</code>	Dead	Replaced by <code>html_to_pdf.py</code>
<code>market_data.py</code>	Deprecated	Legacy duplicate of <code>pages/market_data.py</code>
<code>companyProfile.py</code>	Dead	Replaced by <code>components/companyProfile.py</code>
<code>__init__.py</code>	Package marker	—

`server_logger.py` — Central Logging

Output: `server-logs/server-log.log`

Format: `{timestamp} | {level} | {rerun_id} | page={page} | {filename}:{funcName} | {message}`

Core API

Category	Functions
Rerun correlation	<code>new_rerun_id</code> , <code>get_rerun_id</code> , <code>set_page_context</code>
Log levels	<code>log_info</code> , <code>log_debug</code> , <code>log_warning</code> , <code>log_error</code> , <code>log_exception</code> , <code>log_critical</code> , <code>log_structured_error</code>
Error handling	<code>error_boundary</code> , <code>safe_execute</code>
Performance	<code>log_timing</code> , <code>log_db_timing</code> , <code>log_render_timing</code> , <code>log_function_timing</code>
Context managers	<code>timed_operation</code> , <code>timed_db_query</code> , <code>timed_render</code> , <code>timed_function</code>
Page tracking	<code>PageTimer</code> , <code>PageLoadTracker</code> , <code>NavigationTracker</code> , <code>track_page_load</code>
Log admin	<code>get_log_content</code> , <code>clear_logs</code> , <code>download_logs</code> , <code>get_log_stats</code> , <code>analyze_slow_operations</code>

Environment variables

Variable	Default	Role
<code>APP_LOG_LEVEL</code>	<code>WARNING</code>	Minimum log level
<code>APP_TIMING</code>	<code>1</code>	Timing-only filter mode
<code>APP_VERBOSE_LOGGING</code>	<code>0</code>	Verbose output

Rule: Stdlib `logging` is forbidden project-wide. All code imports from `utils.server_logger` only.

`cache_manager.py` — Streamlit Cache Control

Purpose

Centralized `@st.cache_data` management: clear, stats, foreground/background warming for news, companies, earnings, Yahoo Finance (YF) topics.

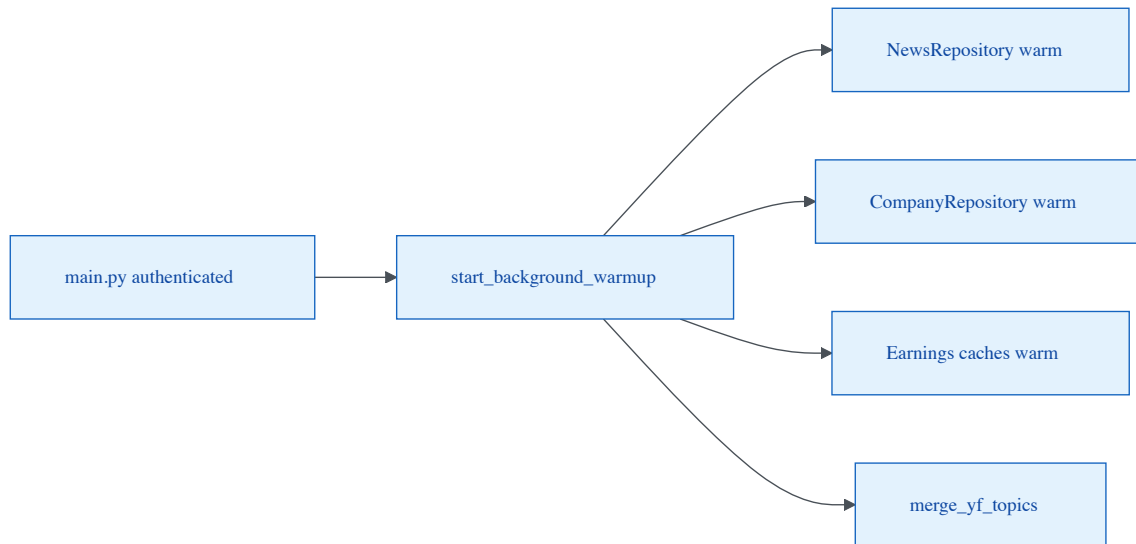
API

Function	Role
<code>clear_news_cache()</code> / <code>clear_company_cache()</code> / <code>clear_all_caches()</code>	Cache invalidation
<code>get_cache_stats()</code>	Hit/miss statistics
<code>warm_caches()</code>	Foreground warmup
<code>start_background_warmup()</code>	Daemon thread warmup
<code>render_cache_controls()</code>	Optional admin User Interface (UI)

Startup integration

Called from `main.py` when authenticated and `ENABLE_BG_WARMUP=1` (default).

Called from `main.py` when authenticated and `ENABLEBGWARMUP=1` (default).



Warms: `data.repository.NewsRepository` , `CompanyRepository` , earnings caches; calls `utils.constants.merge_yf_topics` .

Azure Blob & Filing Cache Subsystem

Architecture

Cross-cutting utilities consumed by pages, core modules, and background warmups.

Figure 2 Part A — Azure source & utils modules

Figure 2 Part A — Azure source & utils modules

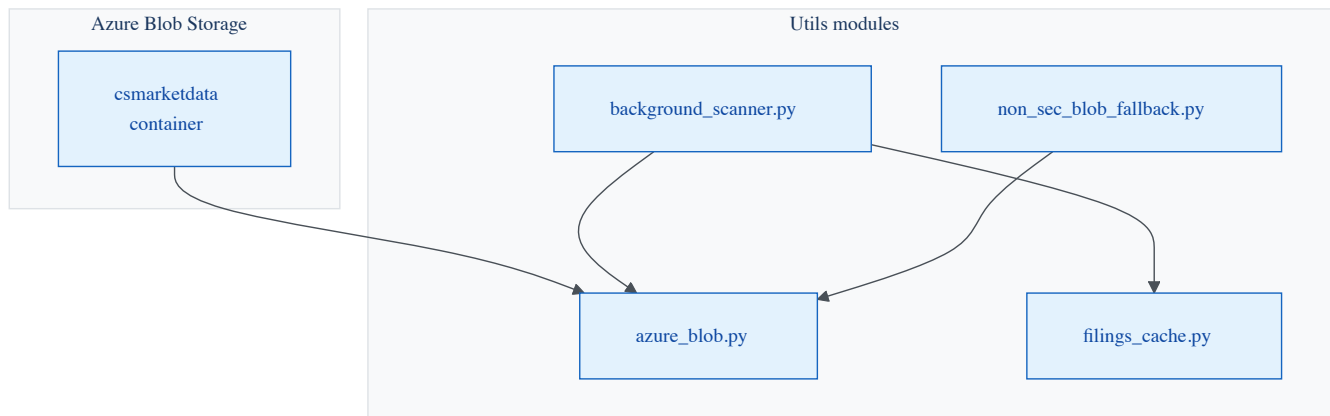
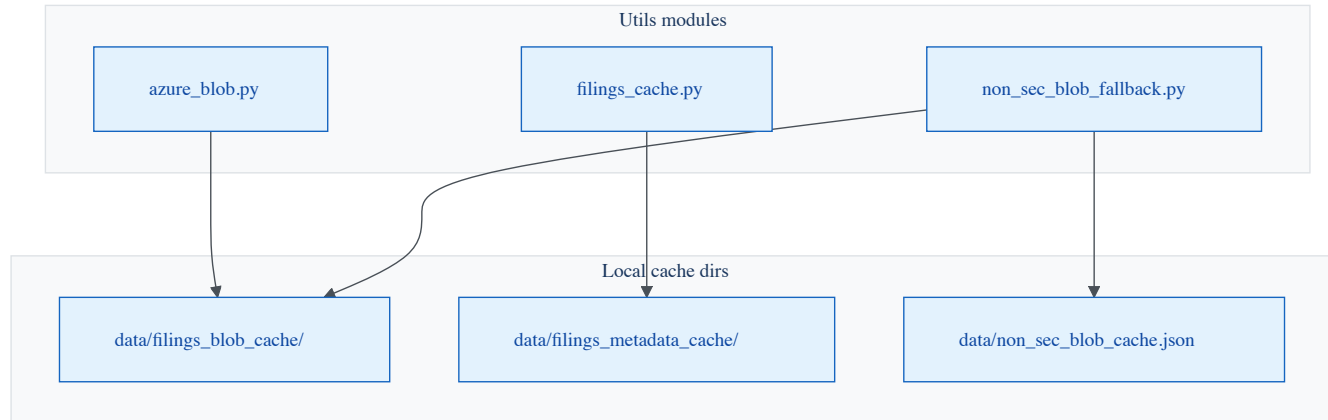


Figure 2 Part B — Local cache paths

Figure 2 Part B — Local cache paths



azure_blob.py

Function	Role
<code>get_container_client()</code>	Azure SDK container client
<code>local_cache_path_for_blob(blob_name)</code>	Local filesystem path
<code>ensure_local_blob_for_transcript(blob_name)</code>	Download if missing
<code>download_blob_to_memory(blob_name)</code>	In-memory bytes

Env vars:

Variable	Default
<code>AZURE_STORAGE_ACCOUNT_NAME</code>	<code>csmarketdata</code>
<code>AZURE_STORAGE_ACCOUNT_KEY</code>	(required)
<code>AZURE_BLOB_CONTAINER</code>	<code>azure-storage-test</code>
<code>AZURE_BLOB_PREFIX</code>	(optional prefix)

Cache root: `{repo}/data/filings_blob_cache/`

Consumers: `earnings_calls.py`, `non_sec_blob_fallback.py`

filings_cache.py

Class / Function	Role
<code>FilingsMetadataCache</code>	DiskCache/SQLite-backed metadata (24h TTL, ETag validation)
<code>InMemoryFallbackCache</code>	Fallback when DiskCache unavailable
<code>get_filings_cache()</code>	Singleton accessor
<code>generate_etag_from_blobs(blobs)</code>	ETag for cache invalidation
<code>invalidate_company_cache(ticker)</code>	Per-ticker purge

Consumers: `background_scanner.py`, `company_filings.py`

background_scanner.py

Purpose: Intelligent background pre-fetch of Azure filing blobs. Skips scan when cache sufficient; discovers years dynamically; parallel downloads.

Function	Role
<code>BackgroundFilingsScanner</code>	Main scanner class
<code>init_background_scanner(auto_start=True)</code>	Start daemon scanner
<code>ensure_cache_purge()</code>	One-time cache purge (<code>.cache_purge_v1_done</code>)
<code>get_scan_progress()</code> / <code>get_cache_audit()</code>	Progress for UI
<code>is_scan_complete()</code>	Completion flag

Startup: `main.py` when `ENABLE_BG_SCANNER=1` (default) and authenticated.

Consumers: `main.py` , `home.py` (progress UI), `company_filings.py`

Data dirs: `data/filings_blob_cache/` , `data/filings_metadata_cache/` , `.blob_container_last_modified`

non_sec_blob_fallback.py

Purpose: Local-first index of non-SEC filing PDFs (annual reports, half-yearly, interim). Azure only as fallback.

Function	Role
<code>get_non_sec_blob_data(ticker)</code>	Blob listing for ticker
<code>warm_non_sec_cache_at_startup(tickers, max_workers=32)</code>	Parallel warm
<code>get_non_sec_tickers_from_db()</code>	Tickers with non-SEC sources
<code>invalidate_non_sec_cache()</code>	Called when scanner downloads new blobs
<code>is_non_sec_source_company(ticker)</code>	Source check

Startup: `main.py` daemon thread when authenticated.

Consumer: `company_filings.py`

PDF Generation

html_to_pdf.py — SEC Filings (Production)

Function	Role
<code>generate_filing_pdf(html_content, title, ticker, ...)</code>	PyMuPDF (fitz) render to PDF bytes
<code>_clean_sec_html(html)</code>	Strip iXBRL/eXtensible Business Reporting Language (XBRL) junk, wrap in branded CSS

Consumer: `company_filings.py` (PDF download button)

transcript_pdf.py — Earnings Transcripts

Function	Role
<code>generate_transcript_pdf(call, segments, ...)</code>	fpdf2 A4 branded PDF

Consumer: `earnings_calls.py`

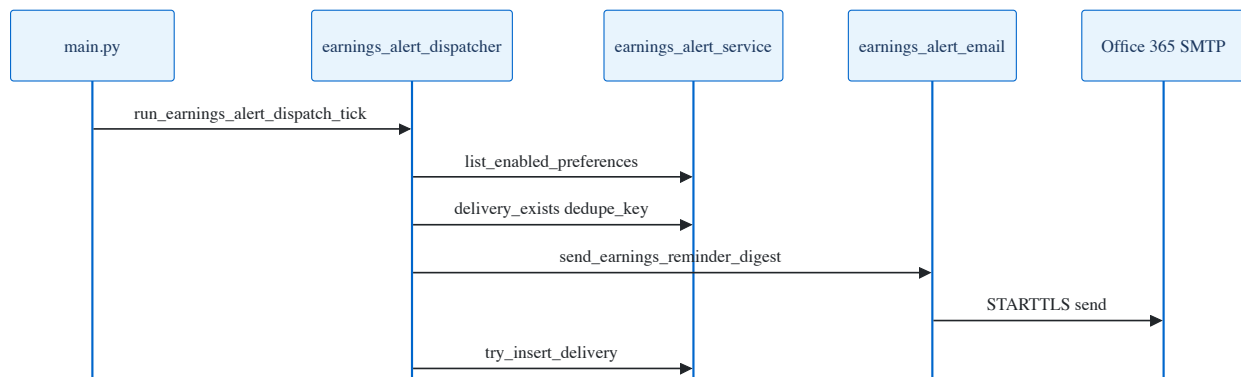
filing_pdf.py — Legacy (Dead)

Simple fpdf2 plain-text generator. **Superseded by** `html_to_pdf.py` . No consumers.

Earnings Alert Email Pipeline

Background dispatcher loads enabled preferences, dedupes deliveries, and sends SMTP alerts for upcoming earnings.

Background dispatcher loads enabled preferences, dedupes deliveries, and sends SMTP alerts for upcoming earnings.



earnings_alert_email.py

Function	Role
<code>send_earnings_alert_preferences_confirmation(...)</code>	User preference saved confirmation
<code>send_earnings_reminder_digest(...)</code>	Reminder email with upcoming earnings

Env vars:

Variable	Default
<code>SMTP_SERVER</code>	<code>smtp.office365.com</code>
<code>SMTP_PORT</code>	<code>587</code>
<code>FROM_EMAIL</code>	<code>dataautomation@coresight.com</code>
<code>EMAIL_PASSWORD</code>	Required — no-op if unset

earnings_alert_dispatcher.py

Function	Role
<code>run_earnings_alert_dispatch_tick()</code>	Throttled dispatch on each authenticated load
<code>start_background_dispatch_thread()</code>	Daemon thread for scheduled sends

Env vars:

Variable	Default
<code>EARNINGS_ALERT_DISPATCH</code>	<code>1</code>
<code>EARNINGS_ALERT_DISPATCH_THREAD</code>	<code>1</code>
<code>EARNINGS_ALERT_DISPATCH_MIN_INTERVAL_SEC</code>	<code>120</code>
<code>EARNINGS_ALERT_DISPATCH_HOUR_UTC</code>	<code>8</code>
<code>EARNINGS_ALERT_DISPATCH_LOOKBACK_DAYS</code>	<code>3</code>
<code>EARNINGS_ALERT_DISPATCH_THREAD_SLEEP_SEC</code>	<code>600</code>

Local Storage & State Persistence

`local_storage.py`

Session-state-backed typed keys via `StorageKey` enum.

Function	Stored value
<code>get/set_sec_filing(...)</code>	SEC filing filter state
<code>get/set_date_range(...)</code>	Date range picker
<code>get/set_marketdata_tab(...)</code>	Active Market Data tab
<code>get/set_selected_company(...)</code>	Global company selection
<code>clear_selected_company()</code>	Reset selection

Consumers: `market_data.py`, `home.py`, `components/toolbar.py`

`local_storage_manager.py`

Browser persistence via `streamlit-local-storage` under key `sip_app_state`.

Function	Role
<code>sync_market_data_state()</code> / <code>save_market_data_state()</code>	Market Data state round-trip
<code>get_persistent_state()</code> / <code>set_persistent_state()</code>	Generic namespaced state
<code>load_earnings_calls_state()</code> / <code>save_earnings_calls_state()</code>	Earnings Calls filters

Consumers: `market_data.py`, `home.py`, `earnings_calls.py`

Note: Degrades gracefully if `streamlit-local-storage` package missing.

ticker_utils.py — Ticker Validation

Function	Role
<code>validate_and_get_ticker(ticker, source)</code>	Database (DB) lookup; fallback to <code>AMZN</code> if invalid
<code>get_ticker_from_source(source)</code>	Extract ticker from URL/session
<code>check_ticker_exists(ticker)</code>	Boolean existence check

Constant: `DEFAULT_FALLBACK_TICKER = "AMZN"`

Consumers: `market_data.py`, `newsroom.py`, `company_filings.py`, `earnings_calls.py`, `earnings_calendar.py`

constants.py — Lookup Tables & Formatters

Key data exports

Constant	Contents
<code>CURRENCY_NAMES</code> / <code>CURRENCY_SYMBOLS</code>	ISO currency display
<code>EXCHANGE_CODE_MAP</code>	Exchange acronym → display name
<code>COUNTRY_NAMES</code>	Country code → name
<code>AV_TOPIC_LABELS</code> / <code>NEWS_TOPIC_LABELS</code>	News topic display labels
<code>SEGMENT_METRIC_GROUPS</code>	XBRL segment groupings for screening

Key functions

Function	Role
<code>get_currency_symbol(code)</code>	<code>\$</code> , <code>€</code> , etc.
<code>format_currency_full(value, code)</code>	Full formatted amount
<code>get_exchange_code(name)</code> / <code>format_exchange(code)</code>	Exchange formatting
<code>format_country(code)</code>	Country name
<code>merge_yf_topics(av_topics, yf_topics)</code>	Combined topic list
<code>get_topic_display_label(key)</code>	Human-readable topic

Consumers: `market_data.py`, `newsroom.py`, `screening.py`, `forecasting.py`, `repository.py`, `components/companyProfile.py`, `excel_export.py`, `cache_manager.py`

excel_export.py — Excel Export

Function	Exports
<code>export_company_profile_excel(...)</code>	Company Profile tab
<code>export_company_profile_simple_excel(...)</code>	Simplified profile
<code>export_stock_quote_excel(...)</code>	Stock quote data
<code>export_company_full_excel(...)</code>	Full company workbook
<code>export_financial_excel(...)</code>	Income/Balance/Cash Flow tabs

Library: openpyxl with Coresight branding

Consumer: `market_data.py`

filing_units.py — Multi-Filing Documents

Supports 8-K, 6-K, DEF14A, and other sub-document types within a filing year.

Function	Role
<code>get_filing_units(ticker, year, doc_type)</code>	List sub-folders from DB or filesystem
<code>get_default_filing_unit(...)</code>	Default unit for display
<code>build_filing_blob_path(...)</code>	Azure blob path builder
<code>get_local_filing_path(...)</code>	Local cache path
<code>has_filing_units(...)</code>	Boolean check

Constant: `FILING_UNIT_DOC_TYPES = {"8-K", "6-K", "DEF14A", "DEFA14A", "DEF 14A", "PRE14A"}`

Consumer: `company_filings.py` (primary)

retailer_forecaster.py — Revenue Forecasting Engine

Class: `RetailerForecaster`

Method	Role
<code>from_dataframe(df)</code>	Construct from revenue history
<code>summary_stats()</code>	Descriptive statistics
<code>backtest()</code>	Holdout-year MAPE evaluation
<code>forecast(periods=5)</code>	Forward projections
<code>run()</code> / <code>run_full_analysis()</code>	Full pipeline

Models (6): linear regression, CAGR, exponential smoothing, Holt, MA trend, weighted average. Best MAPE model selected; ensemble combines top performers.

Consumers: `revenue_forecast_service.py`, `forecast_admin_service.py`, `forecast_refresh_service.py`, forecast scripts

cache_cleaner.py — Bytecode Purge

Function	Role
<code>clean_pycache()</code>	Delete <code>__pycache__</code> under <code>app/</code>
<code>ensure_fresh_code()</code>	Run once per page load
<code>clean_pycache_once()</code>	Idempotent single purge

Sets `PYTHONDONTWRITEBYTECODE=1` at runtime.

Consumer: `earnings_calls.py` (`ensure_fresh_code` at page load)

Note: `main.py` also purges `__pycache__` on every startup.

blob_doc_type_discovery.py — Maintenance Tool

Scans Azure blobs for `doc_type` folder names not in the app's known list. **Not wired into the app** — callable from scripts/REPL.

Function	Role
<code>discover_unknown_doc_types()</code>	Full scan with watermark
<code>get_last_scan_result()</code>	Last scan JSON
<code>format_discovery_report()</code>	Human-readable report

State file: `data/blob_doc_type_scan_state.json`

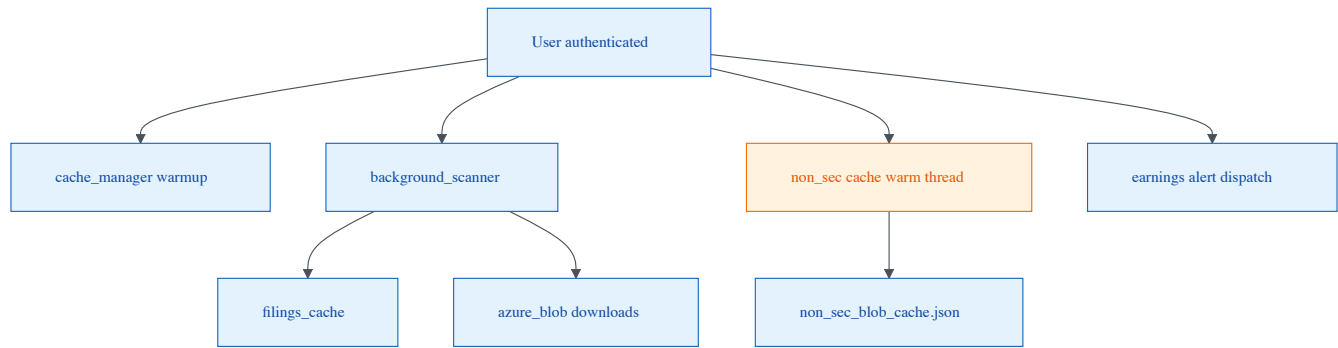
Legacy / Dead Modules

Module	Replacement	Notes
<code>market_data.py</code>	<code>pages/market_data.py</code>	~1,900-line deprecated duplicate (2025-07-10)
<code>filing_pdf.py</code>	<code>html_to_pdf.py</code>	Simple fpdf2 stub
<code>companyProfile.py</code>	<code>components/companyProfile.py</code>	No imports anywhere

Startup Orchestration Summary

Utility	Trigger	Env gate
<code>cache_manager.start_background_warmup</code>	Authenticated session	<code>ENABLE_BG_WARMUP=1</code>
<code>background_scanner.ensure_cache_purge</code> + <code>init_background_scanner</code>	Authenticated session	<code>ENABLE_BG_SCANNER=1</code>
<code>non_sec_blob_fallback.warm_non_sec_cache_at_startup</code>	Authenticated daemon thread	Once per process
<code>earnings_alert_dispatcher</code> tick + thread	Authenticated	<code>EARNINGS_ALERT_DISPATCH=1</code>

Architecture diagram



Dependency Graph (High Level)

```

server_logger ← (used by almost everything)
constants      ← market_data, screening, newsroom, repository, excel_export
azure_blob     ← earnings_calls, non_sec_blob_fallback
filings_cache  ← background_scanner, company_filings
background_scanner → filings_cache, non_sec_blob_fallback
earnings_alert_dispatcher → earnings_alert_email → SMTP
local_storage  ← market_data, toolbar, home
retailer_forecaster ← revenue_forecast_service, forecast_admin_service
  
```

— End of Utilities & Caching ('app/utis/') —

PART 7 OF 22

Login Page — Technical Documentation

OIDC PKCE login flow, token exchange, and cookie handoff to Home.

Continued from previous section →

Source: `app/pages/login.py`

URL: `/login` (default landing page)

Purpose: Authenticate Coresight employees via OpenID Connect (OIDC) Single Sign-On (SSO) (all environments) or JSON Web Token (JWT) email/password (legacy production path when `IS_OIDC_ENV=False`)

Overview

The login page is the application entry point and the OIDC authorization-code callback handler. It supports two authentication modes gated by `core.auth_environment.IS_OIDC_ENV`:

Mode	When active	User experience
OIDC	<code>IS_OIDC_ENV=True</code> (current default)	"Sign in with Coresight" → redirect to Identity Provider (IdP) → callback with <code>?code=</code>
JWT	<code>IS_OIDC_ENV=False</code> (legacy)	Email + password form → WordPress JWT endpoint

Domain restriction: Only `@coresight.com` email addresses are accepted. Non-Coresight users are redirected to IdP logout to clear the SSO session.

The page performs substantial work at **module import time** (before `main()`): callback capture, handoff processing, fast cookie restore, and OIDC token exchange. The visible User Interface (UI) is only shown when the user is not already authenticated and no callback is in progress.

URL / Route

Route	Query params	Meaning
<code>/login</code>	—	Show login UI or fast-restore redirect
<code>/</code> (default)	Same as login	Default page in <code>main.py</code>
<code>/login</code>	<code>code</code> , <code>state</code>	OIDC authorization callback
<code>/login</code>	<code>error</code> , <code>error_description</code>	OIDC error callback
<code>/login</code>	<code>oidc_handoff=<key></code>	Cross-host login token handoff
<code>/login</code>	<code>oidc_post_logout=<key></code>	Post-logout return to original host
<code>/login</code>	<code>auth_error=cookie_write_failed</code>	Cookie handoff failure from <code>main.py</code>

Registered in `main.py`: `st.Page("pages/login.py", title="Login", url_path="login", default=True)`

Dependencies

Python modules

Import	Role
<code>core.auth_environment</code>	<code>IS_OIDC_ENV</code> , <code>is_production_deploy()</code>
<code>core.auth_manager</code>	<code>login_user</code> , <code>is_authenticated</code> , <code>get_current_domain</code> , <code>get_or_create_auth_flow_id</code> , <code>get_auth_flow_id</code> , <code>render_auth_cookie_handoff_redirect</code> , <code>_auth_request_meta</code>
<code>components.styles.hide_sidebar</code>	Hide Streamlit sidebar/chrome
<code>utils.server_logger</code>	<code>log_structured_error</code> , <code>log_error</code> , <code>log_warning</code> , <code>log_timing</code>
<code>requests</code>	OIDC token exchange, userinfo, JWT auth POST
<code>streamlit /</code> <code>streamlit.components.v1</code>	UI, SSO button parent-frame navigation
<code>dotenv.load_dotenv</code>	Environment loading

HyperText Markup Language (HTML)/CSS components (`app/components/`)

Loaded via `load_html_component(filename)` from `APP_DIR / "components"` :

File	Purpose
<code>login_layout.css</code>	Page layout styles (wrapped in <code><style></code> if needed)
<code>login_header.html</code>	Fixed top header bar (Coresight branding, external links)
<code>login_footer.html</code>	Footer widgets (social, terms, contact)

External integrations

Service	Endpoint / config	Usage
Coresight OIDC IdP	<code>IDP_AUTHORIZE_URL</code> (default <code>https://coresight.com/csr-idp/authorize</code>)	SSO redirect
Coresight OIDC token	<code>IDP_TOKEN_URL</code> (default <code>.../csr-idp/token</code>)	Code → tokens
Coresight userinfo	<code>IDP_USERINFO_URL</code> (default <code>.../wp-json/csr-idp/v1/userinfo</code>)	Email/profile fallback
Coresight IdP logout	<code>https://coresight.com/csr-idp/logout/</code>	Access-denied cleanup
WordPress JWT (legacy)	<code>{wp-json}/jwt-auth/v1/token</code> via <code>AUTH_URL</code> / <code>AUTH_URL_PAID</code>	Production password login
Azure CDN	Coresight logo image URL in overlays	Loading spinners

OIDC configuration constants

Constant	Value / source
<code>OIDC_CLIENT_ID</code>	<code>market-data</code>
<code>OIDC_CLIENT_SECRET</code>	Hardcoded in source (also sent in token exchange)
<code>OIDC_REDIRECT_URI</code>	<code>marketdata.coresight.com</code> (prod) or <code>marketdata-stg.coresight.com</code> (non-prod)
<code>OIDC_SCOPE</code>	<code>openid profile email</code> (env override <code>OIDC_SCOPE</code>)
<code>OIDC_STATE_MAX_AGE_SECONDS</code>	900 (15 min)
<code>OIDC_HANDOFF_MAX_AGE_SECONDS</code>	180
<code>POST_LOGIN_REDIRECT_DELAY_MS</code>	300

UI Components

Layout structure (`main()`)

```
login_layout.css (global)
login_header.html (fixed header)
├ .content-wrapper.login-content
│   └ st.columns [2, 5, 2]
│       └ center column:
│           └ Welcome h4 + Portal h1
│           └ [OIDC] SSO section OR [JWT] form
│           └ "Contact Us" trial access footer
login_footer.html
```

OIDC mode UI elements

Element	Type	Behavior
Welcome heading	HTML <code><h4></code>	"Welcome to the Coresight Research"
Portal title	HTML <code><h1></code>	"Coresight Market Data Portal"
Premium member copy	<code>st.markdown</code>	Markdown + link to coresight.com/research
Sign in with Coresight	HTML <code><button id="csr-sso-btn"></code>	Click shows overlay, JS navigates to pre-built authorize URL
SSO overlay	<code>#csr-sso-overlay</code>	Logo + spinner + "Redirecting to SSO..."
<code>st.components.v1.html</code>	Hidden iframe script	Attaches click handler in parent document (bypasses iframe sandbox)
Error display	<code>st.error</code>	From <code>_oidc_login_error</code> session key (one-shot pop)
Trial access footer	HTML	Link to coresight.com/contact

JWT mode UI elements (legacy)

Element	Streamlit widget	Key
Email	<code>st.text_input</code>	default key
Password	<code>st.text_input(type="password")</code>	default key
Log in button	<code>st.button</code>	<code>login_button</code>
Auth error	<code>st.error</code>	<code>auth_error</code> session (cleared after display)

Loading overlays (module-level)

Overlay	Trigger	Message
<code>_make_loading_overlay</code>	Fast restore, OIDC callback	"Loading your workspace..." / "Completing sign-in..."
Callback spinner	<code>__oidc_cb_captured</code>	"Completing sign-in..."

CSS side effects

- Hides `[data-testid="stHeaderActionElements"]`
- Full-width red SSO button (`#d32f2f` / hover `#b71c1c`)
- Hides `stCustomComponentV1` container (height-0 iframe)

Session State Keys

OIDC flow

Key	Set when	Purpose
<code>__oidc_cb_code</code>	Callback capture	Authorization code
<code>__oidc_cb_state</code>	Callback capture	Encoded state payload
<code>__oidc_cb_error</code>	Callback capture	IdP error code
<code>__oidc_cb_error_desc</code>	Callback capture	IdP error description
<code>__oidc_cb_captured</code>	Callback capture	Flag: callback params stored
<code>__oidc_processing_code</code>	Processing start	In-flight code guard
<code>__oidc_processing_started_at</code>	Processing start	Stale processing detection (>8s)
<code>__completed_oidc_code</code>	Success	Dedup processed codes
<code>__completed_oidc_token_data</code>	Success	Cached normalized token data
<code>__oidc_exchange_result</code>	Token exchange	Session cache of raw token response
<code>__oidc_denied_codes_dict</code>	Access denied	Map code → True; skip reprocessing
<code>__completed_oidc_handoff_key</code>	Host handoff	Dedup handoff keys
<code>__completed_oidc_handoff_token_data</code>	Host handoff	Cached handoff token data
<code>__failed_oidc_handoff_key</code>	Handoff failure	Prevent retry loops
<code>__completed_oidc_post_logout_key</code>	Post-logout return	Dedup post-logout redirects
<code>__force_prompt_login</code>	(pop on use)	Force <code>prompt=login</code> on authorize URL
<code>__oidc_login_error</code>	Handoff/cookie failures	User-visible error message
<code>__logout_guard</code>	After logout	Block auto-restore from cookie
<code>__just_logged_out</code>	(pop)	Sets logout guard
<code>auth_flow_id</code>	SSO journey	Correlation ID (via auth_manager)

JWT flow (legacy)

Key	Purpose
<code>is_authenticating</code>	Button disabled + spinner label
<code>auth_error</code>	Display once then clear

Shared (post-login)

Key	Purpose
<code>authenticated</code>	Set by <code>login_user</code> / fast restore
<code>auth_data</code>	Full auth payload

Data Layer

Database

Table: market_data_user_sessions

Written by: `auth_manager.login_user()` → `AuthManager._store_session_in_db()`

Not read during login page execution — audit only.

```
INSERT INTO market_data_user_sessions
    (user_email, user_nicename, user_display_name, token, login_at, last_activity)
VALUES (:user_email, :user_nicename, :user_display_name, :token, NOW(), NOW())
```

Ephemeral file caches (`/tmp`)

Prefix	File pattern	TTL	Purpose
return	_oidc_return_{hash}.json	OIDC_RETURN_CONTEXT_MAX_AGE_SECONDS	Original request origin for cross-host handoff
handoff	_oidc_handoff_{hash}.json	OIDC_HANDOFF_MAX_AGE_SECONDS	Token data for oidc_handoff query param
logout_return	_oidc_logout_return_{hash}.json	OIDC_RETURN_CONTEXT_MAX_AGE_SECONDS	Post-logout origin return
exchange	_oidc_xch_{hash}.json	300s	Authorization code → token response dedup

In-process caches (OIDC only)

- `_oidc_exchange_lock` — threading lock (25s acquire timeout)
- `_oidc_exchange_cache` — dict keyed by authorization code

Business Logic

Module initialization order

Module initialization order



OIDC authorization URL (`_build_authorize_url`)

1. Generate `nonce` , `code_verifier` (`cv`), timestamp
2. Encode state: `{nonce, cv, ru: redirect_uri, ts, rk?: return_key}`
3. Stash request origin in `/tmp` if allowed (`_stash_return_origin`)
4. Proof Key for Code Exchange (PKCE): `code_challenge=cv` , `code_challenge_method=plain`
5. Optional `prompt=login` if `_force_prompt_login`

OIDC callback processing

1. **Denied code check** — if code in `_oidc_denied_codes_dict` , meta-refresh to `/`
2. **Dedup** — if `_completed_oidc_code == code` , reuse cached token data
3. **In-progress** — if same code processing, wait/retry or complete from cache
4. **New code:**
 - Decode and validate state (`_decode_state_payload` , max age 900s)
 - `_exchange_code_for_tokens` with lock + triple cache (session, process, file)
 - `_build_token_data` — decode id_token JWT, optional userinfo fetch
 - `_maybe_redirect_oidc_handoff` if login started on different host (e.g. localhost)
 - `_complete_oidc_login`

`_complete_oidc_login`

1. Reject non-`@coresight.com` → mark code denied, IdP logout redirect, `st.stop()`
2. `login_user(...)` with email, token, id_token, wp_user_id
3. On success: clear OIDC transient keys, `render_auth_cookie_handoff_redirect("/home")` , `st.stop()`

Fast restore (already logged in)

Reads `auth_session` from `st.context.cookies` synchronously:

- Valid cookie → loading overlay → set session_state → `st.switch_page("pages/home.py")`
- Session_state auth without HTTP cookie → cookie handoff redirect
- OIDC: respects `_logout_guard` and `_auth_invalidated`

Cross-host handoff

When OIDC redirect URI is STG but user started on `localhost` :

1. Login captures origin in state (`rk` key)
2. After token exchange on STG, if origins differ → write handoff file, redirect to `{origin}?oidc_handoff={key}`
3. Origin host `_process_oidc_handoff` consumes file and completes login locally

JWT authentication (legacy `_authenticate_user`)

Tries token endpoint with three payload encodings in order:

1. JSON wire with escaped quotes (`_send_json_wire`)
2. Raw JSON (`_send_json_raw`)
3. Form URL-encoded (`_send_form`)

Special handling for passwords containing quotes via `_escape_for_unslash` .

Error Handling

Scenario	User feedback	Log / recovery
OIDC <code>error</code> query param	<code>st.error</code> with description	Clears callback session keys
Invalid/expired state	"Invalid login state"	Clears processing marker
Token exchange non-200	<code>st.error</code> with payload	Retries file cache on <code>invalid_grant</code>
Exchange lock timeout	"Login timed out"	25s lock wait
Missing PKCE verifier	"Missing PKCE verifier"	—
Unresolved email from tokens	Contact support message	—
Non-Coresight email	IdP logout redirect (no message)	Code marked denied
<code>login_user</code> failure	<code>result.error_message</code>	—
Handoff expired	<code>_oidc_login_error</code>	<code>__failed_oidc_handoff_key</code>
Cookie write failed	Via <code>auth_error</code> query	Set in <code>main.py</code>
Page render exception	"Something went wrong"	<code>log_structured_error</code>
JWT bad credentials	"Incorrect username or password"	—
JWT non-Coresight	"Access restricted to Coresight employees only"	—

Access Control List (ACL) / Permissions

The login page itself has **no page-level ACL** — it is public. Post-login, access to other pages may be gated by `coreiq_page_access_control` on individual pages.

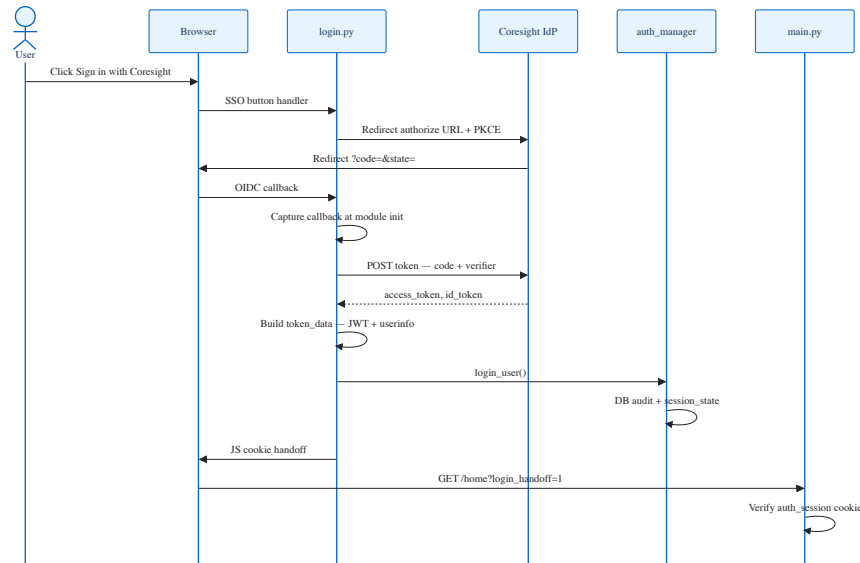
Hard gate at login: `user_email.lower().endswith("@coresight.com")` in both OIDC and JWT paths.

Flowcharts

OIDC login (happy path)

Standard SSO journey from button click through token exchange to cookie handoff and server verification.

Standard SSO journey from button click through token exchange to cookie handoff and server verification.



User interaction — SSO button

The SSO button injects parent-frame JavaScript to navigate to the IdP authorize URL after a brief overlay.

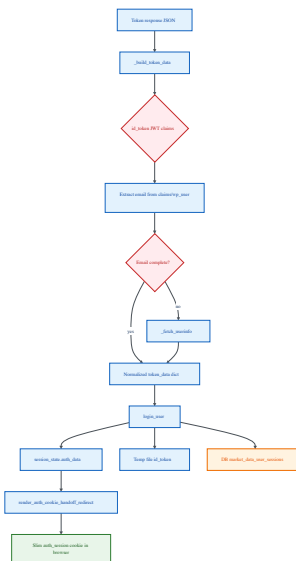
The SSO button injects parent-frame JavaScript to navigate to the IdP authorize URL after a brief overlay.



Data flow — token to session

Token response is normalized to user identity, written to session state and audit Database (DB), then slim cookie is set in the browser.

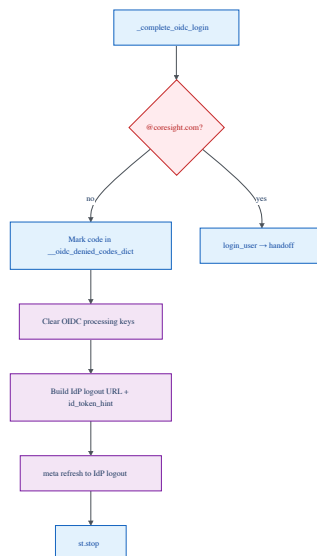
Token response is normalized to user identity, written to session state and audit Database (DB), then slim cookie is set in the browser.



Error handling — access denied

Non-@coresight.com emails trigger IdP logout and block reprocessing of the denied authorization code.

Non-@coresight.com emails trigger IdP logout and block reprocessing of the denied authorization code.



Key code references

Function / block	Lines (approx.)	Description
OIDC config	29–48	IdP URLs, client, redirect
<code>_build_authorize_url</code>	427–455	SSO URL builder
<code>_exchange_code_for_tokens</code>	469–563	Locked token exchange
<code>_complete_oidc_login</code>	672–769	Domain check + login_user
Callback capture	900–921	Module-level query capture
Fast restore	953–1087	Cookie → home redirect
OIDC callback processing	1109–1219	Full exchange pipeline
<code>main()</code> UI	1238–1501	Visible login form
JWT <code>authenticate_user</code>	863–893	Legacy password auth

Related pages

Page	Relationship
<code>main.py</code>	Default route, <code>login_handoff</code> verification, logout routing
<code>logout_bridge.py</code>	Clears session; may redirect back with <code>oidc_post_logout</code>
<code>home.py</code>	Post-login destination (<code>/home</code>)

Generated from source analysis of `app/pages/login.py`. MCP `code-review-graph` was unavailable; documentation based on direct file reads.

— End of Login Page — Technical Documentation —

PART 8 OF 22

Logout Bridge Page — Technical Documentation

RP-initiated OIDC logout and IdP session termination.

Continued from previous section →

Source: `app/pages/logout_bridge.py`

URL: `/logout_bridge`

Purpose: Invisible backend handler that performs **RP-initiated OpenID Connect (OIDC) logout** — recovers `id_token`, clears local auth state and cookies, then redirects the browser to the Coresight Identity Provider (IdP) logout endpoint.

Important: This page intentionally has **minimal visible User Interface (UI)** ("Logging out..."). Do not add interactive Streamlit widgets here.

Overview

When a user clicks **Logout** in the navigation header (`?action=logout`), `main.py` calls `stash_logout_context()` and routes to this page. `logout_bridge.py` then:

1. **Recovers** `id_token_hint` from multiple sources (before any clearing).
2. **Clears** server-side session state and sets `__just_logged_out`.
3. **Injects** parent-frame JavaScript to atomically clear browser cookies and navigate to either:
 - IdP logout URL (with `id_token_hint` + `post_logout_redirect_uri`), or
 - Login page (local-only fallback when no `id_token`).

The page exists because Streamlit 1.54+ strips `<script>` via DOMPurify in `st.html()`, and `components.html()` iframes cannot top-navigate reliably. The solution uses **parent-document script injection** (same pattern as login Single Sign-On (SSO) button and auth cookie handoff).

URL / Route

Route	Access
<code>/logout_bridge</code>	Via <code>st.switch_page</code> from <code>main.py</code> (<code>?action=logout</code>) or <code>auth_manager.logout()</code>

Not linked in navigation — `st.navigation` registers it with `position="hidden"`.

Query parameters: None required; all context comes from session state and cookies.

Dependencies

Python modules

Import	Role
<code>core.auth_environment.is_production_deploy</code>	Chooses <code>OIDC_REDIRECT_URI</code> host
<code>core.auth_manager.build_cookie_clear_js_lines</code>	JS lines to expire cookies (host + <code>.coresight.com</code>)
<code>streamlit</code> / <code>streamlit.components.v1</code>	Minimal UI + iframe script injection
<code>utils.server_logger</code>	<code>log_warning</code> , <code>log_error</code> , <code>log_timing</code>
<code>dotenv.load_dotenv</code>	Environment loading

Constants (must match `login.py`)

Constant	Value
<code>IDP_BASE_URL</code>	<code>https://coresight.com</code>
<code>OIDC_REDIRECT_URI</code>	<code>https://marketdata.coresight.com</code> (prod) or <code>https://marketdata-stg.coresight.com</code> (non-prod)
<code>OIDC_CLIENT_ID</code>	<code>market-data</code>
<code>COOKIE_NAME</code>	<code>auth_session</code>
<code>LOGOUT_BRIDGE_COOKIE_NAME</code>	<code>auth_logout_bridge</code>
<code>OIDC_RETURN_CONTEXT_MAX_AGE_SECONDS</code>	900 (default)
<code>LOGOUT_COOKIE_SYNC_WAIT_MS</code>	250 (defined, legacy)
<code>LOGOUT_COOKIE_SYNC_POLL_MS</code>	50 (defined, legacy)

External integrations

Integration	Usage
Coresight IdP logout	<code>GET https://coresight.com/csr-idp/logout/?id_token_hint=...&post_logout_redirect_uri=...</code>
Browser cookies	Read then clear <code>auth_session</code> , <code>auth_logout_bridge</code>
<code>/tmp</code> <code>id_token</code> files	<code>_oidc_idtok_{hash}.json</code> fallback when slim cookie has no <code>id_token</code>

UI Components

Visible UI (intentionally minimal)

Element	Description
Inline CSS	Hides <code>stHeader</code> , footer, <code>#MainMenu</code> , sidebar
Centered text	"Logging out..." — full viewport flex center
Background	White <code>#fff</code>

No Streamlit buttons, forms, or sidebar. The page should appear as a brief transitional screen before browser navigation.

Invisible machinery

Element	Purpose
<code>streamlit.components.v1.html</code>	Zero-height iframe injecting parent-frame script
Parent script	Cookie clear lines + <code>window.location.href = dest</code>

Session State Keys

Read

Key	Purpose
<code>__logout_bridge_data</code>	Primary source: <code>{id_token, user_email, created_at}</code> from <code>stash_logout_context</code>
<code>auth_data</code>	Fallback <code>id_token</code> / <code>session_id</code>
<code>__logout_oidc_redirect_started</code>	Guard against duplicate renders

Written / cleared

Key	Action
<code>__just_logged_out</code>	Set <code>True</code> — login page sets <code>__logout_guard</code>
<code>__logout_oidc_redirect_started</code>	Set when IdP redirect path taken
Cleared:	<code>auth_data</code> , <code>authenticated</code> , <code>__auth_invalidated</code> , <code>__completed_oidc_code</code> , <code>__completed_oidc_token_data</code> , <code>__oidc_cb_*</code> , <code>__oidc_exchange_result</code> , <code>__logout_bridge_data</code> , handoff keys, <code>__completed_oidc_post_logout_key</code>

Data Layer

No database queries

This page performs **zero** MySQL operations. Session invalidation happens earlier in `stash_logout_context()` via temp-file blacklist (`invalidate_session` in `auth_manager.py`).

Ephemeral files

Operation	File	Purpose
Write	<code>_oidc_logout_return_{hash}.json</code>	Stores origin for cross-host post-logout return
Read	<code>_oidc_idtok_{hash}.json</code>	Recover <code>id_token</code> from <code>session_id</code> (slim cookie deployments)

`_build_post_logout_redirect_uri` logic:

- If current origin is allowed and differs from canonical `OIDC_REDIRECT_URI`, append `oidc_post_logout={return_key}` to redirect URI
- `login.py` `_process_post_logout_return()` consumes this and meta-refreshes browser back to original host

Business Logic

Step 1 — Recover `id_token` (priority order)

Logout bridge walks a fixed priority list of session, cookie, and temp-file sources to recover `id_token_hint`.

Figure 1 Part A — Priority sources 1-3 (session & cookies)

Figure 1 Part A — Priority sources 1–3 (session & cookies)

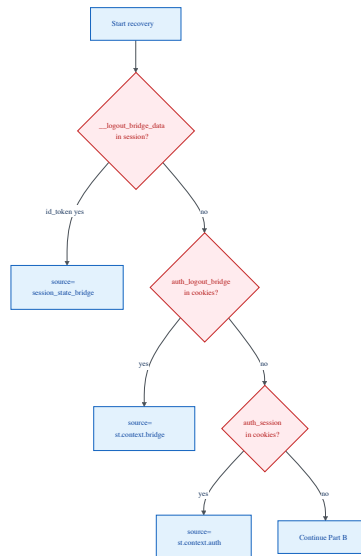
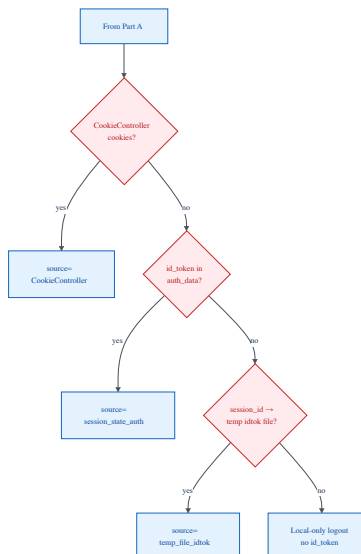


Figure 1 Part B — Priority sources 4–6 (fallbacks)

Figure 1 Part B — Priority sources 4–6 (fallbacks)



Logs `OIDC_LOGOUT_BRIDGE_RECOVER` timing with `source` and `has_id_token`.

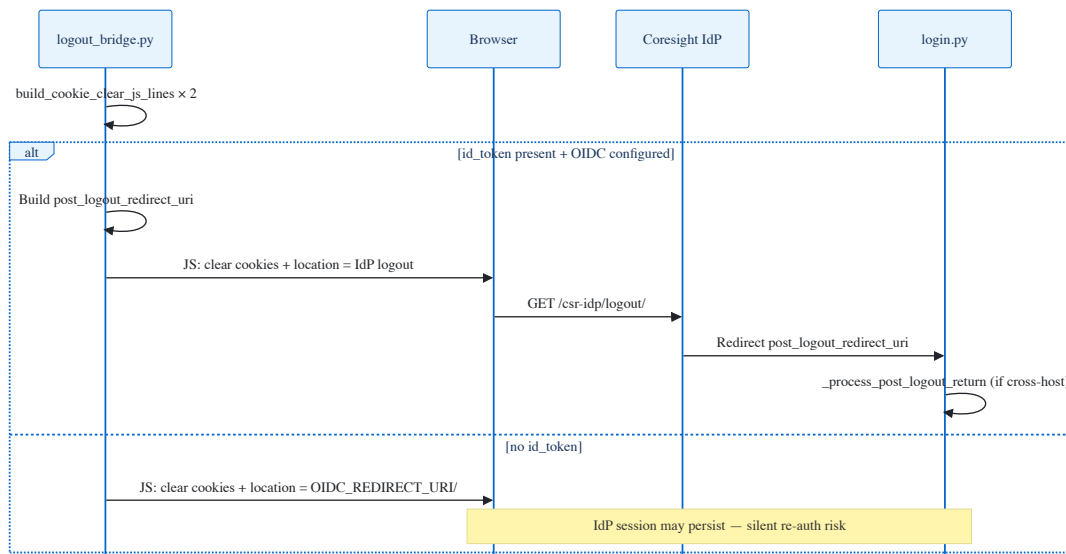
Step 2 — Clear session state

Sets `__just_logged_out` and pops all auth/OIDC keys (see table above).

Step 3 — Atomic JS clear + redirect

Parent-frame JavaScript clears cookies atomically, then navigates to IdP logout or login fallback.

Parent-frame JavaScript clears cookies atomically, then navigates to IdP logout or login fallback.



IdP logout URL shape:

```
https://coresight.com/csr-idp/logout/?id_token_hint={jwt}&post_logout_redirect_uri={encoded}
```

Duplicate render guard: If `__logout_oidc_redirect_started` already set at page entry → log and `st.stop()`.

Fallback: On JS injection exception → `st.switch_page("pages/login.py")`.

Error Handling

Scenario	Behavior
Duplicate page render	<code>st.stop()</code> immediately
<code>id_token</code> not found in any source	Local-only logout to login URL; warning logged
JS injection fails	<code>switch_page(login.py)</code>
Bridge cookie parse error	Log warning; try next source
Temp file read error	Log warning; continue to local-only
Unsafe post-logout origin	Ignored in <code>login.py</code> handler

Local-only logout implication: Without `id_token_hint`, the IdP WordPress session is **not** destroyed. User may be silently re-authenticated on next SSO attempt. This is the documented safe fallback.

Access Control List (ACL) / Permissions

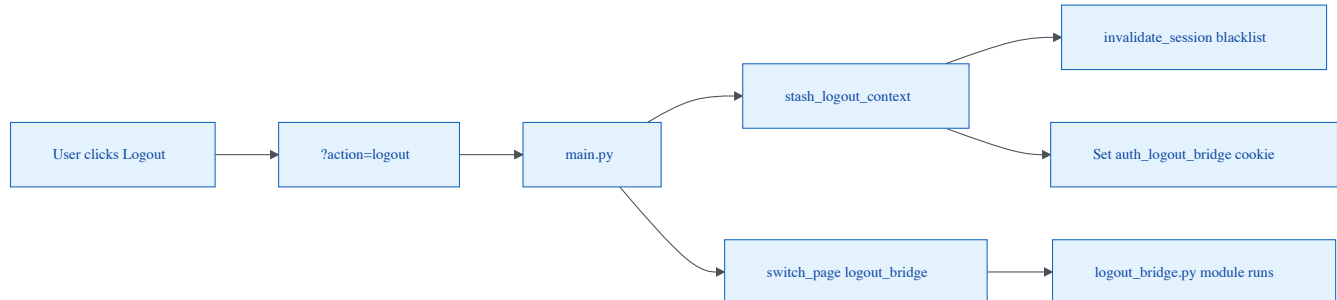
No ACL check — any authenticated user reaching this page via `?action=logout` may log out. The page is not directly bookmarkable for harm: without prior `stash_logout_context`, `id_token` recovery may fail (local-only logout).

Flowcharts

Entry from navigation

User logout starts in navigation, stashes OIDC context in `main.py`, then routes to the invisible logout bridge page.

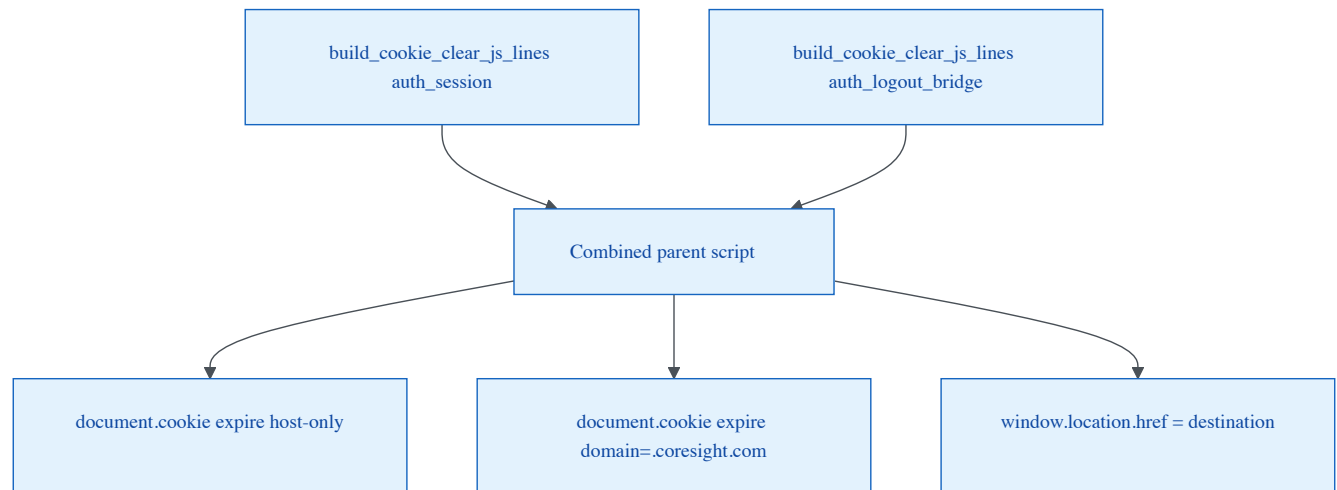
User logout starts in navigation, stashes OIDC context in `main.py`, then routes to the invisible logout bridge page.



Cookie clearing (JS)

Both auth cookies are expired on host-only and `.coresight.com` paths before redirect.

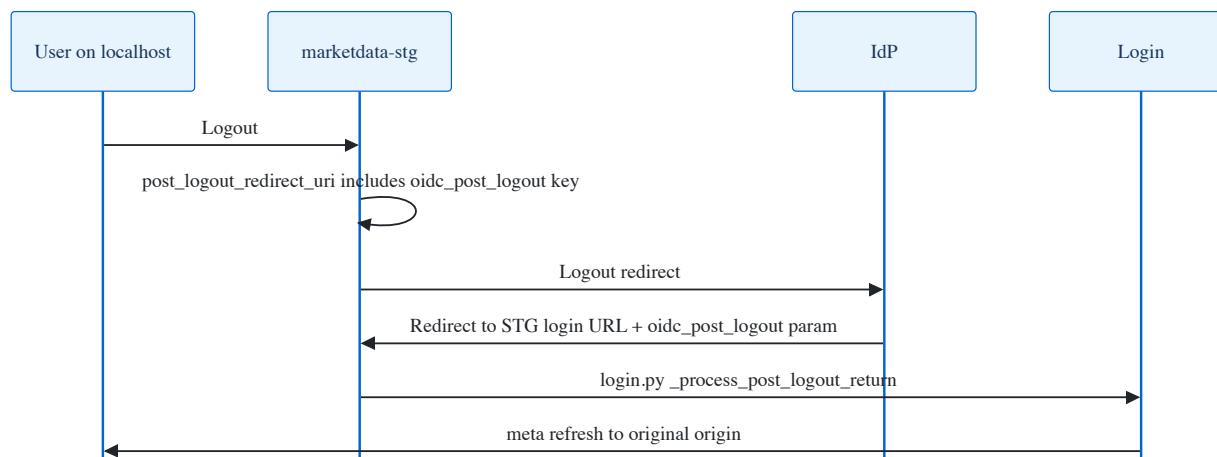
Both auth cookies are expired on host-only and `.coresight.com` paths before redirect.



Cross-host post-logout return

Cross-host logout stores return origin in a temp file; login page meta-refreshes the browser back to the original host.

Cross-host logout stores return origin in a temp file; login page meta-refreshes the browser back to the original host.



Key code references

Section	Lines (approx.)
Module docstring / design rationale	1–20
Ephemeral cache helpers	58–74
Origin helpers	77–149
<code>_build_post_logout_redirect_uri</code>	152–177
Minimal UI CSS	180–191
Duplicate guard	198–200
Step 1 id_token recovery	203–333
Step 2 session clear	347–359
Step 3 JS clear + redirect	361–422

Related documentation

- [App Bootstrap and Auth](#) — `stash_logout_context` , logout routing in `main.py`
- [Login Page](#) — `_process_post_logout_return` , OIDC callback

Generated from source analysis of `app/pages/logout_bridge.py` .

— End of Logout Bridge Page — Technical Documentation —

PART 9 OF 22

Home Page — Technical Documentation

Landing dashboard, company picker, and background filings scan initialization.

Continued from previous section →

Source: `app/pages/home.py`

URL: `/home`

Purpose: Post-login landing dashboard — company/sector entry points to the Market Data portal. Primary navigation hub with "View by Company" (active) and "View by Sector" (coming soon).

Overview

The home page presents the **CORESIGHT MARKET DATA** title and two side-by-side cards:

1. **View by Company** — selectbox of all companies from `coreiq_companies` ; navigates to Market Data → Company Profile tab.
2. **View by Sector** — User Interface (UI) placeholder only; selectbox and button are **disabled** with "Coming Soon" tooltip.

On module load, the page also triggers **intelligent background filings scan** initialization (non-blocking) and loads the company list from the database (cached 5 minutes).

Auth note: Authentication is enforced via `main.py` central bootstrap (`is_authenticated()` hydrates session from cookie) and the user's journey through login. Page-level `require_auth()` may be enabled for direct-route protection.

URL / Route

Route	Parameters	Behavior
<code>/home</code>	—	Render home dashboard
<code>/home</code>	<code>login_handoff=1</code>	Handled by <code>main.py</code> before page runs; params stripped after cookie verify

Registered in `main.py` : `st.Page("pages/home.py", title="Home", url_path="home", default=False)`

Dependencies

Python modules

Import	Role
<code>components.styles.hide_sidebar</code>	Hide Streamlit sidebar
<code>components.styles.render_styles</code>	Inject global design-token CSS
<code>components.navigation.render_header</code>	Coresight nav header (<code>current_page="home"</code>)
<code>components.navigation.render_coresight_footer</code>	Full-width footer with staging/admin links
<code>core.auth_manager.require_auth</code>	Available for page-level enforcement
<code>data.repository.CompanyRepository</code>	<code>get_companies()</code> for selectbox
<code>utils.local_storage.set_marketdata_tab</code>	Persist selected Market Data tab
<code>utils.local_storage_manager.set_persistent_state</code>	Imported (available for cross-session state)
<code>utils.local_storage_manager.save_market_data_state</code>	Sync session → local storage
<code>utils.server_logger</code>	<code>log_structured_error</code> , <code>log_error</code> , <code>error_boundary</code>
<code>utils.background_scanner</code>	<code>init_background_scanner</code> , <code>get_cache_audit</code> , <code>ensure_cache_purge</code>

Component deep-dive: `components/styles.py`

Function	Used on home?	Behavior
<code>hide_sidebar()</code>	Yes (module level)	Cached CSS hides <code>[data-testid="stSidebar"]</code> , removes top padding
<code>render_styles()</code>	Yes (<code>main()</code>)	Injects <code>get_global_css()</code> once per session (<code>_cached_global_css</code>)
<code>COLORS</code> , <code>TYPOGRAPHY</code> , etc.	Indirect	Design tokens in global CSS

Component deep-dive: `components/navigation.py`

Function	Parameters on home	Behavior
<code>render_header</code>	<code>full_width=True</code> , <code>current_page="home"</code>	80px sticky header, logo, nav links (Market Data, Earnings Calls, Earnings Calendar, Screening, News), Logout (<code>?action=logout</code>)
<code>render_coresight_footer</code>	<code>full_width=True</code> , <code>stick_to_bottom=True</code>	4-column footer; staging-only links (Azure File Storage, Add Retailers); Access Management for admins

Header side effects:

- Sets `st.session_state["_active_page"] = "home"`
- Injects `_inject_transition_js()` for client-side nav overlay (parent document)
- Renders hidden `st.page_link` widgets for SPA-style navigation

Footer Access Control List (ACL) (conditional links):

Link	Condition
Azure File Storage	<code>APP_ENV=staging</code>
Add Retailers	Staging + email in retailer allowlist
Access Management	<code>_can_access_mgmt(user_email)</code> — admin/super_user role

UI Components

Page structure

```
hide_sidebar()                [module init]
Background scanner init       [module init, try/except]
_load_companies() → COMPANIES [module init, @st.cache_data 300s]

main():
  |— Inline page-specific CSS (Montserrat/Roboto, cards, selectboxes)
  |— render_styles()
  |— render_header(current_page="home")
  |— H1 "CORESIGHT MARKET DATA"
  |— st.columns [1, 2, 2, 1]
  |   |— col1: View by Company card
  |   |— col2: View by Sector card (disabled)
  |— Spacer 100px
  |— render_coresight_footer(stick_to_bottom=True)
```

Card 1 — View by Company

Element	Widget / HyperText Markup Language (HTML)	Key / ID
Card title	HTML <code><p></code>	"View by Company"
Company select	<code>st.selectbox</code>	<code>company_select</code>
Options	<code>[ticker for (ticker, name) in COMPANIES]</code>	<code>format_func</code> shows <code>name</code>
Placeholder	"Select a Company"	<code>index=None</code>
View button	<code>st.button("View ↗")</code>	<code>view_company_btn</code> — only if company selected
Disabled View	HTML grey button + SVG	Shown when no company selected

Selectbox styling: White background, 40px height, red focus border `#D62E2F`.

Button styling: Red `#D62E2F`, Montserrat 700, 41px height, full width.

Card 2 — View by Sector

Element	State
Sector select	<code>disabled=True</code> , <code>key="sector_select"</code>
Options	<code>SECTORS</code> constant (4 retail sectors)
View button	Static HTML grey + "Coming Soon" hover tooltip

SECTORS constant:

```
[ "Apparel & Footwear", "Department Stores", "Discount Stores", "Luxury Goods" ]
```

Typography / layout CSS (inline in `main()`)

- Title: Montserrat 700, 39px, `#D62E2F`, centered, 96px top margin
- Cards: columns 2 & 3 get `#EBEBEB` background, 357px width, 8px radius
- Fonts loaded: Google Fonts Roboto + Montserrat

Session State Keys

Key	Set by	Purpose
<code>home_company</code>	Selectbox change	Last selected company ticker (or <code>None</code>)
<code>home_sector</code>	Module init in <code>main()</code>	Default <code>SECTORS[0]</code> (sector UI unused)
<code>active_ticker</code>	When company selected	Cross-page ticker sync for header/nav
<code>selected_tab_market_data</code>	View button click	Set to <code>"company_profile"</code> before navigation
<code>company_select</code>	Streamlit widget state	Selectbox value
<code>sector_select</code>	Streamlit widget state	Disabled selectbox
<code>_active_page</code>	<code>render_header</code>	<code>"home"</code>
<code>_cached_global_css</code>	<code>render_styles</code>	Cached global CSS string
<code>_cached_sidebar_css</code>	<code>hide_sidebar</code>	Cached sidebar hide CSS

Local storage (browser)

Function	Key / effect
<code>set_marketdata_tab("company_profile")</code>	<code>StorageKey.MARKETDATA_SELECTED_TAB</code> via <code>utils.local_storage</code>
<code>save_market_data_state()</code>	Persists market data related session fields to local storage manager

Ticker is **not** saved to local storage on home — comment notes it is passed via query params on navigation (`market_data.py` reads session `active_ticker`).

Data Layer

Primary query path

Figure 1 Part A — Repository call chain

Figure 1 Part A — Repository call chain

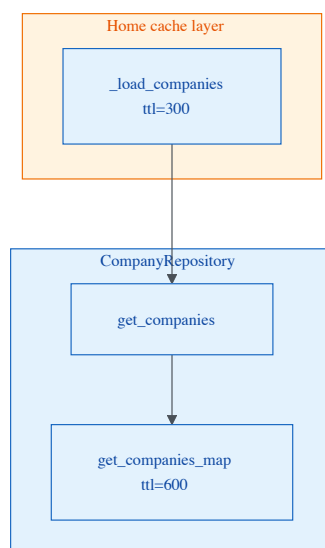
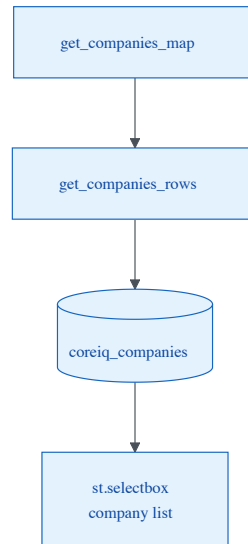


Figure 1 Part B — Database read

Figure 1 Part B — Database read



CompanyRepository.get_companies()

- **Cache:** `@st.cache_data(ttl=600)` ON `get_companies_map` / rows (home wraps with own `@st.cache_data(ttl=300)` ON `_load_companies`)
- **Returns:** `List[Dict[str, str]]` with `ticker` and `name` (display = `name_coresight`)
- **Deduplication:** Skips base-ticker duplicates when composite ticker exists (e.g. prefers `ADS.DE` over `ADS`)
- **Sort:** Alphabetical by display name (case-insensitive)

Underlying Structured Query Language (SQL) (`get_companies_rows`)

```

SELECT ticker,
       name,
       name_coresight,
       exchange,
       source,
       exchange_acronym,
       primary_industry_coresight,
       country_of_incorporation
FROM coreiq_companies;

```

Table: `coreiq_companies`

Access: `db_manager.execute_query_readonly` (read engine, AUTOCOMMIT, 1 RTT)

Home-level cache

```

@st.cache_data(ttl=300)
def _load_companies():
    rows = CompanyRepository.get_companies()
    return [(r['ticker'], r['name']) for r in rows]

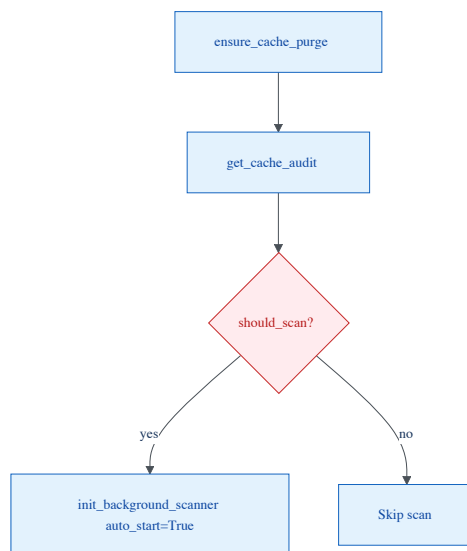
```

On Database (DB) failure: logs error, returns `[]` (empty selectbox).

Business Logic

Module init — background filings scanner

Module init — background filings scanner



- `ensure_cache_purge` : One-time stale cache purge (marker file `.cache_purge_v1_done`)
- `get_cache_audit` : Checks local blob/metadata cache health
- **Non-blocking**: Runs in try/except; failure logged, page still renders

Also started from `main.py` when authenticated (`APP_BG_SCANNER_STARTED` guard) — home init is a **second intelligent entry point** for users landing here first.

View by Company navigation

View by Company navigation



Sector card

No business logic — purely presentational placeholder for future sector-level dashboards.

Error Handling

Location	Failure	User impact
<code>bg_scan_init</code>	Exception	Logged; page continues
<code>_load_companies</code>	DB error	Empty company list
Module <code>COMPANIES</code> load	Exception	<code>COMPANIES = []</code>
<code>main()</code>	Exception	<code>st.error</code> "An unexpected error occurred" + <code>st.stop()</code>
Outer <code>main()</code> call	Exception	<code>st.error</code> "Something went wrong"

All errors use `log_structured_error` with `page="home"` and component/operation tags.

ACL / Permissions

Page-level: Identity via `main.py` cookie hydrate; `require_auth()` optional at page level.

Footer / header: Conditional admin/staging links per `navigation.py` (see above).

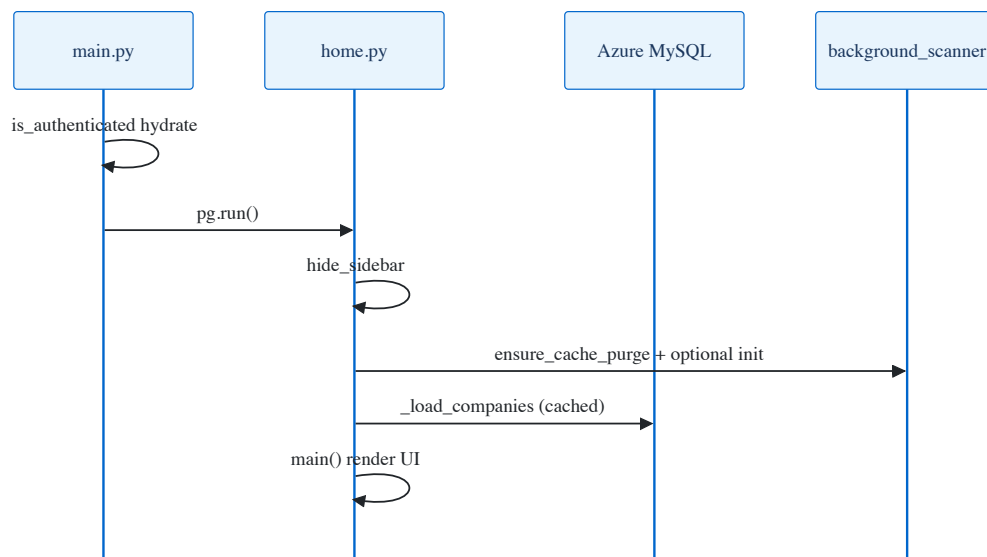
Data: Company list is not filtered per-user on home — full `coreiq_companies` universe. Page-level ACL on other routes may restrict features after navigation.

Flowcharts

Page load sequence

Visual reference for cross-module authentication and request lifecycle flows.

Visual reference for cross-module authentication and request lifecycle flows.



User interaction — company → market data

Figure 2 Part A — Company selection

Figure 2 Part A — Company selection

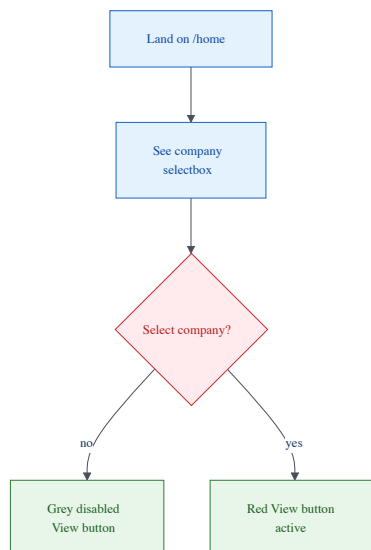
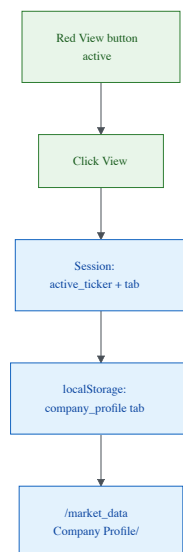


Figure 2 Part B — Navigate to market data

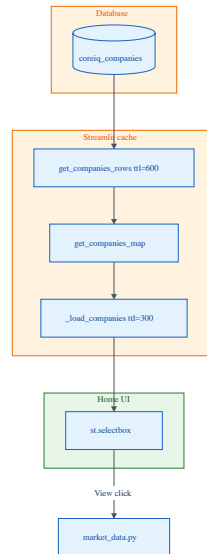
Figure 2 Part B — Navigate to market data



Data flow

Company Profile loads overview data from the repository layer and renders HTML table blocks for the active ticker.

Company Profile loads overview data from the repository layer and renders HTML table blocks for the active ticker.



Key code references

Concern	Lines (approx.)
Module imports and auth setup	1–14
Background scanner init	16–41
<code>_load_companies</code> + COMPANIES	43–58
<code>main()</code> UI + navigation	60–248
Sector placeholder	198–241
Error wrappers	245–254

Environment variables

Variable	Effect on home
<code>APP_ENV=staging</code>	Footer shows staging-only links
<code>ENABLE_BG_SCANNER</code>	Also gates scanner in <code>main.py</code> (redundant with home init)

Related pages

Page	Relationship
<code>login.py</code>	Redirect target after auth (<code>/home</code>)
<code>market_data.py</code>	Destination on "View" click
<code>main.py</code>	Auth bootstrap, scanner warmup when authenticated

Generated from source analysis of `app/pages/home.py` and traced components.

— End of Home Page — Technical Documentation —

PART 10 OF 22

Market Data — Technical Documentation

Financial statements, key stats, estimates, and cross-page ticker
synchronization.

Continued from previous section →

Application: Market Data Portal (MDP)

Page module: `app/pages/market_data.py` (~4,163 lines)

URL route: `/market_data`

Registered in: `app/main.py` as `st.Page("pages/market_data.py", title="Market Data", url_path="market_data")`

1. Overview

The Market Data page is the primary financial dashboard for a single selected company. It presents ten sub-tabs covering company profile, financial statements, derived metrics, analyst estimates, model forecasts, segment breakdowns, and supplemental ratings/store data. The User Interface (UI) is built to match Coresight Figma specifications: custom HyperText Markup Language (HTML) tables (not `st.dataframe`), Roboto typography, Coresight red (`#D62E2F`), sticky first columns, and client-side Excel downloads.

Data sources: Pre-ingested Alpha Vantage (AV) and Yahoo Finance (YF) tables in Azure MySQL. No live vendor Application Programming Interface (API) calls. Routing between U.S. Securities and Exchange Commission (SEC) and YFinance sources is automatic per ticker via `source_router.get_company_source()`.

Key design choices:

- Shared filter bar (period, dates, sort, currency, units) across most financial tabs
- Isolated filter state for **Estimates** and **Forecasting** tabs (do not overwrite shared date/period keys)
- Session-state HTML caching keyed by ticker + filter inputs (avoids re-rendering large tables on reruns)
- Cross-page ticker sync via `active_ticker` and browser `localStorage` (`sip_app_state`)
- Financial values stored in Database (DB) at **millions** scale; User Interface (UI) applies units scaling (mm / bn / k) and FX conversion

URL and deep-linking

Base URL: `/market_data`

Query parameters:

Parameter	Values	Purpose
<code>ticker</code>	Valid symbol (e.g. <code>WMT</code> , <code>ADS.DE</code>)	Selected company; validated against <code>coreiq_companies</code>
<code>tab</code>	See tab keys below	Active sub-tab; default <code>company_profile</code>
<code>period_type</code>	<code>Annual</code> <code>Quarterly</code>	Shared period type (financial tabs)
<code>start_dt</code>	ISO date <code>YYYY-MM-DD</code>	Shared range start
<code>end_dt</code>	ISO date <code>YYYY-MM-DD</code>	Shared range end
<code>sort_ord</code>	<code>Earliest</code> <code>Latest</code>	Column sort direction
<code>to_curr</code>	Currency code	Target display currency
<code>conv_mode</code>	<code>Today's Spot Rate</code> <code>Historical</code>	FX conversion mode
<code>units_sel</code>	<code>Millions (mm)</code> <code>Billions (bn)</code> <code>Thousands (k)</code>	Display units

Example deep link: `/market_data?ticker=WMT&tab=income_statement&period_type=Annual&start_dt=2020-01-01&end_dt=2025-12-31`

Tab keys (`render_tabs` in `app/components/toolbar.py`):

Key	UI label	Dev badge
<code>company_profile</code>	Company Profile	—
<code>key_stats</code>	Key Stats	—
<code>income_statement</code>	Income Statement	—
<code>balance_sheet</code>	Balance Sheet	—
<code>cash_flow</code>	Cash Flow	—
<code>ratios</code>	Ratios	—
<code>estimates</code>	Estimates	—
<code>forecasting</code>	Forecasting	Testing in Progress
<code>segment_data</code>	Segment	Testing in Progress
<code>ratings</code>	Additional Data	Testing in Progress

2. Dependencies

2.1 Page entry and layout

Module	Symbols	Role
<code>components.styles</code>	<code>hide_sidebar</code> , <code>render_styles</code> , <code>set_page_layout</code> , <code>COLORS</code>	Global CSS tokens; sidebar hidden at import
<code>components.navigation</code>	<code>render_header</code> , <code>render_coresight_footer</code> , <code>render_company_header</code>	App header, company selector, footer
<code>components.toolbar</code>	<code>render_tabs</code>	10-tab navigation row
<code>components.companyProfile</code>	<code>get_company_css</code> , <code>get_profile_rows_for_excel</code> , <code>render_info_table</code> , <code>render_business_description</code> , <code>render_reference_table</code>	Company Profile tab HTML + Excel rows

2.2 Core infrastructure

Module	Symbols	Role
<code>core.auth_manager</code>	<code>require_auth</code>	Optional at page level; identity via <code>main.py</code> cookie hydrate
<code>core.database</code>	<code>db_manager</code>	Inline Structured Query Language (SQL) (period-type probe, shares/market cap for Excel)

2.3 Data layer

Module	Symbols	Role
<code>data.repository</code>	<code>CompanyOverviewRepository</code> , <code>IncomeStatementRepository</code> , <code>BalanceSheetRepository</code> , <code>CashFlowRepository</code> , <code>KeyStatsRepository</code> , <code>RatiosRepository</code> , <code>AnalystEstimatesRepository</code> , <code>ModelForecastsRepository</code> , <code>ForexRepository</code> , <code>StockQuoteRepository</code> , <code>SegmentDataRepository</code> , <code>RatingsDataRepository</code> , <code>ExecutiveCompensationRepository</code> , <code>preload_edgartools_ratings</code>	All financial data access
<code>data.source_router</code>	<code>get_company_source</code>	SEC vs YFinance routing (lazy import in page)
<code>data.models</code>	<code>IncomeStatementData</code> , <code>Company</code> , <code>BalanceSheetData</code>	Imported but unused in page logic

2.4 Utilities

Module	Symbols	Role
<code>utils.server_logger</code>	<code>log_structured_error</code> , <code>log_error</code> , <code>new_rerun_id</code> , <code>log_timing</code>	Structured logging + perf timings
<code>utils.local_storage</code>	<code>get/set_marketdata_company</code> , <code>get/set_marketdata_tab</code> , <code>get/set_marketdata_date_range</code>	Python-side persistence
<code>utils.local_storage_manager</code>	<code>sync_market_data_state</code> , <code>save_market_data_state</code> , <code>get/set_persistent_state</code>	Browser <code>localStorage</code> sync
<code>utils.ticker_utils</code>	<code>validate_and_get_ticker</code> , <code>DEFAULT_FALLBACK_TICKER</code>	Ticker validation
<code>utils.constants</code>	<code>get_currency_symbol</code> , <code>format_currency_display</code> , <code>SEGMENT_METRIC_GROUPS</code>	Currency display; segment metric order
<code>utils.excel_export</code>	<code>export_financial_excel</code> , <code>export_company_profile_simple_excel</code>	XLSX generation

2.5 Third-party

- **Streamlit** — UI, `st.cache_data`, query params, widgets
- **Plotly** (`plotly.graph_objects`, `make_subplots`) — stock price chart on Company Profile
- **concurrent.futures.ThreadPoolExecutor** — parallel DB fetches

3. UI Components

3.1 Page chrome

```
main()
├─ new_rerun_id("market_data")
├─ render_styles() + set_page_layout()
├─ render_header(current_page="market_data")
├─ render_page()
└─ render_coresight_footer()
```

`render_company_header()` (inside `render_page`) provides:

- Page title: **CORESIGHT MARKET DATA**

- Company `selectbox` (`key="company_selector_header"`)
- **Company Documents** button → `/company_filings?ticker=...`
- Writes `active_ticker` and `selected_ticker_market_data` on change

3.2 Tab bar (`render_tabs`)

- Two-row layout: dev labels above Segment / Additional Data / Forecasting
- Active tab: red underline + bold label (HTML)
- Inactive tabs: `st.button` with `on_click` → sets `st.query_params["tab"]` + `set_marketdata_tab()`
- Horizontal rule separator below tabs

3.3 Filter row (financial tabs)

Rendered for all tabs **except** Company Profile. Hidden or simplified per tab:

Control	Widget key	Notes
Period Type	<code>period_type_select_shared</code>	Hidden for Ratings, Forecasting; disabled when only one type available
Start Date	<code>start_dt_shared</code>	Month labels from <code>available_dates</code>
End Date	<code>end_dt_shared</code>	
Sort	<code>sort_order_select_shared</code>	Earliest / Latest
Conversion	<code>conversion_mode_select</code>	Hidden for Ratios, Ratings
From Currency	<code>currency_from_unified</code>	Locked to reported currency
To Currency	<code>currency_to_unified</code>	Reported + <code>ForexRepository.get_to_currencies()</code>
Units	<code>units_select</code>	Millions / Billions / Thousands

Estimates / Forecasting: use isolated keys `_period_type_isolated_{tab}` and `_dr_isolated_{tab}_{annual|quarterly}` so filter changes do not mutate shared `date_range_market_data_shared` or URL date params.

3.4 Tables

Custom HTML `<table class="data-table">` via `st.html()` :

- Sticky first column (line item labels)
- Indent levels for subtotals (Income Statement, Balance Sheet, Cash Flow)
- Bold subtotals, grey separators, underlines per Figma (`is_bold_row` , `has_grey_separator` , `has_underline` , `get_indent_level`)
- `format_value()` applies conversion rate, units scale, comma formatting

3.5 Charts

Company Profile only: `render_stock_quote()` → Plotly line chart, ~60 months (`days=1825`) of daily close prices from `StockQuoteRepository.get_price_history()` . Left panel: quote metrics (close, open, change, 52-week H/L, market cap, shares, dividend yield).

3.6 Key Stats badges

Columns may be tagged **A** (Actual), **E** (Estimate), **F** (Forecast) with legend. Forecast columns link to `/forecasting?ticker=...` .

3.7 Excel export

`_render_excel_js_download(excel_bytes, filename, label)` — client-side Blob download via `streamlit.components.v1.html` . Bypasses Streamlit `/media/` (fails on some proxied deployments). Uses Material Symbols "table" icon and Coresight red border styling.

Per-tab builders:

- Financial tabs → `export_financial_excel()`
- Company Profile → `export_company_profile_simple_excel()` (profile + price history + shares/market cap + compensation)

3.8 Loading indicator

`_tab_loading_hint` spinner shown at start of `render_page()` ; cleared after tab content renders.

4. Session State

4.1 Ticker and navigation

Key	Purpose
<code>active_ticker</code>	Cross-page company sync
<code>selected_ticker_market_data</code>	Page-local ticker
<code>selected_tab_market_data</code>	Active tab
<code>_prev_ticker_market_data</code>	Detect company change → reset filters
<code>_prev_rendered_tab</code>	Tab change logging
<code>_md_run_counter</code>	Rerun storm guard (>80 logs warning)

4.2 Shared filters

Key	Purpose
<code>date_range_market_data_shared</code>	<code>(start_iso, end_iso)</code> tuple
<code>sort_order_market_data_shared</code>	<code>Earliest</code> / <code>Latest</code>
<code>period_type_market_data_shared</code>	<code>Annual</code> / <code>Quarterly</code>
<code>target_currency</code> , <code>from_currency</code>	FX display
<code>conversion_mode</code>	Spot vs Historical
<code>units</code>	Millions / Billions / Thousands
<code>start_dt_shared</code> , <code>end_dt_shared</code>	Date widget keys
<code>sort_order_select_shared</code> , <code>period_type_select_shared</code>	Widget keys
<code>conversion_mode_select</code> , <code>currency_from_unified</code> , <code>currency_to_unified</code> , <code>units_select</code>	Widget keys

4.3 localStorage race-fix keys (`_mdlc_*`)

`_mdlc_date_range` , `_mdlc_sort` , `_mdlc_period` , `_mdlc_conv` , `_mdlc_from_curr` , `_mdlc_to_curr` , `_mdlc_units` — prevent browser hydration from overwriting in-flight widget state.

4.4 Isolated state (Estimates / Forecasting)

Pattern	Purpose
<code>_period_type_isolated_{tab}</code>	Period without affecting shared state
<code>_dr_isolated_{tab}_{annual\ quarterly}</code>	Date range per tab + period

4.5 HTML cache keys (input-hash suffixes)

Pattern	Tab
<code>_bs_html_{...}</code>	Balance Sheet
<code>_cf_html_{...}</code>	Cash Flow
<code>_seg_html_{...}</code>	Segment
<code>_ratings_html_{...}</code>	Ratings
<code>_is_html_{...}</code> (+ <code>_x1</code>)	Income Statement
<code>_ks_html_v2_{...}</code> (+ <code>_x1</code>)	Key Stats
<code>_ratios_html_{...}</code> (+ <code>_notes</code> , <code>_x1</code>)	Ratios
<code>_est_html_{...}</code>	Estimates
<code>_fcst_html_v3_{...}</code>	Forecasting

5. Data Layer

5.1 Source routing (`app/data/source_router.py`)

```
@st.cache_data(ttl=600)
def get_company_source(ticker: str) -> Optional[Literal['SEC', 'YFinance']]
```

- Reads `coreiq_companies.source`
- Composite tickers (`TSCO.L`) resolved via `exchange_acronym`
- Returns `None` if company not found

5.2 Repository classes used

Repository	Primary methods
<code>CompanyOverviewRepository</code>	<code>get_company_overview(ticker)</code>
<code>IncomeStatementRepository</code>	<code>get_date_range</code> , <code>get_available_dates</code> , <code>get_income_statement_data</code> , <code>get_reported_currency</code>
<code>BalanceSheetRepository</code>	<code>get_date_range</code> , <code>get_available_dates</code> , <code>get_balance_sheet_data</code>
<code>CashFlowRepository</code>	<code>get_date_range</code> , <code>get_available_dates</code> , <code>get_cash_flow_data</code>
<code>KeyStatsRepository</code>	<code>get_date_range</code> , <code>get_available_dates</code> , <code>get_key_stats_data</code>
<code>RatiosRepository</code>	<code>get_date_range</code> , <code>get_available_dates</code> , <code>get_ratios_data</code>
<code>AnalystEstimatesRepository</code>	<code>get_date_range</code> , <code>get_available_dates</code> , <code>get_estimates_data</code>
<code>ModelForecastsRepository</code>	<code>get_date_range</code> , <code>get_available_dates</code> , <code>get_forecasts_data</code>
<code>ForexRepository</code>	<code>get_conversion_rate</code> , <code>get_to_currencies</code> , <code>get_conversion_rates_bulk</code>
<code>SegmentDataRepository</code>	<code>has_quarterly_segment_data</code> , <code>get_date_range</code> , <code>get_available_dates</code> , <code>get_segment_data</code>
<code>RatingsDataRepository</code>	<code>get_date_range</code> , <code>get_available_dates</code> , <code>get_ratings_data</code>
<code>StockQuoteRepository</code>	<code>get_latest_quote</code> , <code>get_price_history</code> , <code>get_overview_data</code>
<code>ExecutiveCompensationRepository</code>	<code>get_sec_latest_compensation</code> , <code>get_yf_latest_compensation</code>

Read path: `db_manager.execute_query_readonly` (AUTOCOMMIT read engine, ~1 RTT on Azure).

SEC pattern: Wide tables with `report_type`, `fiscal_date_ending`, column-per-metric.

YFinance pattern: Long-format pivoted with `MAX(CASE WHEN line_item = ...)` grouped by `period_end`, `frequency`. Composite tickers use `yf_symbol` column.

5.3 Database tables

Table	Usage
<code>coreiq_companies</code>	Source routing; <code>name_coresight</code> for display
<code>coreiq_av_company_overview</code>	SEC company profile
<code>coreiq_yf_company_overview</code>	Yahoo Finance (YF) company profile + officers JSON
<code>coreiq_av_financials_income_statement</code>	SEC income + key stats base
<code>coreiq_yf_financials_income_statement</code>	YF income
<code>coreiq_av_financials_balance_sheet</code>	SEC balance sheet
<code>coreiq_yf_financials_balance_sheet</code>	YF balance sheet
<code>coreiq_av_financials_cash_flow</code>	SEC cash flow
<code>coreiq_yf_financials_cash_flow</code>	YF cash flow
<code>coreiq_av_financials_earnings_estimates</code>	SEC analyst estimates
<code>coreiq_yf_financials_earnings_estimates</code>	YF analyst estimates
<code>coreiq_model_forecasts</code>	Revenue model forecasts
<code>coreiq_av_forex_daily</code>	FX conversion rates
<code>coreiq_av_time_series_daily</code> / <code>coreiq_yf_time_series_daily</code>	Stock prices
<code>coreiq_av_shares_outstanding</code> / <code>coreiq_yf_shares_outstanding</code>	Shares for market cap Excel
<code>coreiq_filing_metrics_v5</code>	Segment data + ratings/store counts
<code>coreiq_executives_compensation</code>	SEC executive compensation

6. Business Logic

6.1 Ticker resolution priority

1. `active_ticker` if newer than URL (cross-page change)
2. URL `ticker` query param
3. `active_ticker` alone
4. `localStorage` stored ticker
5. `DEFAULT_FALLBACK_TICKER`

On change: reset to **5-year default** date window, Annual period, reported currency 1:1; clear legacy per-tab keys and HTML caches (new hash inputs).

6.2 Period type fallback

`get_available_period_types(ticker, tab)` probes DB for annual/quarterly availability. If user selects Quarterly but tab has no quarterly data → `st.info()` + snap to Annual.

6.3 Currency conversion

- **From:** locked to `reported_currency` (`_get_company_reported_currency` , cached 300s)
- **To:** reported + forex list
- **Spot:** single `ForexRepository.get_conversion_rate(from, to, today)`
- **Historical:** `get_conversion_rates_bulk(from, to, fiscal_dates_in_range)` applied per column

6.4 Units scaling

UI label	units_scale
Millions (mm)	1.0
Billions (bn)	0.001
Thousands (k)	1000.0

DB values are in millions; `format_value(value, conversion_rate, units_scale)` multiplies before display.

6.5 Tab-specific behavior

Tab	Filters	Data source	Export
Company Profile	None	Overview + quote + compensation	Profile Excel
Income Statement	Full shared	<code>IncomeStatementRepository</code>	Yes
Balance Sheet	Full shared	<code>BalanceSheetRepository</code>	Yes
Cash Flow	Full shared	<code>CashFlowRepository</code>	Yes
Key Stats	Full shared	<code>KeyStatsRepository</code> (A/E/F columns)	Yes
Ratios	Dates + sort only	<code>RatiosRepository</code> (derived)	Yes
Estimates	Isolated dates/period	<code>AnalystEstimatesRepository</code>	Yes
Forecasting	Isolated; annual only	<code>ModelForecastsRepository</code>	Yes
Segment	Full shared	<code>SegmentDataRepository</code> + <code>SEGMENT_METRIC_GROUPS</code>	Yes
Additional Data	Annual only; no FX	<code>RatingsDataRepository</code> + edgartools preload	Yes

6.6 Background preload

`preload_edgartools_ratings(ticker)` runs on every tab load (daemon thread, in-process cache). Powers ratings/store/square-footage sections without blocking UI.

7. Structured Query Language (SQL) and Query Patterns

7.1 Period type probe (in-page)

```
SELECT DISTINCT frequency -- or report_type for SEC
FROM coreiq_{yf|av}_financials_{table}
WHERE ticker = :ticker -- or yf_symbol for composite
AND frequency IN ('annual', 'quarterly')
```

7.2 Shares + market cap (Company Profile Excel)

```
SELECT s.report_date, s.shares_outstanding_basic, s.shares_outstanding_diluted, ts.close
FROM coreiq_av_shares_outstanding s
INNER JOIN coreiq_av_time_series_daily ts
    ON ts.ticker = s.ticker AND ts.day_date = s.report_date
WHERE s.ticker = :ticker
ORDER BY s.report_date DESC
```

7.3 Repository patterns

- **Latest row per period:** `ROW_NUMBER() OVER (PARTITION BY ticker ORDER BY date DESC)`
- **Income statement optimization:** `_fetch_all_annual_rows` cached once, reused for date range + table body
- **Segment metrics:** `coreiq_filing_metrics_v5` with edgartools fallback in `SegmentDataRepository`
- **Access Control List (ACL):** Not applied in SQL on this page (no row-level user filter)

8. Caching

Layer	Mechanism	TTL
Page	<code>@st.cache_data</code> on <code>_get_company_reported_currency</code> , <code>get_available_period_types</code>	300s
Repositories	<code>@st.cache_data</code> on overview, statement fetches, key stats, ratios, estimates, ratings	300–600s
<code>source_router</code>	<code>get_company_source</code>	600s
Session state	HTML + Excel bytes keyed by inputs	Until inputs change
Module	<code>_edgar_ratings_cache</code> + preload daemon	In-process
Company map	<code>CompanyRepository.get_companies_map()</code>	Repository cache

Performance: `_timings` dict + `log_timing("PAGE_*")` summary every `render_page()` run.

9. Error Handling

Pattern	Behavior
Per-section <code>try/except</code>	<code>log_structured_error(e, page="market_data", component=..., operation=...)</code>
User messages	<code>st.error()</code> , <code>st.info("No extracted data available...")</code>
<code>main()</code> top-level	<code>log_exception</code> + re-raise
Defensive defaults	Currency → "USD", conversion → 1.0, format → "-"
Rerun guard	<code>_md_run_counter > 80</code> → log possible rerun storm
Best-effort	<code>preload_edgartools_ratings</code> swallows errors; exec comp section fails silently

Invalid ticker → `validate_and_get_ticker` falls back to default; `st.stop()` only if default also fails.

10. Access Control List (ACL)

Mechanism	Status
<code>require_auth(page="market_data")</code>	Optional at page level
<code>AccessControlManager.check_access("market_data", ...)</code>	Not called
<code>main.py</code> OpenID Connect (OIDC) bootstrap	Session required for background warmups; <code>pg.run()</code> unconditional

Effective access: Portal-wide OIDC session (cookie + `auth_manager.is_authenticated()`). No per-page permission levels on Market Data.

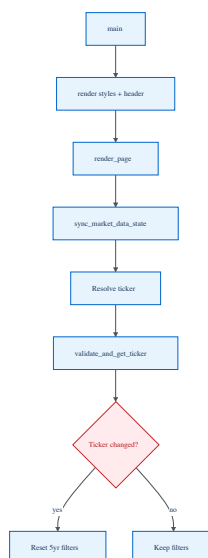
11. Flowcharts

11.1 Page load

End-to-end user journeys on this page, from load through interaction to data fetch.

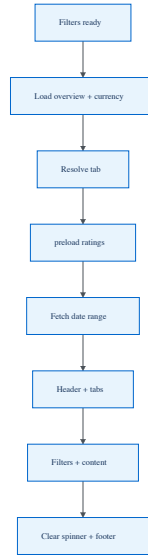
Page load — Part A: Bootstrap & ticker resolution

Page load — Part A: Bootstrap & ticker resolution



Page load — Part B: Data fetch & tab render

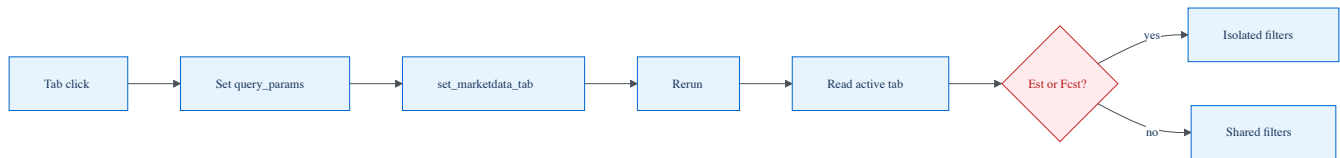
Page load — Part B: Data fetch & tab render



11.2 Tab switching

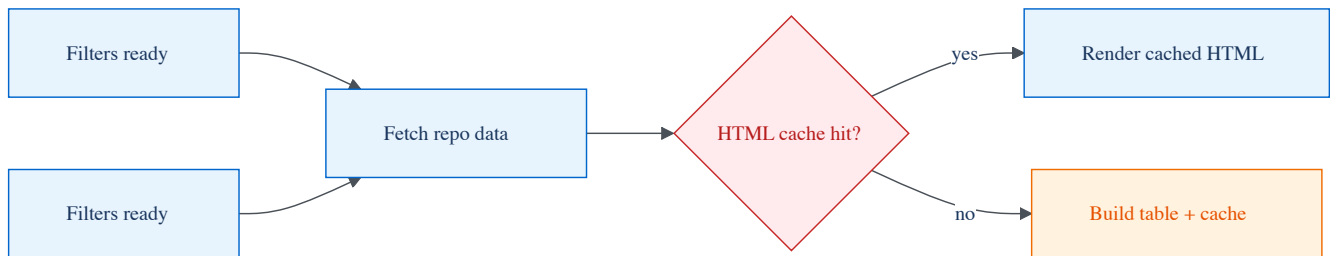
Tab switch — Part A: Navigation & filter routing

Tab switch — Part A: Navigation & filter routing



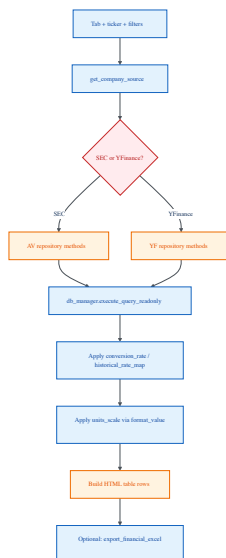
Tab switch — Part B: Data fetch & HTML cache

Tab switch — Part B: Data fetch & HTML cache



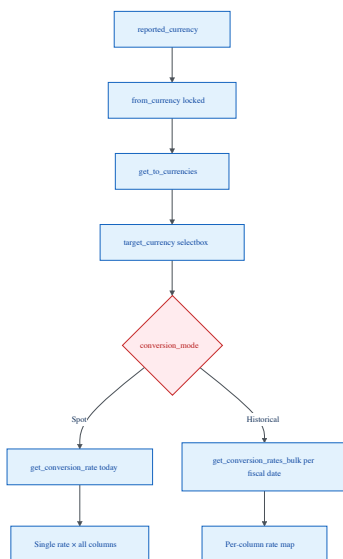
11.3 Data fetch (financial tab)

11.3 Data fetch (financial tab)



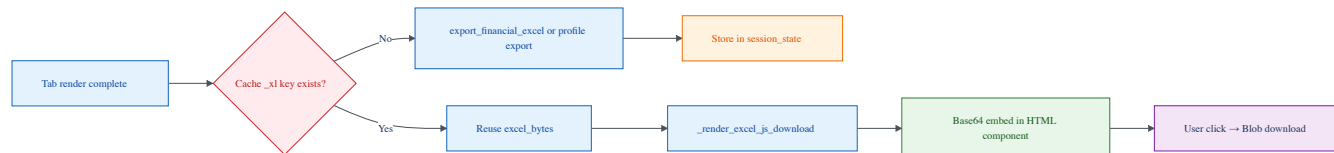
11.4 Currency conversion

11.4 Currency conversion



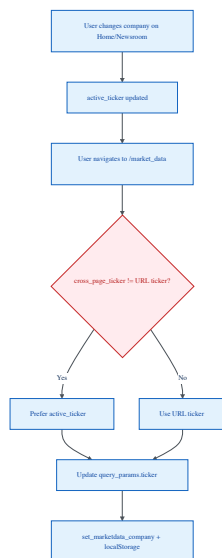
11.5 Excel export

11.5 Excel export



11.6 Cross-page ticker sync

11.6 Cross-page ticker sync



12. Feature reference by tab

12.1 Company Profile

- Info table, business description, executive compensation (SEC multi-column or YF total pay)
- Stock quote metrics + Plotly 60-month chart
- "For Reference" expander (`render_reference_table`)
- No date/currency filters

12.2 Income Statement

- Figma row styling: bold subtotals, grey separators, underlines, indent levels
- Full filter bar + Excel export

12.3 Balance Sheet / Cash Flow

- `render_balance_sheet()` / `render_cash_flow()` with indent levels 0/1/2
- Full filter bar + Excel export

12.4 Key Stats

- Combined actuals, analyst estimates, model forecast columns
- A/E/F legend; links to `/forecasting`

12.5 Ratios

- Derived from cached statements; assumption notes in `st.caption`
- No currency/units filters

12.6 Estimates

- Analyst consensus sections; EPS in reported currency; revenue converted
- Isolated filter state

12.7 Forecasting

- Ensemble + scenario bands from `coreiq_model_forecasts`
- Annual only; link to full `/forecasting` page

12.8 Segment

- Business + Geographic tables from filing metrics
- Metrics per `SEGMENT_METRIC_GROUPS` : Revenues, Operating Profit, Assets, D&A, CapEx
- Quarterly only if `has_quarterly_segment_data`

12.9 Additional Data (Ratings)

- Credit ratings (S&P, Moody's, Fitch), store counts, stores by country, square footage
- Annual only; background edgartools preload

Related pages

Page	Link
Company Filings	<code>/company_filings?ticker=...</code> (Company Documents button)
Forecasting (full)	<code>/forecasting?ticker=...</code> (from Key Stats / Forecasting tab)
Home / Newsroom	Share <code>active_ticker</code> for company sync

Generated from source analysis of `app/pages/market_data.py` and traced dependencies. Last reviewed against codebase structure as of project documentation pass.

— End of Market Data — Technical Documentation —

PART 11 OF 22

Newsroom — Technical Documentation

News feed with sentiment scoring, ticker filters, and sector browsing.

Continued from previous section →

Application: Market Data Portal (MDP)

Page module: `app/pages/newsroom.py`

URL route: `/newsroom`

Registered in: `app/main.py` as `( "pages/newsroom.py", "Newsroom", "newsroom", False )`

1. Purpose and scope

The Newsroom page is a financial news feed that merges **Alpha Vantage (AV)** and **Yahoo Finance (YF)** pre-ingested news from MySQL. It provides:

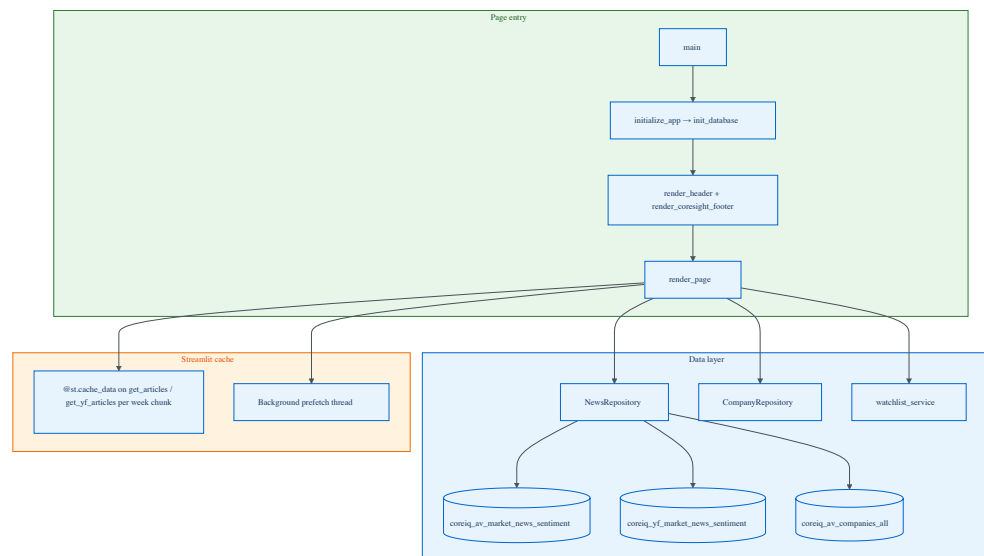
- Date-bounded article loading with **progressive week-chunked fetching** and background cache warming
- **Sentiment** display per tagged company (relevance, label, score)
- **Sector, category (topic), watchlist**, and **keyword** filters
- A two-column UI: left **search results panel**, right **news cards**
- Cross-links from tagged companies to `/market_data?ticker=...`

The page does **not** call AV/YF APIs live; all data is read from `coreiq_*` tables.

2. Architecture overview

Page architecture: UI orchestration, data repositories, and external services for this feature.

Page architecture: UI orchestration, data repositories, and external services for this feature.



3. Dependencies

Import	Role
<code>components.styles</code>	<code>hide_sidebar</code> , <code>set_page_layout</code> , <code>render_styles</code> , design tokens
<code>components.navigation</code>	<code>render_header</code> , <code>render_coresight_footer</code>
<code>core.auth_manager</code>	<code>require_auth</code> , <code>get_current_user</code>
<code>core.database</code>	<code>init_database</code>
<code>data.models</code>	<code>NewsArticle</code> , <code>TickerSentiment</code>
<code>data.repository</code>	<code>NewsRepository</code> , <code>CompanyRepository</code> , <code>_format_company_name</code>
<code>data.watchlist_service</code>	<code>get_user_watchlists</code> , <code>get_watchlist_companies</code>
<code>utils.constants</code>	<code>NEWS_TOPIC_LABELS</code> , <code>get_topic_display_label</code> , <code>merge_yf_topics</code>
<code>utils.server_logger</code>	<code>PageLoadTracker</code> , <code>new_rerun_id</code> , <code>log_structured_error</code> , <code>log_warning</code>

4. Database tables

4.1 `coreiq_av_market_news_sentiment`

Primary AV news store. Key columns used by `NewsRepository.get_articles()` :

Column	Usage
<code>id</code>	Article identity
<code>title</code> , <code>summary</code> , <code>url</code>	Card rendering
<code>source</code> , <code>source_domain</code>	Header metadata
<code>time_published_utc</code>	Sorting, date filters, relative time
<code>overall_sentiment_score</code> , <code>overall_sentiment_label</code>	Article-level sentiment (model field; cards emphasize per-ticker sentiment)
<code>ticker_sentiment_json</code>	Parsed into <code>List[TickerSentiment]</code>
<code>topics_json</code>	Category filter (<code>topics[].topic</code>)
<code>ticker</code>	Row-level ticker (deduped in repository)
<code>banner_image</code> , <code>category_within_source</code>	Model fields

Indexes (documented in repository): `ft_av_title` (FULLTEXT on title), `idx_av_time_id`, `idx_ticker_time`, etc.

Dedup rule: Same title → one card; `ticker_sentiment` merged across duplicate rows (highest relevance per ticker wins).

4.2 `coreiq_yf_market_news_sentiment`

YF news store. `NewsRepository.get_yf_articles()` :

- Groups by `news_id` server-side (`GROUP BY` + `GROUP_CONCAT` tickers)
- No sector on rows — sector filter treats YF as **"Unknown Sector"** only
- Topics may include YF-only keys merged into `NEWS_TOPIC_LABELS` via `get_yf_topic_keys()`

4.3 Supporting tables

Table	Usage
<code>coreiq_companies</code>	<code>CompanyRepository.get_companies_map()</code> → display names on cards
<code>coreiq_av_companies_all</code>	Sector dropdown + <code>_filter_av_by_sector()</code> via <code>get_ticker_sector_map()</code>
<code>coreiq_watchlists</code> , <code>coreiq_watchlist_companies</code>	Watchlist filter

5. Data models

5.1 `NewsArticle` (`app/data/models.py`)

```
@dataclass
class NewsArticle:
    id: int
    title: str
    summary: str
    url: str
    source: str
    source_domain: str
    time_published: datetime
    time_published_raw: str
    overall_sentiment_score: float
    overall_sentiment_label: str
    banner_image: Optional[str]
    ticker_sentiment: List[TickerSentiment]
    topics: List[Dict[str, str]]
    category_within_source: str
    source_tag: str = 'av' # 'av' | 'yf'
```

Properties: `formatted_date`, `company_tickers`.

5.2 `TickerSentiment`

Field	UI usage
<code>ticker</code>	Link to market data (if in company map)
<code>relevance_score</code>	Tooltip: "Relevance: X%"
<code>ticker_sentiment_label</code>	Tooltip: Bullish/Bearish/Neutral
<code>ticker_sentiment_score</code>	Tooltip numeric score
<code>display_name</code>	Optional YF override

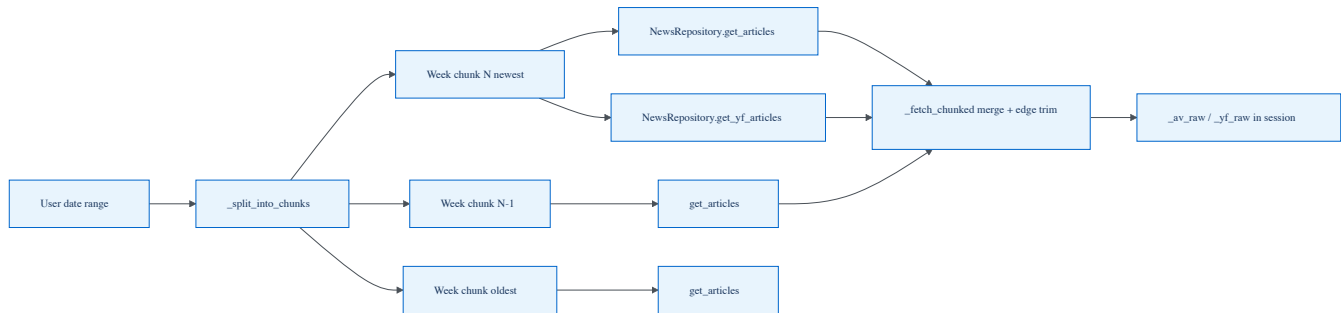
6. Progressive loading engine

6.1 Week-chunk strategy

`_CHUNK_DAYS = 7`, `_MAX_WORKERS = 6`, `_BG_PREFETCH_WEEKS = 4`.

`_split_into_chunks(d_from, d_to)` aligns chunks to **ISO weeks (Mon-Sun)** so `@st.cache_data` keys are stable across overlapping date ranges.

splitintochunks(dfrom, dto) aligns chunks to ISO weeks (Mon–Sun) so @st.cachedata keys are stable across overlapping date ranges.



6.2 `_fetch_chunked`

- Single chunk: direct repository calls (no thread pool)
- Multiple chunks: `ThreadPoolExecutor` with up to `_MAX_WORKERS` workers; each chunk fires AV + YF in parallel
- **Edge trim**: Full-week chunks may extend past `date_from / date_to`; articles outside range filtered after merge
- DB fetch is always **unfiltered** (`sector=None` , `company_ticker=None`) for cache stability; sector/sort applied client-side

6.3 Background prefetch

`_spawn_background_prefetch()` starts a **daemon thread** after initial load:

- Prefetches `_BG_PREFETCH_WEEKS` (4) of weeks **before** `date_from`
- Only calls `@st.cache_data` repository methods (no `st.session_state`)
- Failures are logged; user gets cold query on expand

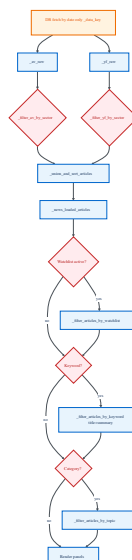
6.4 First-load behavior

1. Default session: last 7 days until `get_news_date_range()` returns
2. `ThreadPoolExecutor` prefetches date bounds + chunked fetch in parallel
3. Spinner shown while `_chunked_future` resolves

7. Filter pipeline

News articles pass through date chunking, ticker/sector filters, and deduplication before render.

News articles pass through date chunking, ticker/sector filters, and deduplication before render.



7.1 Two-level cache keys

Key	Triggers
<code>_news_data_key</code>	<code>date_from\ date_to</code> — DB refetch
<code>_news_view_key</code>	<code>adds query_sector\ sort_ascending</code> — client re-filter only

7.2 Filter widgets (filter row)

Widget	Session / state	DB vs client
Search (form)	<code>news_search</code> , <code>news_search_input</code>	Client: title + summary, min 2 chars
From / To	<code>date_from</code> , <code>date_to</code>	DB via chunk fetch
Sort	<code>news_sort_order</code> → <code>sort_ascending</code>	Client re-sort
Sector	<code>news_selected_sector</code>	Client on <code>_av_raw</code> / <code>_yf_raw</code>
Category	<code>news_selected_category</code> , URL <code>category</code> , <code>localStorage</code>	Client on <code>topics</code>
Watchlist	<code>news_active_watchlist_id</code> , <code>news_watchlist_filter</code>	Client ticker match

Company filter: Commented out in UI; `query_company` always `None` .

7.3 Sector semantics

Sector value	AV behavior	YF behavior
<code>All</code>	All articles	All articles
<code>"Unknown Sector"</code>	Empty (AV always has sectors)	All YF articles
<code>"None"</code>	Tickers with no/null sector	Empty
Named sector	Match via <code>get_ticker_sector_map()</code>	Empty

7.4 Category (topic)

- `NEWS_TOPIC_LABELS` from `utils.constants` + dynamic YF keys
- `uncategorized` → articles with empty `topics` (typical YF)
- Persisted: `localStorage.newsroom_category` + optional URL `?category=`

7.5 Watchlist filtering

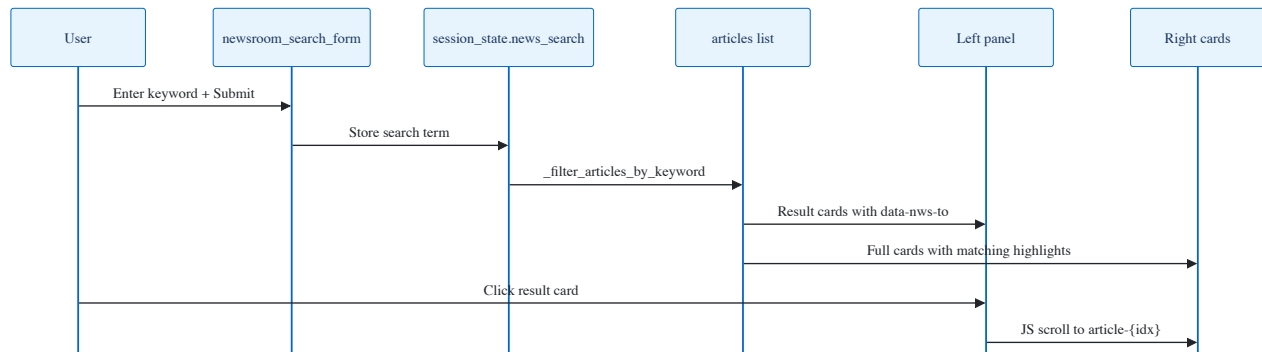
- `_news_ticker_variants` : exact ticker + base before `.` (e.g. `ADS.DE` → `ADS`)
- Match if any `article.ticker_sentiment` ticker variant intersects watchlist
- Limitation:** Cannot disambiguate same symbol across companies (documented in code)

8. Search

8.1 Keyword search flow

Keyword search queries MySQL FULLTEXT indexes and merges AV + YFinance article sources.

Keyword search queries MySQL FULLTEXT indexes and merges AV + YFinance article sources.



- Search is **in-memory** over loaded date range (not DB FULLTEXT on newsroom page)
- `_highlight_keyword` : wraps each word in `<mark>` (brand red theme)
- Left panel shows count: `N matching (in M most recent)`
- Scroll handler: `streamlit.components.v1.html` installs click listener on `[data-nws-to]`

8.2 AV repository keyword path (when used)

`NewsRepository.get_articles(keyword=...)` supports FULLTEXT (`USE_AV_FULLTEXT=1`) or LIKE; newsroom passes `keyword=None` to DB and filters client-side.

9. News cards UI

9.1 `render_news_card`

HTML structure per article:

1. **Header**: source (left), formatted date + relative time (right)
2. **Title**: external link (`article.url`), stretched-link pattern for card hover
3. **Summary**: indented body text
4. **Tagged companies**: links to `/market_data?ticker=` with sentiment tooltips

9.2 Chunked rendering

`_render_cards_chunked(articles, company_map, keyword, chunk_size=500)` — multiple `st.markdown` calls to avoid single huge HTML blob.

9.3 CSS

`get_news_css()` — Figma-aligned Roboto/Montserrat, `content-visibility: auto` on cards, left sidebar scroll styling, keyword `<mark>` styling.

9.4 Layout

- Columns: `[0.25, 0.75]`, height `900px` bordered containers
- Footer caption: loaded count and day span; shows `matching / loaded` when keyword active

10. Sentiment display

Sentiment is **per tagged company**, not a single badge on the card:

Tooltip: Relevance: 85.0% | Sentiment: Bullish (0.42)

- Active companies (in `company_name_map`): red link to Market Data
- Unknown tickers: inactive gray tag

Article-level `overall_sentiment_score` / `overall_sentiment_label` exist on the model but are not prominently rendered on cards.

11. Pagination

Historical offset pagination (`_news_offset` , `_news_has_more` , `_PAGE_SIZE = 1000`) is **disabled** for the current architecture:

- Chunked fetch loads **all articles** in the selected date range
- `_news_has_more` set to `False` after load
- `_PAGE_SIZE` remains in code but is unused for "load more"

12. Session state reference

Key	Purpose
<code>date_from</code> , <code>date_to</code>	Active date range
<code>news_sort_order</code>	"Latest" "Earliest"
<code>news_search</code>	Active keyword
<code>news_selected_sector</code>	Sector filter
<code>news_selected_category</code>	Topic filter
<code>news_active_watchlist_id</code> , <code>news_active_watchlist_name</code>	Watchlist
<code>company_name_map</code>	Ticker → display name cache
<code>_news_data_key</code> , <code>_news_view_key</code>	Filter change detection
<code>_news_loaded_articles</code>	Merged filtered list for display pipeline
<code>_av_raw</code> , <code>_yf_raw</code>	Unfiltered chunk merge
<code>_yf_loaded</code>	YF load flag
<code>_newsroom_url_ticker</code>	URL ticker tracking
<code>_news_offset</code> , <code>_news_has_more</code> , <code>_news_loading_more</code>	Legacy pagination keys (cleared on data change)

Stale keys removed on render: `news_full_search` , `news_full_search_input` , `_news_fullsearch_kw_too_short` .

13. URL query parameters

Param	Behavior
<code>ticker</code>	Clears <code>news_company_select</code> when removed; header nav fallback
<code>category</code>	Restores category dropdown; synced from <code>localStorage</code> on first load

14. Performance instrumentation

`PageLoadTracker("newsroom")` steps:

- `PAGE_TITLE` , `LOCALSTORAGE_RESTORE` , `PREFETCH_SETUP` , `DATE_RANGE_FETCH`
- `CSS_INJECT` , `FILTER_WIDGETS` , `DATA_LOAD` , `COMPANY_NAME_MAP` , `RENDER_LAYOUT`

15. Error handling, entry flow, and related files

- `main()` wrapped in try/except → `st.error` + `log_structured_error` ; chunk fetch failures return empty lists with warning logs.
- Entry: `new_rerun_id` → `styles` → `initialize_app` → `header` → `render_page` → footer (see diagram below).

- Entry: `newrerunid` → `styles` → `initializeapp` → `header` → `renderpage` → footer (see diagram below).



File	Relevance
<code>app/data/repository.py</code> → <code>NewsRepository</code>	All SQL, caching, dedup
<code>app/utils/constants.py</code>	Topic labels
<code>app/utils/cache_manager.py</code>	App-wide warmup from <code>main.py</code>
<code>app/data/watchlist_service.py</code>	Watchlist reads

16. Authentication and access control

Check	Status
<code>require_auth(page="newsroom")</code>	Commented out at module level
<code>AccessControlManager.check_access</code>	Not used on this page
<code>get_current_user()</code>	Used for watchlist-scoped filtering when a watchlist is active
Identity source	<code>main.py</code> central auth bootstrap (cookie hydrate)

No page-level ACL row is required for Newsroom in `coreiq_page_access_control` ; access is implied by portal authentication and navigation visibility.

17. Design decisions (summary)

- Week-aligned chunks** maximize `@st.cache_data` reuse when users widen date range.
- Client-side sector/sort/keyword** avoid cache key explosion and extra DB round-trips.
- Dual-source union** presents one chronological feed; YF sector gap handled explicitly.
- No live APIs** — all latency is MySQL + merge + render.

— End of Newsroom — Technical Documentation —

PART 12 OF 22

Earnings Calls — Technical Documentation

Earnings transcript browser with TF-IDF search and PDF download.

Continued from previous section →

Application: Market Data Portal (MDP)

Page module: `app/pages/earnings_calls.py`

URL route: `/earnings_calls`

Registered in: `app/main.py` as `( "pages/earnings_calls.py", "Earnings Calls", "earnings_calls", False )`

1. Purpose and scope

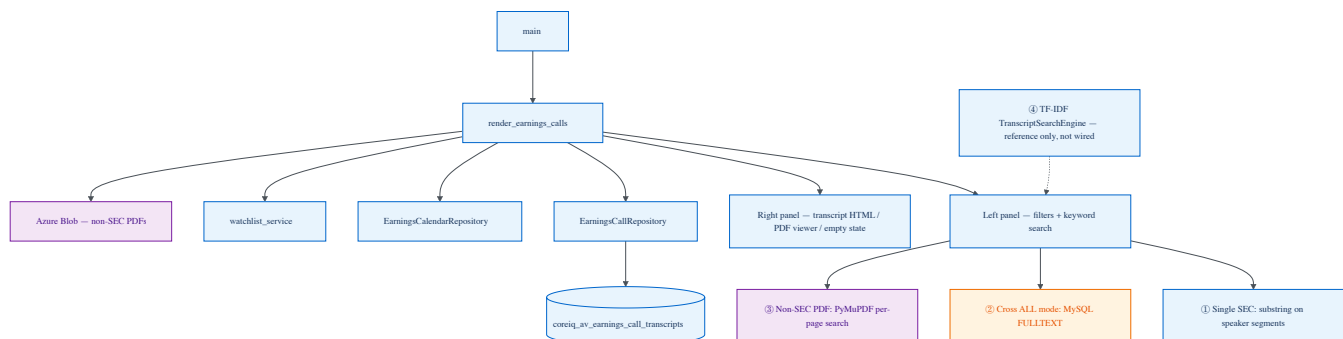
The Earnings Calls page lets users browse **earnings call transcripts** for SEC and select non-SEC companies. Capabilities include:

- **Company / Year / Quarter** filters with cross-transcript **"ALL"** mode
- **Keyword search** in three modes: single transcript, cross-transcript (MySQL FULLTEXT), non-SEC PDF
- **Transcript display** with speaker segmentation and keyword highlighting
- **PDF download** (generated for SEC text transcripts; blob PDF for non-SEC)
- **Excel export** of keyword hits (inline or background thread for large cross-search)
- **Watchlist-scoped** cross-transcript search
- Deep links from **Earnings Calendar** and cross-search result cards

2. Architecture overview

Top-down layout: page orchestration loads repositories and storage, then the left search panel dispatches to one of three active search paths (plus an unused Term Frequency–Inverse Document Frequency (TF-IDF) reference engine).

Top-down layout: page orchestration loads repositories and storage, then the left search panel dispatches to one of three active search paths (plus an unused Term Frequency–Inverse



3. Term Frequency–Inverse Document Frequency (TF-IDF) search engine (`app/core/search_engine.py`)

3.1 Status in this page

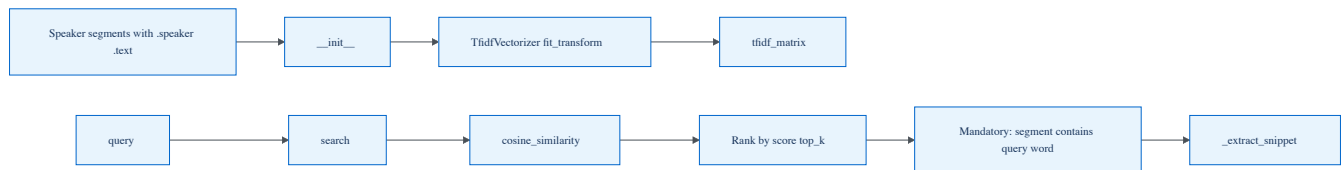
`TranscriptSearchEngine` is **implemented and documented** as the project's Term Frequency–Inverse Document Frequency (TF-IDF) transcript search utility, but `earnings_calls.py` **does not import it** (as of current codebase). The page uses:

Mode	Mechanism
Single transcript	Case-insensitive substring scan on parsed speaker segments
Cross-transcript	MySQL FULLTEXT (<code>search_transcripts_fulltext</code>) + segment extraction
Non-SEC PDF	PyMuPDF per-page text + substring search

The Term Frequency–Inverse Document Frequency (TF-IDF) engine remains the canonical design for **relevance-ranked segment search** and is listed in `requirements.txt` (scikit-learn). Integrating it would replace or augment single-transcript `matches` loop with ranked `SearchResult` objects.

3.2 TranscriptSearchEngine design

3.2 'TranscriptSearchEngine' design



Class: TranscriptSearchEngine

Dataclass: SearchResult(index, speaker, text, score, snippet)

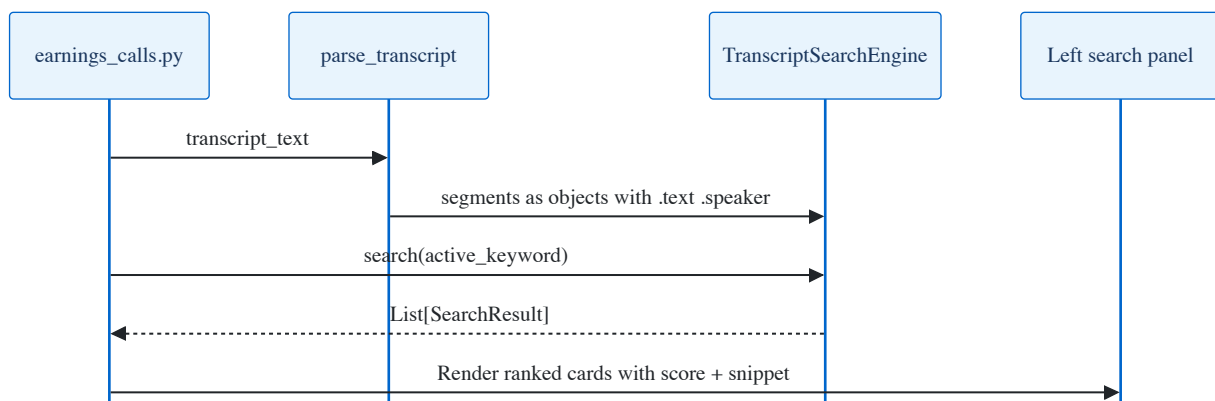
Parameter	Value
stop_words	'english'
max_features	10000
ngram_range	(1, 2) unigrams + bigrams
min_df	1
sublinear_tf	True
min_score default	0.01
top_k default	20

Fallback: If sklearn missing or fit fails → `_fallback_search` counts substring occurrences normalized by word count.

Snippet extraction: Centers on full query or first matching word; adds `...` ellipses.

3.3 Hypothetical integration (not current code)

3.3 Hypothetical integration (not current code)



4. Database: coreiq_av_earnings_call_transcripts

Primary table for SEC-sourced (and ingested) transcript text.

Column	Type role
<code>id</code>	Primary key
<code>source</code>	Data provenance
<code>ticker</code>	Company symbol
<code>quarter</code>	Compact key e.g. <code>2025Q4</code>
<code>year</code>	Fiscal/report year
<code>q</code>	Quarter integer 1–4
<code>transcript_text</code>	LONGTEXT body
<code>has_transcript</code>	Filter flag (= 1 for UI lists)
<code>title</code>	Metadata
<code>event_datetime_utc</code>	Event time
<code>fetch_at_utc</code>	Ingest timestamp

Index: `idx_transcript_fulltext_search` — FULLTEXT on `transcript_text` for cross-search.

4.1 `EarningsCall` model

```
@dataclass
class EarningsCall:
    id: int
    source: Optional[str]
    ticker: str
    quarter: str
    year: int
    q: int
    transcript_text: Optional[str]
    has_transcript: bool
    title: Optional[str]
    event_datetime_utc: Optional[datetime]
    fetch_at_utc: datetime
```

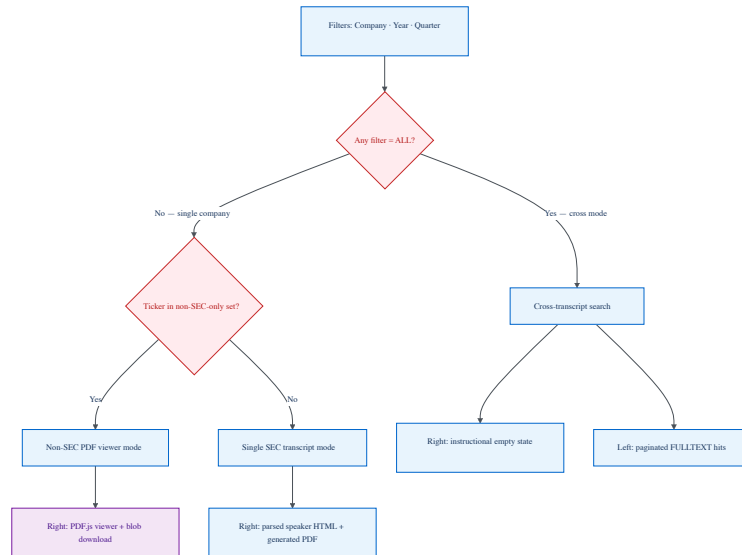
4.2 Key repository methods

Method	Cache TTL	Purpose
<code>get_companies_with_earnings()</code>	600s	Distinct tickers with transcripts
<code>get_earnings_calls(ticker, year, quarter)</code>	60s	Fetch transcript row(s)
<code>get_years_and_quarters(ticker)</code>	—	Combined year→quarters map
<code>get_all_available_years()</code>	—	For Company=ALL
<code>search_transcripts_fulltext(...)</code>	120s	Cross-transcript keyword
<code>search_transcript_windows_for_export(...)</code>	—	Excel export (up to 10k rows)
<code>get_non_sec_transcript_companies()</code>	—	YFinance-only transcript tickers
<code>get_non_sec_transcript_years_and_quarters(ticker)</code>	—	Non-SEC period map

5. Page modes

Filter combination determines which right-panel renderer runs. Cross mode keeps instructional empty state on the right while results populate the left panel.

Filter combination determines which right-panel renderer runs. Cross mode keeps instructional empty state on the right while results populate the left panel.



5.1 Cross-transcript mode

Triggered when `company == 'ALL'` OR `year == 'ALL'` OR `quarter == 'ALL'`.

- Left panel: `_get_cross_search_results` (paginated, 15 per page)
- Right panel: `render_cross_search_panel` instructional empty state
- Watchlist dropdown shown **above** filter row when all three are ALL

5.2 Single SEC transcript mode

- Fetches `EarningsCallRepository.get_earnings_calls`
- Parses `transcript_text` → speaker segments
- Optional earnings calendar metadata: report date, fiscal period end, display period label
- Right: HTML transcript card + generated PDF download

5.3 Non-SEC transcript mode

Companies only in `get_non_sec_transcript_companies()` (not in SEC earnings table).

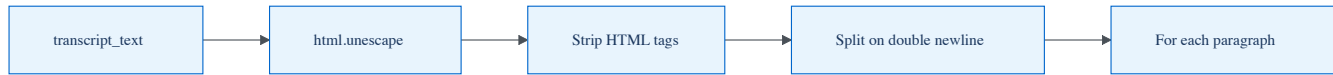
- PDF path: Azure blob `{ticker}/{year}/transcript-{quarter}/transcript.pdf`
- Local cache via `utils.azure_blob.ensure_local_blob_for_transcript`
- Right: `_render_ec_pdf_viewer` (PDF.js) + raw PDF download
- Search: page-level text extraction, not speaker parse

6. Transcript parsing

6.1 `parse_transcript(transcript_text)` — `@st.cache_data(ttl=600)`

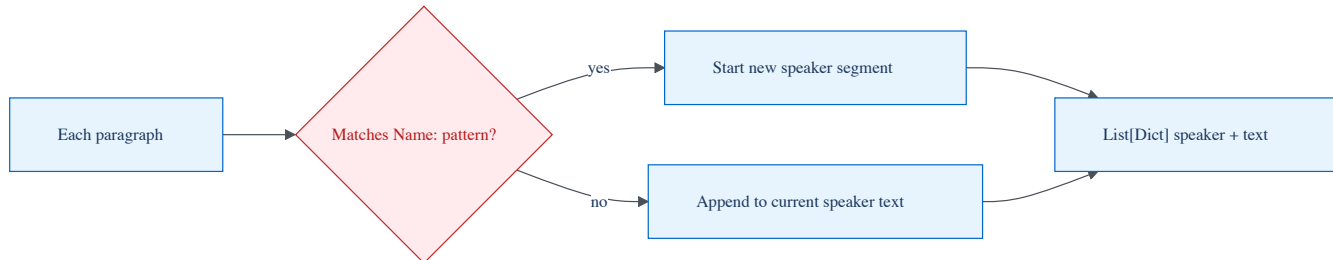
Part 1 — Normalize raw text

Part 1 — Normalize raw text



Part 2 — Speaker detection

Part 2 — Speaker detection



Speaker regex: `^([A-Z][a-zA-Z\s\.\,]+(?:\s+[A-Z][a-zA-Z]+)*):\s*(.*)$` — returns picklable `List[Dict]` with keys `'speaker'`, `'text'` for Streamlit cache.

6.2 Rendering

Function	Output
<code>render_speaker_section</code>	HTML block <code>#seg-{index}</code> with optional <code><mark></code> highlights
<code>render_transcript_card</code>	Wrapper <code>.ec-transcript-body-wrap</code> + scrollable body
<code>_render_transcript_header_html</code>	Title, year, quarter, report/QE dates, fiscal period label

7. Search implementations (actual)

7.1 Single-transcript keyword search

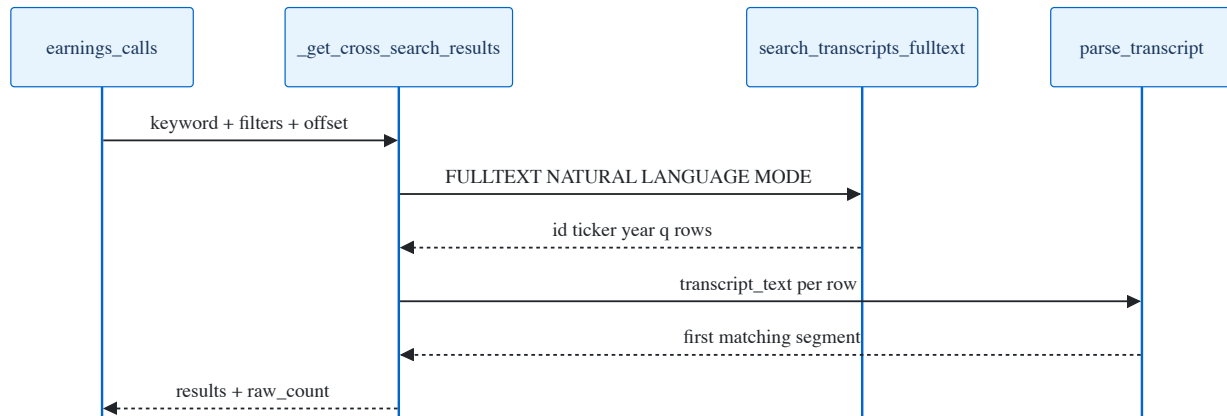
```

for i, seg in enumerate(segments):
    if active_keyword.lower() in seg['text'].lower():
        # build snippet ±40 chars around first match
  
```

- Left panel cards link to `#seg-{index}` anchor in right column
- Max 30 cards displayed in loop (no pagination within single transcript)
- Excel: `_build_keyword_results_excel` when matches exist

7.2 Cross-transcript search

7.2 Cross-transcript search



Pagination session keys:

Key	Purpose
<code>ec_cross_sig</code>	<code>(keyword, company, year, quarter, watchlist_id)</code>
<code>ec_cross_acc</code>	Accumulated result list
<code>ec_cross_offset</code>	Next DB offset
<code>ec_cross_has_more</code>	<code>raw_count == page_size</code>

View → navigation: `/earnings_calls?ticker=&year=&quarter=&highlight=`

7.3 FULLTEXT query strategy (`search_transcripts_fulltext`)

- Keywords ≥ 3 chars: `MATCH(transcript_text) AGAINST (:kw IN NATURAL LANGUAGE MODE)`
- Short keywords: `LIKE '%keyword%'`
- Two-phase fetch:** light columns + sort/limit first; `transcript_text` lookup by `id` IN (...) second — avoids sorting LONGTEXT
- Watchlist: `allowed_tickers` → `ticker IN (...) when watchlist_id set`

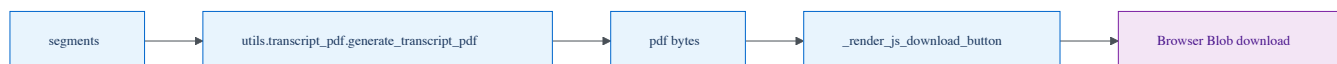
7.4 Non-SEC PDF search

- `_extract_pdf_text_by_page(pdf_path)` — PyMuPDF, cached 300s
- `_search_pdf_pages` — substring per page, snippet ± 60 chars
- View link includes `pdf_page` and `keyword` query params
- PDF viewer scrolls to `target_page` and highlights keyword on all rendered pages (red overlay boxes)

8. PDF download

8.1 SEC transcripts (generated)

8.1 SEC transcripts (generated)



- Avoids Streamlit `/media/` endpoint (proxy issues)

- Base64 embedded in `streamlit.components.v1.html` iframe button

8.2 Non-SEC transcripts (file)

- Reads cached blob file → `_render_js_download_button`
- Filename: `{ticker}_{year}_{quarter}_Transcript.pdf`

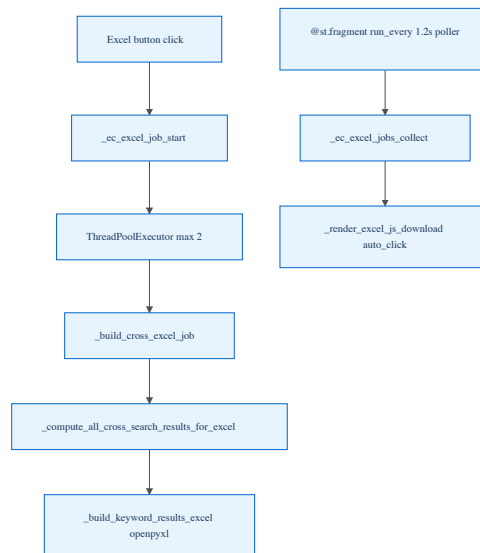
8.3 PDF viewer (`_render_ec_pdf_viewer`)

- PDF.js 3.11.174 from CDN
- Lazy page render via `IntersectionObserver`
- Keyword highlight on every visible page
- Session cache: `ec_pdf_b64_{pdf_path}`

9. Excel export

9.1 Cross-transcript (background)

9.1 Cross-transcript (background)



- Module-level `_EC_EXCEL_JOBS` dict (survives reruns)
- Max 12 jobs retained
- Columns: Company, Ticker, Year, Quarter, Reporter Name, Whole Paragraph

9.2 Single-transcript

- Synchronous build cached in `ec_single_excel_bytes` / `ec_single_excel_sig`

10. Filters and session state

10.1 Filter row widgets

Widget	Key	Callback
Search	<code>ec_search_input</code> → <code>ec_search</code>	—
Company	<code>ec_company_select</code>	<code>on_company_change</code>
Year	<code>ec_year_select_{company}</code>	<code>on_year_change</code>
Quarter	<code>ec_quarter_select_{company}_{year}</code>	<code>on_quarter_change</code>

Dynamic widget keys force Streamlit to reset dropdown display when company/year changes.

10.2 Session state reference

Key	Purpose
<code>ec_company</code> , <code>ec_year</code> , <code>ec_quarter</code>	Filter values
<code>ec_search</code>	Keyword
<code>_ec_quarter_user_set</code>	Preserve user quarter vs auto-default
<code>_ec_nav_id</code>	One-time URL nav consumption
<code>_ec_last_url_ticker</code>	Fresh ticker detection
<code>ec_calls_active_watchlist_id</code>	Cross-search watchlist
<code>active_ticker</code>	Shared nav ticker
<code>ec_cross_*</code>	Cross-search pagination
<code>ec_single_excel_*</code>	Single Excel cache
<code>_earnings_cleanup_done</code>	One-time <code>cache_cleaner</code> flag

10.3 Local storage persistence

`utils.local_storage_manager` :

- `load_earnings_calls_state()` — before widget init
- `save_earnings_calls_state()` — end of `render_earnings_calls` (not in callbacks — avoids header DOM glitch)

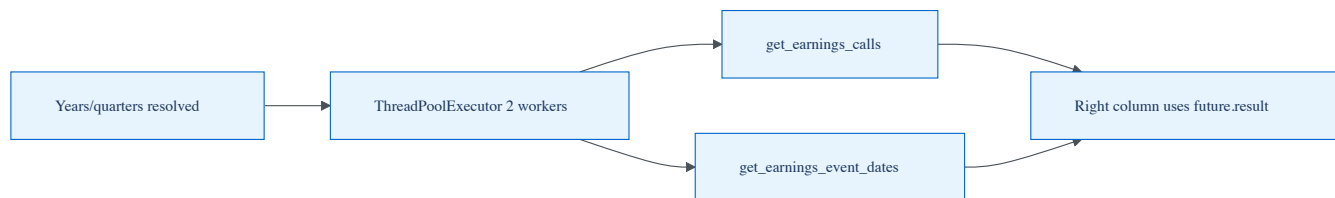
11. URL query parameters

Param	Purpose
<code>ticker</code>	Set company
<code>year</code> , <code>quarter</code>	Set period
<code>highlight</code>	Keyword → <code>ec_search</code>
<code>keyword</code>	Preserved on PDF view links
<code>from=calendar</code>	Calendar navigation bundle
<code>pdf_page</code>	Scroll PDF to page (non-SEC)
<code>clear_cache=1</code>	Admin: <code>st.cache_data.clear()</code>

12. Prefetch optimization

When not in cross-search and not non-SEC:

When not in cross-search and not non-SEC:



13. Company list construction

1. Parallel fetch: SEC companies + non-SEC transcript companies
2. Merge with dedupe by ticker and normalized name
3. `_non_sec_only_tickers` — routing for YQ map and PDF path
4. Dropdown: `( 'ALL', 'All Companies' )` + sorted list

14. CSS and layout

- `get_earnings_css()` — extensive Streamlit overrides, 520px matched panels
- Left: `st.container(border=True, height=520)`
- Right: transcript card or PDF viewer (~820px iframe height)
- `PageLoadTracker` steps: `FETCH_COMPANIES` , `MERGE_COMPANIES` , `FETCH_YEARS_QUARTERS` , `FETCH_TRANSCRIPT` , `RENDER_CONTENT`

15. Module init: cache cleanup

On first session run per process:

- `utils.cache_cleaner.ensure_fresh_code()` removes stale `__pycache__`
- Results stored in `_earnings_cleanup_results` , timing in `_earnings_cleanup_time_ms`

16. Authentication and access control

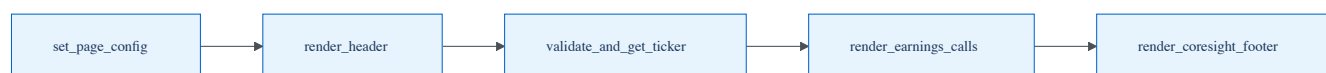
Check	Status
<code>require_auth(page="earnings_calls")</code>	Optional at page level; identity via <code>main.py</code> cookie hydrate
<code>AccessControlManager.check_access</code>	Not used — any authenticated portal user with nav access may view
Identity source	<code>main.py</code> cookie hydrate + optional <code>get_current_user()</code> for watchlist scoping

Direct `/earnings_calls` URLs should be protected by enabling `require_auth()` so only valid OpenID Connect (OIDC) sessions can render the page.

17. Entry point, errors, and related files

`main()` runs at module import: `set_page_config` → `render_header` → ticker validation for nav → `render_earnings_calls` → footer. Errors use `log_structured_error` with `page="earnings_calls"`. PDF generation failures hide the download button; transcript prefetch futures fall back serially after a 30s timeout.

main() runs at module import: setpageconfig → renderheader → ticker validation for nav → renderearningscalls → footer. Errors use logstructurederror with page="earningscalls". PDF



File	Role
<code>app/core/search_engine.py</code>	Term Frequency–Inverse Document Frequency (TF-IDF) engine (reference, not wired)
<code>app/utils/transcript_pdf.py</code>	Securities and Exchange Commission (SEC) transcript PDF generation
<code>app/utils/azure_blob.py</code>	Non-SEC PDF local cache
<code>app/data/repository.py</code>	<code>EarningsCallRepository</code> , <code>EarningsCalendarRepository</code>
<code>app/utils/local_storage_manager.py</code>	Filter persistence across sessions

18. Summary: search technology map

User action	Technology	Ranked?
Search one SEC call	Python substring on segments	No
Search all calls (ALL mode)	MySQL FULLTEXT + first segment hit	DB relevance (NL mode)
Search non-SEC PDF	PyMuPDF + substring	No
Available not used	<code>TranscriptSearchEngine</code> Term Frequency–Inverse Document Frequency (TF-IDF) cosine	Yes

— End of Earnings Calls — Technical Documentation —

PART 13 OF 22

Live Earnings Transcript — Technical Documentation

Live audio transcription pipeline and earnings call ingestion.

Continued from previous section →

Application: Market Data Portal (MDP)

Page module: `app/pages/live_earnings_transcript.py`

URL route: `/live_earnings_transcript`

Registered in: `app/main.py` as `( "pages/live_earnings_transcript.py", "Live Transcript", "live_earnings_transcript", False )`

1. Purpose and scope

The Live Earnings Transcript page is a **chunked transcription workspace** for earnings calls. It:

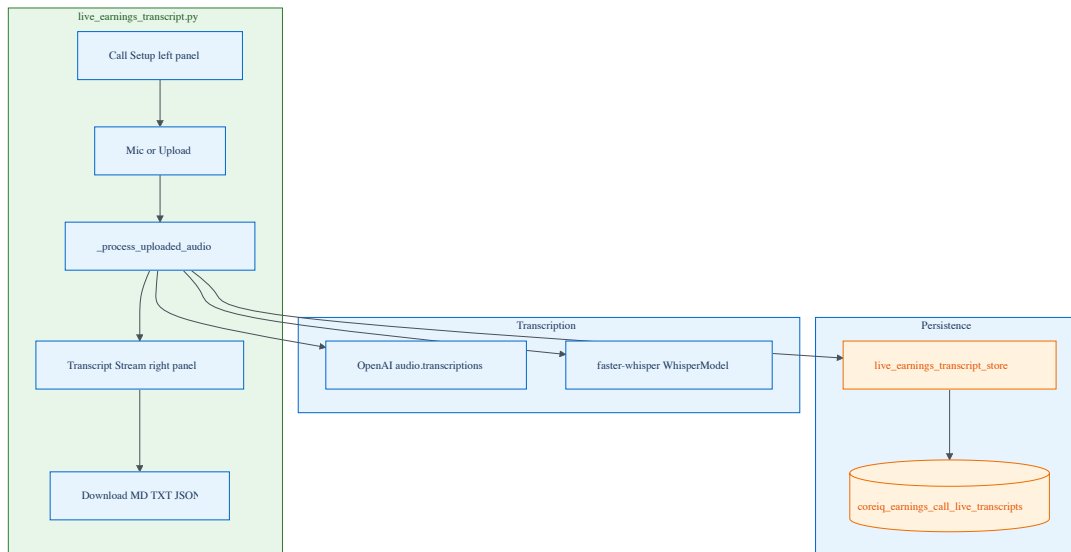
- Accepts audio via **browser microphone** (`st.audio_input`) or **file upload**
- Transcribes each completed clip with **OpenAI API** or **local faster-whisper**
- Appends segments to an in-session transcript stream
- **Persists** structured JSON to MySQL (`coreiq_earnings_call_live_transcripts`)
- Exports **MD / TXT / JSON** downloads

Important: This is **not** a live captioning or polling UI. Text appears only **after** each audio blob finishes uploading and transcription completes. There is **no auto-refresh**, **no WebSocket**, and **no periodic** `st.rerun` **polling loop** for incoming transcript text.

2. Architecture overview

Page architecture: UI orchestration, data repositories, and external services for this feature.

Page architecture: UI orchestration, data repositories, and external services for this feature.



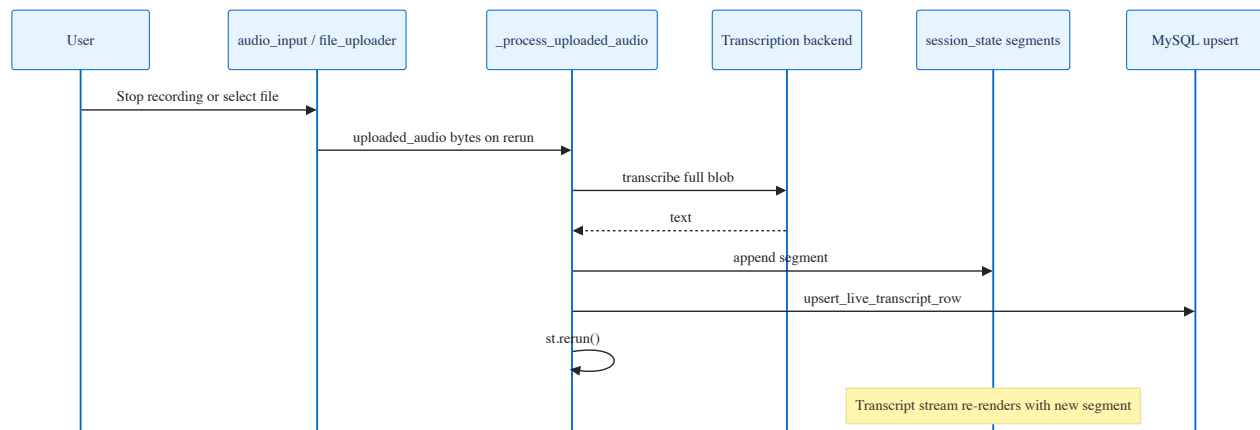
3. Polling and refresh behavior

3.1 What the page does NOT do

Expected "live" pattern	Actual behavior
Poll DB for new transcript rows	No — only writes, no reads from DB on page
<code>st.fragment(run_every=...)</code> auto-refresh	No
Stream partial text while recording	No — mic must stop/upload first
Server-side HLS/ffmpeg chunking	Planned (documented in UI expander), not implemented

3.2 What triggers UI updates

3.2 What triggers UI updates



Explicit reruns:

1. `st.rerun()` after successful transcription
2. Start / End / Clear buttons
3. "Reload run" button (mic upload pickup)
4. Environment check / clear log buttons

Timer display: `live_call_started_at` updated on Start; elapsed shown in metrics but **does not** trigger periodic refresh.

4. Database: `coreiq_earnings_call_live_transcripts`

4.1 Table usage

Written exclusively via `app/data/live_earnings_transcript_store.py` → `upsert_live_transcript_row()`.

Column	Value source
<code>source</code>	<code>'live_streamlit'</code> (<code>LIVE_TRANSCRIPT_SOURCE</code>)
<code>ticker</code>	Normalized symbol (<code>normalize_ticker</code>)
<code>quarter</code>	Compact period e.g. <code>2024Q1</code> (<code>export_period_key</code>)
<code>year</code>	Fiscal year integer
<code>q</code>	1-4 from <code>Q1</code> – <code>Q4</code> label
<code>transcript_text</code>	Plain text: segments joined <code>\n\n</code>
<code>has_transcript</code>	<code>1</code> if non-empty plain text
<code>title</code>	<code>{SYMBOL} {period} live transcript</code> (max 512)
<code>event_datetime_utc</code>	<code>NULL</code> (not set by Streamlit page)
<code>fetch_at_utc</code>	UTC naive timestamp
<code>raw_json</code>	Structured payload JSON
<code>raw_json_sha1</code>	SHA1 of <code>raw_json</code>
<code>last_error</code>	Cleared on successful upsert

Upsert key: `(source, ticker, quarter)` — `ON DUPLICATE KEY UPDATE` refreshes all content fields.

4.2 `raw_json` schema

Built by `build_earnings_transcript_payload()` :

```

{
  "symbol": "AAPL",
  "quarter": "2024Q1",
  "transcript": [
    {
      "speaker": "",
      "title": "",
      "content": "segment text",
      "sentiment": "0"
    }
  ]
}
```

Live page segments use keys: `id`, `recorded_at`, `company`, `quarter`, `fiscal_year`, `backend`, `model`, `text` in session — mapped to `content` in DB payload (speaker/title empty unless extended later).

5. Session state

Initialized in `_init_state()` :

Key	Type	Purpose
<code>live_call_active</code>	<code>bool</code>	Timer pill "on" (not live captions)
<code>live_call_started_at</code>	<code>float</code> <code>None</code>	<code>time.time()</code> epoch for elapsed metric
<code>live_transcript_segments</code>	<code>List[Dict]</code>	All transcribed segments
<code>live_processed_audio_hashes</code>	<code>set</code>	SHA256 digest prefix dedupe
<code>live_audio_widget_version</code>	<code>int</code>	Bumps to reset audio widgets after success
<code>live_activity_log</code>	<code>List[str]</code>	Pipeline log (max 100 lines)

5.1 Segment dict shape

```
{
  "id": digest,          # sha256 hex[:20]
  "recorded_at": "YYYY-MM-DD HH:MM:SS",
  "company": "AAPL",
  "quarter": "Q1",
  "fiscal_year": 2024,
  "backend": "OpenAI API" | "Local faster-whisper",
  "model": "gpt-4o-transcribe" | "small.en" | ...,
  "text": "transcribed content",
}
```

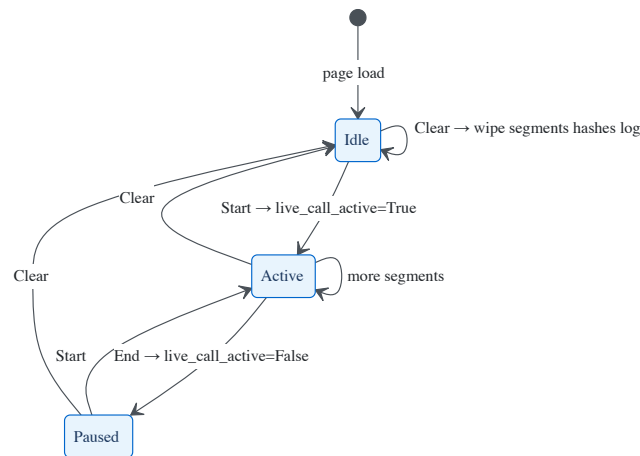
6. Call setup (left panel)

6.1 Metadata inputs

Field	Widget	Default
Company / ticker	<code>st.text_input</code>	URL <code>ticker</code> or <code>active_ticker</code>
Quarter	<code>st.selectbox</code> Q1-Q4	Q1
Fiscal year	<code>st.number_input</code>	Current calendar year
Transcription backend	<code>selectbox</code>	<code>Local faster-whisper</code>
OpenAI model	<code>conditional</code>	<code>gpt-4o-transcribe</code>
faster-whisper model	<code>conditional</code>	<code>small.en</code>
Device	<code>conditional</code>	<code>auto</code> → cuda if torch else cpu
Compute type	<code>conditional</code>	<code>int8</code>
Language	<code>text</code>	<code>en</code>
Call context	<code>textarea</code>	Speaker/product hints for prompt

6.2 Control buttons

6.2 Control buttons



Button	Action
Start	<code>_start_call()</code> — sets active + started_at if missing; <code>st.rerun()</code>
End	<code>_end_call()</code> — active=False only
Clear	<code>_clear_call()</code> — full reset + increment widget version

7. Audio ingestion

7.1 Modes

Mode	Constant	Widget
Microphone	<code>INGESTION_MIC</code>	<code>st.audio_input</code> key <code>live_audio_input_{version}</code>
Upload	<code>INGESTION_UPLOAD</code>	<code>st.file_uploader</code> wav/mp3/m4a/webm/ogg/flac

7.2 Processing gate

When `uploaded_audio` is not None :

1. Read bytes via `getvalue()`
2. Dedupe by `_audio_digest` (SHA256[:20])
3. Build domain prompt via `_build_transcription_prompt`
4. Run transcription inside `st.status` or `st.spinner`
5. On success: append segment, upsert DB, bump widget version, `st.rerun()`

Note: Transcription runs even if user clicked **End** before upload completed (comment in code — avoids dropping in-flight clips).

7.3 Format detection

`_guess_audio_suffix()` — extension from filename or magic bytes (RIFF/WAVE, OggS, ftyp → webm).

7.4 Large file warning

Clips > 40 MB log warning in pipeline (proxy/runtime limits).

8. Transcription backends

8.1 OpenAI API

8.1 OpenAI API



- Requires `OPENAI_API_KEY`
- Models: `gpt-4o-transcribe`, `gpt-4o-mini-transcribe`, `whisper-1`
- Optional `language`, `prompt` kwargs

8.2 Local faster-whisper

8.2 Local faster-whisper



- `@st.cache_resource` on `_load_faster_whisper_model` — model loaded once per size/device/compute_type
- `vad_filter=False` — helps short browser chunks classified as non-speech
- Temp file required (BytesIO breaks webm format detection)

8.3 Shared prompt

`_build_transcription_prompt` instructs model to preserve financial terms, ticker symbols, KPIs, and optional company/quarter/context from UI.

9. Transcript stream (right panel)

9.1 Metrics bar

Metric	Source
Segments	<code>len(segments)</code>
Words	sum of <code>_word_count</code> per segment
Elapsed	<code>_elapsed_label(live_call_started_at)</code> — static until rerun

9.2 Segment rendering

`_render_transcript()` — HTML `.segment` blocks with meta:

Segment NN | recorded_at | backend | model

Empty state explains no live rolling transcript during recording.

9.3 Pipeline log

- Expander auto-expands if recent lines contain FAIL/EXCEPTION/ABORT/ERROR

- **Run environment check** — imports faster_whisper, av, OPENAI_API_KEY, `db_manager.health_check()`
- **Clear log** — resets `live_activity_log`

10. Export downloads

When `segments` non-empty:

Button	Content	MIME
Download MD	<code>_transcript_markdown</code>	text/markdown
Download TXT	<code>_transcript_text</code> numbered blocks	text/plain
Download JSON	<code>build_earnings_transcript_payload</code>	application/json

Filename base: `{TICKER}_{YYYYQn}_live_transcript`

11. Dependencies

Module	Role
<code>components.styles</code>	Layout, sidebar hide
<code>components.navigation</code>	Header, footer
<code>core.auth_manager</code>	<code>require_auth</code> optional at page level
<code>data.live_earnings_transcript_store</code>	DB upsert, payload builders
<code>core.database.db_manager</code>	Health check in env verify
<code>openai</code>	Optional OpenAI backend
<code>faster_whisper</code>	Optional local backend
<code>torch</code>	CUDA detection for device=auto

12. CSS

`_render_css()` — `.live-wrap`, metrics grid, `.segment` left border brand red `#d62e2f`, responsive breakpoints at 900px.

Status pill:

- Inactive: "Timer off"
- Active: "Timer on (not live captions)" with green styling

13. Authentication and access control

Check	Status
<code>require_auth(page="live_earnings_transcript")</code>	Optional at page level
<code>AccessControlManager.check_access</code>	Not used
DB writes	Any session user can upsert rows to <code>coreiq_earnings_call_live_transcripts</code> if the page is reachable

Identity is established via `main.py` cookie hydrate. Enable `require_auth()` at page level if live-transcript ingestion should be restricted to authenticated analysts.

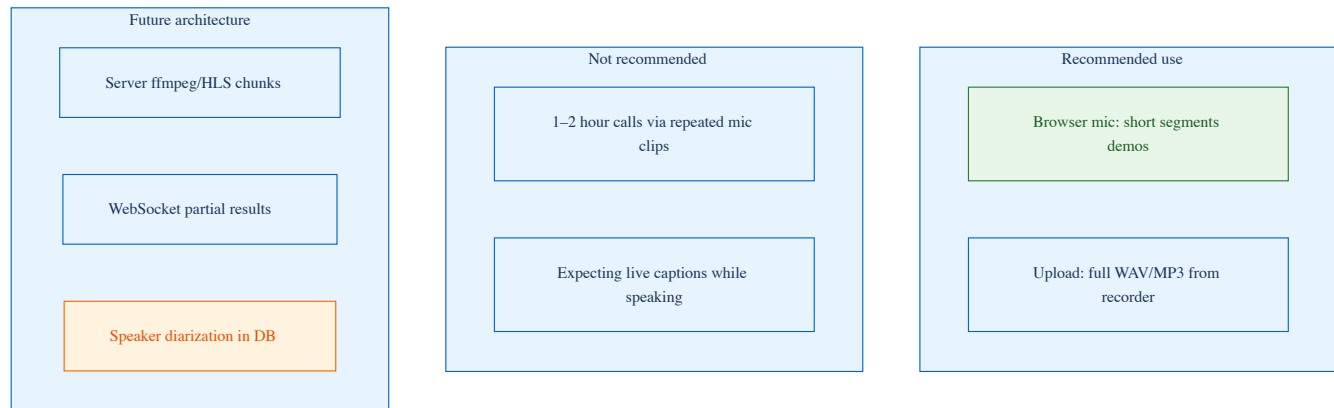
14. Error handling

Failure	UX
Zero audio bytes	Warning + pipeline ABORT log
Duplicate digest	Info skip message
Empty model text	Warning with troubleshooting tips
DB upsert fail	Warning; segment kept in session
Exception in transcribe	<code>st.error</code> + traceback expander + <code>log_structured_error</code>

`main()` top-level catch: generic refresh message.

15. Operational constraints (documented in UI)

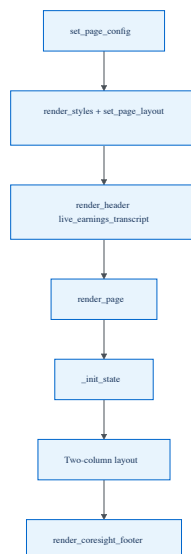
15. Operational constraints (documented in UI)



16. Main entry flow

`main()` entry point: bootstrap UI, read session/query state, fetch data, and render widgets.

main() entry point: bootstrap UI, read session/query state, fetch data, and render widgets.



Header ticker: query `ticker` or `active_ticker` or fallback `"M"`.

17. Comparison with Earnings Calls page

Aspect	<code>earnings_calls.py</code>	<code>live_earnings_transcript.py</code>
Data source	Pre-ingested AV / Azure PDF	User audio → transcription
DB table	<code>coreiq_av_earnings_call_transcripts</code>	<code>coreiq_earnings_call_live_transcripts</code>
Display	Full historical transcript	Growing segment list
Search	Keyword / FULLTEXT	None
Polling	None	None

18. Environment variables

Variable	Required for
<code>OPENAI_API_KEY</code>	OpenAI backend
<code>STG_DB_*</code> / <code>PROD_DB_*</code>	MySQL upsert (via app config)

19. Related files

File	Role
<code>app/data/live_earnings_transcript_store.py</code>	INSERT upsert SQL, JSON schema
<code>requirements.txt</code>	faster-whisper, openai optional deps
<code>app/core/database.py</code>	<code>db_manager.execute_insert</code>

20. Testing checklist

1. Upload small WAV → segment appears, pipeline log shows byte count and timing
2. Repeat same file → duplicate skip message
3. **Clear** → segments and log empty
4. Run environment check → faster_whisper, av, MySQL lines
5. Verify row in `coreiq_earnings_call_live_transcripts` for ticker/quarter key
6. Download JSON matches `raw_json` column structure

— End of Live Earnings Transcript — Technical Documentation —

PART 14 OF 22

Earnings Calendar — Technical Documentation

FullCalendar earnings schedule with watchlists and email alerts.

Continued from previous section →

Application: Market Data Portal (MDP)

Page module: `app/pages/earnings_calendar.py` (~2,127 lines)

URL route: `/earnings_calendar`

Registered in: `app/main.py` as `( "pages/earnings_calendar.py", "Earnings Calendar", "earnings_calendar", False)`

1. Purpose and scope

The Earnings Calendar page is a **FullCalendar**-powered view of upcoming and historical earnings announcements across the Market Data Portal (MDP) universe. It also overlays **completed mergers and acquisitions (M&A)** closing events from key developments data.

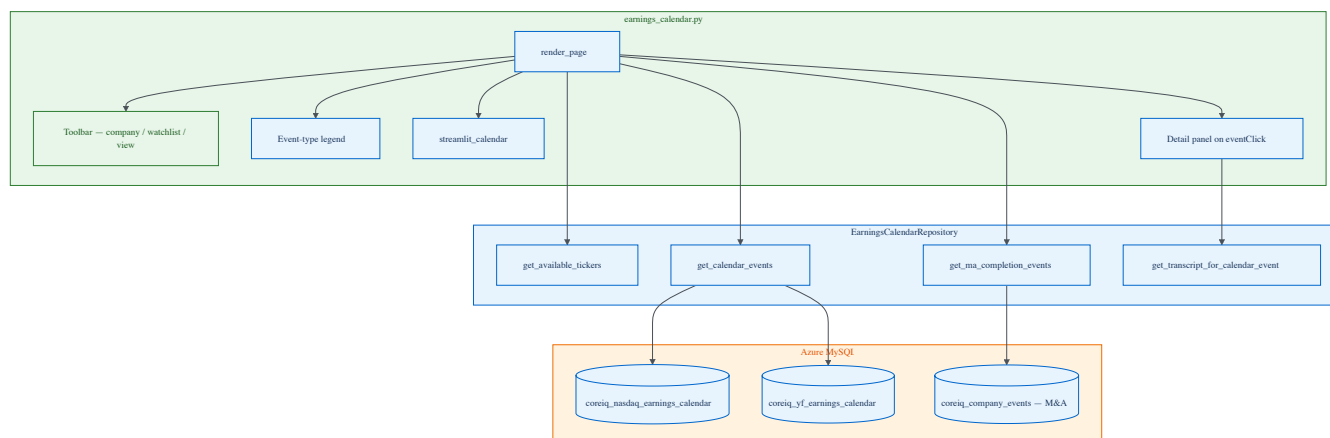
Core capabilities:

- **Month view** (`dayGridMonth`) and **Year view** (`multiMonthYear`) with list-month toggle
- **Company filter** (All Companies or one ticker) with cross-page ticker synchronization via `?ticker=`
- **Watchlist filter** (from Screening watchlists) with ticker + company-name disambiguation
- **Event-type legend** — clickable chips for Fiscal Quarter 1–4 (Q1–Q4) and M&A Completion; combines with company/watchlist filters
- **Event detail panel** — Earnings per share (EPS) actual/forecast, surprise %, market cap, investor relations (IR) webcast link, deep link to Earnings Calls transcript
- **Email alert preferences** — modal dialog to schedule reminders (companies, sectors, or watchlist scope)
- **Parallel data prefetch** on page load (tickers, date range, all events, M&A events)

2. Architecture overview

Top-down: the page orchestrates toolbar filters, prefetches repository data in parallel, converts rows to FullCalendar JSON, and renders `streamlit_calendar` with click-to-detail.

Top-down: the page orchestrates toolbar filters, prefetches repository data in parallel, converts rows to FullCalendar JSON, and renders `streamlitcalendar` with click-to-detail.



3. URL, route, and deep links

Entry	Behavior
<code>/earnings_calendar</code>	Default — All Companies, month view
<code>/earnings_calendar?ticker=AAPL</code>	Validates ticker via <code>validate_and_get_ticker</code> ; dropdown still defaults to All Companies but URL ticker is preserved on company change
Navigation bar	<code>navigation.py</code> links with <code>?ticker={active_ticker}</code>
Detail panel → transcript	<code>/earnings_calls?ticker={T}&year={Y}&quarter=Q{Q}&from=calendar</code>
Earnings Calls → calendar	Reverse deep link documented in earnings-calls.md

4. Dependencies

4.1 Python modules

Module	Role
<code>components/styles.py</code>	<code>render_styles()</code> , <code>hide_sidebar()</code>
<code>components/navigation.py</code>	<code>render_header()</code> , <code>render_coresight_footer()</code>
<code>core/auth_manager.py</code>	<code>require_auth()</code> optional; cookie hydrate via <code>main.py</code>
<code>data/repository.py</code>	<code>CompanyRepository</code> , <code>EarningsCalendarRepository</code>
<code>data/watchlist_service.py</code>	<code>get_user_watchlists</code> , <code>get_watchlist_companies</code>
<code>utils/ticker_utils.py</code>	<code>validate_and_get_ticker</code> , <code>DEFAULT_FALLBACK_TICKER</code>
<code>utils/earnings_alert_email.py</code>	<code>send_earnings_alert_preferences_confirmation</code>
<code>utils/server_logger.py</code>	Structured logging, <code>log_timing</code> , <code>new_rerun_id</code>
<code>streamlit_calendar</code>	FullCalendar widget (<code>st_calendar</code>)

4.2 External libraries

Library	Use
FullCalendar (via <code>streamlit-calendar</code>)	Month grid, multi-month year, list view
ThreadPoolExecutor	Parallel prefetch of 4 independent DB calls

5. User Interface (UI) components

5.1 Toolbar

Control	Session key	Notes
Email Alerts button	<code>ec_open_email_alerts</code>	Opens <code>@st.dialog</code> modal
Alert status pill	—	HTML badge: Alerts on / Alerts off
Company selectbox	<code>ec_company_filter</code>	"All Companies" or "Name (TICK)" labels
Watchlist selectbox	<code>ec_watchlist_filter</code>	Disabled when user has no watchlists
Month Wise / Year Wise	<code>ec_view</code>	Toggles <code>calendar</code> vs <code>year</code> FullCalendar mode

5.2 Event-type legend

Five clickable chips (Q1, Q2, Q3, Q4, M&A Completion) using Coresight brand colors from `_KIND_COLORS`. Toggling a chip updates `ec_active_kinds`, clears `ec_selected_event`, and increments `ec_cal_version` to remount FullCalendar.

5.3 FullCalendar views

Mode	<code>initialView</code>	Key options
Month	<code>dayGridMonth</code>	<code>height: 680</code> , <code>dayMaxEvents: 4</code> , list-month toggle
Year	<code>multiMonthYear</code>	<code>multiMonthMaxColumns: 3</code> , <code>dayMaxEvents: 3</code>

Callbacks restricted to `["eventClick"]` only — avoids `eventsSet` rerun storms.

5.4 Detail panel

Shown in right column when `ec_selected_event` is set:

- **Earnings events** — EPS actual/forecast, surprise %, fiscal quarter end, market cap, report time, transcript link, IR webcast link
- **M&A events** — acquirer, target, deal type, transaction value, announce/close dates, source link

Close button clears selection and bumps `ec_cal_version`.

5.5 Email alerts dialog

`@st.dialog("Email alerts for upcoming earnings calls", width="large")` — coverage modes:

1. **Select Companies** — all calendar companies or specific multiselect
2. **Select Sectors** — all sectors or specific multiselect (matched in-memory against ticker metadata)
3. **Select Watchlist** — one watchlist; encoded in database (DB) as sentinel `watchlist_id:<id>`

Reminders sent at **08:00 UTC**; `days_before` 0–60.

6. Session state

Key	Type	Purpose
<code>ec_view</code>	<code>str</code>	"calendar" or "year"
<code>ec_selected_event</code>	<code>dict \ None</code>	FullCalendar event object from click
<code>ec_cal_version</code>	<code>int</code>	Bumped to remount calendar widget (new <code>key</code>)
<code>_ec_current_date</code>	<code>str</code> (ISO)	<code>initialDate</code> for FullCalendar — updated on event click
<code>ec_active_watchlist_id</code>	<code>int \ None</code>	Active watchlist filter
<code>ec_active_watchlist_name</code>	<code>str</code>	Display name for caption
<code>ec_active_kinds</code>	<code>list[str]</code>	Active legend kinds: Q1–Q4, <code>ma</code>
<code>ec_company_filter</code>	<code>str</code>	Selectbox label value
<code>ec_watchlist_filter</code>	<code>str</code>	Watchlist selectbox label
<code>ec_alert_send_email</code>	<code>bool</code>	Checkbox in dialog
<code>ec_alert_days_before</code>	<code>int</code>	Number input
<code>ec_alert_company_scope</code>	<code>str</code>	Radio: Companies / Sectors / Watchlist
<code>ec_alert_save_in_progress</code>	<code>bool</code>	Disables Save during persist
<code>ec_alert_save_success</code>	<code>bool</code>	Flash success on next dialog open
<code>ec_alert_save_mailed</code>	<code>bool</code>	Whether confirmation email sent
<code>_active_page</code>	<code>str</code>	Used to reset filters on fresh navigation
<code>active_ticker</code>	<code>str</code>	Cross-page ticker sync

Reset on fresh page entry (`_active_page != "earnings_calendar"`): clears `ec_company_filter` , `ec_watchlist_filter` , watchlist ids, resets `ec_active_kinds` to all.

7. Data layer

7.1 Primary tables

Table	Source	Content
<code>coreiq_nasdaq_earnings_calendar</code>	Nasdaq ingest	SEC-lineage earnings dates, EPS, fiscal quarter ending
<code>coreiq_yf_earnings_calendar</code>	Yahoo Finance (YF) ingest	Non-Nasdaq / supplemental earnings
<code>coreiq_company_events</code>	Key developments	M&A Closing rows (<code>event_subtype = 'M&A Closing'</code> , <code>ma_is_closed = 1</code>)
<code>coreiq_companies</code>	Company master	Names, sectors for filters
<code>coreiq_ir_websites</code>	IR URLs	Webcast links joined by ticker + source (SEC vs YFinance)
<code>coreiq_earnings_alert_preferences</code>	Alert prefs	Per-user email reminder configuration
<code>coreiq_earnings_alert_dedupe</code>	Dispatcher	Prevents duplicate sends
<code>coreiq_earnings_alert_delivery_log</code>	Dispatcher	Delivery audit trail
<code>coreiq_av_earnings_call_transcripts</code>	Transcripts	Mapped via <code>get_transcript_for_calendar_event</code>

7.2 `EarningsCalendarRepository` key methods

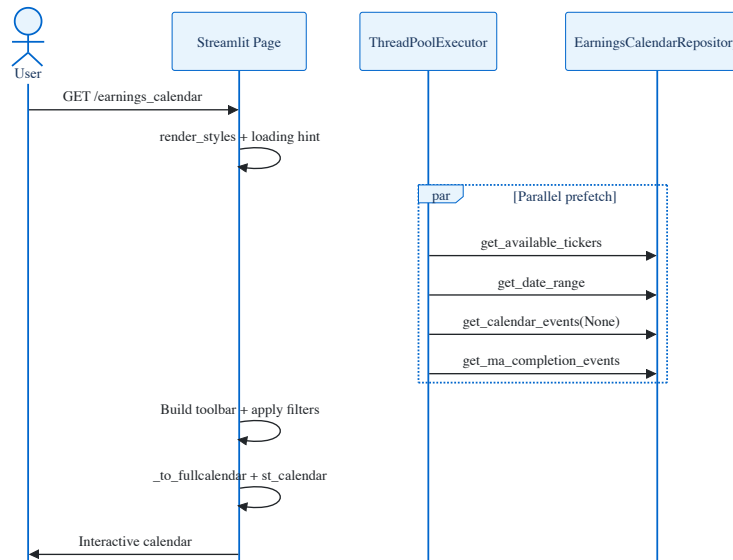
Method	Cache TTL	Description
<code>get_available_tickers()</code>	300s	Dropdown metadata: name, ticker, sector
<code>get_date_range()</code>	300s	Min/max earnings dates across both feeds
<code>get_calendar_events(tickers)</code>	300s	Union Nasdaq + YF, deduped per (<code>ticker</code> , <code>fiscal_quarter_ending</code>) — prefers latest <code>fetch_at_utc</code> , then Nasdaq over YF
<code>get_ma_completion_events()</code>	300s	M&A completion overlay from <code>coreiq_company_events</code>
<code>get_transcript_for_calendar_event(ticker, date)</code>	600s	Maps calendar date → transcript year/quarter
<code>get_earnings_alert_preferences(email)</code>	120s session	Loads prefs via <code>earnings_alert_store</code>
<code>upsert_earnings_alert_preferences(...)</code>	—	Persists prefs; invalidates session cache

7.3 Dedup and fiscal quarter logic

- `fiscal_quarter_ending` stored as `Mon/YYYY` (e.g. `Sep/2025`)
- Quarter derived: Jan–Mar = Q1 ... Oct–Dec = Q4
- Event colors use Coresight palette: Q1 dark blue, Q2 green, Q3 plum, Q4 gold, M&A red (`#D62E2F`)

8. Page load and data fetch

8. Page load and data fetch

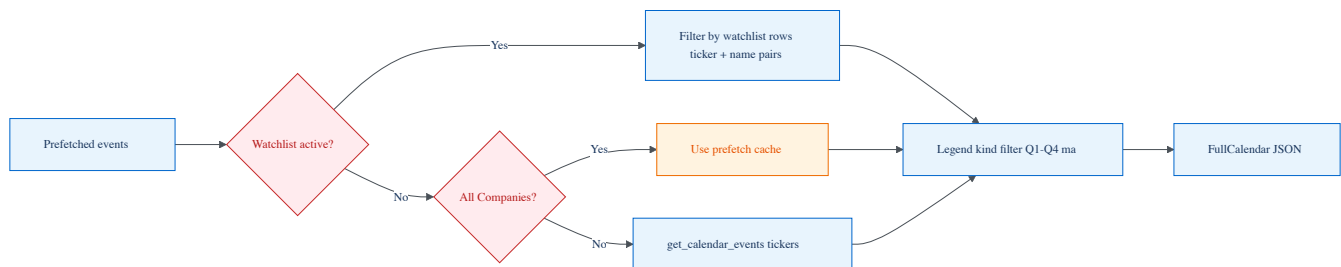


Timing logged as `EC_PAGE_FETCH_*` and `EC_PAGE_RENDER_TOTAL` (see server logs).

9. Filter pipeline

Filters apply **in sequence** (each narrows the in-memory event lists):

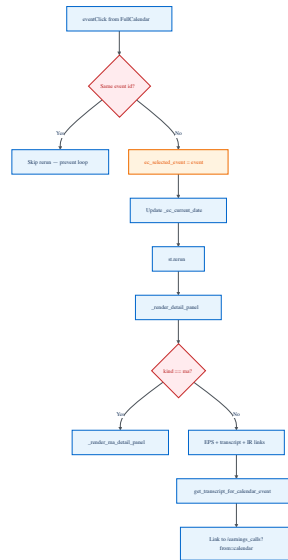
Filters apply in sequence (each narrows the in-memory event lists):



Watchlist disambiguation: For shared tickers (e.g. JD, LULU, TSCO), when both watchlist row and event carry `company_name`, a `(ticker_variant, normalized_name)` match is required.

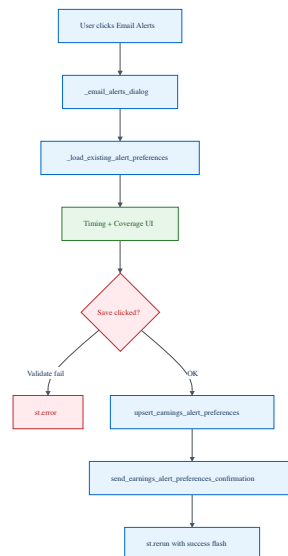
10. Event click and detail flow

10. Event click and detail flow



11. Email alert preferences

11. Email alert preferences



Watchlist encoding: `selection_mode == "watchlist"` stores `tickers_json = [{"watchlist_id":<id>}]`. The background dispatcher (`utils/earnings_alert_dispatcher.py`) resolves tickers at send time.

12. FullCalendar event shape

Earnings events (`_to_fullcalendar`):

```
{
  "id": "12345",
  "title": "Apple Inc Q1",
  "start": "2025-01-30",
  "color": "#005F8F",
  "extendedProps": {
    "kind": "Q1",
    "ticker": "AAPL",
    "fiscal_q": 1,
    "fiscal_year": 2025,
    "eps_actual": 2.18,
    "beat_miss": "beat",
    "ir_website_url": "https://..."
  }
}
```

M&A events use `id` prefix `ma_` and `kind: "ma"` .

13. Error handling

Area	Behavior
Parallel fetch failure	Per-future <code>log_structured_error</code> ; returns empty list / None
No calendar data	<code>st.error("No earnings calendar data found.")</code>
No events after filters	HTML message with hint to re-enable legend chips
Top-level render	<code>st.error("Earnings calendar is temporarily unavailable.")</code>
Alert save failure	<code>st.error(message)</code> from <code>_save_alert_preferences</code>
All exceptions	<code>log_structured_error(..., page="earnings_calendar", ...)</code>

14. Authentication and access control

Check	Status
<code>require_auth()</code>	Cookie hydrate via <code>main.py</code> ; optional page-level enforcement
<code>main.py</code> cookie hydrate	Provides identity when auth active
Email alerts	Requires signed-in user email from <code>auth_data</code> / session
Page Access Control List (ACL)	No dedicated ACL gate — available to all authenticated portal users

15. Performance notes

- **Parallel prefetch** — 4 workers on initial load; subsequent company-specific fetch hits `@st.cache_data` on repository methods
- **Warmup** — `cache_manager.py` pre-warms `get_available_tickers` , `_get_fiscal_year_end_map` , `get_calendar_events` on app boot
- **Calendar remount** — `ec_cal_version` in widget `key` prevents stale `eventClick` state loops
- **Navigation** — `navigation.py` queues page switch if calendar load is mid-run

16. Related files

File	Role
<code>app/data/repository.py</code>	<code>EarningsCalendarRepository</code> — all SQL
<code>app/data/earnings_alert_store.py</code>	Alert preference persistence
<code>app/utils/earnings_alert_dispatcher.py</code>	Scheduled email delivery
<code>app/utils/earnings_alert_email.py</code>	Simple Mail Transfer Protocol (SMTP) confirmation
<code>app/pages/earnings_calls.py</code>	Transcript viewer (deep-link target)
<code>app/components/navigation.py</code>	Nav link with ticker param
<code>app/utils/cache_manager.py</code>	Background warmup

17. Key code references

Function	Lines (approx.)	Purpose
<code>render_page()</code>	1605–2118	Main page orchestration
<code>_to_fullcalendar()</code>	1252–1296	Earnings → FullCalendar JSON
<code>_ma_to_fullcalendar()</code>	1299–1345	M&A → FullCalendar JSON
<code>_render_detail_panel()</code>	1421–1521	Earnings detail card
<code>_render_email_alert_preferences_inner()</code>	880–1206	Alert dialog body
<code>_filter_events_by_watchlist()</code>	1594–1598	Watchlist scope filter

— End of Earnings Calendar — Technical Documentation —

PART 15 OF 22

Screening — Technical Documentation

Multi-criteria financial screener with saved criteria and watchlists.

Continued from previous section →

Application: Market Data Portal (MDP)

Page module: `app/pages/screening.py` (~6,149 lines)

URL route: `/screening`

Registered in: `app/main.py` as `st.Page("pages/screening.py", title="Screening", url_path="screening")`

1. Overview

The Screening page implements a **Capital IQ-style progressive screener**. Users build an ordered stack of criteria (industry, geography, financial metrics, key developments, segments, estimates, forecasts). Criteria are applied sequentially to annotate a company universe; companies that fail a filter remain in the result set with **N/A** in the metric column rather than being removed (non-filtering annotation model).

Three active modes (`scr_screen_for`):

Mode	Subtitle	Results renderer
Companies	Company Screening	<code>_render_results()</code> — AG Grid
Key Devs	Key Developments Screening	<code>_render_keydevs_results()</code> — event-per-row grid
People	People Screening	<code>_render_people_results()</code> — executive compensation table

Additional modes (Equities, Fixed Income, Transactions, Projects/Portfolios) show a "coming soon" banner.

Architecture principles (from module docstring):

- All Database (DB) access delegated to `data.screening_service`
- All metric/table mappings in `data.screening_config`
- Session keys prefixed `scr_` to avoid collisions
- Criteria stack fingerprinting for cache invalidation
- Watchlists pre-filter universe before criteria run

2. URL, entry point, and dependencies

URL: `/screening` — no query-parameter deep linking; all state lives in `st.session_state` and optional saved-criteria database records.

Entry point:

```
def main():
    new_rerun_id("screening")
    _init_state()
    _flush_pending_toasts()
    _ensure_db_tables()
    _render_page_header()
    _render_screen_for()
    # ... mode-specific UI ...
    render_coresight_footer(full_width=True, stick_to_bottom=True)

main() # module bottom
```

Core / auth

Module	Symbols	Role
<code>core.auth_manager</code>	<code>require_auth</code> , <code>get_current_user</code> , <code>get_auth_data</code>	Identity via <code>main.py</code> OIDC cookie hydrate

Data — configuration

Module	Symbols	Role
<code>data.screening_config</code>	<code>OPERATORS</code> , <code>STATEMENT_CONFIG</code> , <code>PERIOD_TYPES</code> , <code>SCREENING_YEARS</code> , <code>FORWARD_SCREENING_YEARS</code> , <code>KEYDEV_CATEGORIES_ALL</code> , <code>get_metric_labels</code> , <code>get_metric_info</code> , <code>DB_SCALE</code>	Single source of truth for metrics, tables, operators

Data — service layer

Module	Symbols	Role
<code>data.screening_service</code>	<code>recompute_working_set</code> , <code>get_base_company_universe</code> , <code>apply_*_criterion</code> , <code>criteria_stack_fingerprint</code> , <code>segment_cache_builders</code> , key devs fetchers	All screening SQL and pipeline logic
<code>data.saved_criteria_service</code>	<code>save_criteria</code> , <code>get_user_criteria</code> , <code>update_criteria</code> , <code>delete_criteria</code> , <code>grant/revoke</code> access	Saved screening sets
<code>data.watchlist_service</code>	Create, Read, Update, Delete (CRUD) + access grants on watchlists	Watchlist persistence
<code>data.portal_users_service</code>	Portal user list	Access grant user picker
<code>data.repository</code>	<code>CompanyRepository</code> , <code>KeyStatsRepository</code> , <code>RatiosRepository</code> , <code>SegmentDataRepository</code> , <code>ExecutiveCompensationRepository</code>	Universe + tabular data + people mode
<code>data.source_router</code>	<code>get_company_source</code>	U.S. Securities and Exchange Commission (SEC) vs YFinance (service uses <code>companies_map["source"]</code> in hot path)
<code>data.segment_aliases</code>	<code>GEO_CANONICAL_GROUPS</code> , <code>canonicalize_geo_label</code> , <code>expand_geo_canonical_segments</code>	Geo segment normalization
<code>data.models</code>	<code>get_fiscal_quarter</code> , <code>parse_fiscal_year_end</code>	Fiscal Quarter (FQ) screening

UI components

Module	Symbols	Role
<code>components.styles</code>	<code>hide_sidebar</code> , <code>set_page_layout</code> , <code>render_styles</code>	Layout
<code>components.navigation</code>	<code>render_header</code> , <code>render_coresight_footer</code>	Chrome

Note: Most UI (~6k lines) is inline in `screening.py` — large `_SCREENING_CSS` , dialogs, AG Grid config, Excel builders. Only styles and navigation are shared components.

Third-party

- `streamlit` , `pandas` , `openpyxl`
- `st_aggrid` (optional; `HAS_AGGRID` flag) — filterable grid with custom `SourceLinkCellRenderer`
- `streamlit.components.v1.html` — JS Excel download

Utils

```
utils.server_logger : new_rerun_id , log_structured_error , log_error , log_timing , log_info
```

4. UI Components (in-page)

4.1 Layout regions (top → bottom)

1. **Page header** — Coresight branding; title from `_SCREEN_TITLES[scr_screen_for]`
2. **Screen For radio** — horizontal mode selector; switching resets criteria via `_reset_criteria()`
3. **Watchlist bar** — dropdown + Edit/Manage (`_render_watchlist_bar`)
4. **Dialogs** (one at a time): watchlist manager OR browse saved criteria
5. **Criteria palette** — Industry, Geography, Financial, Key Devs + "Saved Screenings"
6. **Active criterion form** — expander per type (`_render_criterion_form`)
7. **Status bar** — universe / working-set count (`_render_status_bar`)
8. **Active criteria stack** — cards with edit/remove (`_render_active_criteria`)
9. **Results** — mode-specific grid/table + Excel

4.2 Criteria palette (`_CRITERIA_OPTIONS`)

Label	type key
Industry Classifications	industry
Geographic Locations	geography
Financial Information	financial
Key Developments by Category	keydevs

4.3 Financial form (`_render_financial_form`)

Multi-step wizard:

1. Statement type (from `STATEMENT_CONFIG` keys + Key Stats, Ratios, Segments, Estimates, Forecasting)
2. Metric selectbox
3. Period type: Fiscal Year (FY), Calendar Quarter (CQ), FQ, TQ (trailing quarters)
4. Year / quarter selectors (single, range, trailing N)
5. Operator + threshold values (\$mm, %, or x per metric unit)
6. Optional segment member multiselect (Business / Geographical Segments)
7. Optional **Additional Data**: Credit Ratings (S&P), Store Counts (hidden child criteria)

Widget keys: `scr_fin_stmt_sel` , `scr_fin_metric_sel` , `scr_fin_period_type_sel` , `scr_fin_year_sel` , `scr_fin_quarter_sel` , `scr_fin_op_sel` , `scr_fin_val1` , `scr_fin_val2` , etc.

4.4 Dialogs

Dialog	Function	Features
Watchlist manager	<code>_dialog_watchlist_manager</code>	Create/edit name, description, companies, sectors; per-row delete; access grants
Browse saved criteria	<code>_dialog_browse_saved_criteria</code>	Load, edit, delete, view detail panel
Save criteria	<code>_dialog_save_criteria</code>	Name + description for current stack

4.5 Results grid (`_render_filterable_results_grid`)

- **AG Grid** when `HAS_AGGRID` : text filters, pinned columns (Company Name, Ticker, etc.)
- **Fallback**: `st.dataframe`
- Row selection → **Save as Watchlist** panel (`_render_save_as_watchlist_panel`)
- Segment criteria → expanded table (`_render_segment_expanded_table`) + dedicated Excel

4.6 Loading overlay

`_render_coresight_loading_overlay` — full-viewport Coresight logo + spinner during Show Results recompute (0.6s paint delay via `scr_results_pending_paint`).

4.7 Excel export

`_render_excel_download` → `_render_excel_js_download` (client-side Blob, same pattern as Market Data).

Mode	Builder	Filename
Companies (flat)	<code>_build_screening_excel</code>	<code>Screening_Results_{timestamp}.xlsx</code>
Companies (segments)	<code>_build_segment_excel</code>	same
People	inline openpyxl	<code>People_Screening_{timestamp}.xlsx</code>
Key Devs	keydevs Excel builder	event export

Module-level `_EXCEL_BUILD_CACHE` — MD5 signature keyed, bounded size.

5. Session State

All keys use `scr_` prefix unless noted.

5.1 Core (`_init_state` defaults)

Key	Type	Purpose
<code>scr_screen_for</code>	str	"Companies" default
<code>scr_active_criteria</code>	list[dict]	Ordered criterion stack
<code>scr_working_df</code>	DataFrame None	Computed universe + annotation columns
<code>scr_debug_trace</code>	list[dict]	Per-criterion timing/metadata
<code>scr_active_form</code>	str None	Open palette form type
<code>scr_show_results</code>	bool	Results section visible
<code>scr_prefill</code> / <code>scr_prefill_idx</code>	dict / int	Edit-mode prefill
<code>scr_fin_stmt</code> , <code>scr_fin_metric</code> , <code>scr_fin_operator</code> , <code>scr_fin_timeframe</code>		Financial form sub-state
<code>scr_geo_available</code>	bool None	Geography column check
<code>scr_criterion_cache</code>	dict	Per-criterion result cache
<code>scr_active_watchlist_id</code> / <code>scr_active_watchlist_name</code>		Active watchlist scope
<code>scr_watchlist_tickers</code>	set None	Ticker set (back-compat)
<code>scr_watchlist_members</code>	list[(ticker, name)]	Composite identity
<code>scr_watchlist_count</code>	int None	True company count
<code>scr_db_tables_ready</code>	bool	DDL bootstrap once
<code>scr_results_loading</code>	bool	Show Results in progress
<code>scr_results_pending_paint</code>	bool	Overlay paint delay
<code>scr_show_results_requested</code>	bool	
<code>scr_results_error</code>	str None	Last pipeline error
<code>scr_criteria_fingerprint</code>	str None	Current stack hash
<code>scr_last_computed_fingerprint</code>	str None	Last successful compute hash

5.2 Watchlist dialog keys

`wl_dlg_open` , `wl_dlg_mode` , `_wl_dlg_preselect` , `wl_dlg_select` , `wl_create_*` , `wl_add_co_ms_{id}`

5.3 Saved criteria dialog keys

`scr_saved_dlg_open` , `scr_dlg_editmode_{cid}` , `scr_dlg_confirm_del_{cid}`

5.4 Fallback store

`_mem_wl_data` — in-memory watchlist when DB unreachable (offline fallback)

6. Data Layer

6.1 `screening_config.py`

Operators: Greater Than, Less Than, Between, Equals, GTE, LTE → `OPERATOR_SQL` map (Between handled separately).

Period types: `FY` , `CQ` , `FQ` , display-only `TQ` (trailing quarters: 4, 8, 12).

Scale: `DB_SCALE = 1_000_000` — UI thresholds in \$mm; SQL compares raw DB units.

STATEMENT_CONFIG — per statement type:

Field	Meaning
<code>sec_table / yf_table</code>	MySQL table names
<code>sec_date_col , sec_period_col , sec_period_value</code>	SEC column mapping
<code>yf_date_col , yf_period_col , yf_period_value</code>	Yahoo Finance (YF) long-format mapping
<code>metrics[]</code>	<code>{label, sec_col, yf_item, unit}</code>

Statement types in config:

- Income Statement, Balance Sheet, Cash Flow
- Key Stats (`tabular_market_data: True` → repository)
- Ratios
- Business Segments, Geographical Segments (`segment_statement: True`)
- Estimates, Forecasting (`forward-looking` year lists)

Key developments: `KEYDEV_CATEGORIES_ALL` (24 Excel-aligned DB categories), `KEYDEV_TIMEFRAMES` , alias maps for legacy migration.

6.2 `screening_service.py` — key functions

Function	Role
<code>get_base_company_universe()</code>	All companies from <code>CompanyRepository.get_companies_rows()</code>
<code>filter_universe_to_members()</code>	Watchlist pre-filter by (<code>ticker, company_name</code>)
<code>recompute_working_set()</code>	Full progressive pipeline
<code>apply_industry_criterion / apply_geography_criterion</code>	In-memory annotation
<code>apply_financial_criterion</code>	SEC/YF SQL + operator filter
<code>apply_key_stats_criterion / apply_ratios_criterion</code>	Repository-backed tabular
<code>apply_segment_statement_criterion</code>	Segment cache reads
<code>apply_estimates_criterion / apply_forecast_criterion</code>	Forward-looking tables
<code>apply_keydevs_criterion</code>	<code>coreiq_company_events</code> filter
<code>build_segment_values_cache</code>	Offline cache population
<code>get_keydevs_events_for_tickers</code>	Key Devs mode event fetch
<code>criteria_stack_fingerprint</code>	Invalidation hash

6.3 `segment_aliases.py`

- `GEO_CANONICAL_GROUPS` — United States, Canada, UK, North America, Non-US, Americas
- `canonicalize_geo_label(raw)` → canonical display label
- `expand_geo_canonical_segments(canonical)` → all raw aliases for SQL `IN` clauses
- `GEO_PENSION_SKIP_LABELS` — exclude pension rows from geo revenue
- `GEO_SUPPRESSED_DROPDOWN_LABELS` — hide duplicate raw variants in UI

6.4 source_router.py

```
get_company_source(ticker) -> 'SEC' | 'YFinance' | None
```

Composite tickers: `TSCO.L` → `ticker='TSCO'`, `exchange_acronym='L'`.

Screening service splits ticker lists by `companies_map[ticker]["source"]` and runs parallel SEC/YF queries in `ThreadPoolExecutor`.

6.5 Repository methods (screening-related)

Class	Methods
<code>CompanyRepository</code>	<code>get_companies_rows()</code> , <code>get_companies_map()</code> , <code>get_all_sectors()</code>
<code>KeyStatsRepository</code>	Tabular key stats for screening
<code>RatiosRepository</code>	Tabular ratios
<code>SegmentDataRepository</code>	<code>_classify_segment_rows</code> (shared with cache build)
<code>ExecutiveCompensationRepository</code>	<code>get_sec_all_compensation()</code> , <code>get_yf_all_compensation()</code>

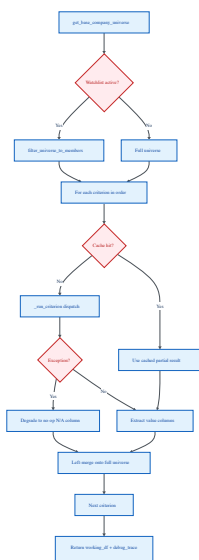
`_get_fiscal_year_end_cached(ticker)` — FQ calendar alignment (1800s TTL).

7. Business Logic

7.1 Progressive pipeline (`recompute_working_set`)

Screening progressively filters the company universe — each active criterion narrows the working set.

Screening progressively filters the company universe — each active criterion narrows the working set.



Non-filtering merge: Even if `apply_financial_criterion` returns a filtered subset internally, the pipeline extracts new columns and `merge(how="left")` onto the full universe. Failed companies show **N/A**.

7.2 Criterion dispatch (`_run_criterion` by `type`)

type	Handler
industry	apply_industry_criterion
geography	apply_geography_criterion
financial	Routed by <code>statement</code> (see below)
keydevs	apply_keydevs_criterion
biz_segments / geo_segments	Legacy segment filters
additional	apply_additional_criterion (credit ratings, store counts)

Financial `statement` routing:

Statement	Function
Income / Balance / Cash Flow	apply_financial_criterion
Key Stats	apply_key_stats_criterion
Ratios	apply_ratios_criterion
Business / Geographical Segments	apply_*_segments_statement_criterion
Estimates	apply_estimates_criterion
Forecasting	apply_forecast_criterion

7.3 Financial criterion dict shape

```
{
  "type": "financial",
  "statement": "Income Statement",
  "metric_info": {...},      # from screening_config
  "metric_label": "Total Revenue",
  "operator": "Greater Than",
  "value1": 1000.0,          # $mm in UI
  "value2": 0.0,             # Between only
  "period_type": "FY",       # FY|CQ|FQ|TQ
  "quarter": "Q1",           # CQ/FQ
  "year": 2024,              # or "Latest"
  "timeframe": "FY 2024",
  "display_col": "Total Revenue FY 2024",
  "summary": "human-readable",
  # optional:
  "num_quarters": 8,          # TQ mode
  "year_range": [2020, 2024],
  "quarter_range": {...},
  "segment_type": "business",
  "selected_segments": ["North America"],
}
```

Threshold scaling: User \$mm × `DB_SCALE` before `_apply_operator`. Equals uses \$1 raw tolerance.

7.4 Show Results flow

1. User clicks **Show Results** → `scr_show_results_requested = True`

2. `_needs_results_recompute()` compares fingerprints
3. If stale: `_trigger_recompute()` → `recompute_working_set()`
4. Loading overlay + 0.6s paint delay
5. `_render_results()` reads `scr_working_df`

Add Criteria on most types triggers immediate `_trigger_recompute()` (background); financial add may defer on failure.

7.5 Watchlist CRUD

DB tables: `coreiq_watchlists`, `coreiq_watchlist_access`, `coreiq_watchlist_companies`

Unique key: `(watchlist_id, ticker, company_name)` — disambiguates shared tickers (JD, TSCO, LULU).

Operation	Service function	Page wrapper
Create	<code>create_watchlist</code>	<code>wl_create</code>
List	<code>get_user_watchlists</code>	<code>wl_get_all</code>
Get companies	<code>get_watchlist_companies</code>	<code>wl_get_companies</code>
Add companies	<code>add_companies_to_watchlist</code>	<code>wl_add_companies</code>
Remove	<code>remove_company_from_watchlist</code>	<code>wl_remove_company</code>
Replace all	<code>replace_watchlist_companies</code>	<code>wl_replace_companies</code>
Update metadata	<code>update_watchlist</code>	<code>wl_update</code>
Delete	<code>delete_watchlist</code>	<code>wl_delete</code>
Grant/revoke	<code>grant_watchlist_access</code> , <code>revoke_watchlist_access</code>	<code>wl_grant_access</code> , <code>wl_revoke_access</code>

Access Control List (ACL) model: All watchlists visible to logged-in users; `is_owner`, `can_edit`, `can_delete` per row. Grants via `coreiq_watchlist_access`.

In-memory fallback: `_InMemoryWatchlistStore` when DB unreachable.

7.6 Saved criteria CRUD

DB tables: `coreiq_saved_criteria`, `coreiq_saved_criteria_access`

- `criteria_json` stores full `scr_active_criteria` list
- Load replaces stack + `_trigger_recompute()`
- Same owner/grant ACL model as watchlists

7.7 Fingerprints

- `_criterion_fingerprint(criterion)` — JSON excluding ephemeral fields (`summary`, `display_col`, `quarter_cols`, `year_cols`)
- `criteria_stack_fingerprint(criteria, allowed_tickers, allowed_members)` — full stack + watchlist scope

8. Structured Query Language (SQL) and Query Patterns

8.1 Company universe

```
SELECT ticker, name, name_coresight, exchange, source, exchange_acronym,
       primary_industry_coresight, country_of_incorporation
FROM coreiq_companies;
```

8.2 SEC financial latest (`_query_sec_latest`)

```
SELECT ticker, {col} AS metric_value FROM (
  SELECT ticker, {col},
    ROW_NUMBER() OVER (PARTITION BY ticker ORDER BY {date_col} DESC) AS rn
  FROM {sec_table}
  WHERE ticker IN ({sanitized_list})
    AND {period_col} = 'annual' -- or 'quarterly'
    AND {col} IS NOT NULL
) ranked WHERE rn = 1;
```

Variants: year `BETWEEN` , CQ with `QUARTER(date_col) = N` , FQ with fiscal year-end logic.

8.3 YF financial (`_query_yf_latest`)

Same `ROW_NUMBER` pattern on long-format tables:

```
WHERE line_item = '{yf_item}' AND frequency = 'annual'
```

8.4 Ticker IN safety (`_build_ticker_in_list`)

Whitelist sanitize: `[A-Za-z0-9.\-/]` , max length 20 per ticker.

8.5 Key developments

```
SELECT DISTINCT ticker FROM coreiq_company_events
WHERE ticker IN (...)
  AND event_category IN (...)
  {date_clause}
```

With headlines: `ROW_NUMBER() OVER (PARTITION BY ticker ORDER BY event_date DESC, event_id DESC)` .

8.6 Segment values cache

```
SELECT ticker, member_label, report_fiscal_year, value_mm
FROM coreiq_screening_segment_values_cache
WHERE ticker IN (...)
  AND segment_type = :type
  AND metric_key = :metric
```

Cache build: Bulk fetch `coreiq_filing_metrics_v5` per ticker chunk → `SegmentDataRepository._classify_segment_rows` → bulk `INSERT` .

8.7 People mode compensation

```
SELECT ticker, executive_name, position, compensation_year,
  salary, bonus, stock_awards, total_compensation
FROM coreiq_executives_compensation
WHERE ticker IN (...)
ORDER BY ticker, compensation_year DESC;
```

Parallel SEC + YF fetches via `ThreadPoolExecutor` .

8.8 Database tables summary

Table	Role
<code>coreiq_companies</code>	Universe, source, sector, country
<code>coreiq_av_financials_*</code> / <code>coreiq_yf_financials_*</code>	Financial criteria
<code>coreiq_model_forecasts</code>	Forecasting criteria
<code>coreiq_av_financials_earnings_estimates</code> / YF equivalent	Estimates
<code>coreiq_filing_metrics_v5</code>	Segment source + additional data
<code>coreiq_company_events</code>	Key developments
<code>coreiq_executives_compensation</code>	People mode
<code>coreiq_screening_segment_values_cache</code>	Precomputed segment metrics
<code>coreiq_screening_segment_member_cache</code>	Segment dropdown members
<code>coreiq_watchlists</code> , <code>coreiq_watchlist_companies</code> , <code>coreiq_watchlist_access</code>	Watchlists
<code>coreiq_saved_criteria</code> , <code>coreiq_saved_criteria_access</code>	Saved screenings

9. Caching

9.1 Page-level (`screening.py`)

Function	TTL	Purpose
<code>_cached_user_criteria</code>	30s	Saved criteria list
<code>_cached_user_watchlists</code>	30s	Watchlist list
<code>_cached_portal_users</code>	300s	Access grant picker

Cleared via `_invalidate_criteria_cache()` / `_invalidate_watchlist_cache()` on writes.

9.2 Service-level (`screening_service.py`)

Data	TTL
<code>_get_segment_names_cached</code>	1800s
<code>get_all_countries</code>	3600s
<code>read_segment_member_options_cache</code>	300s
<code>get_keydevs_events_for_tickers</code>	300s
Estimates/forecast/additional helpers	1800s

9.3 Repository

Function	TTL
<code>get_companies_rows()</code>	3600s
<code>get_companies_map()</code>	3600s (derived)
<code>_get_fiscal_year_end_cached</code>	1800s

9.4 Session-level

- `scr_criterion_cache` — keyed by `(criterion_fingerprint, frozenset(universe_tickers))`
- `_EXCEL_BUILD_CACHE` — module-level MD5 keyed
- Watchlist service: 120s session TTL per user email

9.5 Startup prewarm (`main.py`)

Background thread ensures watchlist, saved_criteria, portal_users DDL before first visit.

10. Error Handling

Pattern	Behavior
<code>try/except</code> + <code>log_structured_error</code>	All major functions
User generic	<code>"Something went wrong. Please try again."</code>
Results failure	<code>"Could not load screening results. Please check logs."</code>
Segment cache miss	<code>SEGMENT_CACHE_UNAVAILABLE_MESSAGE</code> warning; background rebuild
Single criterion failure	Degrade to no-op; universe preserved
Watchlist DB down	<code>_InMemoryWatchlistStore</code> fallback
<code>DEBUG</code> enabled	Technical details in expander (diagnostics)
Toasts	<code>_queue_toast</code> / <code>_flush_pending_toasts</code> survive reruns

11. ACL (Access Control)

11.1 Page access

- Identity via `main.py` OIDC cookie hydrate; optional `require_auth(page="screening")` at page level
- No `coreiq_page_access_control` check for `"screening"`

11.2 Watchlist ACL

- All active watchlists visible to authenticated users
- Owner: full edit/delete; grants via `coreiq_watchlist_access` (`can_edit` , `can_delete`)
- UI disables actions when permissions false

11.3 Saved criteria ACL

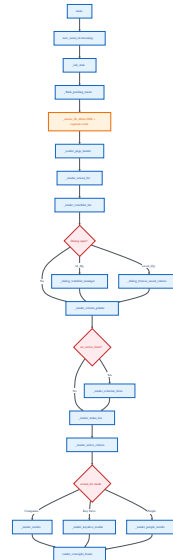
- Same owner/grant model as watchlists
- Non-owners get read-only **View** in browse dialog

12. Flowcharts

12.1 Page load

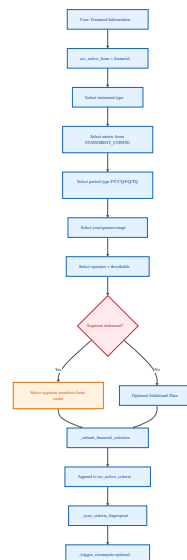
End-to-end user journeys on this page, from load through interaction to data fetch.

End-to-end user journeys on this page, from load through interaction to data fetch.



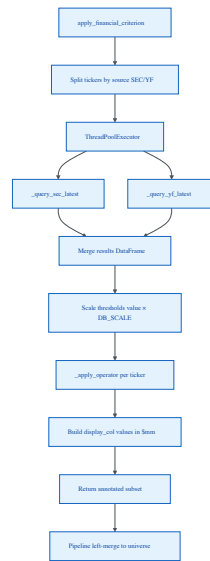
12.2 Criteria build (financial)

12.2 Criteria build (financial)



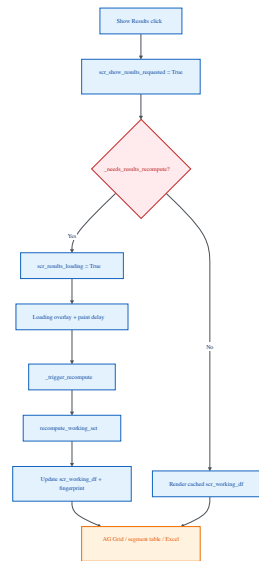
12.3 SQL execution (financial criterion)

12.3 SQL execution (financial criterion)



12.4 Show Results / recompute

12.4 Show Results / recompute



12.5 Watchlist CRUD

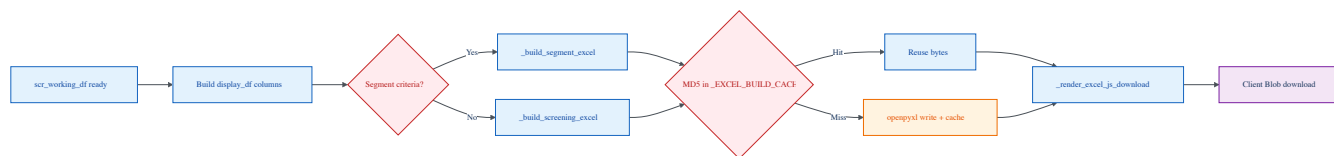
12.5 Watchlist CRUD



12.6 Export flow

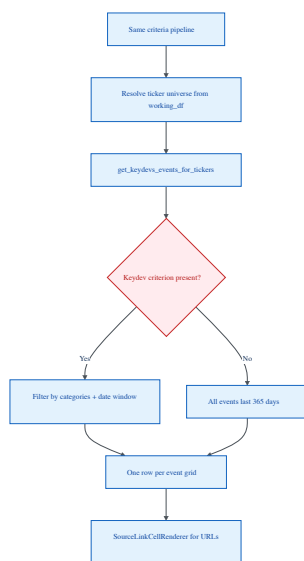
Export builds a CSV from the current working set and triggers a browser download.

Export builds a CSV from the current working set and triggers a browser download.



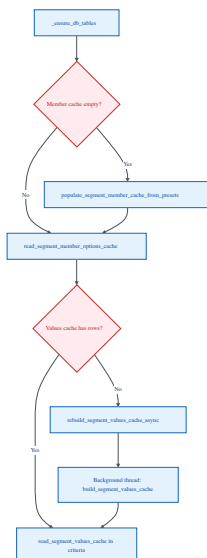
12.7 Key Devs mode

12.7 Key Devs mode



12.8 Segment cache lifecycle

12.8 Segment cache lifecycle



13. Feature reference

13.1 Companies mode

- Progressive criteria stack with non-filtering N/A columns
- Industry (in-memory sector match), Geography (in-memory country match)
- Financial: all statement types in `STATEMENT_CONFIG` + Key Stats, Ratios, Segments, Estimates, Forecasting
- Key Devs criterion annotates/filter-marks companies with matching events
- AG Grid with filters; row selection → watchlist save
- Segment expanded view + dedicated Excel when segment criteria active

13.2 Key Devs mode

- Reuses company universe from criteria pipeline
- One row per event (not per company)
- Columns: date, type, company name(s), headline, summary, sources, link
- Default: last 365 days, all categories if no keydev criterion

13.3 People mode

- Industry + geography (+ optional financial) narrow universe
- All historical executive compensation (SEC detail + YF total pay)
- Year filter dropdown; dynamic money columns
- Excel sheet "People"

13.4 Saved Screenings

- Save/load/edit/delete criteria sets with sharing grants
- Detail panel with per-filter inline edit

13.5 Segment screening specifics

- Requires `coreiq_screening_segment_values_cache`

- Geo members canonicalized via `segment_aliases`
- Metrics: Revenues, Operating Profit Before Tax, Assets, D&A, Capex
- Admin cache status in financial form; async rebuild on empty cache

Related configuration files

File	Do not duplicate logic in page
<code>app/data/screening_config.py</code>	All metric → table/column mappings
<code>app/data/screening_service.py</code>	All SQL and pipeline logic
<code>app/data/segment_aliases.py</code>	Geo canonical groups only
<code>app/data/watchlist_service.py</code>	Watchlist persistence
<code>app/data/saved_criteria_service.py</code>	Saved criteria persistence

Generated from source analysis of `app/pages/screening.py` and traced dependencies. Last reviewed against codebase structure as of project documentation pass.

— End of Screening — Technical Documentation —

PART 16 OF 22

Company Filings — Technical Documentation

SEC filing browser with XBRL metrics and Azure blob PDF retrieval.

Continued from previous section →

Application: Market Data Portal (MDP)

Page module: `app/pages/company_filings.py`

URL: `/company_filings`

Registered in: `app/main.py` as `( "pages/company_filings.py", "Company Filings", "company_filings", False )`

Product name: Company Filing Documents (Market Data Portal)

1. Purpose and scope

The Company Filings page is the primary SEC and non-SEC filing browser for the portal. It combines:

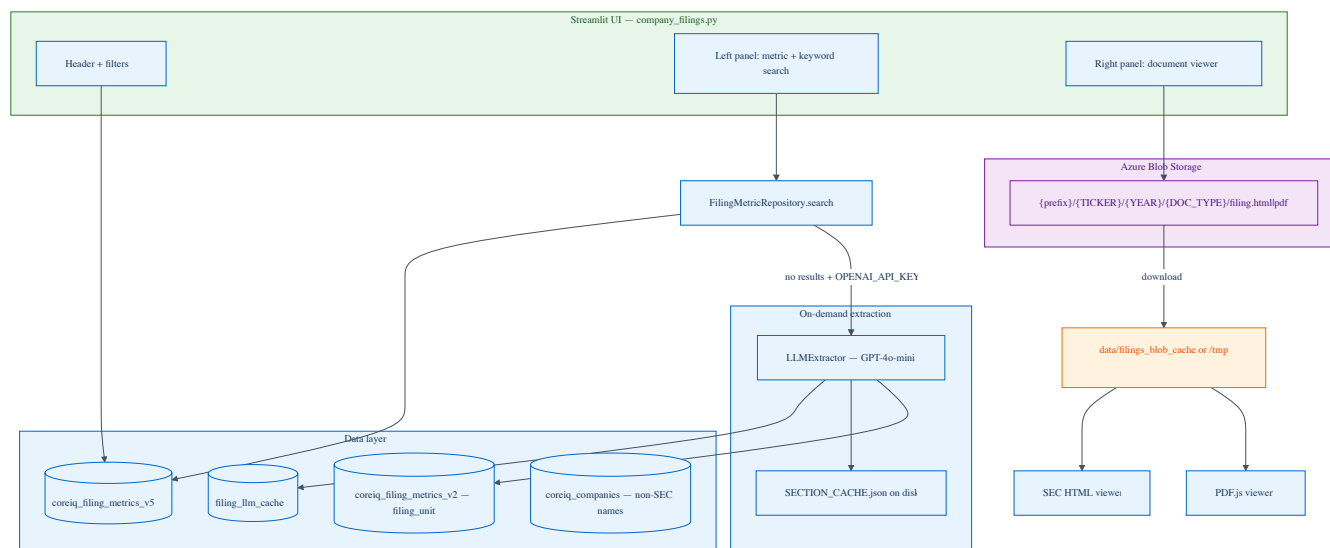
- **Filter-driven navigation** — company, document type, optional filing unit (8-K / 6-K / proxy), fiscal/storage year
- **Metric search** — pre-ingested eXtensible Business Reporting Language (XBRL) metrics from MySQL (`coreiq_filing_metrics_v5`)
- **Document viewing** — SEC ieXtensible Business Reporting Language (XBRL) HTML in a sandboxed iframe; non-SEC PDFs via PDF.js
- **Keyword search** — full-text hits in the loaded document (HTML or PDF)
- **LLM fallback** — GPT-4o-mini extraction when DB search returns no rows (`app/core/llm_extractor.py`)
- **Azure Blob Storage** — source of truth for filing HTML/PDF files

The page does **not** call `edgartools` at runtime. eXtensible Business Reporting Language (XBRL) data is ingested offline into `coreiq_filing_metrics_v5`; metrics with `source='edgartools'` are displayed with an EDGAR badge when present in the DB.

2. High-level architecture

Filing viewer architecture: SEC eXtensible Business Reporting Language (XBRL) path, non-SEC blob PDFs, and LLM fallback extraction.

Filing viewer architecture: SEC eXtensible Business Reporting Language (XBRL) path, non-SEC blob PDFs, and LLM fallback extraction.



3. UI layout and controls

3.1 Page structure

Region	Width	Contents
Global header	Full width	<code>render_header(..., current_page="company_filings")</code>
Title row	1/3 + 2/3	"Company Filing Documents" + four filter dropdowns
Main content	30% / 70%	Search Metrics (left) + Document Viewer (right)
Footer	Full width	<code>render_coresight_footer(full_width=True, stick_to_bottom=True)</code>

There are **no Streamlit tabs** on this page. Navigation is entirely via dropdowns plus the search panel.

3.2 Filter dropdowns

#	Widget key	Session key	Callback	Data source
1	<code>cf_company_select</code>	<code>cf_company</code> , <code>active_ticker</code>	<code>_on_company_change</code>	<code>_load_companies_from_db()</code> → <code>coreiq_filing_metrics_v5</code> + non-SEC blob scan
2	<code>cf_doc_type_select</code>	<code>cf_doc_type</code>	<code>_on_doc_type_change</code>	<code>_get_available_doc_types_from_db(ticker)</code>
3	<code>cf_filing_unit_select</code>	<code>cf_filing_unit</code>	<code>_on_filing_unit_change</code>	<code>get_filing_units()</code> — only for <code>FILING_UNIT_DOC_TYPES</code>
4	<code>cf_year_select</code>	<code>cf_year</code>	<code>_on_year_change</code>	<code>_get_available_years_for_doc_type(ticker, doc_type)</code>

Filing unit doc types (`app/utils/filing_units.py`): `8-K`, `6-K`, `DEF14A`, `DEFA14A`, `DEF 14A`, `PRE14A`.

Display numbers (1, 2, 3...) map to sorted `filing_N` folders; default is the **highest** filing number (most recent).

Year display: `format_func` uses `_fiscal_label_for()` to show DEI fiscal year while plumbing uses **storage/bucket year** (`COALESCE(storage_year, report_fiscal_year, fiscal_year)`).

3.3 Deep links (query params)

Param	Example	Effect
<code>ticker</code>	<code>?ticker=AAPL</code>	Resolved via <code>validate_and_get_ticker()</code>
<code>doc_type</code>	<code>?doc_type=10-Q-Q1</code>	Sets <code>cf_doc_type</code> if in <code>VALID_DOC_TYPES</code>
<code>year</code>	<code>?year=2025</code>	Sets <code>cf_year</code> if 4-digit

`st.query_params["ticker"]` is kept in sync when the user changes the company dropdown.

3.4 Left panel — Search Metrics

- Text input** — key `cf_search_input_{cf_search_gen}`; placeholder "eg., Revenue"
- Metric results** — cards from `FilingMetricRepository.search()`; each has **View in Document**
- Keyword hits** — `_kw_search_document()` when local file exists; separate **KEYWORD** badge section

Source badges on metric cards:

source value	Badge
calculated	CALC (blue)
llm	AI (green)
edgartools	EDGAR (orange)

3.5 Right panel — Document viewer

- Filing header** — company name, DEI fiscal year, period chip, filing date, fiscal period label (`FilingMetricRepository.get_header_metadata`)
- Download button** — SEC: `utils.html_to_pdf.generate_filing_pdf` ; non-SEC: raw PDF bytes
- Viewer** — `render_sec_html_viewer()` or `_render_pdf_viewer()`

4. Session state reference

Key	Type	Purpose
<code>cf_company</code>	str	Selected ticker
<code>cf_doc_type</code>	str	DB doc type (e.g. <code>10-Q-Q1</code> , <code>annual-report</code>)
<code>cf_year</code>	str	Storage/bucket year for blob path + DB filter
<code>cf_filing_unit</code>	str	e.g. <code>filing_94</code> for multi-filing docs
<code>cf_search</code>	str	Current search term
<code>cf_search_gen</code>	int	Incremented to reset search widget on filter change
<code>cf_highlight_fact_id</code>	str None	eXtensible Business Reporting Language (XBRL) element id, <code>TEXT:...</code> , or <code>PDFPAGE:n:hit</code>
<code>cf_view_metric</code>	str None	Display label for highlight banner
<code>cf_company_select</code>	str	Widget key for company selectbox
<code>cf_doc_type_select</code>	str	Widget key for doc type
<code>cf_year_select</code>	str	Widget key for year
<code>cf_filing_unit_select</code>	int	1-based display index for filing unit
<code>active_ticker</code>	str	Global ticker used by header
<code>_highlight_just_set</code>	bool	Prevents clearing highlight on view-button rerun
<code>_view_button_click_time</code>	float	Performance timing for autoscroll
<code>_kw_highlight_keyword</code>	str	Keyword for PDF highlight overlay
<code>pdf_bytes_{ticker}_{year}_{doc_type}</code>	bytes	Generated PDF awaiting download
<code>pdf_ready_{...}</code>	bool	Triggers auto-download on next rerun
<code>pdf_b64_{path}</code>	str	Base64 PDF cached for PDF.js component
<code>_TIMING_DATA</code>	dict	Internal perf: reruns, searches, company changes

Filter change behavior: When company, doc type, year, or filing unit changes, search is cleared and highlight state reset (unless `_highlight_just_set`).

5. Components and utilities

5.1 UI components (`app/components/`)

Import	Usage
<code>styles.hide_sidebar</code>	Module load
<code>styles.render_styles</code> , <code>set_page_layout</code>	Page chrome
<code>navigation.render_header</code>	Top nav with ticker
<code>navigation.render_coresight_footer</code>	Bottom footer
<code>loading.inject_red_spinner_css</code>	Brand spinner (#D62E2F) when cache available

5.2 Domain utilities

Module	Role
<code>utils.ticker_utils</code>	<code>validate_and_get_ticker</code> , <code>DEFAULT_FALLBACK_TICKER</code>
<code>utils.filing_units</code>	Filing unit list, blob path builder, <code>FILING_UNIT_DOC_TYPES</code>
<code>utils.filings_cache</code>	DiskCache metadata (<code>get_filings_cache</code> , ETag)
<code>utils.background_scanner</code>	Background blob scan, <code>ensure_cache_purge</code>
<code>utils.non_sec_blob_fallback</code>	Parallel Azure scan for non-SEC tickers
<code>utils.html_to_pdf</code>	SEC HTML → branded PDF (PyMuPDF)
<code>utils.server_logger</code>	Structured logging (<code>log_structured_error</code> , etc.)

5.3 Data layer

Module	Role
<code>data.repository.FilingMetricRepository</code>	Metric search, header metadata, synonym expansion
<code>data.models.FilingMetricResult</code>	Search result dataclass
<code>core.llm_extractor.LLMExtractor</code>	On-demand LLM extraction
<code>core.database.db_manager</code>	Read-only queries, SQLAlchemy engine for LLM writes

5.4 Azure helpers (in-page)

`company_filings.py` implements its own Azure Blob client (not `app/utils/azure_blob.py`) with performance tuning:

- `_get_azure_config()` — env: `AZURE_STORAGE_ACCOUNT_NAME` , `AZURE_STORAGE_ACCOUNT_KEY` , `AZURE_BLOB_CONTAINER` , `AZURE_BLOB_PREFIX`
- `_get_optimized_blob_service_client()` — 64MB single-get, 8MB chunks, pool size 50
- `_ensure_local_blob_optimized()` — parallel download `max_concurrency=4` , atomic temp→rename
- `_local_cache_path_for_blob()` — maps to `data/filings_blob_cache/{relative_path}`

`app/utils/azure_blob.py` mirrors cache layout for **other pages** (e.g. earnings transcripts) and is documented here for path consistency only.

6. Database tables

6.1 `coreiq_filing_metrics_v5` (primary)

Role: Company list, year/doc_type filters, metric search, filing header metadata.

Column (conceptual)	Usage on page
<code>ticker</code> , <code>company_name</code>	Company dropdown labels
<code>doc_type</code>	Document type filter
<code>storage_year</code>	Physical Azure folder year (blob path)
<code>fiscal_year</code> , <code>report_fiscal_year</code>	DEI display labels
<code>filing_unit</code>	8-K/6-K sub-filings
<code>original_label</code> , <code>standard_concept</code> , <code>dimension_label</code>	Search targets
<code>numeric_value</code> , <code>unit_ref</code> , <code>value</code>	Display + formatting
<code>ixbrl_id</code>	HTML autoscroll target (<code>document.getElementById</code>)
<code>source</code>	<code>xbrl</code> , <code>calculated</code> , <code>llm</code> , <code>edgartools</code> , <code>store_count</code> , <code>credit_rating</code>
<code>llm_query</code>	Formula string (calculated) or JSON detail (store/credit)
<code>period_*</code> , <code>statement_type</code>	Card metadata
<code>filing_date</code> , <code>filing_date_sec</code>	Header metadata

Indexes referenced in code: `idx_ticker_fiscal_doctype` (covering prefetch).

6.2 `coreiq_filing_metrics_v2`

Used by:

- `filing_units.get_filing_units_from_db()` — `DISTINCT filing_unit` for 8-K/6-K
- `llm_extractor.insert_llm_metric()` — inserts new LLM rows with `source='llm'`

Runtime search reads **v5**; LLM writes go to **v2** (legacy split).

6.3 `filing_llm_cache`

Column	Purpose
<code>ticker</code> , <code>fiscal_year</code> , <code>doc_type</code> , <code>query_normalized</code>	Cache key
<code>status</code>	<code>found</code> <code>not_found</code>
<code>metric_id</code>	Link to inserted v2 row
<code>searched_at</code>	Timestamp

6.4 `coreiq_companies`

Non-SEC company names via `get_non_sec_tickers_from_db()` in `utils.non_sec_blob_fallback` .

6.5 `coreiq_ir_websites` / `company_ir_websites`

Not used directly on this page. IR URLs are resolved in `EarningsCalendarRepository._build_ir_website_lookup()` (`app/data/repository.py`) for the earnings calendar. Documented here because filing metrics pipeline and IR discovery share the same company universe.

7. Document types and Azure blob layout

7.1 VALID_DOC_TYPES

SEC: 10-K, 10-Q-Q1 ... 10-Q-Q3, 8-K, 20-F, 6-K, DEF14A, DEFA14A, DEF 14A, PRE14A

Non-SEC (Capital Q): annual-report, interim-report-Q1 ... Q5, half-yearly

7.2 Blob path conventions

```
{AZURE_BLOB_PREFIX}/{TICKER}/{YEAR}/{DOC_TYPE}/filing.html # SEC default
{AZURE_BLOB_PREFIX}/{TICKER}/{YEAR}/{DOC_TYPE}/filing_94/filing.html # 8-K unit
{AZURE_BLOB_PREFIX}/{TICKER}/{YEAR}/{annual-report}/filing.pdf # non-SEC
```

Parser: `_parse_blob_path()` expects at least 4 path segments after prefix.

Scanner: `scan_filings_directory()` groups blobs by `(ticker, year, doc_type_dir)` and picks best file via `_pick_best_blob_for_docdir()` (prefers `filing.html` / `filing.pdf`).

7.3 SEC vs non-SEC routing

Routing is by **doc_type**, not company flag:

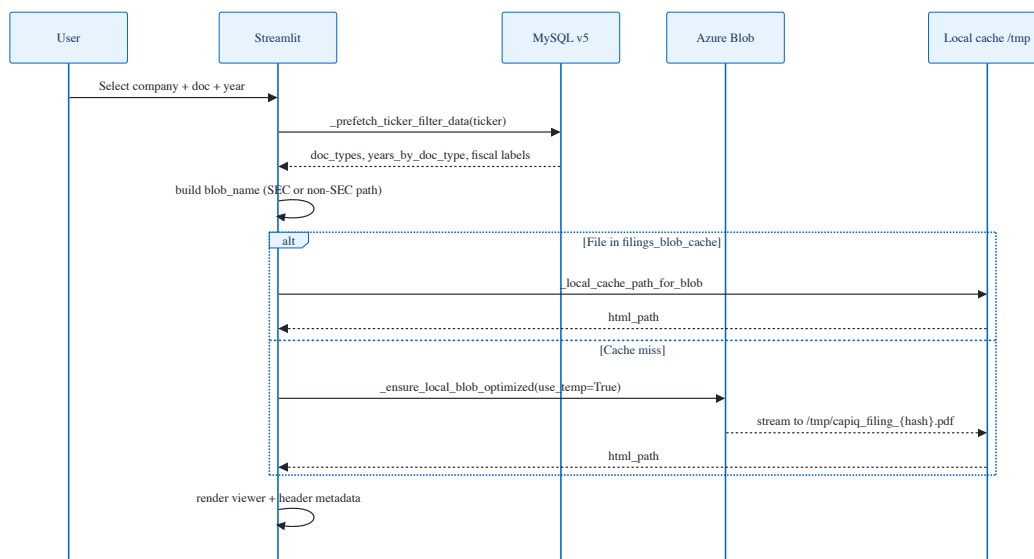
Doc type set	File	Viewer
NON_SEC_DOC_TYPES	filing.pdf	_render_pdf_viewer (PDF.js)
All other valid SEC types	filing.html	render_sec_html_viewer

10-Q storage year fallback: If blob missing at `{year}`, retries `{year-1}` for non-December fiscal year ends (e.g. LULU).

8. Filing load flow

Selecting a filing loads metrics from XBRL tables, with optional LLM extraction on cache miss.

Selecting a filing loads metrics from XBRL tables, with optional LLM extraction on cache miss.



9. XBRL / iXBRL handling

9.1 Ingestion (offline)

XBRL facts are extracted during ETL into `coreiq_filing_metrics_v5` with `source='xbrl'`. Calculated metrics use `source='calculated'`. Some supplemental metrics use `edgartools` during batch jobs (see `app/data/repository.py` `segment/ratings` paths) — not invoked from this page.

9.2 iXBRL HTML processing (runtime)

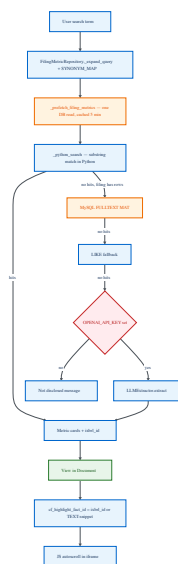
`_load_and_process_html()` pipeline:

1. Read UTF-8 from local path
2. Strip XML declaration (force HTML5 mode)
3. Remove `<script>` tags (malformed SEC JS breaks iframe)
4. `_convert_ixbrl_to_spans()` — namespace tags → `<span>` with preserved `id` for `ixbrl_id` navigation
5. Inject `content-visibility: auto` CSS for lazy rendering
6. `_resolve_sec_edgar_base_url()` — rewrite relative links to SEC EDGAR absolute URLs
7. Inject click handler: hash links scroll in-iframe; external links open new tab

9.3 XBRL extract / metric search flow

Metric search checks filing tables first, then may invoke GPT-4o-mini extraction for missing values.

Metric search checks filing tables first, then may invoke GPT-4o-mini extraction for missing values.



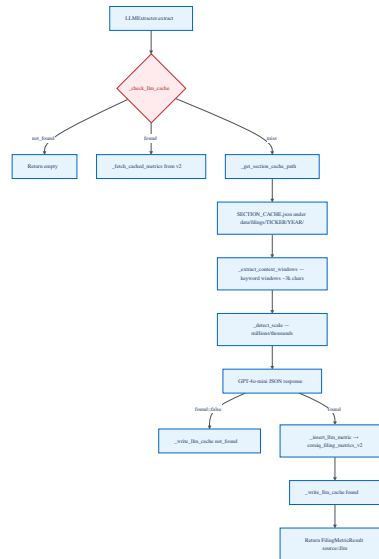
9.4 `FilingMetricRepository.search()` strategy

1. Expand query with `SYNONYM_MAP` (100+ financial terms)
2. **Fast path:** prefetch all numeric rows for `(ticker, year, doc_type)` → Python filter/dedup/sort (0ms after first call)
3. **DB fallback:** `MATCH ... AGAINST ON (original_label, standard_concept, dimension_label) then LIKE`
4. Dedup: latest `period_end` / `period_instant` per `(label, dimension, dimension_label)`
5. Exclude `standard_concept LIKE '%Text Block'`

10. LLM extraction (`app/core/llm_extractor.py`)

Triggered from `main()` when `FilingMetricRepository.search()` returns empty and `OPENAI_API_KEY` is set.

Triggered from main() when FilingMetricRepository.search() returns empty and OPENAIAPIKEY is set.



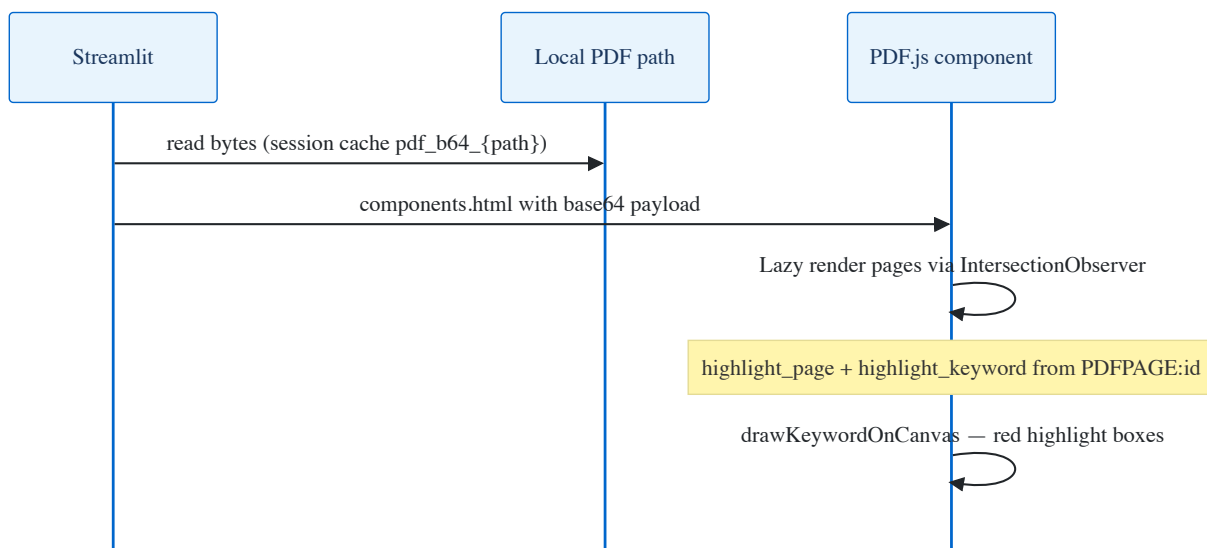
Cost note (from module docstring): ~\$0.00012 per unique query; cached queries \$0.

SECTION_CACHE.json sections prioritized: `part_ii_item_8` , `part_ii_item_7` , `part_i_item_1` , etc.

11. PDF viewing flow (non-SEC)

Non-SEC PDFs stream from Azure Blob via SAS URL embedded in the Streamlit viewer.

Non-SEC PDFs stream from Azure Blob via SAS URL embedded in the Streamlit viewer.



Why PDF.js: Chrome blocks native PDF plugins inside Streamlit's sandboxed iframe. PDF.js renders to `<canvas>` without plugins.

Text index: `_build_doc_text_index()` uses PyMuPDF (`fitz`) per page; `_kw_search_document()` returns snippets with physical + printed page labels.

12. SEC HTML → PDF download flow

SEC HTML filings are converted server-side to PDF for download via the PDF utility.

SEC HTML filings are converted server-side to PDF for download via the PDF utility.



Non-SEC downloads skip generation — existing `filing.pdf` bytes are served directly.

13. Caching and performance

Layer	Mechanism	TTL / scope
Streamlit <code>@st.cache_data</code>	Companies, prefetch, HTML process, text index, keyword search	300–3600s
Streamlit <code>@st.cache_resource</code>	Azure clients, <code>scan_filings_directory</code>	Process lifetime
<code>filings_cache</code> <code>DiskCache</code>	Per-ticker blob metadata from background scanner	Disk-backed
<code>data/filings_blob_cache/</code>	Downloaded blobs (when not <code>use_temp</code>)	Filesystem + mtime vs blob <code>last_modified</code>
<code>/tmp/capiq_filing_*</code>	On-demand downloads (<code>use_temp=True</code>)	OS-managed
Session state	PDF base64, generated PDF bytes	Per session

Load order in `main()` : Header renders first → companies load → session init → dropdowns → two-column content. Early spinner: "Loading filing data..."

14. Authentication and access

- Identity via `main.py` OIDC cookie hydrate; optional `require_auth(page="company_filings")` at page level.
- Page is registered without ACL flag `False` in `main.py` — default authenticated access.

15. Error handling

- Module-level `try/except` around `main()` shows generic Streamlit error
- Azure download failures: warning + retry button
- Missing filing: `st.error` with blob path + placeholder viewer
- All significant exceptions logged via `log_structured_error` with `page="company_filings"`

16. Environment variables

Variable	Purpose
<code>AZURE_STORAGE_ACCOUNT_NAME</code>	Blob account (default <code>csmarketdata</code> in code)
<code>AZURE_STORAGE_ACCOUNT_KEY</code>	Blob credential
<code>AZURE_BLOB_CONTAINER</code>	Container name (default <code>azure-storage-test</code>)
<code>AZURE_BLOB_PREFIX</code>	Optional path prefix inside container
<code>OPENAI_API_KEY</code>	Enables LLM metric fallback

17. Related files (quick index)

Path	Relationship
<code>app/pages/company_filings_add_files.py</code>	Admin Azure upload for non-SEC PDFs
<code>app/utils/azure_blob.py</code>	Shared blob cache helpers (transcripts)
<code>app/utils/filing_units.py</code>	Multi-filing path logic
<code>app/data/repository.py</code>	<code>FilingMetricRepository</code> class
<code>app/core/llm_extractor.py</code>	LLM on-demand extraction
<code>app/utils/html_to_pdf.py</code>	SEC download PDF generation
<code>edgartools/</code> (repo root)	Offline SEC parsing; not imported by this page

18. Key functions index (`company_filings.py`)

Function	Responsibility
<code>main()</code>	Page entry, session init, layout
<code>_on_company_change</code> / <code>_on_doc_type_change</code> / <code>_on_year_change</code> / <code>_on_filing_unit_change</code>	Filter callbacks
<code>_prefetch_ticker_filter_data</code>	Single-query filter data per ticker
<code>_get_non_sec_blob_path</code>	Non-SEC PDF blob path
<code>build_filing_blob_path</code> (filing_units)	SEC path with filing unit
<code>_ensure_local_blob_optimized</code>	Azure download
<code>FilingMetricRepository.search</code>	Metric search
<code>render_sec_html_viewer</code>	SEC HTML iframe + autoscroll
<code>_render_pdf_viewer</code>	PDF.js viewer
<code>_kw_search_document</code>	Keyword hits in document
<code>scan_filings_directory</code>	Full Azure inventory (fallback company list)

— End of Company Filings — Technical Documentation —

PART 17 OF 22

Company Filings Add Files — Technical Documentation

Admin upload tool for non-SEC filings to Azure Blob Storage.

Continued from previous section →

Application: Market Data Portal (MDP)

Page module: `app/pages/company_filings_add_files.py`

URL: `/company_filings_add_files`

Registered in: `app/main.py` as `( "pages/company_filings_add_files.py", "Add Files", "company_filings_add_files", False)`

UI title: Company Filings File Manager

1. Purpose and scope

This page is an **administrative Azure Data Lake Storage (ADLS Gen2) file manager** for company filing PDFs (primarily non-SEC / Capital Q documents). It allows authorized users to:

- Create folder hierarchies under a configured root prefix
- Upload files (always renamed to `filing.pdf`)
- Browse folder contents (list, download)
- Delete files or folders recursively (restricted role)
- Audit every upload/delete in MySQL
- Send SMTP email notifications on successful uploads

It complements `app/pages/company_filings.py`, which **reads** blobs the upload page **writes**. The filings viewer expects non-SEC paths like:

```
{TICKER}/{YEAR}/{annual-report|interim-report-Q*}/filing.pdf
```

2. Environment and access guards

2.1 Environment restriction

```
if config.env not in (Environment.LOCAL, Environment.STAGING):
    st.switch_page("pages/home.py")
    st.stop()
```

Production is blocked — this tool runs only in `LOCAL` and `STAGING`.

2.2 Authentication

- OIDC via `require_auth()` at page entry.

2.3 ACL model (`coreiq_page_access_control`)

Capability	Minimum permission	Constant
View page / browse / download	Authenticated (default)	—
Upload	<code>edit</code>	<code>AccessControlManager.check_access(..., "edit")</code>
Delete file/folder	<code>delete_admin</code>	<code>AccessControlManager.check_access(..., "delete_admin")</code>

```
CAN_UPLOAD = AccessControlManager.check_access("company_filings_add_files", LOGGED_IN_EMAIL, "edit")
CAN_DELETE = AccessControlManager.check_access("company_filings_add_files", LOGGED_IN_EMAIL, "delete_admin")
```

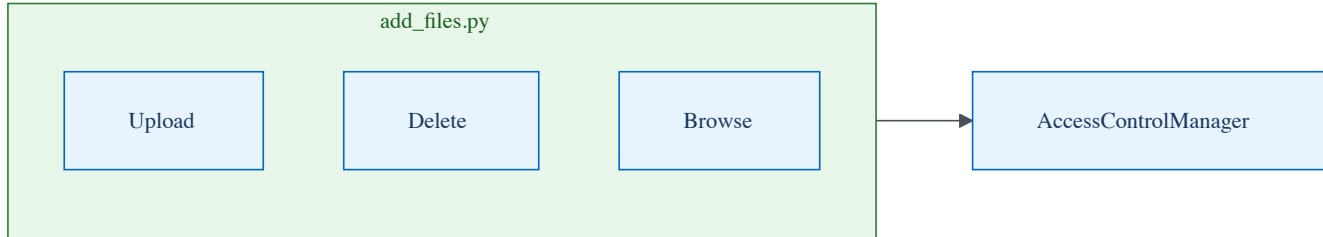
Note: Module docstring mentions `shashankgupta@coresight.com` for delete; actual enforcement is ACL-based `delete_admin`, not a hardcoded email.

3. High-level architecture

Admin upload page: ACL gate, Azure Blob storage, and audit logging for non-SEC filings.

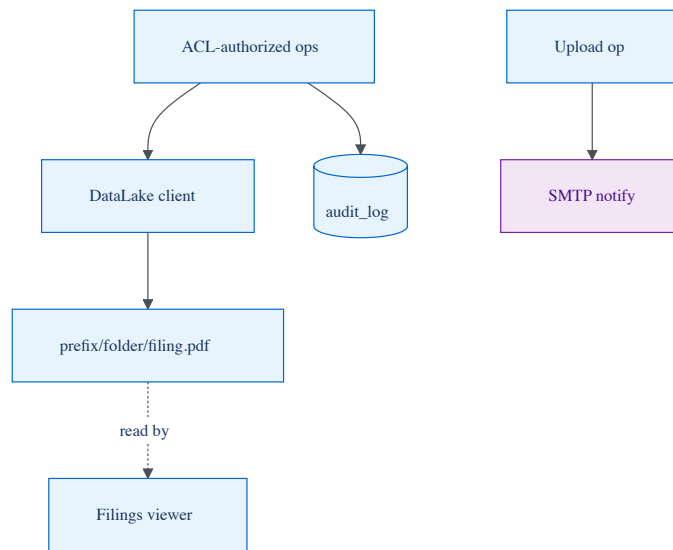
Architecture — Part A: UI and access control

Architecture — Part A: UI and access control



Architecture — Part B: Storage, audit, and viewer

Architecture — Part B: Storage, audit, and viewer



Important distinction: This page uses `azure.storage.filedatalake.DataLakeServiceClient` (hierarchical namespace / ADLS Gen2). `company_filings.py` uses `azure.storage.blob.BlobServiceClient`. Both target the same storage account/container env vars but different SDK surfaces.

4. UI sections (no tabs)

The page is a **single vertical layout** with dividers — not tabbed.

Section	Visible when	Controls
Current User Context (expander)	Always	Email, <code>CAN_UPLOAD</code> , <code>CAN_DELETE</code>
Target Location	Always	Folder path text input, disabled root prefix display, target path info
Upload Files	Always	File uploader, overwrite checkbox, primary upload button
Delete Folder	<code>CAN_DELETE</code> only	Folder path, type <code>DELETE</code> confirm, recursive delete button
Browse Existing Folder	Always	Browse path, "Show folder contents", per-item download/delete

4.1 Upload section details

Control	Key / behavior
Folder path input	Free text, e.g. <code>BRBY/2025/annual-report</code>
Root prefix	Read-only display of <code>AZURE_BLOB_PREFIX</code>
Target preview	<code>info box: {container}/{full_path}/filing.pdf</code>
File uploader	<code>accept_multiple_files=False</code> , disabled if <code>!CAN_UPLOAD</code>
Overwrite	Checkbox, default <code>True</code>
Upload button	<code>"Create folder and upload file"</code> , <code>type="primary"</code>

Forced filename: All uploads saved as `FORCED_UPLOAD_FILENAME = "filing.pdf"` regardless of original name.

4.2 Delete section details

- Recursive directory delete via `delete_directory_recursive()`
- Requires typing `DELETE` exactly in confirmation field
- Hidden entirely when `CAN_DELETE` is false

4.3 Browse section details

Action	Permission	Behavior
Show folder contents	All	Sets <code>show_folder_contents=True</code> , stores <code>last_browse_full_path</code>
List items	All	Up to 300 entries, non-recursive
Download file	All	<code>download_file_bytes()</code> → <code>st.download_button</code>
Delete folder/file	<code>CAN_DELETE</code>	Per-row buttons, <code>st.rerun()</code> on success

5. Session state

Key	Type	Purpose
<code>show_folder_contents</code>	bool	Whether browse results are shown
<code>last_browse_full_path</code>	str	Full ADLS path after <code>build_full_path()</code>
<code>delete_folder_input</code>	str	Widget key for delete path
<code>delete_confirm_text</code>	str	DELETE confirmation
<code>browse_path</code>	str	Widget key for browse path

No ticker/doc_type session keys — paths are free-form strings.

6. Configuration

6.1 Environment variables

Variable	Default / notes
<code>AZURE_STORAGE_ACCOUNT_NAME</code>	Required
<code>AZURE_STORAGE_ACCOUNT_KEY</code>	Required
<code>AZURE_BLOB_CONTAINER</code>	<code>azure-storage-test</code> — ADLS filesystem name
<code>AZURE_BLOB_PREFIX</code>	Optional root prefix inside filesystem
<code>SMTP_SERVER</code>	<code>smtp.office365.com</code>
<code>SMTP_PORT</code>	<code>587</code>
<code>FROM_EMAIL</code>	<code>dataautomation@coresight.com</code>
<code>EMAIL_PASSWORD</code>	SMTP auth

Loaded via `load_dotenv(find_dotenv(usecwd=True))` plus `core.config.config` for DB.

6.2 Constants (in-module)

Constant	Value
<code>AUDIT_TABLE_NAME</code>	<code>coreiq_azure_blob_audit_log</code>
<code>FORCED_UPLOAD_FILENAME</code>	<code>filing.pdf</code>
<code>UPLOAD_SUCCESS_RECIPIENTS</code>	Fixed list of 4 Coresight emails

6.3 DB connection

`get_db_connection()` uses **PyMySQL** directly with `config.database.connect_args` (SSL/CA parity with main app). This page does **not** use `db_manager` singleton.

7. Azure helpers

7.1 Client initialization

```
account_url = f"https://{AZURE_STORAGE_ACCOUNT_NAME}.dfs.core.windows.net"
service_client = DataLakeServiceClient(account_url=account_url, credential=AZURE_STORAGE_ACCOUNT_KEY)
filesystem_client = service_client.get_file_system_client(AZURE_BLOB_CONTAINER)
```

Endpoint uses `.dfs.core.windows.net` (Data Lake), not `.blob.core.windows.net`.

7.2 Path helpers

Function	Behavior
<code>normalize_path_part(value)</code>	Strip slashes
<code>build_full_path(folder_path)</code>	<code>{AZURE_BLOB_PREFIX}/{folder}</code> or folder alone
<code>ensure_directory_exists(fs, path)</code>	Create each path segment; ignore <code>ResourceExistsError</code>

7.3 File operations

Function	SDK call
<code>list_paths(fs, prefix, max_items=200)</code>	<code>get_paths(recursive=False)</code>
<code>download_file_bytes(fs, path)</code>	<code>get_file_client().download_file().readall()</code>
Upload (inline)	<code>get_file_client(path).upload_data(bytes, overwrite=)</code>
<code>delete_file</code>	<code>get_file_client().delete_file()</code>
<code>delete_directory_recursive</code>	<code>get_directory_client().delete_directory()</code>

8. Upload flow

Upload writes the PDF to Azure Blob, records metadata in MySQL, and logs the blob audit entry.

Diagram 8.1 — Part A: Auth & validation

Diagram 8.1 — Part A: Auth & validation

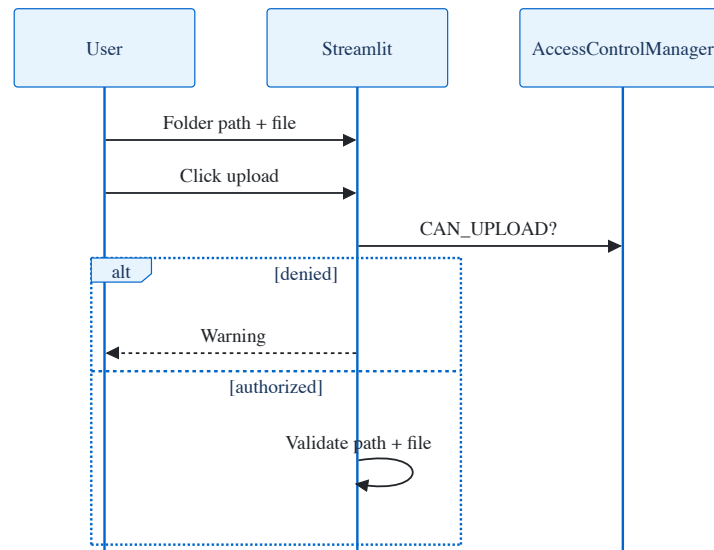
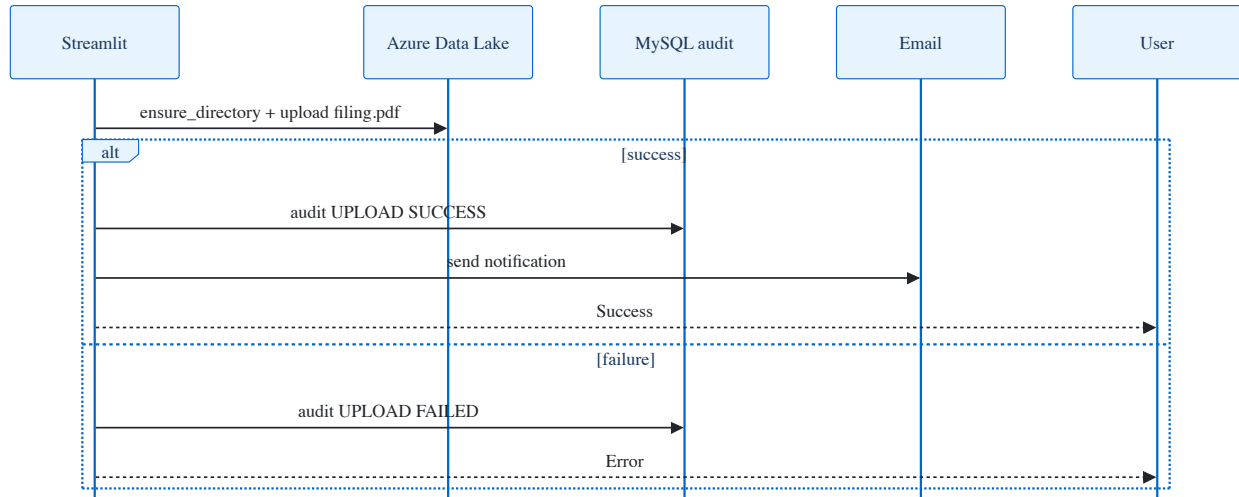


Diagram 8.1 — Part B: Azure write & audit

Diagram 8.1 — Part B: Azure write & audit



8.1 Upload path resolution example

User input	AZURE_BLOB_PREFIX	Resulting ADLS path
AAPL/2024/annual-report	(empty)	AAPL/2024/annual-report/filing.pdf
BRBY/2025/interim-report-Q1	filings	filings/BRBY/2025/interim-report-Q1/filing.pdf

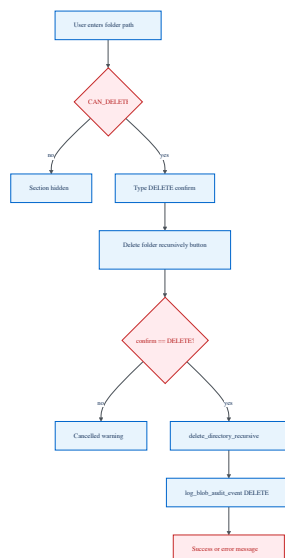
8.2 Email behavior

- **TO:** `UPLOAD_SUCCESS_RECIPIENTS` (normalized, deduplicated)
- **CC:** Logged-in uploader (unless already in TO)
- **Subject:** `Company Filings Upload Success - 1 file`
- **Body:** uploader email, container, folder, overwrite flag, original vs saved filename, byte size, azure path
- Email failure → `st.warning` but upload still succeeds

9. Delete flow

Delete removes the blob object, soft-deletes metadata, and records an audit log row.

Delete removes the blob object, soft-deletes metadata, and records an audit log row.



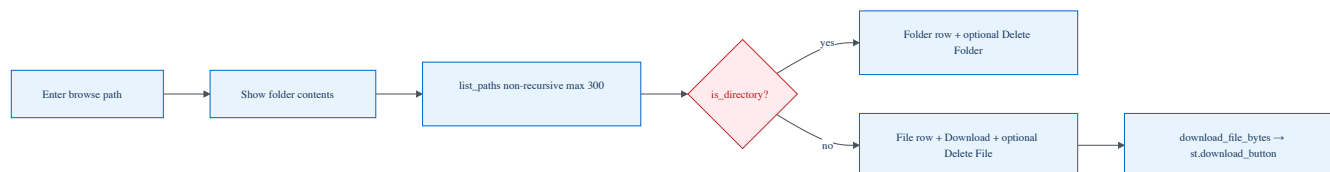
Browse-row deletes (single file or subfolder) follow the same audit pattern with `item_type` `FILE` or `FOLDER`.

`ensure_delete_access()` raises `PermissionError` if `CAN_DELETE` is false — defense in depth beyond UI hiding.

10. Browse / download flow

Browse lists non-SEC filings from the DB; download generates a time-limited blob SAS URL.

Browse lists non-SEC filings from the DB; download generates a time-limited blob SAS URL.



Downloads use `mime="application/octet-stream"` and basename as `file_name`.

11. Audit logging — `coreiq_azure_blob_audit_log`

11.1 Table bootstrap

`create_audit_table_if_not_exists()` runs on page load (`CREATE TABLE IF NOT EXISTS`). Failure → `st.error` + `st.stop()`.

11.2 Schema

Column	Type	Description
<code>id</code>	BIGINT AUTO_INCREMENT	Primary key
<code>event_time</code>	DATETIME	Default <code>CURRENT_TIMESTAMP</code>
<code>user_email</code>	VARCHAR(255)	Actor
<code>action_type</code>	VARCHAR(50)	<code>UPLOAD</code> , <code>DELETE</code>
<code>item_type</code>	VARCHAR(50)	<code>FILE</code> , <code>FOLDER</code>
<code>container_name</code>	VARCHAR(255)	Filesystem/container
<code>root_prefix</code>	VARCHAR(500)	<code>AZURE_BLOB_PREFIX</code>
<code>target_path</code>	TEXT	Full ADLS path
<code>target_name</code>	VARCHAR(500)	Filename or folder leaf
<code>file_size_bytes</code>	BIGINT	Upload size (nullable)
<code>overwrite_flag</code>	TINYINT(1)	Upload overwrite
<code>status</code>	VARCHAR(50)	<code>SUCCESS</code> , <code>FAILED</code>
<code>error_message</code>	TEXT	Failure detail
<code>created_at</code>	DATETIME	Insert timestamp

Indexes: `idx_coreiq_blob_audit_time` , `idx_coreiq_blob_audit_user` , `idx_coreiq_blob_audit_action`

11.3 Failure isolation

`log_blob_audit_event()` catches all exceptions and shows `st.warning` — **audit failure never blocks** the primary Azure operation.

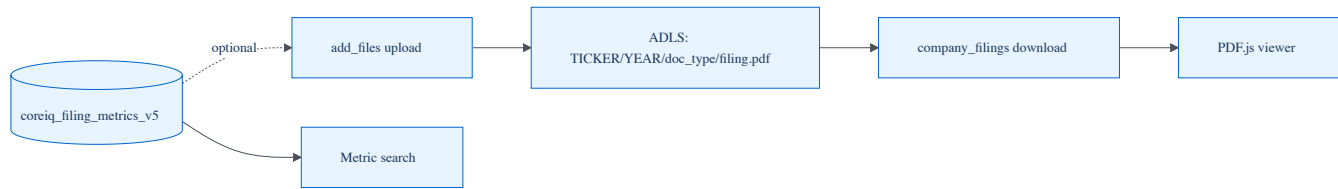
12. Components used

Module	Usage
<code>components.styles.hide_sidebar</code>	Sidebar hidden
<code>components.styles.render_styles</code>	Global CSS
<code>components.styles.set_page_layout</code>	Layout
<code>components.navigation.render_header</code>	"Company Filings File Manager"
<code>components.navigation.render_coresight_footer</code>	Footer
<code>core.auth_manager.get_current_user</code>	<code>LOGGED_IN_EMAIL</code>
<code>core.access_control.AccessControlManager</code>	Upload/delete gates
<code>core.config.config</code> , <code>Environment</code>	Staging/local guard + DB config
<code>utils.server_logger.new_rerun_id</code>	Log correlation

Not used: `app/utils/azure_blob.py` , `FilingMetricRepository` , `edgartools` , LLM extractor.

13. Relationship to Company Filings viewer

13. Relationship to Company Filings viewer



Concern	add_files page	company_filings page
SDK	<code>filedatalake</code>	<code>storage.blob</code>
Write	Yes	No
Read/list	Browse section	Viewer + scanner
Filename	Always <code>filing.pdf</code>	Expects <code>filing.pdf</code> (non-SEC) or <code>filing.html</code> (SEC)
Metric DB	No	Yes

Uploading via `add_files` **does not** insert rows into `coreiq_filing_metrics_v5`. Metric data requires separate ETL. The viewer can still show PDFs if blob exists; search may return empty until metrics are ingested.

14. Suggested folder conventions (non-SEC)

Align uploads with `VALID_DOC_TYPES` / `DOC_TYPE_TO_FOLDER` in `company_filings.py`:

Document	Example folder segment
Annual report	<code>{TICKER}/{YEAR}/annual-report</code>
Q1 interim	<code>{TICKER}/{YEAR}/interim-report-Q1</code>
Half-yearly	<code>{TICKER}/{YEAR}/half-yearly</code>

Example: `LULU/2025/interim-report-Q3` → ADLS file `.../LULU/2025/interim-report-Q3/filing.pdf`

15. Security considerations

- Path injection:** User-supplied folder paths passed to ADLS APIs — rely on Azure ACLs; no `..` sanitization in this module (unlike `filing_units.build_filing_blob_path`).
- Secrets:** Azure key and SMTP password from environment only.
- Production guard:** Page redirects away on prod `config.env`.
- Delete:** Recursive delete is destructive; gated by `delete_admin` + `DELETE` confirmation for bulk folder delete.
- Auth:** OIDC via `require_auth()` at page entry.

16. Error handling patterns

Scenario	User feedback
Azure connection fail on init	<code>st.error</code> + <code>st.stop()</code>
Audit table setup fail	<code>st.error</code> + <code>st.stop()</code>
Upload validation (no path/file)	<code>st.warning</code>
Upload Azure error	<code>st.error</code> + audit FAILED row
Audit write fail	<code>st.warning</code> (action completed)
Email fail	<code>st.warning</code> (upload succeeded)
Browse list fail	<code>st.error</code>
Delete permission	<code>PermissionError</code> → <code>st.error</code>

17. Function index

Function	Responsibility
<code>get_db_connection()</code>	PyMySQL with shared SSL config
<code>create_audit_table_if_not_exists()</code>	DDL for audit table
<code>log_blob_audit_event()</code>	Insert audit row (best-effort)
<code>get_datalake_clients()</code>	ADLS service + filesystem clients
<code>build_full_path()</code>	Prefix + user folder
<code>ensure_directory_exists()</code>	Recursive mkdir on ADLS
<code>list_paths()</code>	Non-recursive directory listing
<code>download_file_bytes()</code>	Full file read
<code>delete_directory_recursive()</code>	ACL-gated folder delete
<code>delete_file()</code>	ACL-gated file delete
<code>send_upload_success_email()</code>	SMTP TLS notification
<code>_normalize_email_list()</code>	Email list cleanup

18. Dependencies (Python packages)

Package	Usage
<code>streamlit</code>	UI framework
<code>azure-storage-file-datalake</code>	<code>DataLakeServiceClient</code>
<code>azure-core</code>	<code>ResourceExistsError</code>
<code>pymysql</code>	Audit DB writes
<code>python-dotenv</code>	Env loading
<code>stdlib</code> <code>smtplib</code> , <code>email.mime</code>	Upload notifications

19. Operational checklist

1. Confirm `config.env` is `LOCAL` or `STAGING` .
2. Set Azure ADLS credentials and container name.
3. Grant `edit` on `company_filings_add_files` for uploaders.
4. Grant `delete_admin` only to trusted admins.
5. Upload to paths matching `company_filings` non-SEC conventions.
6. Verify blob appears in Company Filings viewer (select matching ticker/year/doc type).
7. Run metric ETL separately if search coverage is required.

— End of Company Filings Add Files — Technical Documentation —

PART 18 OF 22

Logs Page — Technical Documentation

Hidden operations surface — server log viewer and segment cache admin.

Continued from previous section →

Application: Market Data Portal (MDP)

Source: `app/pages/logs.py`

URL path: `/logs` (registered in `app/main.py` as hidden navigation)

Purpose: Internal server log viewer, screening segment-cache admin panel, and stealth shell command runner.

Audience: Platform operators and engineers with direct URL access.

Overview

The Logs page is a **debug / operations** surface for the Market Data Portal (MDP). It is **not linked from the main navigation** (`st.navigation(..., position="hidden")` in `main.py`). Users reach it only by navigating directly to `/logs` .

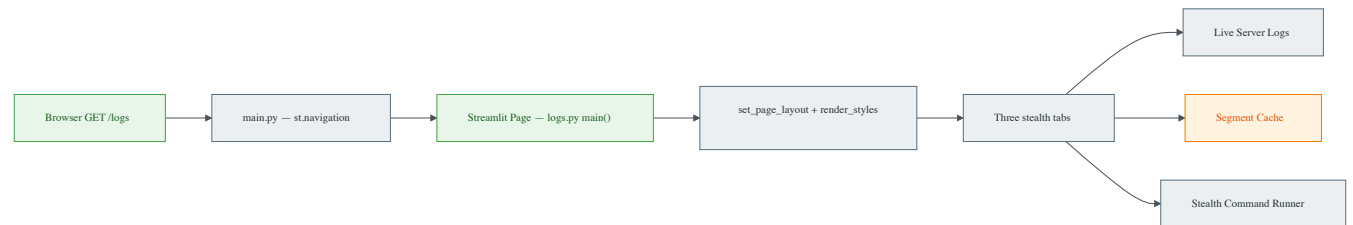
Security note: This hidden route does not enforce page-level `require_auth()` or ACL. Restrict reachability via network controls and monitor access in production.

Registration & Routing

Property	Value
File	<code>app/pages/logs.py</code>
<code>st.Page</code> title	Logs
<code>url_path</code>	<code>logs</code>
Default page	No
Sidebar	Hidden via <code>hide_sidebar()</code>

Hidden route: `/logs` is registered but not linked from navigation — direct URL access only.

Hidden route: `/logs` is registered but not linked from navigation — direct URL access only.



Authentication & Access Control

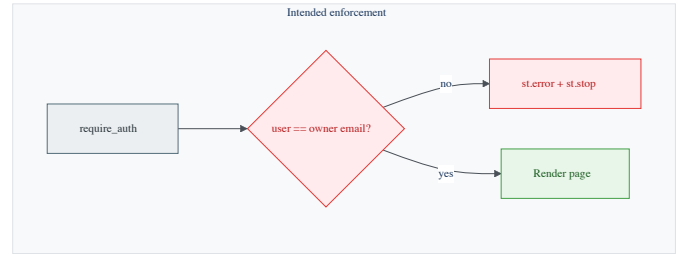
Design

1. `require_auth()` — OpenID Connect (OIDC) session check at page entry.
2. **Owner-only gate** — restrict to authorized operator email or admin role.

Current behavior

- No `AccessControlManager` check.
- No `UserRolesManager` check.
- Page renders for any visitor who can hit the Streamlit route.

- Page renders for any visitor who can hit the Streamlit route.



Recommendation for operators: Enforce `require_auth()` and gate via `UserRolesManager.is_admin()` or a dedicated `coreiq_page_access_control` row for page `logs`.

UI Structure

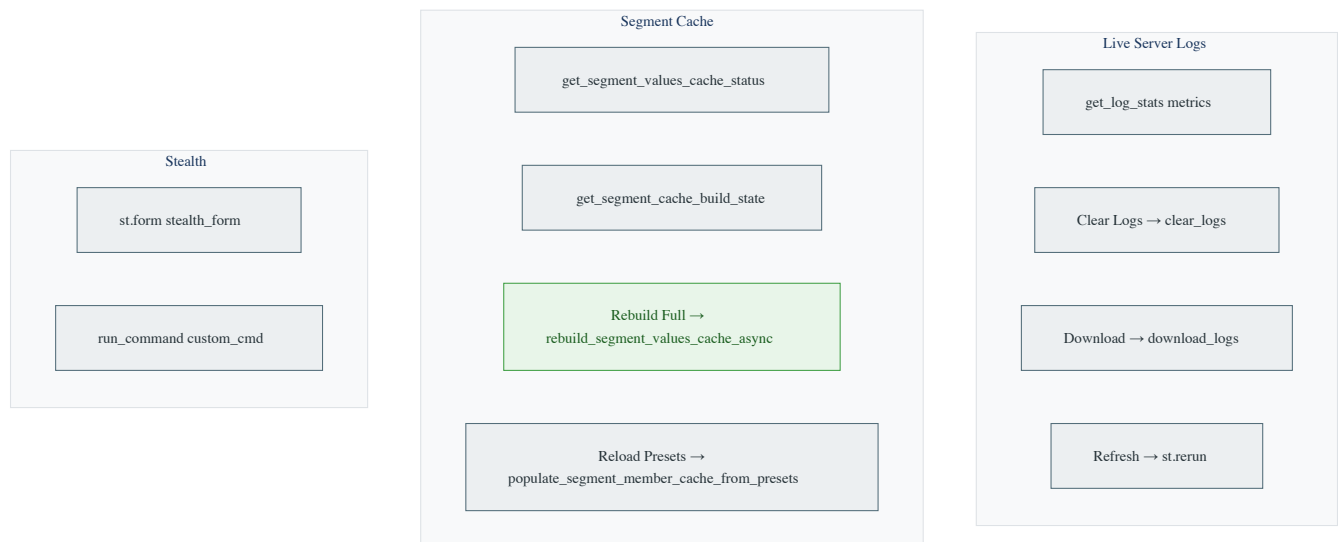
Layout

- `set_page_layout(header_full_width=True, footer_full_width=True, body_padding="20px 40px", max_content_width="1400px")`
- `render_styles()` from `components.styles`
- Custom CSS hides the **third tab label** (stealth tab) — invisible 2–6px wide tab button aligned to the far right.

Tabs

Tab label (visible)	Internal key	Function
Live Server Logs	<code>tab1</code>	Metrics, clear/download/refresh log file
Segment Cache	<code>tab_seg</code>	Screening segment cache rebuild UI
• (stealth)	<code>tab2</code>	Shell command form

Architecture diagram



Tab 1 — Live Server Logs

Dependencies

Module	Symbols used
<code>utils.server_logger</code>	<code>clear_logs</code> , <code>download_logs</code> , <code>get_log_stats</code> , <code>log_structured_error</code> , <code>SERVER_LOG_FILE</code>
<code>components.styles</code>	<code>hide_sidebar</code> , <code>render_styles</code> , <code>set_page_layout</code>

Log file location

- Directory: `server-logs/` (repo root, sibling of `app/`)
- File: `server-log.log` (`SERVER_LOG_FILE` in `server_logger.py`)

Log format (written by `server_logger`)

```
{timestamp} | {level} | {rerun_id} | page={page} | {filename}:{funcName} | {message}
```

User actions

Button	Handler	Side effect
Clear Logs	<code>clear_logs()</code>	Stops queue listener, truncates file, writes IST "Logs cleared" marker
Download Logs	<code>st.download_button(data=download_logs())</code>	Full file bytes, filename <code>server-logs-{YYYYMMDD_HHMMSS}.log</code>
Refresh Now	<code>st.rerun()</code>	Re-reads stats

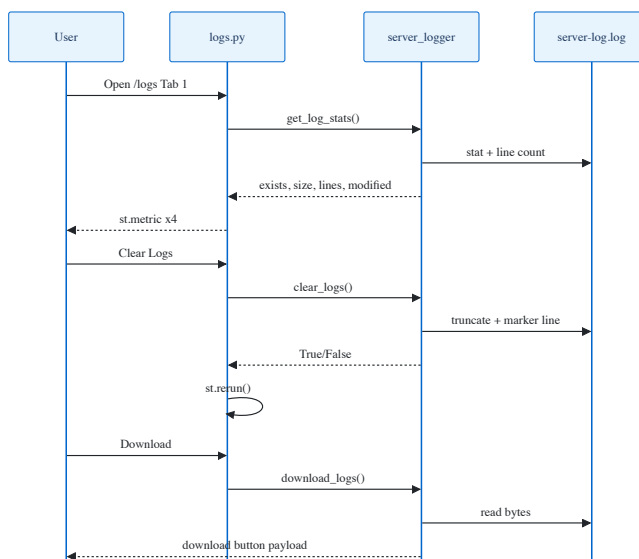
Metrics displayed

- File size (bytes)
- Total line count
- Last modified timestamp
- Static label `server-log.log`

Note: Tab 1 does **not** render inline log tail content in the current UI — only file metadata and actions. Full content is available via download.

`get_log_content()` exists in `server_logger` but is unused on this page.

Note: Tab 1 does not render inline log tail content in the current UI — only file metadata and actions. Full content is available via download. `getlogcontent()` exists in `serverlogg`



Tab 2 — Segment Cache Manager

Powers the **Business / Geographical Segments** screening filter. Data lives in MySQL tables managed by `app/data/screening_service.py`.

Imported functions

```

from data.screening_service import (
    get_segment_values_cache_status,
    get_segment_cache_build_state,
    rebuild_segment_values_cache_async,
    populate_segment_member_cache_from_presets,
)

```

Database tables

Table	Role
<code>coreiq_screening_segment_values_cache</code>	Per-ticker segment metric values for screening
<code>coreiq_screening_segment_member_cache</code>	Segment member label options (presets + derived)

Status metrics

- `total_rows` — rows in values cache
- `distinct_tickers` — unique tickers covered
- `business_rows_count` / `geographical_rows_count`
- `last_updated_timestamp`

Build state (`st.session_state` + module lock)

Module-level `_segment_cache_build_state` dict:

Key	Meaning
<code>running</code>	Background thread active
<code>progress</code>	Current step
<code>total</code>	Total steps (typically 2)
<code>last_error</code>	Last failure message

Actions

1. Rebuild Full Segment Cache — `rebuild_segment_values_cache_async()`

- Replays segment classifier over full company universe (~15–25 min).
- Uses `coreiq_filing_metrics_v5` (per UI help text).
- Non-blocking background thread.

2. Reload Preset Members — `populate_segment_member_cache_from_presets()`

- Instant upsert of hardcoded preset labels (~0 ms).

- Instant upsert of hardcoded preset labels (~0 ms).



Tab 3 — Stealth Command Runner

UX

- Third tab styled invisible (1px font, transparent).
- `st.form("stealth_form")` with collapsed text input.
- Submit button hidden via CSS (`opacity: 0; height: 0`).
- Output: `st.code(stdout)` or `st.error(stderr)` .

`run_command(cmd, timeout=15)`

- Executes `subprocess.run(cmd, shell=True, ...)` .
- Docstring claims only predefined commands — **implementation accepts any string** from the form.
- Timeout: 15 seconds default.

Security implications

This is effectively a **remote shell** if the page is reachable without auth. Path traversal guards exist in `list_directory` / `read_file_safe` helpers, but those helpers are **not wired to the stealth tab** in current `main()`.

Unused helpers (present in file)

The module defines extensive Linux diagnostics helpers not rendered in `main()`:

- `get_system_info`, `get_environment_vars`, `get_meminfo`, `get_cpu_info`
- `parse_df`, `parse_ps`, `parse_network_interfaces`, `parse_open_ports`
- `list_directory`, `read_file_safe`, `get_file_download_bytes`
- etc.

These appear to be legacy from a fuller "System Logs Explorer" and may be re-enabled in future tabs.

Session State

Key	Set by	Purpose
<code>stealth_cmd</code>	Streamlit widget	Last command typed in stealth form

No other `st.session_state` keys are used on this page. Segment build state lives in **module globals** inside `screening_service.py`, not Streamlit session.

Error Handling

All major blocks wrap `try/except` and call:

```
log_structured_error(e, page="logs", component="...", operation="...")
```

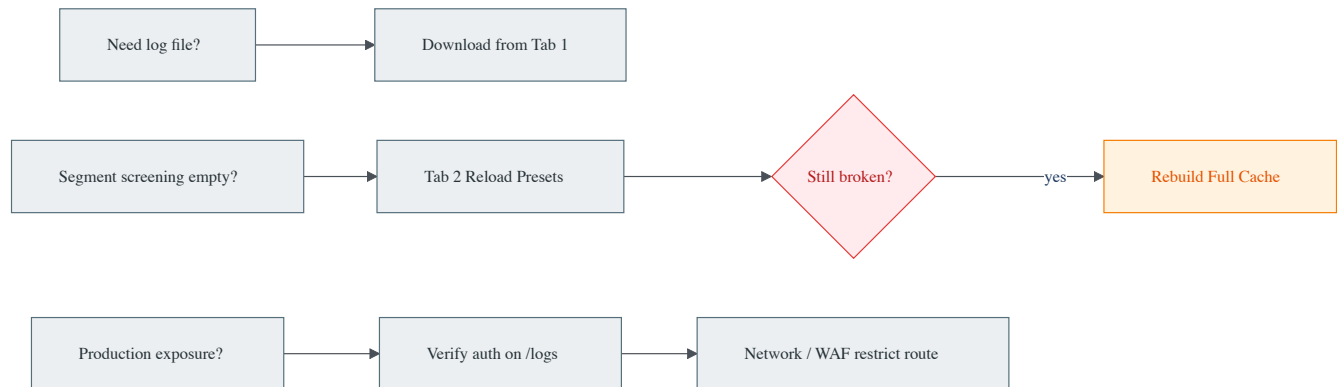
Components: `module_init`, `run_command`, `main`, `_render_log_body`, etc.

Related Documentation

- Shell commands reference: `docs/market-data-commands.md` (cited in stealth tab comment)
- Logging standard: `CLAUDE.md` → Logging section
- Segment cache implementation: `app/data/screening_service.py` (~lines 1530–2900)

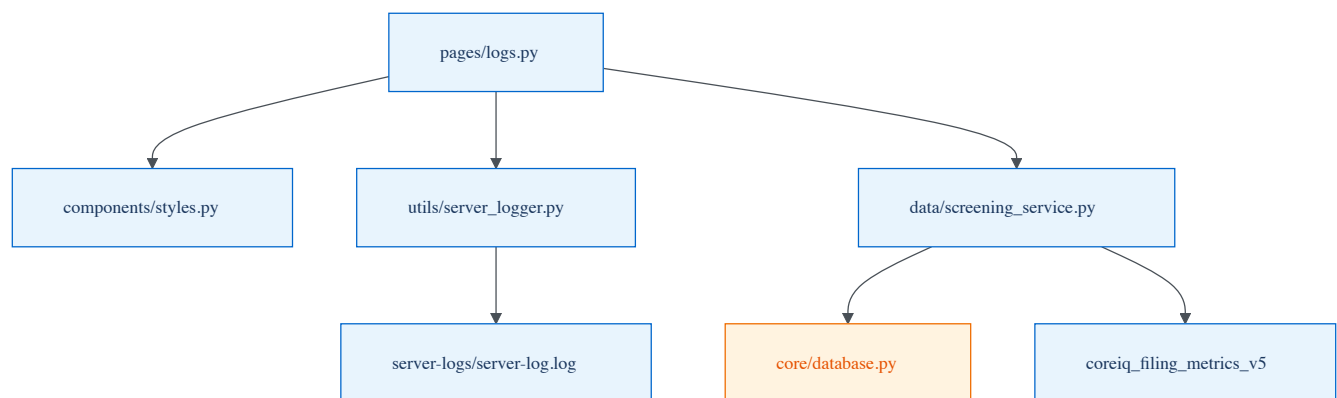
Operational Checklist

Operational Checklist



File Dependency Graph

File Dependency Graph



Generated from source analysis of the Market Data Portal (MDP). Last reviewed against codebase structure as of project documentation pass.

— End of Logs Page — Technical Documentation —

PART 19 OF 22

Retail Adding Page — Technical Documentation

Retail company master data management with ACL gating and audit trail.

Continued from previous section →

Application: Market Data Portal (MDP)

Source: `app/pages/retailer_adding.py`

URL path: `/retailer_adding`

UI title: Retailer Manager

Purpose: CRUD and bulk upload for retail company master records in `coreiq_companies`.

Overview

The Retailer Manager page lets authorized users **view, edit, insert, and bulk-upload** rows in the company master table. It is restricted to **local and staging** environments, gated by **page-level Access Control List (ACL)**, and writes an **audit trail** plus **email notifications** on successful mutations.

Registration & Discovery

Property	Value
Registered in <code>main.py</code>	Yes (<code>url_path=retailer_adding</code>)
Navigation link	Footer Quick Links on staging only, for <code>_RETAILER_ALLOWLIST</code> emails (<code>navigation.py</code>)
Header <code>current_page</code>	<code>retailer_manager</code>

Diagram — Part A: Route & auth

Diagram — Part A: Route & auth

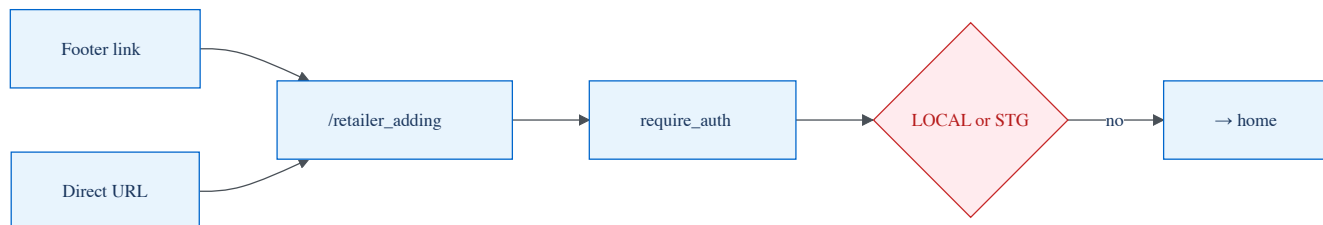
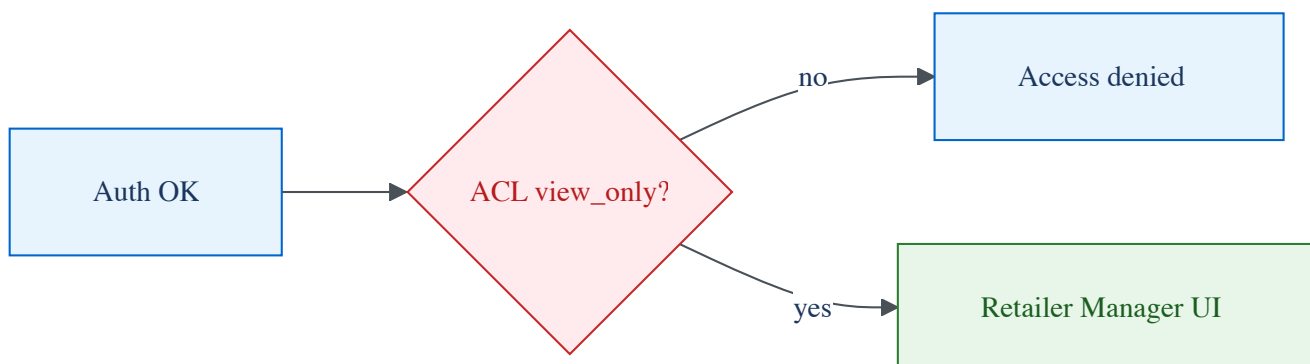


Diagram — Part B: ACL gate

Diagram — Part B: ACL gate



Authentication & Access Control

Auth bootstrap

```
auth_user = require_auth()
user_email = get_current_user()
```

Environment guard

```
if config.env not in (Environment.LOCAL, Environment.STAGING):
    st.switch_page("pages/home.py")
```

Production users are redirected to Home — the page must not run against prod DB from this entry point.

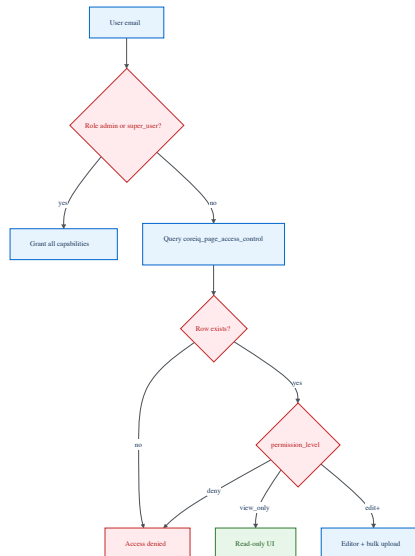
Page ACL (`coreiq_page_access_control`)

Check	Permission required	Effect
Page entry	<code>view_only</code>	Can load page and view data
Edit / save / upload / download	<code>edit</code>	<code>CAN_EDIT_RETAILER = True</code>

```
has_access = AccessControlManager.check_access("retailer_adding", user_email, "view_only")
CAN_EDIT_RETAILER = AccessControlManager.check_access("retailer_adding", user_email, "edit")
```

Important: `retailer_adding` has **no default access** — users without an ACL row are denied unless they hold `admin` or `super_user` role (global bypass in `AccessControlManager.check_access`).

Important: retaileradding has no default access — users without an ACL row are denied unless they hold admin or superuser role (global bypass in AccessControlManager.checkaccess).



Footer allowlist (navigation only)

Separate from ACL — controls **link visibility** in staging footer:

`_RETAILER_ALLOWLIST` in `navigation.py` (5 emails including `dataautomation@coresight.com`).

Database Architecture

Connection pattern

Uses **direct** `pymysql` connection — **not** `DatabaseManager` / SQLAlchemy stack used elsewhere.

Env var	Purpose
<code>STG_DB_HOST</code> , <code>STG_DB_PORT</code> , <code>STG_DB_NAME</code>	MySQL target
<code>STG_DB_USER</code> , <code>STG_DB_PASSWORD</code>	Credentials
<code>SSL_CA</code> , <code>ENABLE_SSL</code>	Azure MySQL TLS

Primary table

`coreiq_companies` — company master (tickers, names, industry, exchange, source, etc.)

Business key (unique constraint)

```
(ticker, source, exchange_acronym)
```

Constraint name in comments: `uq_coreiq_companies_name_source`

Audit table

`coreiq_companies_audit_log` — auto-created via `ensure_audit_table()` if missing.

Column	Type	Notes
<code>id</code>	BIGINT AUTO_INCREMENT	PK
<code>event_time</code>	DATETIME	Default CURRENT_TIMESTAMP
<code>user_email</code>	VARCHAR(255)	Actor
<code>action_type</code>	VARCHAR(50)	e.g. UPDATE, INSERT_EDITOR, INSERT_UPLOAD
<code>table_name</code>	VARCHAR(255)	Default <code>coreiq_companies</code>
<code>row_count</code>	INT	Rows affected
<code>status</code>	VARCHAR(50)	SUCCESS / FAILED
<code>details</code>	TEXT	Truncated row summaries
<code>error_message</code>	TEXT	On failure

Reference tables (read-only union export)

Table	Purpose
<code>coreiq_sec_companies_all</code>	SEC company universe
<code>coreiq_non_sec_companies_all</code>	Non-SEC company universe

`build_union_df()` concatenates both with `company_source` = `SEC` / `NON_SEC` for CSV download.

Column Management

System-managed columns (never user-editable)

```
SYSTEM_MANAGED_COLUMNS = {"data_inserted_at"}
```

Also excluded dynamically when:

- `AUTO_INCREMENT` or `GENERATED` in `INFORMATION_SCHEMA.COLUMNS.EXTRA`
- `COLUMN_DEFAULT = CURRENT_TIMESTAMP`

Schema introspection

- `get_table_columns()` — full column metadata from `INFORMATION_SCHEMA`
- `get_primary_key_column()` — PK name (typically `id`)
- `get_pk_and_insertable_columns()` → `(pk, all_cols, upload_cols)`

`upload_cols` = insertable columns minus PK — used for bulk upload template and editor inserts.

Session State

Key	Initial	Purpose
<code>df</code>	On load / refresh	Cached <code>coreiq_companies</code> rows
<code>df_limit</code>	Matches <code>limit</code> input	Invalidates cache when limit changes
<code>editor_save_in_progress</code>	<code>False</code>	Prevents double-submit on Save
<code>bulk_insert_in_progress</code>	<code>False</code>	Prevents double-submit on bulk insert

Widget keys: `retailer_editor` (`st.data_editor`), file uploader, search input.

Method 1 — Inline Editor

Load

```
df = load_data(int(limit)) # SELECT * FROM coreiq_companies LIMIT n
```

Default limit: 500 (UI allows 10–1000).

Search

`filter_editor_df(df, search_text)` — case-insensitive match across all columns (stringified).

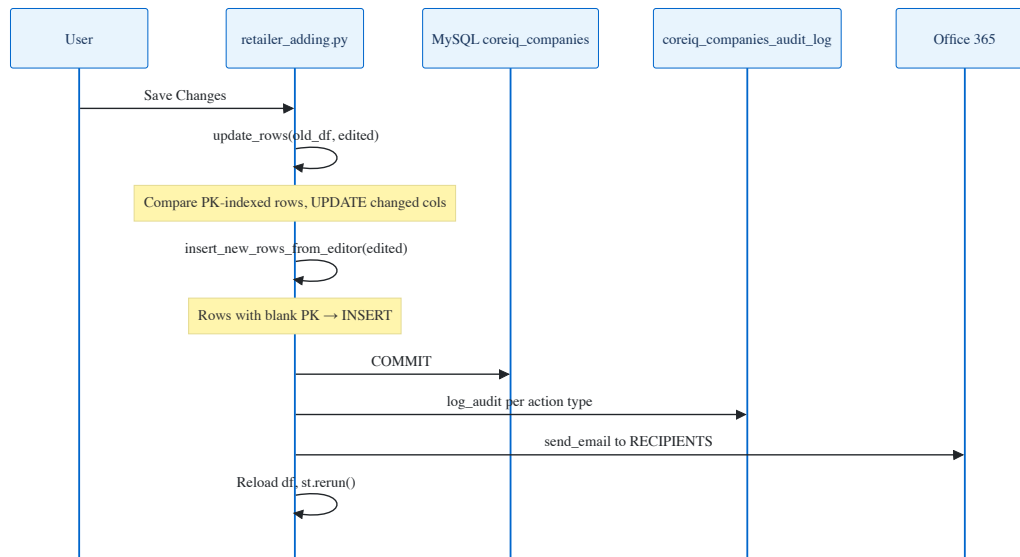
Editor

- `st.data_editor` with `num_rows="dynamic"` when `CAN_EDIT_RETAILER`
- PK column disabled via `build_editor_column_config`
- Read-only `st.dataframe` for view-only users

Save pipeline

Retailer save validates ACL, upserts company rows, and writes an audit log entry.

Retailer save validates ACL, upserts company rows, and writes an audit log entry.



```
update_rows(old_df, new_df, pk, upload_cols)
```

- Indexes by primary key.
- Builds `SET col=%s` for changed editable columns.
- Normalizes compare via `normalize_for_compare()` (handles `Timestamp`, `NaN`, strings).

```
insert_new_rows_from_editor
```

- Selects rows where PK is blank (`is_blank_value`).
- **Deduplicates** within payload by business key (`ticker`, `source`, `exchange_acronym`).
- Skips rows already in DB (`fetch_existing_business_keys`).
- Catches `IntegrityError` for race / constraint duplicates.

Downloads (edit permission)

Button	Output
Download Screen Data (CSV)	Current edited dataframe
Download All SEC + NON_SEC (CSV)	<code>build_union_df()</code>

Method 2 — Bulk Upload

Supported formats

`.xlsx`, `.csv`

Flow

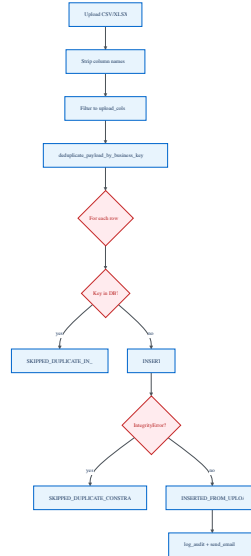
1. User uploads file → preview in `st.dataframe`
2. Invalid columns warned and ignored
3. **Insert Bulk Rows** → `insert_rows(upload_df, upload_cols)`
4. Same dedup / business-key / integrity handling as editor inserts
5. Audit + email on success

Template

Download Bulk Upload Template (CSV) — empty CSV with `upload_cols` headers only.

Bulk CSV upload validates rows, inserts companies, and reports per-row results.

Bulk CSV upload validates rows, inserts companies, and reports per-row results.



Email Notifications

Setting	Value
SMTP	<code>smtp.office365.com:587</code>
From	<code>dataautomation@coresight.com</code>
Password	<code>EMAIL_PASSWORD</code> env
Recipients	shashankgupta, RisthaVilinaDsa, PhilipMoore, dataautomation @ coresight.com

Triggered on: `UPDATE` , `INSERT_EDITOR` , `INSERT_UPLOAD` when row count > 0.

Body includes action, row count, performer email, and up to 200 detail lines.

UI Components

Component	Source
<code>hide_sidebar</code> , <code>render_styles</code> , <code>set_page_layout</code>	<code>components.styles</code>
<code>render_header(current_page="retailer_manager")</code>	<code>components.navigation</code>
<code>render_coresight_footer()</code>	<code>components.navigation</code>
Custom <code>.rm-card</code> CSS	Inline in page

`HAS_SHARED_UI` flag — page degrades gracefully if imports fail.

Error Handling

- Save failures → `log_audit(..., status="FAILED", error_message=str(e))`
- Audit logging failures → `st.warning` (non-fatal)
- Email failures → `st.warning` (non-fatal)

Security Notes

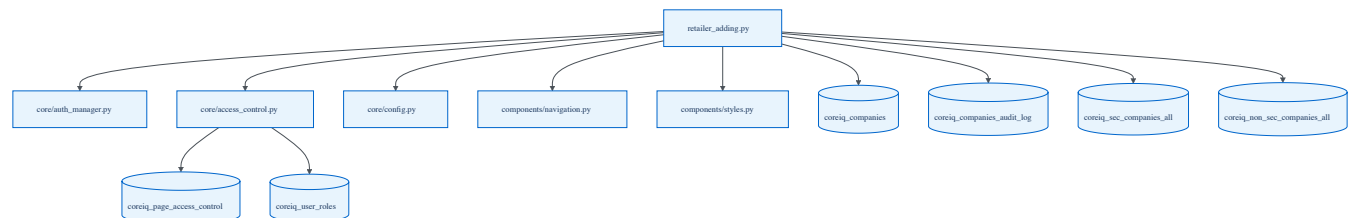
1. **Staging-only env guard** — primary prod safety mechanism.
2. **ACL** — no implicit access; explicit `coreiq_page_access_control` row required (except admin/super_user).
3. **Direct pymysql** — bypasses centralized `db_manager` read/write patterns; uses manual transactions (`autocommit=False`).
4. **No DELETE UI** — users can update and insert only; no row deletion exposed.

Permission Matrix

Role / ACL	View data	Edit rows	Bulk upload	SEC+NON_SEC export
admin / super_user	Yes	Yes	Yes	Yes
<code>view_only</code>	Yes	No	No	No
<code>edit</code> / <code>delete_admin</code> / <code>allow</code>	Yes	Yes	Yes	Yes
No row / <code>deny</code>	No	No	No	No

Dependency Graph

Dependency Graph



Generated from source analysis of the Market Data Portal (MDP). Last reviewed against codebase structure as of project documentation pass.

— End of Retailer Adding Page — Technical Documentation —

PART 20 OF 22

Forecasting Page (Revenue Estimates) — Technical Documentation

Revenue forecast viewer, refresh workflows, and quarterly estimates.

Continued from previous section →

Application: Market Data Portal (MDP)

Source: `app/pages/forecasting.py`

URL path: `/forecasting`

UI title: Revenue Forecasting

Purpose: Database-backed revenue forecast viewer — annual and quarterly modes, statistical models, ensemble, scenarios, backtesting, and Excel export.

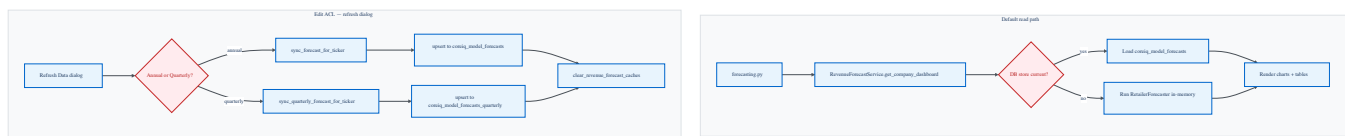
Overview

The Forecasting page is the primary **read-only analytics cockpit** for revenue projections at **annual** and **quarterly** cadence. It:

1. Loads historical actuals from staging income-statement tables (annual or quarterly frequency).
2. Runs (or loads from cache) `RetailerForecaster` (six annual models) or `RetailerQuarterlyForecaster` (seven models including seasonal naive).
3. Displays interactive Plotly charts, methodology popups (`st.dialog`), and exportable Excel workbooks.
4. Optionally allows authorized users to refresh stored forecasts via **Refresh Data** (writes to `coreiq_model_forecasts` or `coreiq_model_forecasts_quarterly` when `edit` Access Control List (ACL) is granted).

Typical viewers never write to the database; writes occur only through the refresh dialog.

Typical viewers never write to the database; writes occur only through the refresh dialog.



Registration

Property	Value
<code>main.py</code>	<code>("pages/forecasting.py", "Forecasting", "forecasting", False)</code>
Header key	<code>current_page='forecasting'</code>
Auth	<code>get_current_user()</code> — no hard <code>require_auth()</code> at page top

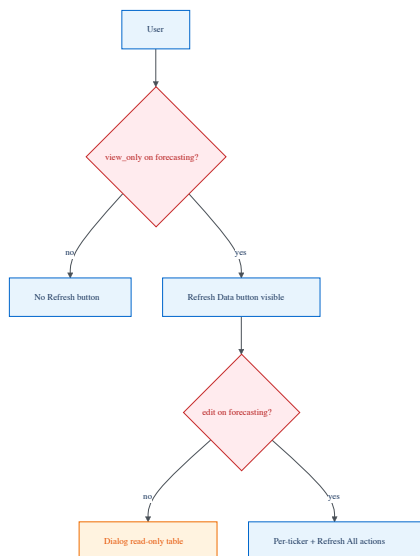
Access Control

Refresh dialog permissions — `_get_refresh_permissions(user_email)`

Capability	Access Control List (ACL) check
Can open Refresh dialog	<code>check_access("forecasting", email, "view_only")</code>
Can run Refresh Now / Refresh All	<code>check_access("forecasting", email, "edit")</code>

`admin` / `super_user` bypass all checks via `AccessControlManager`.

admin / superuser bypass all checks via AccessControlManager.



Page viewing itself is **not Access Control List (ACL)-gated** in `forecasting.py` — any authenticated portal user with nav access can browse estimates (subject to global auth in `main.py`).

Architecture Layers

Layer	Module	Responsibility
Page UI	<code>app/pages/forecasting.py</code>	Layout, period toggle, charts, tabs, Excel, refresh dialog
Data service	<code>app/data/revenue_forecast_service.py</code>	Companies list, annual + quarterly dashboard payloads
Annual store	<code>app/data/forecast_store.py</code>	Staleness, upsert, read <code>coreiq_model_forecasts</code>
Quarterly store	<code>app/data/quarterly_forecast_store.py</code>	Table init, staleness, upsert <code>coreiq_model_forecasts_quarterly</code>
Admin sync	<code>app/data/forecast_admin_service.py</code>	Annual + quarterly write paths from refresh
Refresh metadata	<code>app/data/forecast_refresh_service.py</code>	Dialog table (annual/quarterly), schema migrations, email
Annual engine	<code>app/utils/retailer_forecaster.py</code>	Six models + ensemble + scenarios
Quarterly engine	<code>app/utils/retailer_quarterly_forecaster.py</code>	Seven models + seasonal decomposition + ensemble
Source routing	<code>app/data/source_router.py</code>	Securities and Exchange Commission (SEC) vs Yahoo Finance per ticker

Database Tables

Actual revenue (read)

Table	Source	Key filters
<code>coreiq_av_financials_income_statement</code>	SEC	<code>report_type='annual'</code> , <code>total_revenue IS NOT NULL</code>
<code>coreiq_yf_financials_income_statement</code>	YFinance	<code>frequency='annual'</code> , <code>line_item='Total Revenue'</code>
<code>coreiq_yf_company_overview</code>	YFinance	<code>payload_json</code> for <code>financialCurrency</code>
<code>coreiq_companies</code>	Master	Ticker, name, source, exchange

Stored annual forecasts (read/write via refresh)

`coreiq_model_forecasts` — one row per (`ticker` , `fiscal_year` , `model_key` , `metric`) ; values in millions (`value_millions`) ; `last_actual_date` drives staleness; fast path for `get_company_dashboard` and Market Data Key Stats.

Stored quarterly forecasts (read/write via refresh)

`coreiq_model_forecasts_quarterly` — managed by `quarterly_forecast_store.py` ; created idempotently on first session via `ensure_quarterly_forecast_table()` ; same staleness pattern as annual store; consumed by `get_quarterly_dashboard` .

Company universe query

`RevenueForecastService.get_companies()` — cached 1 hour:

- Joins `coreiq_companies` with SEC/YF stats CTEs
- Requires ≥ 3 distinct annual revenue periods
- Builds **display ticker**: `TSCO.L` when `exchange_acronym` present

Forecasting Models

Annual — six core models (`MODEL_LABELS` / `RetailerForecaster`)

Key	Display name	Approach
<code>linear</code>	Linear Regression	OLS trend line extended forward
<code>cagr</code>	CAGR	Compound growth from first to last historical year
<code>exp_smoothing</code>	Exponential Smoothing	Weighted toward recent years
<code>holt</code>	Holt's Linear Trend	Level + slope state space
<code>ma_trend</code>	Moving Average Trend	Mean of last 3 YoY growth rates
<code>weighted_avg</code>	Weighted Average Growth	Weighted historical growth rates

Quarterly — seven models (`RetailerQuarterlyForecaster`)

Same six annual method keys plus `seasonal_naive` (seasonal baseline). All methods run on a **de-seasonalised** series (ratio-to-trend / centred moving average), then re-apply seasonal indices (Q1–Q4). Requires ≥ 8 **quarters** of revenue history. Ensemble still averages **top 3** by backtest mean absolute percentage error (MAPE).

Key	Display name	Notes
<code>seasonal_naive</code>	Seasonal Naive	Quarter-over-quarter seasonal baseline
<i>(others)</i>	Same keys as annual	Applied after seasonal adjustment

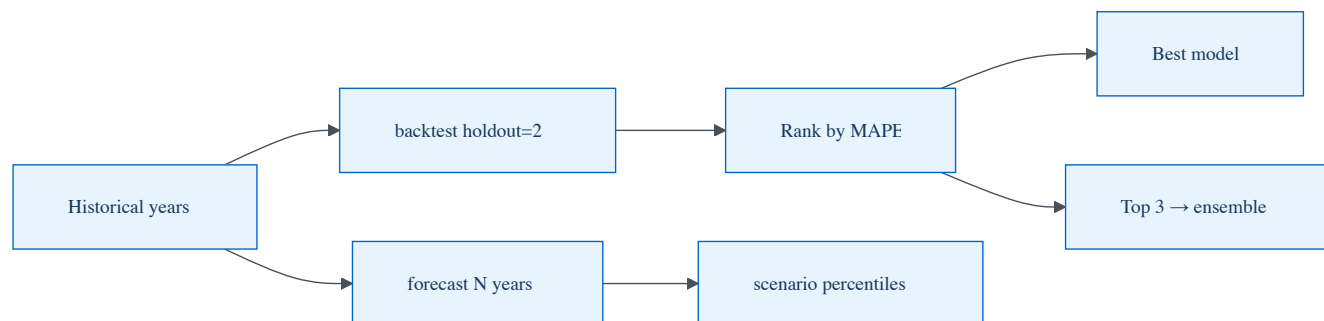
Derived outputs (annual and quarterly)

Key	Description
<code>ensemble</code>	Average of top 3 models by backtest MAPE (not all 6)
<code>scenario_pessimistic</code>	25th percentile historical growth path
<code>scenario_baseline</code>	50th percentile (median) growth path
<code>scenario_optimistic</code>	75th percentile growth path

Backtesting

- Hold out **2 most recent years**
- Rank models by **MAPE** (mean absolute percentage error)
- Best model shown in summary card; top 3 feed ensemble

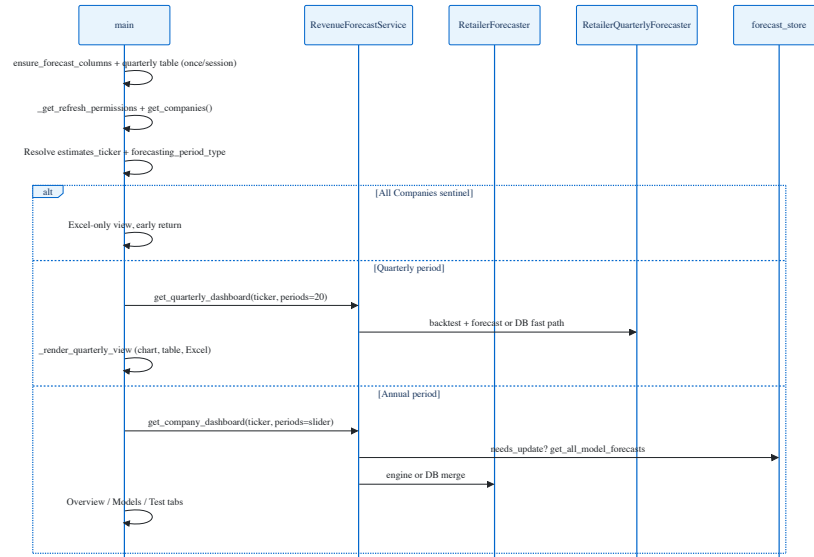
- Best model shown in summary card; top 3 feed ensemble



Page Load Flow — `main()`

After company selection, flow branches on **Period** radio (`Annual` | `Quarterly`). Quarterly uses a dedicated `_render_quarterly_view()` path with no annual regression.

After company selection, flow branches on Period radio (Annual / Quarterly). Quarterly uses a dedicated `renderquarterlyview()` path with no annual regression.



Lazy schema init (session once)

```

if not st.session_state.get("_forecast_schema_initiated"):
    ensure_forecast_columns()
    backfill_company_info()
    ensure_quarterly_forecast_table() # from quarterly_forecast_store
    st.session_state["_forecast_schema_initiated"] = True
    
```

Session State

Key	Purpose
<code>_forecast_schema_initiated</code>	One-time DB column + quarterly table init
<code>estimates_ticker</code>	Selected company (<code>_ALL_SENTINEL</code> , <code>_SELECT_SENTINEL</code> , or ticker)
<code>forecasting_period_type</code>	<code>Annual</code> or <code>Quarterly</code> — radio below header
<code>active_ticker</code>	Header breadcrumb ticker
<code>estimates_forecast_periods</code>	Annual slider 1–5 years
<code>_est_all_xl_bytes</code>	Cached all-companies Excel bytes
<code>_est_all_xl_just_built</code>	Auto-trigger download flag
<code>_est_sgl_xl_{ticker}_{periods}</code>	Per-ticker annual Excel cache
<code>refresh_dialog_period_type</code>	Annual/Quarterly radio inside refresh dialog

Query params: `?ticker=TSKO` synced on company change; `?period_type=Quarterly` seeds `forecasting_period_type` on first visit only (never overrides user selection).

Sentinel constants

- `_ALL_SENTINEL` — "All Companies" download-only mode
- `_SELECT_SENTINEL` — Placeholder "— Select Company —"

UI Structure

Header row

- Company selectbox (hidden label, custom styled title row with company name)
- **Period** radio — `Annual` | `Quarterly` (individual ticker only; hidden in All Companies mode)
- **Forecast Horizon** slider (1–5 years) — **annual mode only** (right column)
- **Refresh Data** button — if `can_view_popup`

Annual hero metrics (5 cards)

1. Latest Actual Revenue (period ending date)
2. Ensemble — Next Year
3. Best Forecasting Model — clickable card opens `_model_popup` dialog (MAPE in meta)
4. Historical Years in Training
5. Forecast Horizon (years through end year)

Annual main tabs

Tab	Content
Overview	Combined chart, forecast table, implied growth, historical stats, per-model explore buttons
Models	Nested tab per model + Scenarios sub-tab with <code>_scenario_chart</code>
Test	Backtest chart/table, scenario analysis, implied growth table, MAPE/Bias/RMSE documentation panels

Quarterly view — `_render_quarterly_view()`

Self-contained path when `forecasting_period_type == 'Quarterly'`:

- Loads `RevenueForecastService.get_quarterly_dashboard(selected_ticker, periods=20)`
- Hero chips: historical quarter count, “Quarterly · seasonally adjusted”
- Five cards: Latest Actual Revenue (quarter label), Ensemble — Next Quarter, Best Model, Quarters in Training, Forecast Horizon (quarters)
- Plotly chart via `_build_quarterly_overview_chart` — historical + faint model lines + dashed ensemble + scenario band
- Forecast table (quarters × models) and backtest MAPE table
- Excel via `_build_quarterly_excel` → `{ticker}_Quarterly_Revenue_Forecasting.xlsx`
- Empty state when < 8 quarters or no stored forecast — prompts **Refresh Data → Quarterly**

Charts (Plotly)

Function	Purpose
<code>_build_overview_chart</code>	Annual historical + all models + scenarios
<code>_build_quarterly_overview_chart</code>	Quarterly historical + models + ensemble + scenario band
<code>_build_model_chart</code>	Single model detail
<code>_build_backtest_chart</code>	MAPE comparison bars
<code>_scenario_chart</code>	Pessimistic / baseline / optimistic paths

Model popup — `_model_popup`

`@st.dialog('Model Details', width='large')` — methodology (`MODEL_BUSINESS_OVERVIEW` , `MODEL_NOTES`), chart, and metrics for one model key. Best-model hero card opens this dialog directly (reliable vs tab-jumping JS).

All Companies Mode

When `estimates_ticker == _ALL_SENTINEL` :

- No per-ticker dashboard load
- **Excel** button builds `_build_forecast_excel_all(companies)` — 3 sheets, all tickers
- Early `return` after footer

Excel Export

Single company — `_build_forecast_excel_single`

Sheets include historical actuals, all model forecasts, scenarios, backtest, implied growth.

Download mechanism — `_render_excel_js_download`

JavaScript blob download via `streamlit.components.v1.html` (same pattern as `market_data.py`) for one-click export without Streamlit's native download delay.

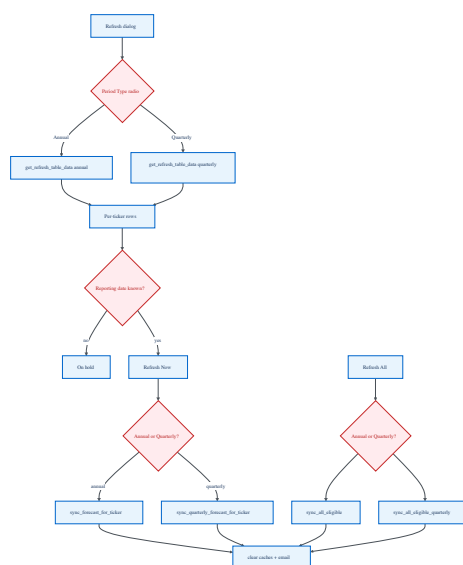
Refresh Data Dialog — `_refresh_dialog`

`@st.dialog("Refresh Forecasting Models", width="large")` with **Period Type** radio (`Annual` | `Quarterly`). Backend routing:

Period	Per-ticker sync	Bulk sync
Annual	<code>sync_forecast_for_ticker</code>	<code>sync_all_eligible</code>
Quarterly	<code>sync_quarterly_forecast_for_ticker</code>	<code>sync_all_eligible_quarterly</code>

`get_refresh_table_data(period_type.lower())` supplies per-ticker company name, exchange, reporting date, fiscal period, last refresh. Tickers **without** a confirmed reporting date show **On hold** (excluded from Refresh All). After success: `clear_revenue_forecast_caches()` , `get_refresh_table_data.clear()` , and `send_model_refresh_email(..., period_type=...)` .

getrefreshabledata(periodtype.lower()) supplies per-ticker company name, exchange, reporting date, fiscal period, last refresh. Tickers without a confirmed reporting date show On



Dashboard Payload Schema

`RevenueForecastService.get_company_dashboard()` returns dict including:

Key	Content
<code>ticker</code> , <code>company_name</code>	Identifiers
<code>source</code> , <code>source_label</code> , <code>source_table</code>	Data provenance
<code>reported_currency</code> , <code>display_currency</code>	No FX conversion
<code>actual_rows</code> , <code>historical_rows</code>	Normalized annual series
<code>forecast_rows</code> / internal <code>forecast_df</code>	Model outputs
<code>backtest_rows</code>	MAPE rankings
<code>raw_forecasts</code>	Engine-native structure
<code>summary</code>	Latest actual, next ensemble, best method, periods
<code>best_method</code>	Winning model key
<code>outlier_years</code>	Excluded from training
<code>timings_ms</code>	Performance breakdown
<code>available_models</code>	Model keys present

Cached: `@st.cache_data(ttl=86400 * 7)` on `get_company_dashboard`.

Data Normalization Pipeline

1. `_fetch_actual_rows(ticker, source)` — raw DB rows
2. `_normalize_actual_rows` — fiscal year, revenue in billions, growth %, ancillary metrics
3. Build `model_df` — dedupe by year (max revenue), drop years < 10% of peak
4. `RetailerForecaster.from_dataframe(model_df)` — outlier detection + clean series

DB fast path

If `not needs_update(forecast_ticker, last_actual_date)`:

- Load `get_all_model_forecasts` from store
- Reconstruct `forecast_df`
- Merge scenario columns from in-memory engine if missing from DB
- Approximate `backtest_df` from stored MAPE values
- Skip full engine recompute (~seconds saved per load)

Formatting Helpers

Function	Output
<code>_fmt_billions</code>	<code>\$X.XXB</code> style
<code>_fmt_mm</code>	Millions with commas
<code>_fmt_pct</code>	Percentage display
<code>_fmt_year</code>	Fiscal year label
<code>get_currency_symbol</code>	From <code>utils.constants</code>

Display currency stored in module global `_EST_DISPLAY_CURRENCY` per payload.

Error Handling

Failure	Behavior
<code>get_companies()</code> / dashboard load	<code>log_structured_error</code> ; user sees empty state or error banner
Refresh dialog sync failure	Per-ticker error row in dialog; email still sent with partial results
Excel build failure	Warning toast; cached bytes not set
Schema backfill (<code>ensure_forecast_schema</code>)	Logged; page continues with best-effort columns
Pre-warm thread exception	Swallowed in daemon thread; logged at WARNING

All exceptions use `utils.server_logger` (`log_structured_error` , `log_error`) with `page="forecasting"` . No bare `except:` blocks.

Logging & Timing

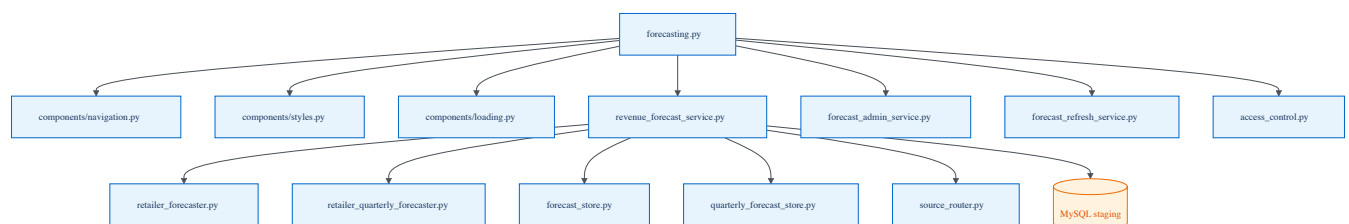
Uses `log_timing` and `log_structured_error` with `page= forecasting` :

- `FORECASTING_schema_init`
- `FORECASTING_get_companies`
- `ESTIMATES_PAGE_LOAD_DASHBOARD`
- `ESTIMATES_PAGE_RENDER`

Service layer logs: `FORECAST_GET_COMPANIES` , `FORECAST_LOAD_ACTUALS` , `FORECAST_DB_FAST_PATH` , etc.

Component Dependencies

Component Dependencies



Model Methodology Reference (in-page)

The page embeds extensive documentation:

- `MODEL_BUSINESS_OVERVIEW` — plain-language per model
- `MODEL_NOTES` — formulas (OLS, CAGR, Holt, etc.)
- `_render_documentation_tab` / `_render_method_notes` — user-facing help
- Test tab — MAPE interpretation table, Bias/RMSE explainer

Screening Integration

`coreiq_model_forecasts` is also consumed by:

- `screening_service.py` — forecasting criterion filters
- `repository.py` / Key Stats — F- columns on Market Data

Refreshing forecasts from this page updates downstream screening and market data after cache clear.

Known Behaviors & Edge Cases

1. **YFinance composite tickers** — Store uses `TICKER.EXCHANGE` (e.g. `ADS.DE`); routing handled in service layer.
2. **Scenario rows** — May be absent in database store; merged from in-memory engine at read time.
3. **Quarterly mode** — Separate render path; fixed 20-quarter horizon in UI; requires ≥ 8 historical quarters.
4. **Quarterly refresh** — Fully supported in refresh dialog (same on-hold rules for missing reporting dates).
5. **Page view not ACL-gated** — Only refresh actions require Access Control List (ACL).
6. **Pre-warm thread** — Background `get_refresh_table_data("annual")` when user can open refresh dialog.
7. **Best model card** — Opens `_model_popup` via Streamlit button (not fragile DOM tab jump).

Generated from source analysis of the Market Data Portal (MDP). Last reviewed against codebase structure as of project documentation pass.

— End of Forecasting Page (Revenue Estimates) — Technical Documentation —

PART 21 OF 22

Access Management Page — Technical Documentation

IAM console for system roles and page-level access control.

Continued from previous section →

Application: Market Data Portal (MDP)

Source: `app/pages/access_management.py`

URL path: `/access_management`

Purpose: Centralized Identity and Access Management (IAM) for **system roles** (`coreiq_user_roles`) and **page-level permissions** (`coreiq_page_access_control`).

Overview

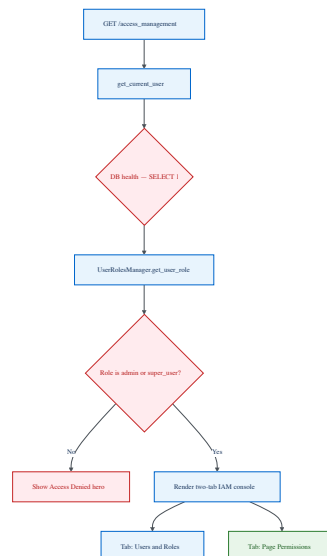
Access Management is the administrative console for who can use restricted portal pages and what **system role** each user holds. Only users with role `admin` or `super_user` may open the page. Within the page, **admins** have additional powers (create admin users, grant delete permission, delete users).

Registration & Discovery

Property	Value
<code>main.py</code> registration	Yes — <code>url_path=access_management</code>
Footer link	Shown when <code>_can_access_mgmt(user_email)</code> is true
Auth	<code>get_current_user()</code> — identity via <code>main.py</code> cookie hydrate

Gate: only `admin` or `super_user` roles reach the Identity and Access Management (IAM) console; others see an access-denied hero.

Gate: only admin or superuser roles reach the Identity and Access Management (IAM) console; others see an access-denied hero.



Core Modules

AccessControlManager (`app/core/access_control.py`)

Manages `coreiq_page_access_control` — per-page, per-user permission rows.

Method	Purpose
<code>check_access(page, email, required_permission)</code>	Runtime gate used by feature pages
<code>get_page_users(page)</code>	List Access Control List (ACL) rows for a page
<code>set_user_access(page, email, permission, created_by)</code>	Upsert Access Control List (ACL) row
<code>remove_user_access(page, email)</code>	Delete Access Control List (ACL) row
<code>get_permission_options()</code>	All valid permission strings

`UserRolesManager` (same file)

Manages `coreiq_user_roles` — global role per email.

Method	Purpose
<code>get_user_role(email)</code>	Returns <code>admin</code> , <code>super_user</code> , <code>user</code> , or None
<code>get_all_users()</code>	Full user list ordered by role hierarchy
<code>set_user_role(email, role, display_name, created_by)</code>	Upsert user
<code>delete_user(email)</code>	Remove user (admin-only in UI)
<code>is_admin</code> / <code>is_admin_or_super_user</code>	Convenience checks

Database Tables

`coreiq_page_access_control`

Column	Purpose
<code>page_name</code>	Page identifier (e.g. <code>retailer_adding</code>)
<code>user_email</code>	Lowercased email
<code>permission_level</code>	See hierarchy below
<code>created_by</code>	Granting admin email
<code>created_at</code> / <code>updated_at</code>	Timestamps

Upsert: `INSERT ... ON DUPLICATE KEY UPDATE permission_level, updated_at`

`coreiq_user_roles`

Column	Purpose
<code>user_email</code>	PK / unique
<code>display_name</code>	Optional friendly name
<code>role</code>	<code>admin</code> <code>super_user</code> <code>user</code>
<code>created_by</code>	Who added the user
<code>created_at</code> / <code>updated_at</code>	Timestamps

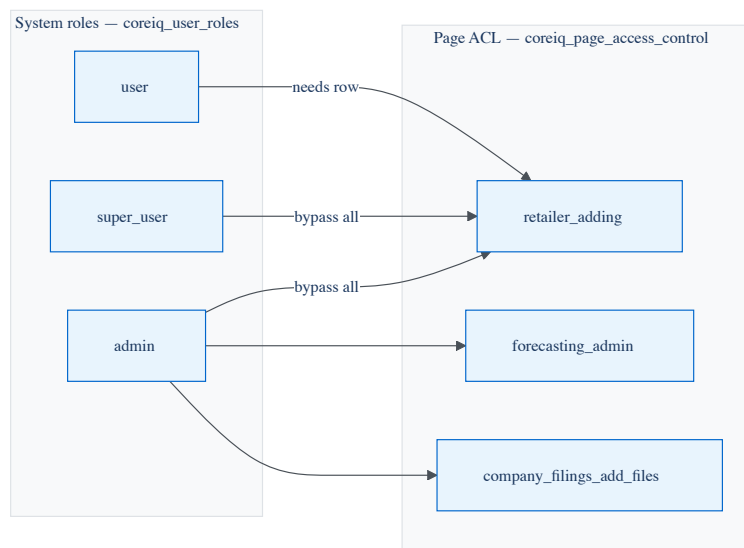
Permission Hierarchy (runtime)

Used by `AccessControlManager.check_access` :

Level	Numeric rank	Meaning
<code>view_only</code>	1	Read-only
<code>upload_only</code>	2	Upload capability
<code>edit</code>	3	Edit content
<code>delete_admin</code> , <code>allow</code>	4	Full page access
<code>deny</code>	—	Explicit block (always false)

Bypass: Users with role `admin` or `super_user` return `True` for any `check_access` call without reading Access Control List (ACL) table.

Bypass: Users with role admin or superuser return True for any checkaccess call without reading Access Control List (ACL) table.



Managed Pages (UI scope)

Configured in `MANAGED_PAGES` :

page_name	Display label
<code>company_filings_add_files</code>	Company Filings File Manager
<code>retailer_adding</code>	Retailer Adding
<code>forecasting_admin</code>	Forecasting Admin

Note: The public **forecasting viewer** (`forecasting` page) uses separate Access Control List (ACL) checks in `forecasting.py` but is **not** listed in Access Management UI.

Tab 1 — Users & Roles

Stats grid

Total users, Admins, Super Users, standard Users — computed from `get_all_users()` .

Add / Update User form

Field	Widget
Email	<code>st.text_input</code> key <code>am_u_email</code>
Display name	<code>am_u_name</code>
Role	<code>am_u_role</code> selectbox
Submit	Add User → <code>UserRolesManager.set_user_role</code>

Rules:

- Only **admins** can assign role `admin` (super_users blocked with error).
- Mutations disabled when `db_ok=False` .

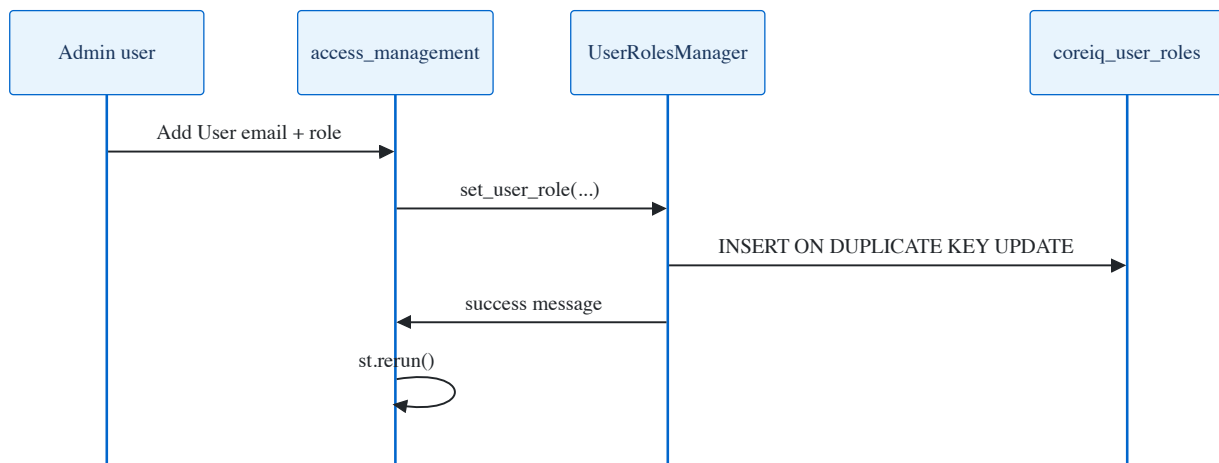
Users table

- Filter: All Roles / Admin / Super User / User
- Columns: Email, Display Name, Role (pill), Since, Added By, Delete
- **Delete flow:** Two-step confirm ( → ☒ Sure?)
- Cannot delete self
- Only admins can delete
- Calls `UserRolesManager.delete_user`

Role legend cards

Describes `admin` , `super_user` , `user` capabilities from `ROLE_DESCS` .

Describes admin, superuser, user capabilities from ROLEDESCS.



Tab 2 — Page Permissions

Page selector

Dropdown of `MANAGED_PAGES` → loads `AccessControlManager.get_page_users(sel_page_id)`.

Stats

Metrics: Total with Access, View Only, Edit, Delete (`delete_admin` count).

Grant Access form

Field	Notes
Email	Manual entry
Permission	<code>view_only</code> , <code>edit</code> , and <code>delete_admin</code> (admin only)
Quick-pick	Selectbox of known users from roles table

Super user limitation: Can grant `view_only` and `edit` only — not `delete_admin` .

Calls `AccessControlManager.set_user_access(sel_page_id, email, perm, current_user)` .

Current access table

Per user row:

- Email (read-only display)
- Permission selectbox — **auto-saves on change** (compares to current, calls `set_user_access`)
- Granted date, Granted by
- Revoke button → `remove_user_access`

Permission legend

UI exposes simplified 3-tier model:

UI label	DB value	Description
View Only	<code>view_only</code>	Read-only
Edit	<code>edit</code>	View + edit
Delete	<code>delete_admin</code>	Full admin on page

Legacy values (`allow` , `deny` , `upload_only`) still exist in DB and `get_permission_options()` but are collapsed in display pills via `PERM_DISPLAY` .

Database Health Check

On page load:

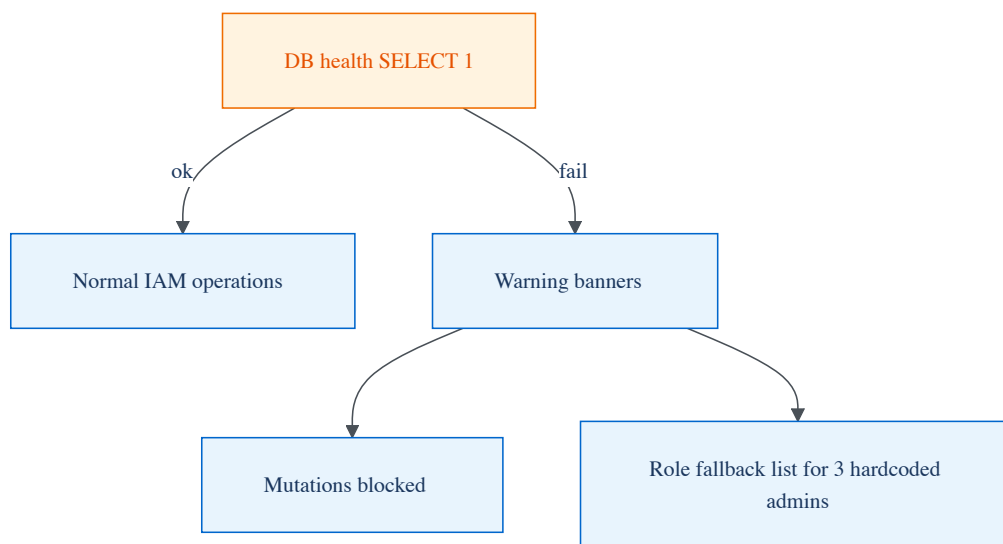
```
db_manager.execute_query_readonly("SELECT 1", {})
```

If fails → `db_ok=False` :

- Warning banner on page
- Read operations show empty/cached states
- **All mutations disabled**

- Role fallback: hard-coded admin emails get `admin` role

- Role fallback: hard-coded admin emails get admin role



Session State Keys

Key pattern	Purpose
<code>am_u_email</code> , <code>am_u_name</code> , <code>am_u_role</code>	Add user form widgets
<code>am_u_filter</code>	Role filter selectbox
<code>_del_confirm_{email}</code>	Two-step delete confirmation
<code>am_p_sel</code>	Selected managed page index
<code>am_p_email</code> , <code>am_p_perm</code> , <code>am_p_pick</code>	Grant access form
<code>am_pp_{idx}_{email}_{page}</code>	Per-row permission selectbox

Role cache busting on write: `UserRolesManager` pops `_urm_role_{email}` from session.

Error Handling

Failure	UX
DB health check fail	Warning banner; mutations blocked; role list falls back to 3 hardcoded admins
<code>set_user_access</code> / <code>remove_user_access</code> exception	<code>st.error</code> + <code>[Access Control List (ACL)]</code> log line
Invalid email on add user	Inline validation before DB write
Non-admin attempting admin-only action	Button hidden or <code>st.error</code>

Page render wrapped in try/except at module level — failures log via `log_structured_error` with `page="access_management"` .

Logging

Uses `log_timing` for:

- `ACCESS_MGMT_auth`
- `ACCESS_MGMT_db_health`
- `ACCESS_MGMT_role_check`
- `ACCESS_MGMT_total_page_render`

ACL mutations log via `log_error` in `access_control.py` with `[Access Control List (ACL)]` prefix.

Relationship to Other Admin UIs

Surface	Overlap
<code>forecasting_admin.py</code> expander	Duplicate mini-IAM for 3 pages — uses same <code>AccessControlManager</code>
<code>admin_iam.py</code>	Unregistered legacy IAM page
Feature pages	Call <code>check_access</code> at module top

Canonical management: Access Management page is the primary full-featured IAM UI.

Admin Capability Matrix

Action	admin	super_user
Open page	Yes	Yes
Add user (any role)	Yes	Yes except admin role
Delete user	Yes	No
Grant view/edit page perm	Yes	Yes
Grant delete_admin page perm	Yes	No
Revoke page access	Yes	Yes

Fallback Admins (DB unavailable)

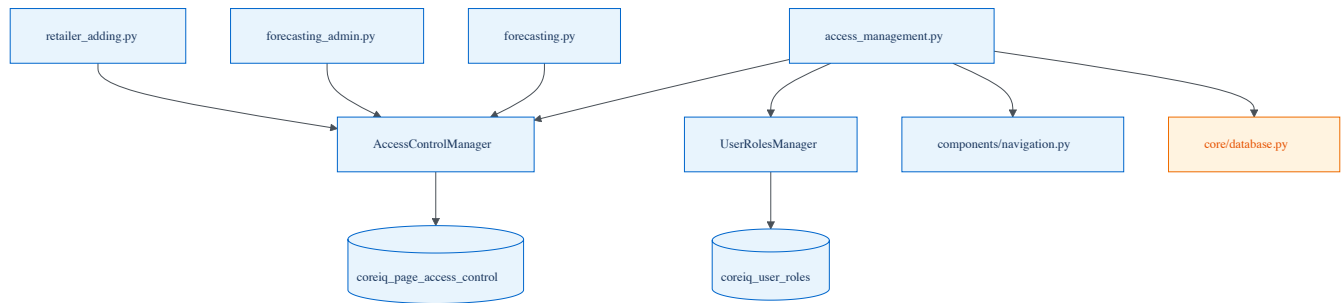
`UserRolesManager._FALLBACK_ADMINS` :

- `mohdsaeedafri@coresight.com`
- `philipmoore@coresight.com`
- `shashankgupta@coresight.com`

Also used in `_can_access_mgmt()` footer helper and Access Management role check fallback.

Dependency Graph

Dependency Graph



Generated from source analysis of the Market Data Portal (MDP). Last reviewed against codebase structure as of project documentation pass.

— End of Access Management Page — Technical Documentation —

PART 22 OF 22

Forecasting Admin Page — Technical Documentation

Unregistered appendix — forecast model administration and bulk refresh.

Continued from previous section →

Application: Market Data Portal (MDP)

Source: `app/pages/forecasting_admin.py`

Documented URL: `/forecasting_admin` (per `CLAUDE.md`)

Purpose: Bulk sync of stored revenue model forecasts into `coreiq_model_forecasts` when annual actuals are newer than cached forecasts.

Registration Status

Important: `forecasting_admin.py` exists under `app/pages/` but is **not listed** in `app/main.py` `_PAGES_CONFIG` as of current source. The page is therefore **not registered in Streamlit navigation** unless added manually or loaded via an alternate entry point.

Related surfaces:

- `app/pages/admin_iam.py` — unregistered duplicate Identity and Access Management (IAM) patterns
- Embedded Identity and Access Management (IAM) expander **inside** `forecasting_admin.py` itself
- `access_management.py` — canonical Identity and Access Management (IAM) for `forecasting_admin` page Access Control List (ACL)

Operators may need to add:

```
("pages/forecasting_admin.py", "Forecasting Admin", "forecasting_admin", False),
```

to `_PAGES_CONFIG` for `/forecasting_admin` to resolve.

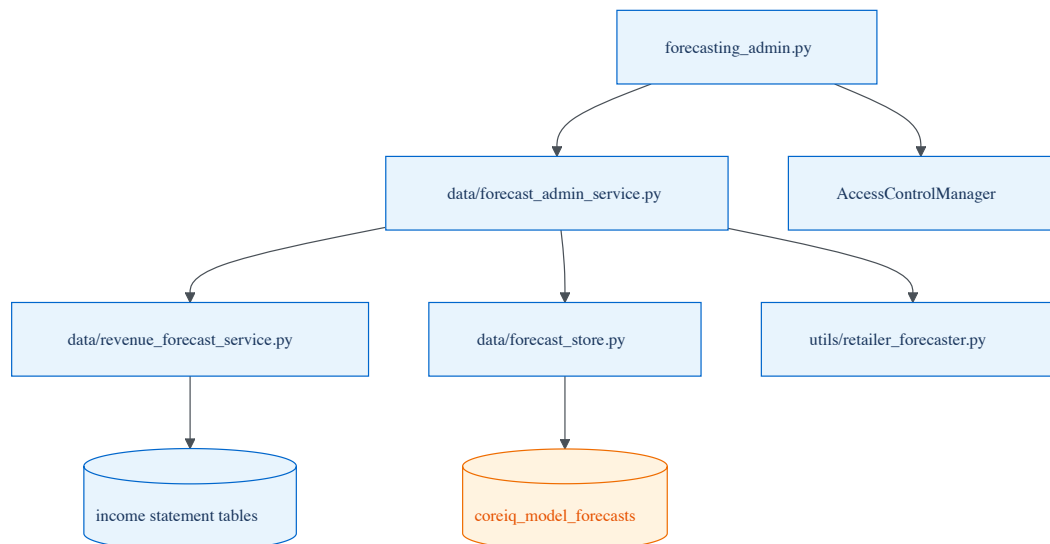
Overview

Forecasting Admin is an **internal write path** for the revenue forecasting pipeline (annual and quarterly). While `forecasting.py` is read-only for most users, this page triggers:

1. **Single-ticker sync** — `sync_forecast_for_ticker` (annual) or `sync_quarterly_forecast_for_ticker`
2. **Bulk stale sync** — `sync_all_eligible` or `sync_all_eligible_quarterly`
3. **Optional embedded IAM** — manage ACL for admin pages (hardcoded admin emails only)

After sync, `clear_revenue_forecast_caches()` invalidates Streamlit caches so Market Data / Estimates pick up new rows.

After sync, `clearrevenueforecastcaches()` invalidates Streamlit caches so Market Data / Estimates pick up new rows.



Authentication & Access Control

Auth

```
user_email = get_current_user()
```

Identity is established via `main.py` cookie hydrate. Page-level `require_auth()` may be enabled for direct-route protection.

Sync permission

```
has_access = AccessControlManager.check_access("forecasting_admin", user_email, "allow")
```

Cached in session:

```
_cache_key = f"fc_admin_access_{user_email.lower()}"
st.session_state[_cache_key] = has_access
```

Requires permission level $\geq$ `allow` (rank 4) or `admin` / `super_user` role bypass.

Users without access see warning: sync buttons hidden; IAM expander may still show for hardcoded admins.

IAM expander admins

Separate from DB roles — hardcoded set:

Email
mohdsaeedafri@coresight.com
shashangupta@coresight.com
risthavilinadsa@coresight.com
philipmoore@coresight.com

When `is_admin` (email in set), expander **Access Control — Page Permissions** is visible.

Legacy allowlist

`forecast_admin_service.FORECAST_ADMIN_EMAILS` (4 emails) + `is_forecast_admin()` — **not used** by current `forecasting_admin.py` UI (ACL replaced it).

UI Layout

Chrome

- `new_rerun_id("forecasting_admin")`
- `st.set_page_config(page_title="Forecasting Admin", layout="wide")`
- `hide_sidebar()`, `render_styles()`, `inject_red_spinner_css()`
- `render_header(current_page="forecasting_admin")`
- Custom CSS: `.fc-admin-hero`, cards, deny state, bold form labels

Sections

- Hero** — "Internal · Forecast Store" / "Forecasting Admin"
- IAM expander** (admin emails only) — mini access management

3. **Sync controls** (if `has_access`)
4. **Results table + metrics** (after sync run)
5. **Page load diagnostics** expander — timing breakdown
6. `render_coresight_footer`

Embedded IAM Expander

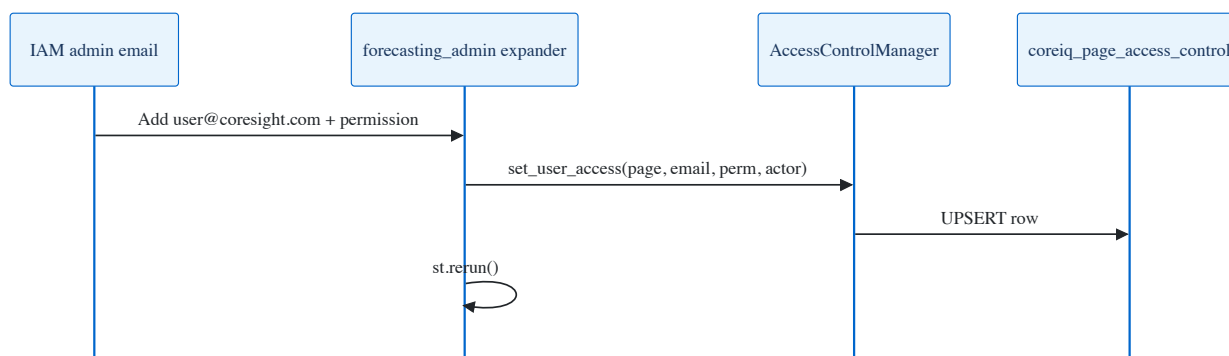
Duplicates patterns from `access_management.py` for three pages:

```
MANAGED_PAGES = ["forecasting_admin", "retailer_adding", "company_filings_add_files"]
```

Features:

- Page selector → `get_page_users(selected_page)`
- Stats: Total Users, Allow Access, Delete Admin
- Add user form → `set_user_access`
- Per-user permission select + Save / Delete
- Permission legend (allow, deny, view_only, upload_only, edit, delete_admin)

- Permission legend (allow, deny, viewonly, uploadonly, edit, deleteadmin)



Sync Operations

Sync all stale tickers

```
run_all = st.button("Sync all stale tickers")
results = sync_all_eligible(force=False, periods=5, progress_callback=_cb)
clear_revenue_forecast_caches()
```

- Progress bar via callback (`cur`, `total`, `display_ticker`)
- `force=False` — respects `needs_update()` staleness check

Sync single ticker

```
run_one = st.button("Sync this ticker only")
row = sync_forecast_for_ticker(ticker.strip())
clear_revenue_forecast_caches()
```

Enriches result with `company_name` from `RevenueForecastService.get_companies()` .

Result metrics

Metric	Counts status
Upserted	updated
Already current	up_to_date
Skipped / no data	skipped , no_data
Errors	error

Dataframe columns: `ticker` , `company_name` , `status` , `rows_upserted` , `message`

Service Layer — `forecast_admin_service.py`

`sync_forecast_for_ticker(display_ticker, force=False, periods=5)`

Diagram — Part A: Source gate & actuals prep

Diagram — Part A: Source gate & actuals prep

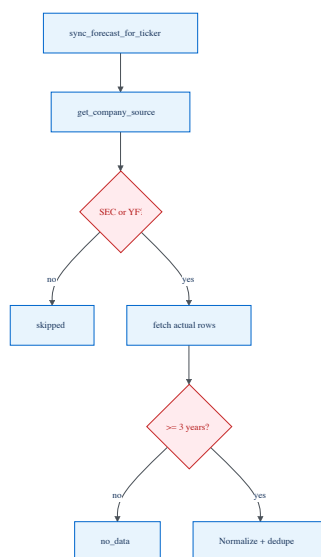
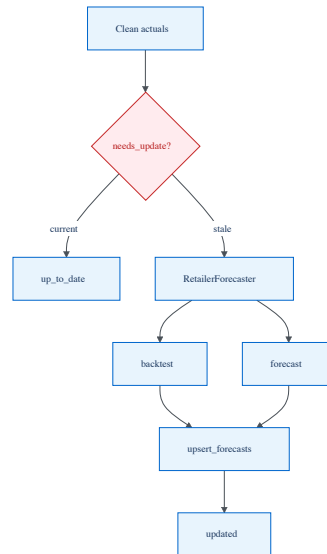


Diagram — Part B: Staleness check & forecast store

Diagram — Part B: Staleness check & forecast store



Return dict keys: `ticker`, `status`, `message`, `rows_upserted`, `store_ticker`, `last_actual_date`, `company_name`, `exchange`

Status values:

Status	Meaning
<code>updated</code>	Engine ran; DB upserted
<code>up_to_date</code>	<code>needs_update</code> false and not forced
<code>skipped</code>	Unsupported source or insufficient years
<code>no_data</code>	No revenue rows
<code>error</code>	Exception captured

`sync_all_eligible(force, periods, progress_callback, exclude_tickers)`

Iterates `RevenueForecastService.get_companies()` — same universe as Revenue Estimates (≥ 3 annual revenue points).

`clear_revenue_forecast_caches()`

```

RevenueForecastService.get_company_dashboard.clear()
RevenueForecastService.get_companies.clear()

```

Database — `coreiq_model_forecasts`

Written by `forecast_store.upsert_forecasts()`.

Staleness — `needs_update(ticker, last_actual_date)`

Compares latest annual actual `fiscal_date_ending` against `MAX(last_actual_date)` stored per ticker/metric.

Also forces recompute if `computed_at < 2026-05-11` (dedup fix cutoff `_DEDUP_FIX_DATE`).

Key columns (conceptual)

Column	Purpose
<code>ticker</code>	Composite for YFinance (e.g. <code>TSCO.L</code>)
<code>fiscal_year</code>	Forecast year
<code>model_key</code>	linear, cagr, ensemble, scenario_*, etc.
<code>value_millions</code>	Forecast value
<code>metric</code>	<code>total_revenue</code>
<code>mape</code>	Backtest accuracy
<code>last_actual_date</code>	Actuals snapshot used
<code>computed_at</code>	When forecast was stored
<code>company_name</code> , <code>exchange</code>	Denormalized (via <code>forecast_refresh_service</code>)

Source actuals tables

Source	Table
SEC	<code>coreiq_av_financials_income_statement</code>
YFinance	<code>coreiq_yf_financials_income_statement</code>

Forecasting Engine

```
utils.retailer_forecaster.RetailerForecaster :
```

- Six models: linear, cagr, exp_smoothing, holt, ma_trend, weighted_avg
- `backtest(holdout_years=2)` — ranks by MAPE
- `forecast(periods)` — includes ensemble (top 3 MAPE) + scenario percentiles
- Data cleaning: duplicate year dedup, 10% of max revenue floor, outlier detection

Session State

Key	Purpose
<code>fc_admin_access_{email}</code>	Cached ACL boolean
<code>iam_page_select</code> , <code>iam_new_email</code> , <code>iam_new_perm</code>	IAM expander widgets
<code>iam_perm_{page}_{email}</code>	Per-user permission selectbox state
<code>fc_one_ticker</code>	Single ticker input
Widget keys for sync buttons	<code>fc_sync_all</code> , <code>fc_sync_one</code>

Performance Instrumentation

Page records `_timings` list:

- `hide_sidebar` + `render_styles`

- `get_current_user`
- `render_header`
- `check_access`
- `render_footer`

Displayed in expander: **Page load diagnostics — total Nms** with color-coded bars (>300ms red).

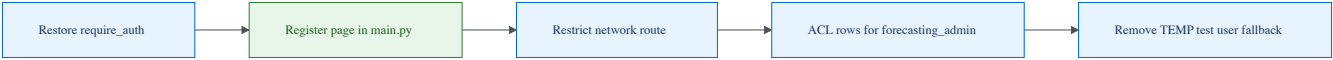
Relationship to `forecasting.py`

Aspect	forecasting.py	forecasting_admin.py
Primary mode	Read / visualize	Write / sync store
DB writes	Via refresh dialog (edit ACL)	Bulk admin sync
ACL page name	<code>forecasting</code>	<code>forecasting_admin</code>
Registered in main.py	Yes	No (currently)
Cache clear after write	Yes (<code>clear_revenue_forecast_caches</code>)	Yes

The Estimates page **Refresh Data** dialog calls the same `sync_forecast_for_ticker` / `sync_all_eligible` for users with `edit` on page `forecasting`.

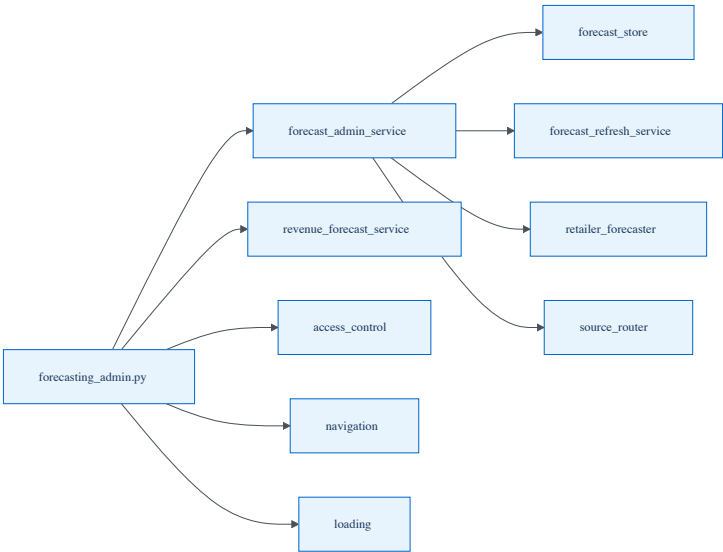
Security Checklist

Security Checklist



File Dependencies

File Dependencies



Generated from source analysis of the Market Data Portal (MDP). Last reviewed against codebase structure as of project documentation pass.

— End of Forecasting Admin Page — Technical Documentation —